

Digital Design & Computer Arch.

Lecture 11: Multi-Cycle and Pipelined Processor Design

Dr. Mohammad Sadrosadati

Prof. Onur Mutlu

ETH Zürich

Spring 2026

26 March 2026

Agenda for Today & Next Few Lectures

- Instruction Set Architectures (ISA): LC-3 and MIPS
- Assembly programming: LC-3 and MIPS
- Microarchitecture (principles & single-cycle uarch)
- Multi-cycle microarchitecture
- Pipelining
- Issues in Pipelining:
 - Control & Data Dependence Handling
 - State Maintenance and Recovery
- Out-of-Order Execution

Problem
Algorithm
Program/Language
System Software
SW/HW Interface
Micro-architecture
Logic
Devices
Electrons

Readings

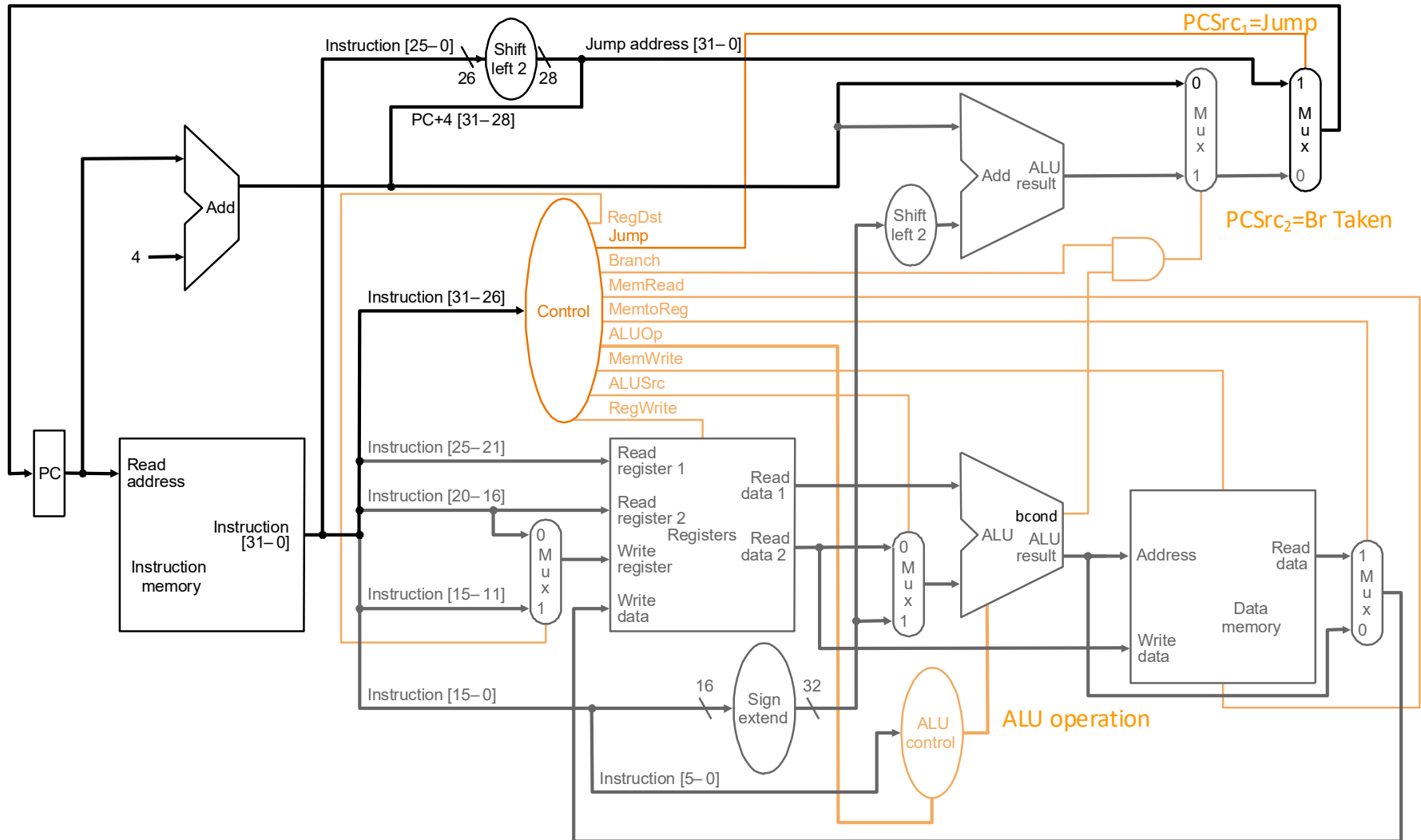
■ This week

- Introduction to microarchitecture and single-cycle microarchitecture
 - H&H, Chapter 7.1-7.3
 - P&P, Appendices A and C
- Multi-cycle microarchitecture
 - H&H, Chapter 7.4
 - P&P, Appendices A and C

■ Also this week

- Pipelining
 - H&H, Chapter 7.5
- Pipelining Issues
 - H&H, Chapter 7.7, 7.8.1-7.8.3

Recall: Single-Cycle Uarch



Performance Analysis of Our Single-Cycle Design

Recall: Example Single-Cycle Datapath Analysis

- Assume (for the design in the previous slide)
 - ❑ memory units (read or write): 200 ps
 - ❑ ALU and adders: 100 ps
 - ❑ register file (read or write): 50 ps
 - ❑ other logic or wire delay: 0 ps

steps	IF	ID	EX	MEM	WB	Delay
resources	mem	RF	ALU	mem	RF	
R-type	200	50	100		50	400
I-type	200	50	100		50	400
LW	200	50	100	200	50	600
SW	200	50	100	200		550
Branch	200	50	100			350
Jump	200					200

Recall: What About Control Logic?

- How does that affect the critical path?
- Food for thought for you:
 - Can control logic be on the critical path?
 - Historical example:
 - CDC 5600: control store access took too long...

What is Really the Slowest Instruction to Process?

- Real world: **Memory is slow (not magic)**
- What if memory *sometimes* takes 150ns to access?
- Does it make sense to have a simple register to register add or jump to take {150ns + all else to perform a memory operation}?
- And, what if you need to access memory more than once to process an instruction?
 - Which instructions require this?

Single Cycle uArch: Complexity

■ Contrived

- All instructions run as slow as the slowest instruction

■ Inefficient

- All instructions run as slow as the slowest instruction
- Must provide worst-case combinational resources in parallel as required by any instruction
- Need to replicate a resource if it is needed more than once by an instruction during different parts of the instruction processing cycle

■ Not necessarily the simplest way to implement an ISA

- Tough for complex instructions, e.g., REP MOVSB (x86) or INDEX (VAX)

■ Not easy to optimize/improve performance

- Optimizing the common case (frequent instructions) does not work
- Need to optimize the worst case all the time

(Micro)architecture Design Principles

■ Critical path design

- Find and **decrease the maximum combinational logic delay**
- Break a path into multiple cycles if it takes too long

■ Bread and butter (common case) design

- **Spend time and resources on where it matters most**
 - i.e., improve what the machine is really designed to do
- Common case vs. uncommon case

■ Balanced design

- **Balance** instruction/data flow through hardware components
- **Design to eliminate bottlenecks:** balance the hardware for the work

Single-Cycle Design vs. Design Principles

- Critical path design
- Bread and butter (common case) design
- Balanced design

How does a single-cycle microarchitecture fare with respect to these principles?

Aside: System Design Principles

- When designing computer systems/architectures, it is important to follow good principles
 - Actually, this is true for **any system design**
 - Real architectures, buildings, bridges, train stations, ...
 - Good consumer products
 - Security & safety-critical systems
 - Decision making systems
 - ...
- Remember: “principled design” from our second lecture
 - Frank Lloyd Wright: “architecture [...] based upon **principle**, and not upon **precedent**”

Aside: Principled Architecture

- “architecture [...] based upon **principle**, and not upon **precedent**”



This



That



Recall: Takeaways

- It all starts from the basic building blocks and design principles
- And, knowledge of how to use, apply, enhance them
- Underlying technology might change (e.g., steel vs. wood)
 - but methods of taking advantage of technology bear resemblance
 - methods used for design depend on the principles employed

Aside: System Design Principles

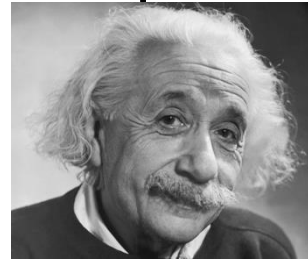
- We will continue to cover key principles in this course
- Here are some references where you can learn more
- Yale Patt, "[Requirements, Bottlenecks, and Good Fortune: Agents for Microprocessor Evolution](#)," Proc. of IEEE, 2001. (Levels of transformation, design point, etc)
- Mike Flynn, "[Very High-Speed Computing Systems](#)," Proc. of IEEE, 1966. (Flynn's Bottleneck → Balanced design)
- Gene M. Amdahl, "[Validity of the single processor approach to achieving large scale computing capabilities](#)," AFIPS Conference, April 1967. (Amdahl's Law → Common-case design)
- Butler W. Lampson, "[Hints for Computer System Design](#)," ACM Operating Systems Review, 1983.

A Key System Design Principle

- **Keep it simple**

- “Everything should be made as simple as possible, **but no simpler.**”

- Albert Einstein (paraphrased)

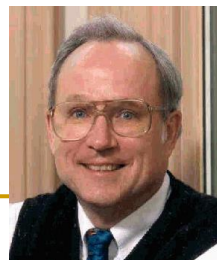


- And, **keep it low cost**: “An engineer is a person who can do for a dime what any fool can do for a dollar.”

- For more, see:

- Butler W. Lampson, “**Hints for Computer System Design**,” ACM Operating Systems Review, 1983, updated 2021.

- <https://arxiv.org/pdf/2011.02455>



Can We Do Better?

Multi-Cycle Microarchitectures

Multi-Cycle Microarchitectures

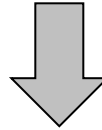
- Goal: Let each instruction take (close to) only as much time it really needs
- Idea
 - Determine clock cycle time independently of instruction processing time
 - Each instruction takes as many clock cycles as it needs to take
 - Multiple state transitions per instruction
 - The states followed by each instruction is different

Recall: The “Process Instruction” Step

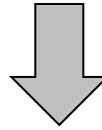
- ISA specifies abstractly what AS' should be, given an instruction and AS
 - It defines an **abstract finite state machine** where
 - State = programmer-visible state
 - Next-state logic = instruction execution specification
 - From ISA point of view, there are no “intermediate states” between AS and AS' during instruction execution
 - One state transition per instruction
- Microarchitecture implements how AS is transformed to AS'
 - There are many choices in implementation
 - We can have programmer-invisible state to optimize the speed of instruction execution: **multiple** state transitions per instruction
 - Choice 1: $AS \rightarrow AS'$ (transform AS to AS' in a single clock cycle)
 - Choice 2: $AS \rightarrow AS+MS1 \rightarrow AS+MS2 \rightarrow AS+MS3 \rightarrow AS'$ (take multiple clock cycles to transform AS to AS')

Multi-Cycle Microarchitecture

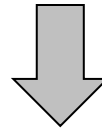
AS = Architectural (programmer visible) state
at the beginning of an instruction



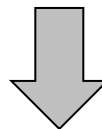
Step 1: Process part of instruction in one clock cycle



Step 2: Process part of instruction in the next clock cycle

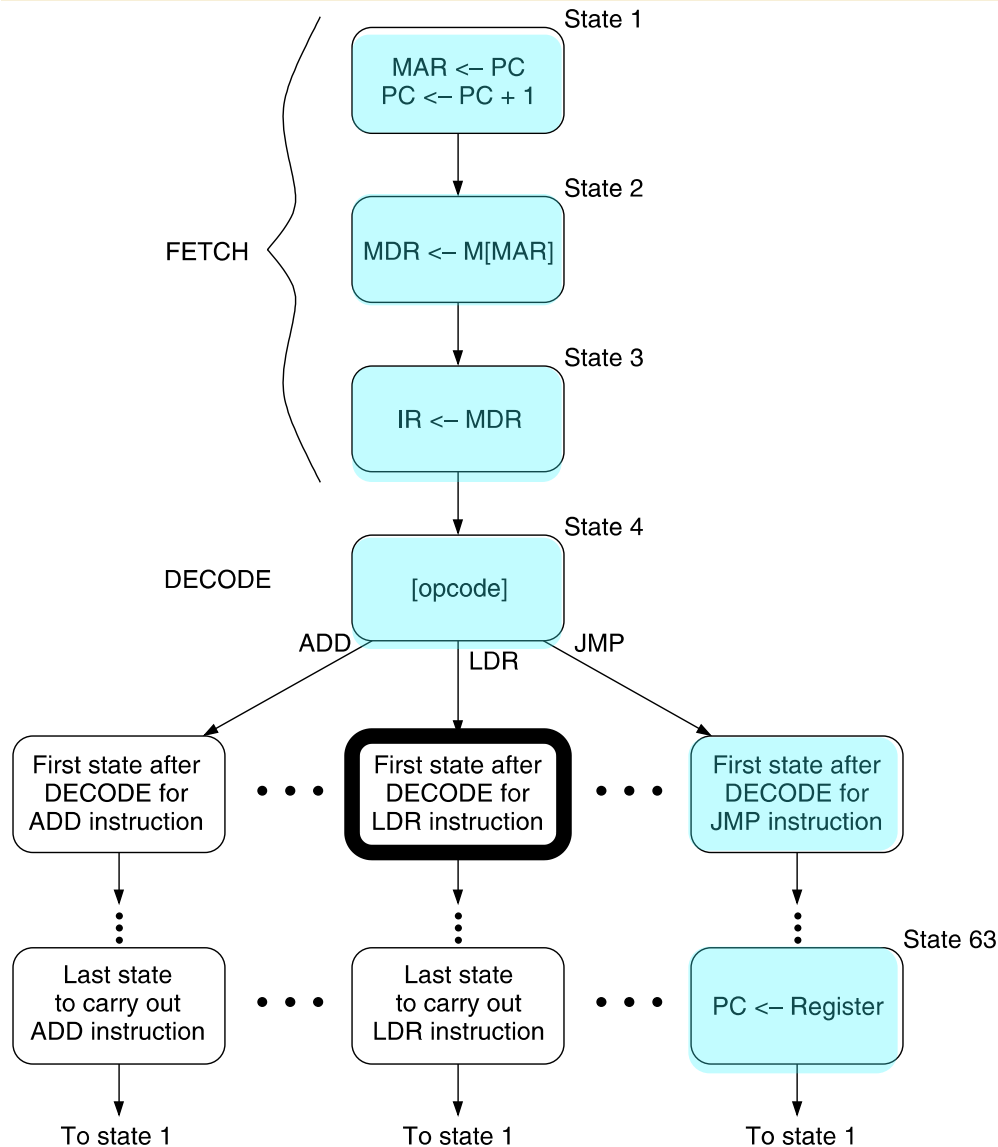


...



AS' = Architectural (programmer visible) state
at the end of a clock cycle

Recall: Control of the Instruction Cycle



- **State 1**
 - The FSM asserts GatePC and LD.MAR
 - It selects input (+1) in PCMUX and asserts LD.PC
- **State 2**
 - MDR is loaded with the instruction
- **State 3**
 - The FSM asserts GateMDR and LD.IR
- **State 4**
 - The FSM goes to next state depending on opcode
- **State 63**
 - JMP loads register into PC
- Full state diagram in Patt&Pattell, Appendix C

Figure 4.4 An abbreviated state diagram of the LC-3

This is an FSM Controlling a Multi-Cycle LC-3 Microarchitecture

Recall: Full State Machine for LC-3b

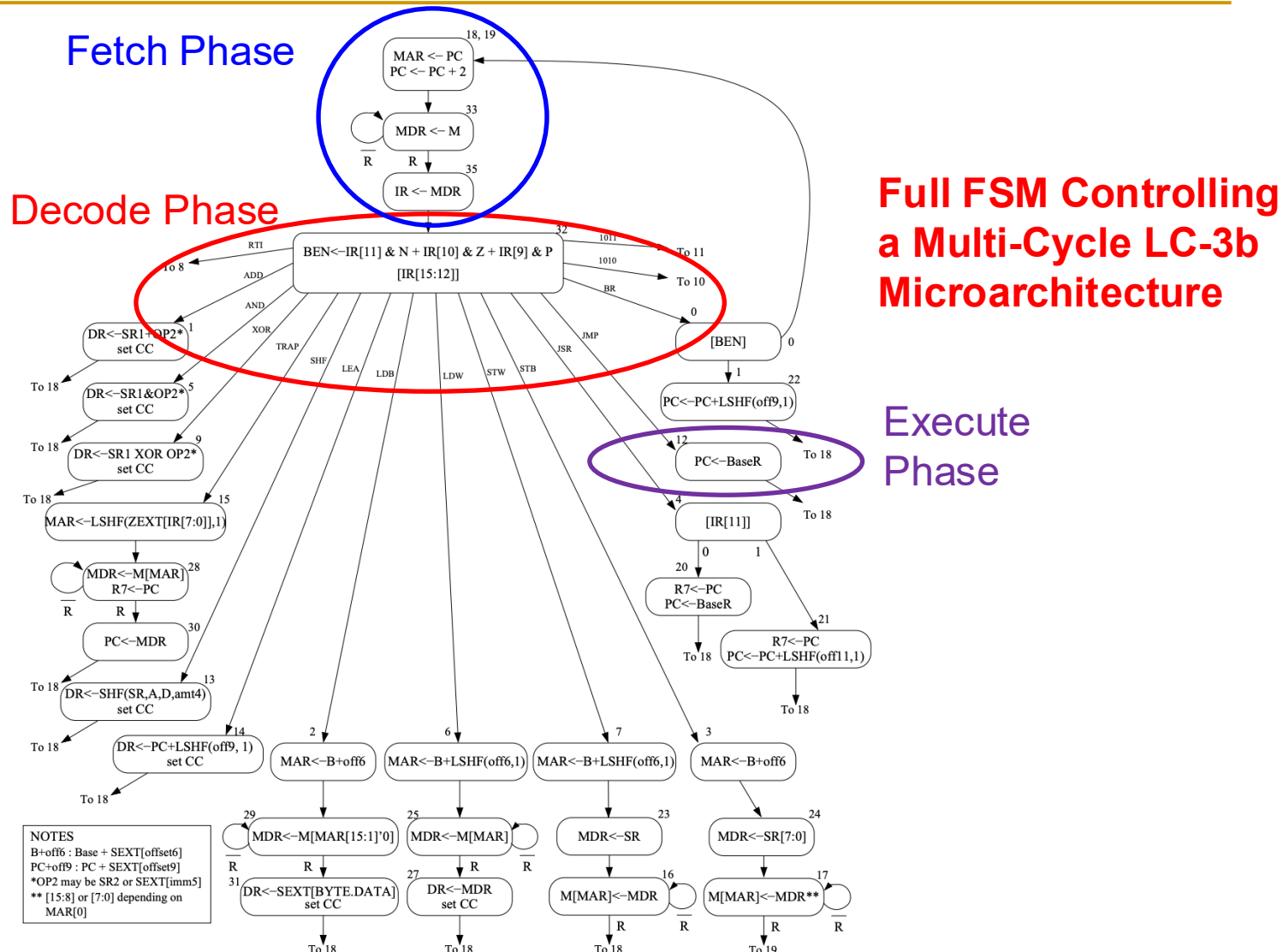


Figure C.2: A state machine for the LC-3b

Recall: LC-3 Multi-Cycle Implementation

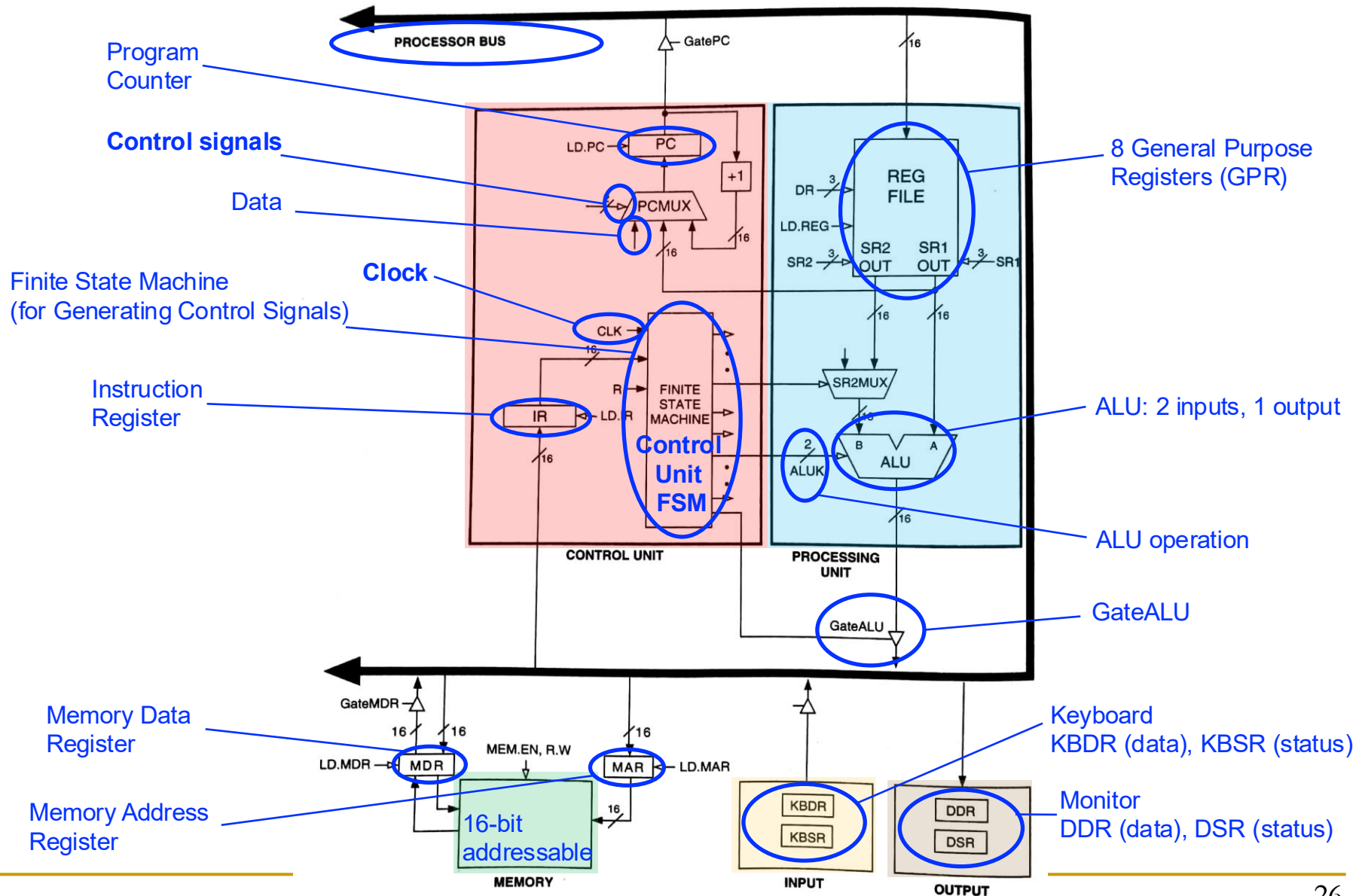


Figure 4.3 The LC-3 as an example of the von Neumann model

Benefits of Multi-Cycle Design

■ Critical path design

- ❑ Can keep reducing the critical path independently of the worst-case processing time of any instruction

■ Bread and butter (common case) design

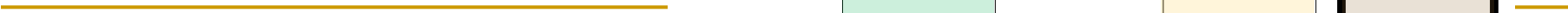
- ❑ Can optimize the number of states it takes to execute “important” instructions that make up much of the execution time

■ Balanced design

- ❑ No need to provide more capability or resources than really needed
 - An instruction that needs resource X multiple times does **not** require multiple X's to be implemented
 - Leads to more efficient hardware: Can reuse hardware components needed multiple times for an instruction

Downsides of Multi-Cycle Design

- **Need to store the intermediate results** at the end of each clock cycle
 - ❑ Hardware overhead for microarchitectural registers
 - ❑ Register setup/hold overhead (i.e., sequencing overhead) is paid multiple times for an instruction
- **Limited concurrency**
 - ❑ Only a small part of the machine is used at any point in time



Remember: Performance Analysis

- Execution time of a single instruction
 - **{CPI} x {clock cycle time}** CPI: Cycles Per Instruction
- Execution time of an entire program
 - Sum over all instructions [**{CPI} x {clock cycle time}**]
 - **{# of instructions} x {Average CPI} x {clock cycle time}**
- Single-cycle microarchitecture performance
 - CPI = 1
 - Clock cycle time = long
- Multi-cycle microarchitecture performance
 - CPI = different for each instruction
 - Average CPI → hopefully small
 - Clock cycle time = short

In multi-cycle, we have two degrees of freedom to optimize independently

A Multi-Cycle Microarchitecture

A Closer Look

An Elegant Multi-Cycle Processor Design

- Maurice Wilkes, “[The Best Way to Design an Automatic Calculating Machine](#),” Manchester Univ. Computer Inaugural Conf., 1951.

THE BEST WAY TO DESIGN AN AUTOMATIC CALCULATING MACHINE

By M. V. Wilkes, M.A., Ph.D., F.R.A.S.



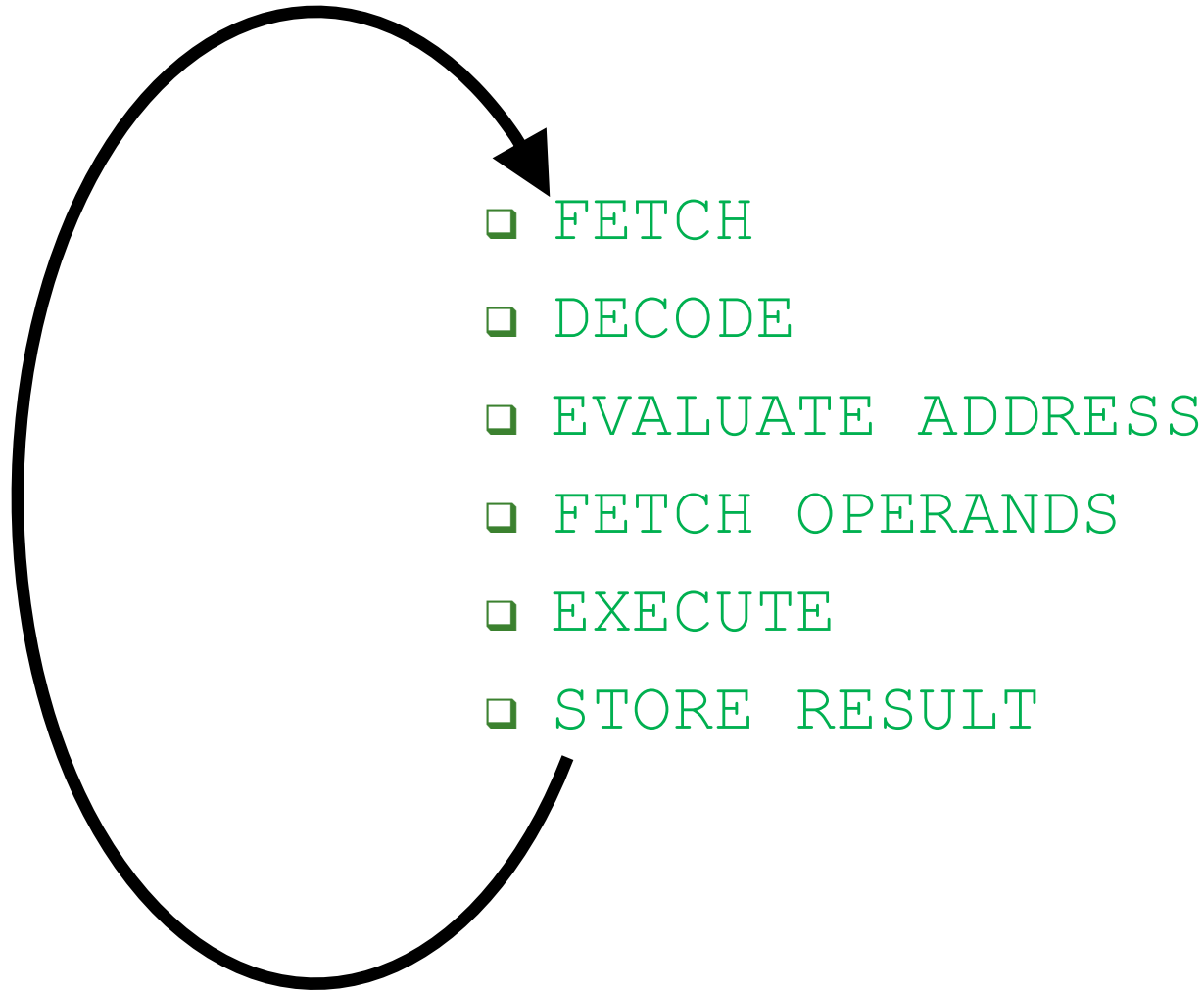
- An elegant implementation:
 - [The concept of microcoded/microprogrammed machines](#)

Multi-Cycle Microarchitectures

- Key Idea for Realization

- One can implement the “process instruction” step as a finite state machine that sequences between states and eventually returns back to the “fetch instruction” state
- A state is defined by the control signals asserted in it
- Control signals for the next state are determined in current state

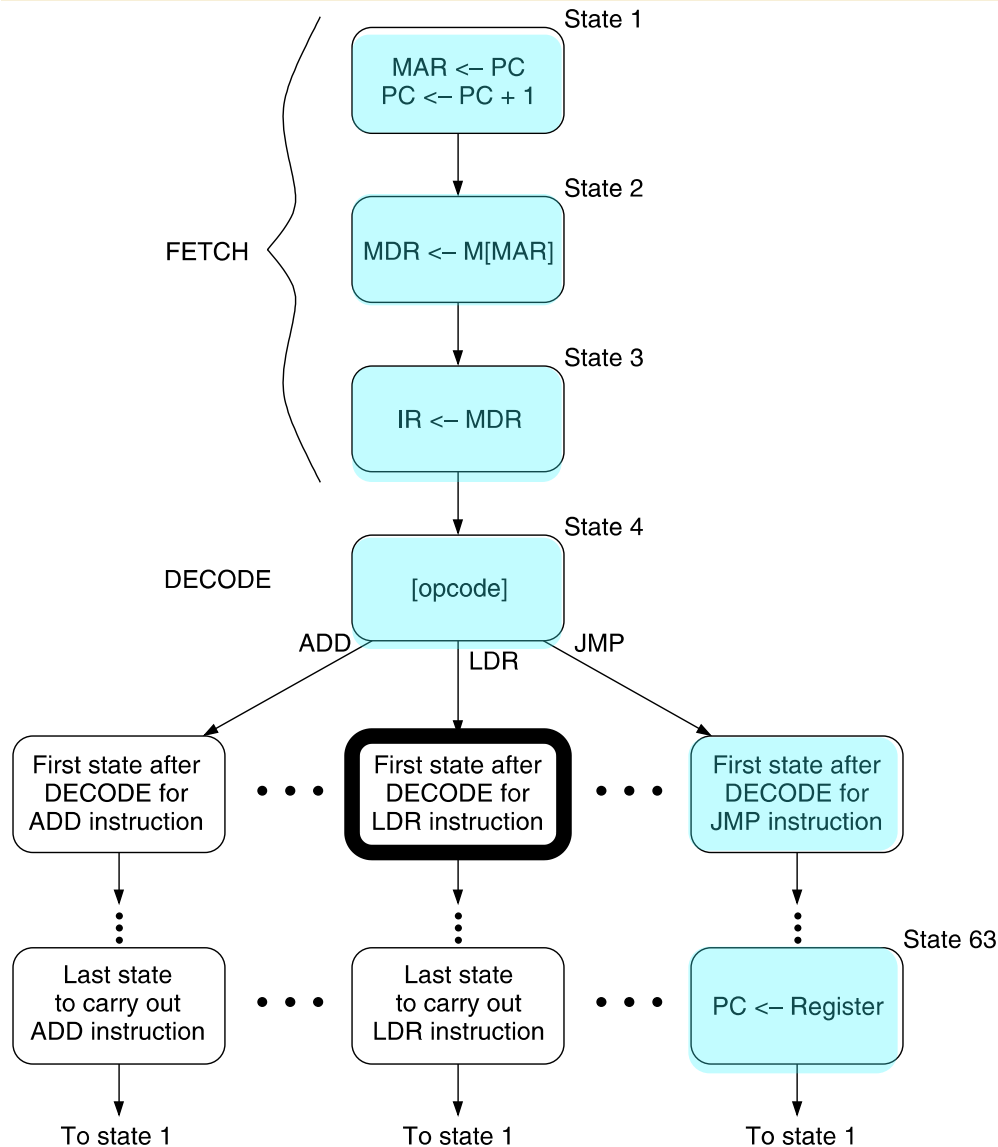
Recall: The Instruction Processing “Cycle”



A Basic Multi-Cycle Microarchitecture

- Instruction processing cycle divided into “states”
 - A stage in the instruction processing cycle can take multiple states
- A multi-cycle microarchitecture sequences from state to state to process an instruction
 - The behavior of the machine in a state is completely determined by control signals in that state
- The behavior of the entire processor is specified fully by a *finite state machine*
- In a state (clock cycle), control signals control two things:
 - How the datapath should process the data
 - How to generate the control signals for the (next) clock cycle

Recall: Control of the Instruction Cycle



- **State 1**
 - The FSM asserts GatePC and LD.MAR
 - It selects input (+1) in PCMUX and asserts LD.PC
- **State 2**
 - MDR is loaded with the instruction
- **State 3**
 - The FSM asserts GateMDR and LD.IR
- **State 4**
 - The FSM goes to next state depending on opcode
- **State 63**
 - JMP loads register into PC
- Full state diagram in Patt&Pattell, Appendix C

Figure 4.4 An abbreviated state diagram of the LC-3

This is an FSM Controlling a Multi-Cycle LC-3 Microarchitecture

Recall: Full State Machine for LC-3b

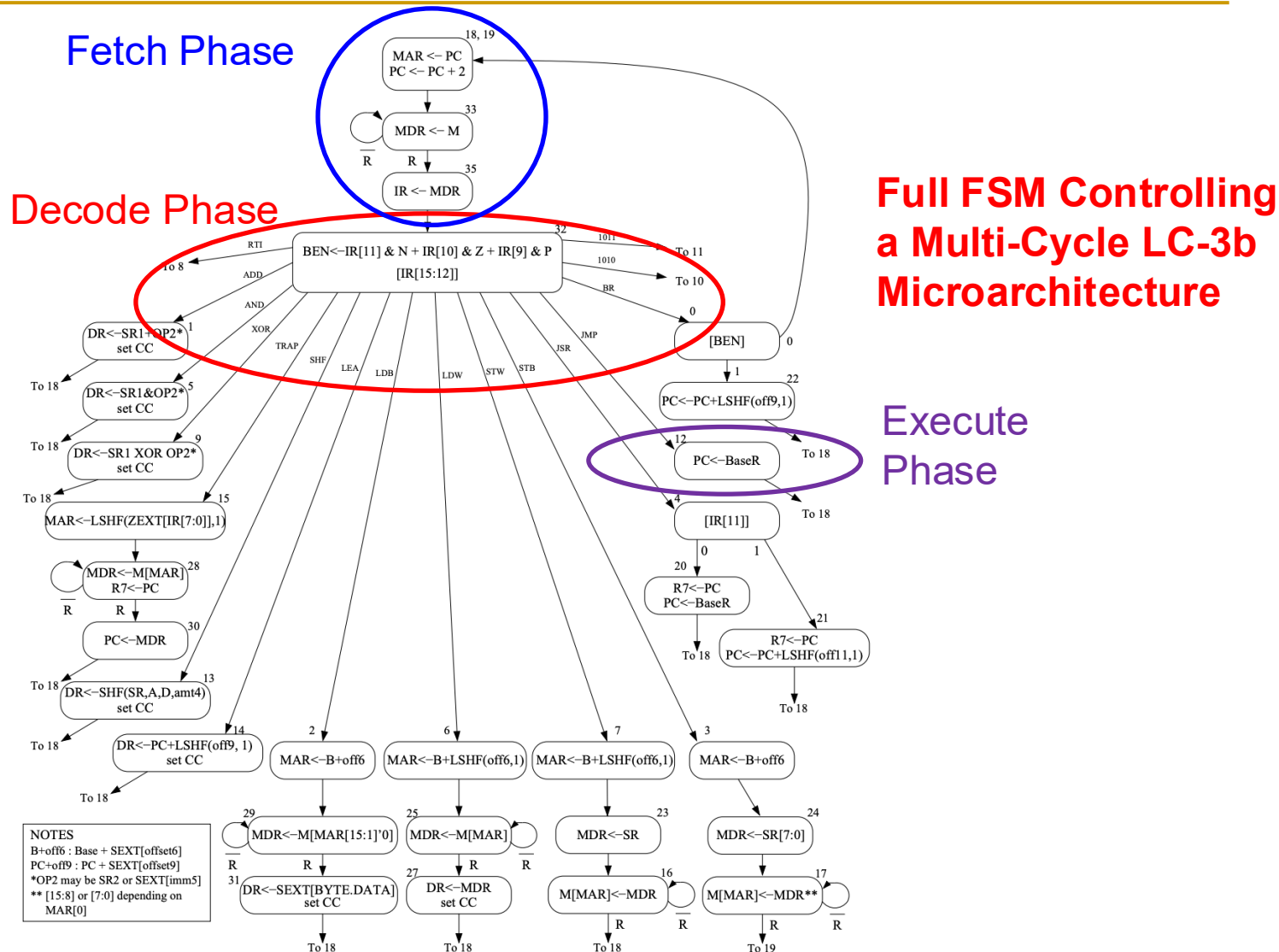
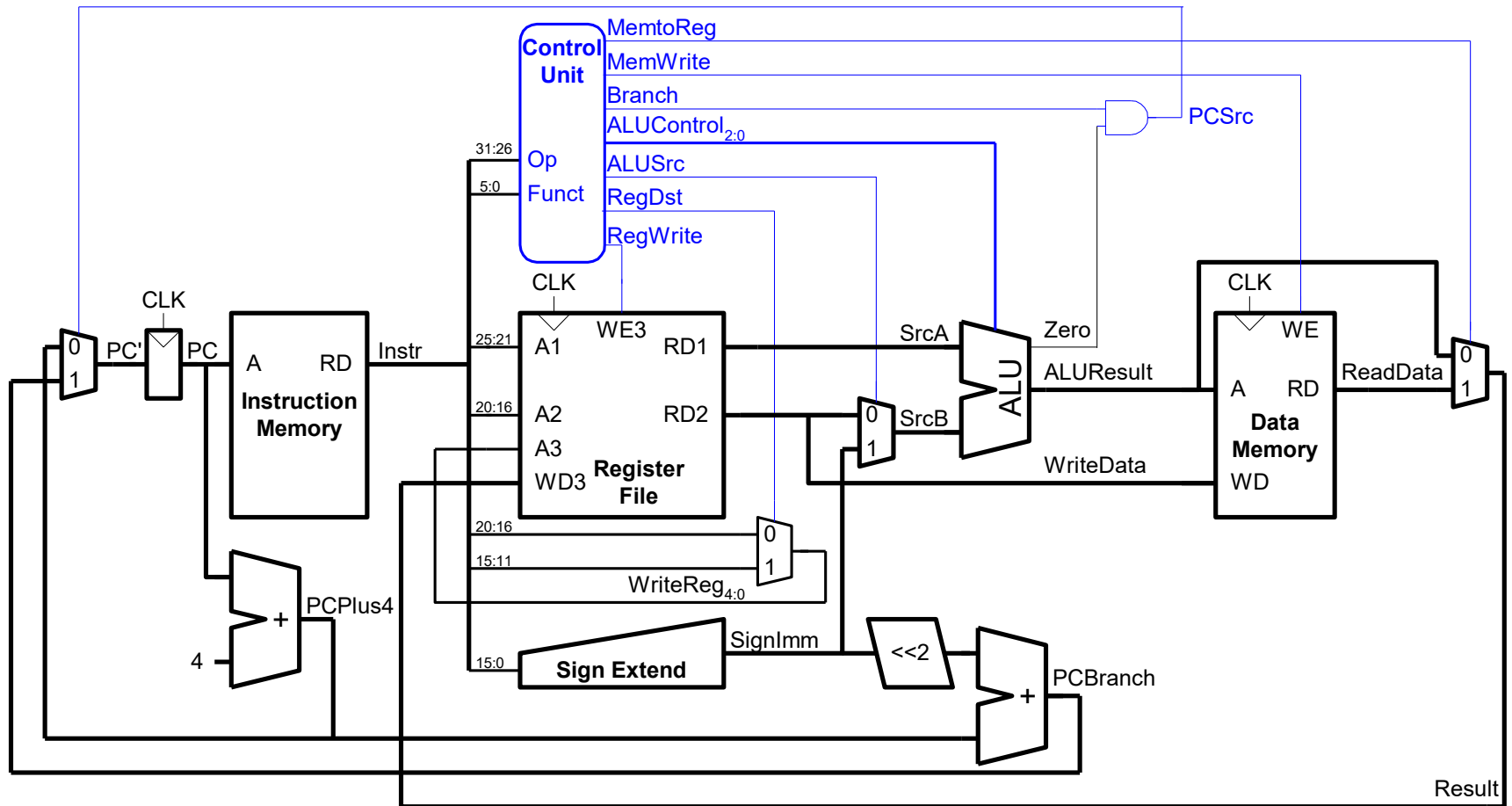


Figure C.2: A state machine for the LC-3b

Another Example Multi-Cycle Microarchitecture

Remember the Single-Cycle Uarch II (Readings)



Why Do We Want Multi-Cycle?

- Single-cycle microarchitecture:
 - cycle time limited by longest instruction (1w) → low clock frequency
 - three adders/ALUs and two memories → high hardware cost
- Multi-cycle microarchitecture:
 - + higher clock frequency
 - + simpler instructions take only a few clock cycles
 - + reuse expensive hardware across multiple cycles
 - hardware overhead for storing intermediate results
 - sequential logic overhead paid many times for each instruction
- Multi-cycle requires the same design steps as single cycle:
 - datapath
 - control logic

What Can We Optimize with Multi-Cycle

- Single-cycle microarchitecture uses **two memories**
 - One memory stores instructions, the other data
 - We want to use a single memory (lower cost)
- Single-cycle microarchitecture needs **three adders**
 - ALU, PC, Branch address calculation
 - We want to use only one ALU for all operations (lower cost)
- Single-cycle microarchitecture: **each instruction takes one cycle**
 - The slowest instruction slows down every single instruction
 - We want to determine clock cycle time independently of instruction processing time
 - Divide each instruction into multiple clock cycles
 - Simpler instructions can be very fast (compared to the slowest)

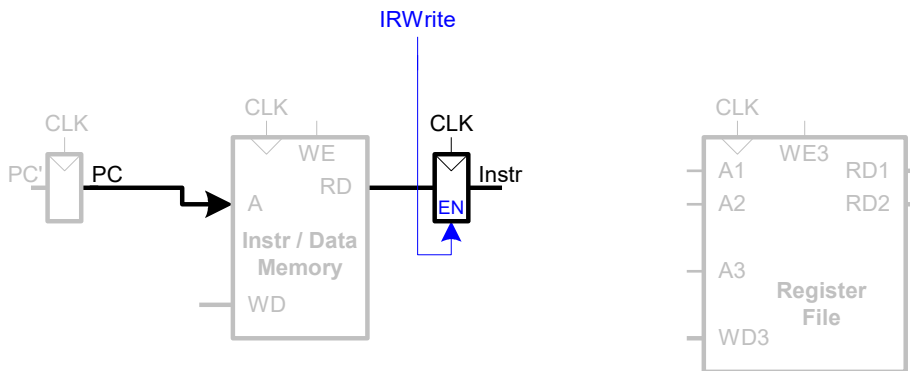
Let's Construct the Multi-Cycle Datapath

Consider the lw Instruction

- For an instruction such as: `lw $t0, 0x20($t1)`
- We need to:
 - Read the instruction from memory
 - Then read `$t1` from register array
 - Add the immediate value (`0x20`) to calculate the memory address
 - Read the content of this address
 - Write to the register `$t0` this content

Multi-Cycle Datapath: Instruction Fetch

- We will consider lw, but fetch is the same for all instructions
 - STEP 1: Fetch instruction

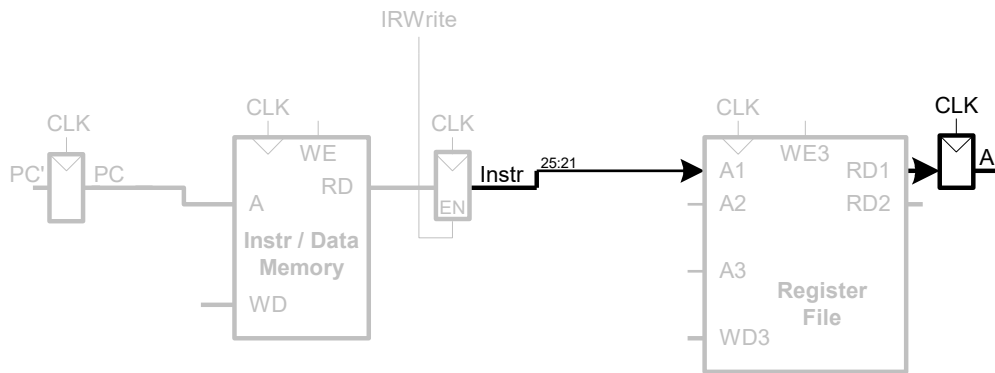


read from the memory location $[rs] + imm$ to location $[rt]$

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

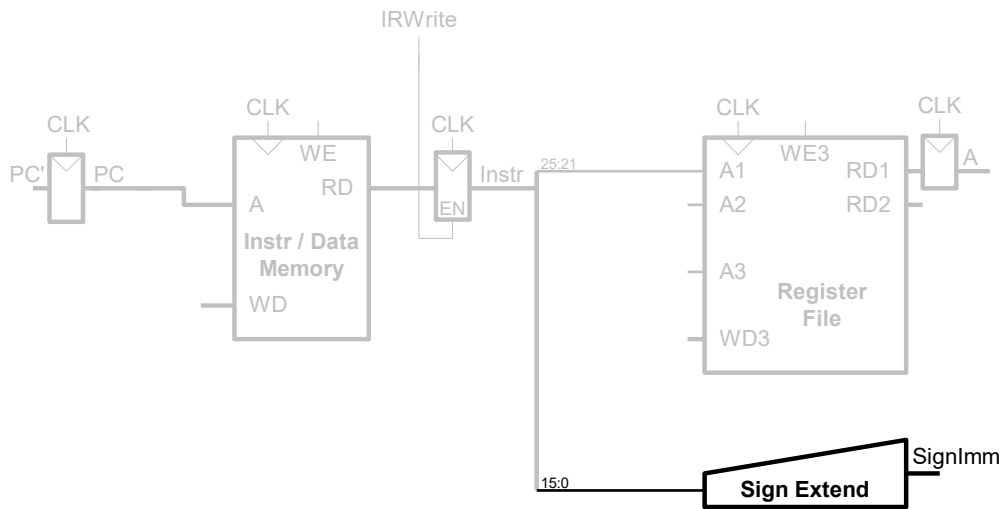
Multi-Cycle Datapath: lw register read



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

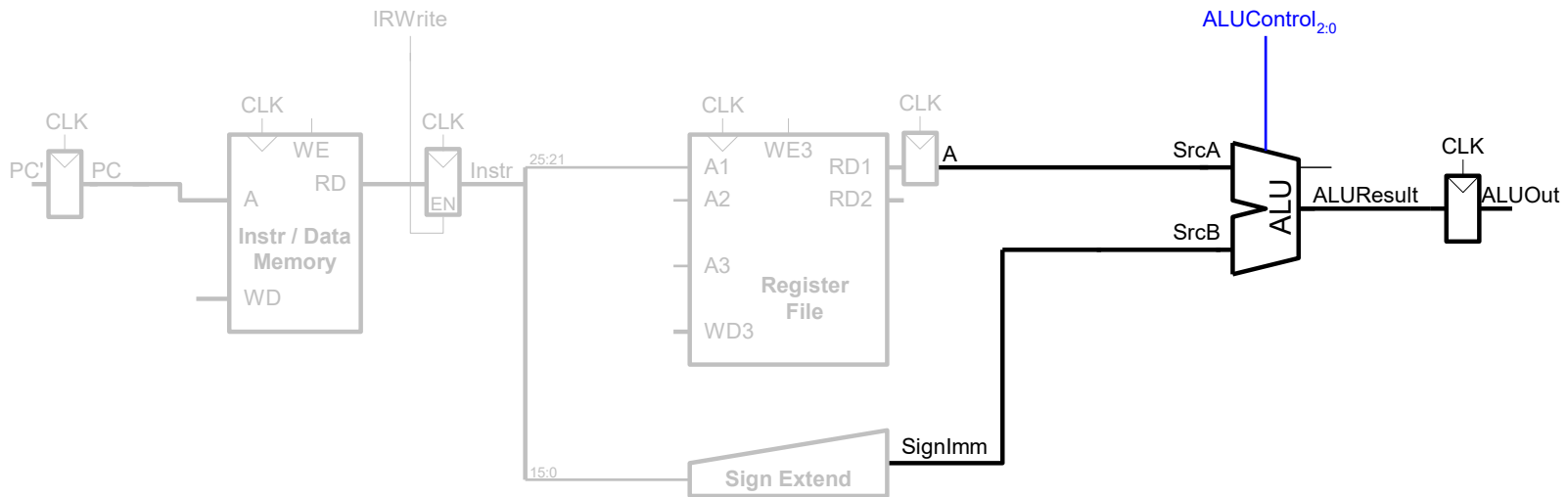
Multi-Cycle Datapath: lw immediate



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw address

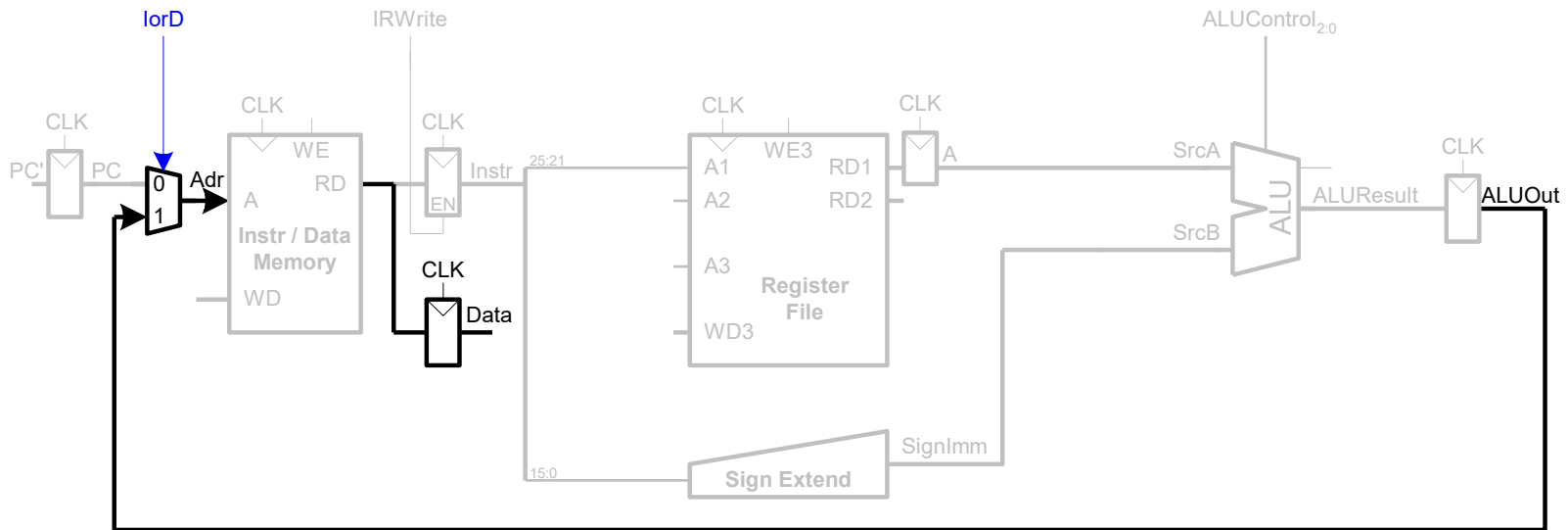


I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw memory read

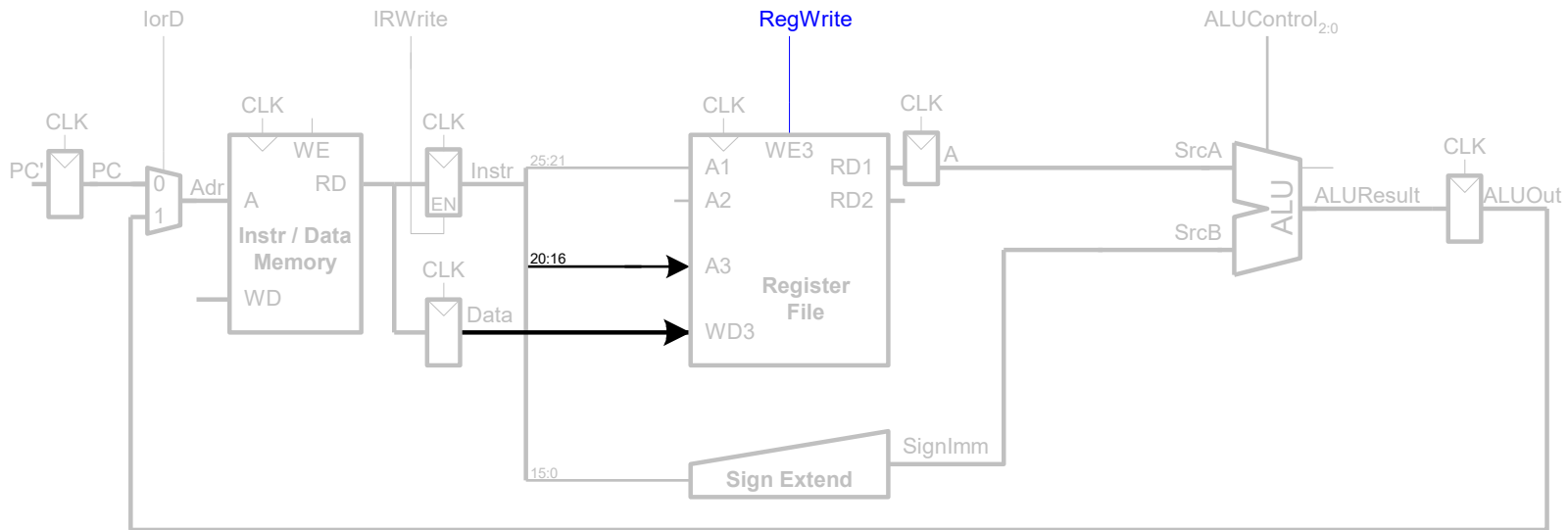
Now we can use a **single memory** to both fetch an instruction and fetch an operand (in different clock cycles)



I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Multi-Cycle Datapath: lw write register

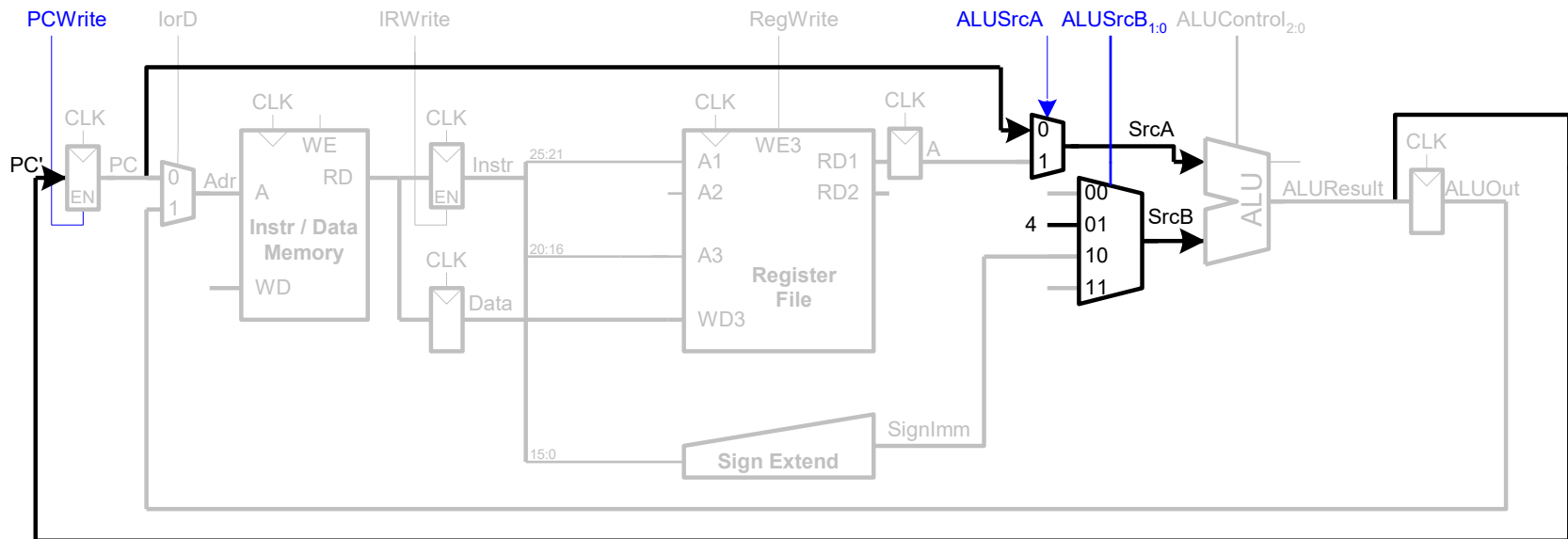


I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

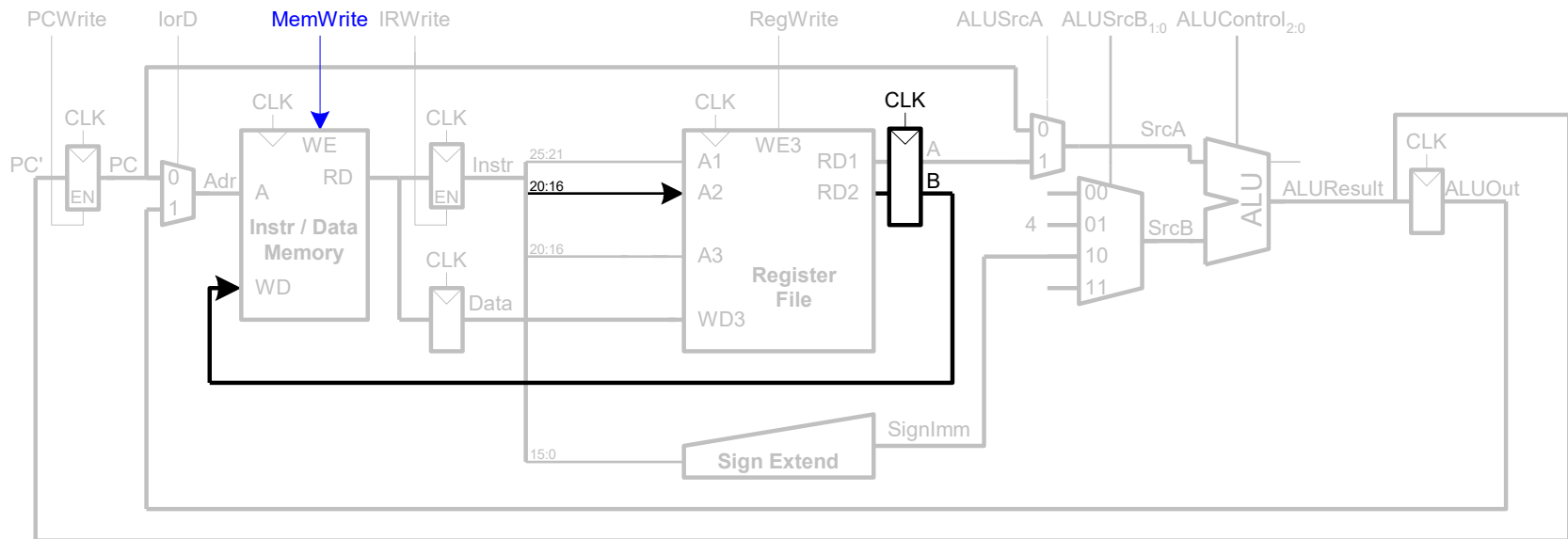
Multi-Cycle Datapath: increment PC

Now we can use **a single ALU** to both increment PC and do address calculation or arithmetic operations (in different clock cycles)



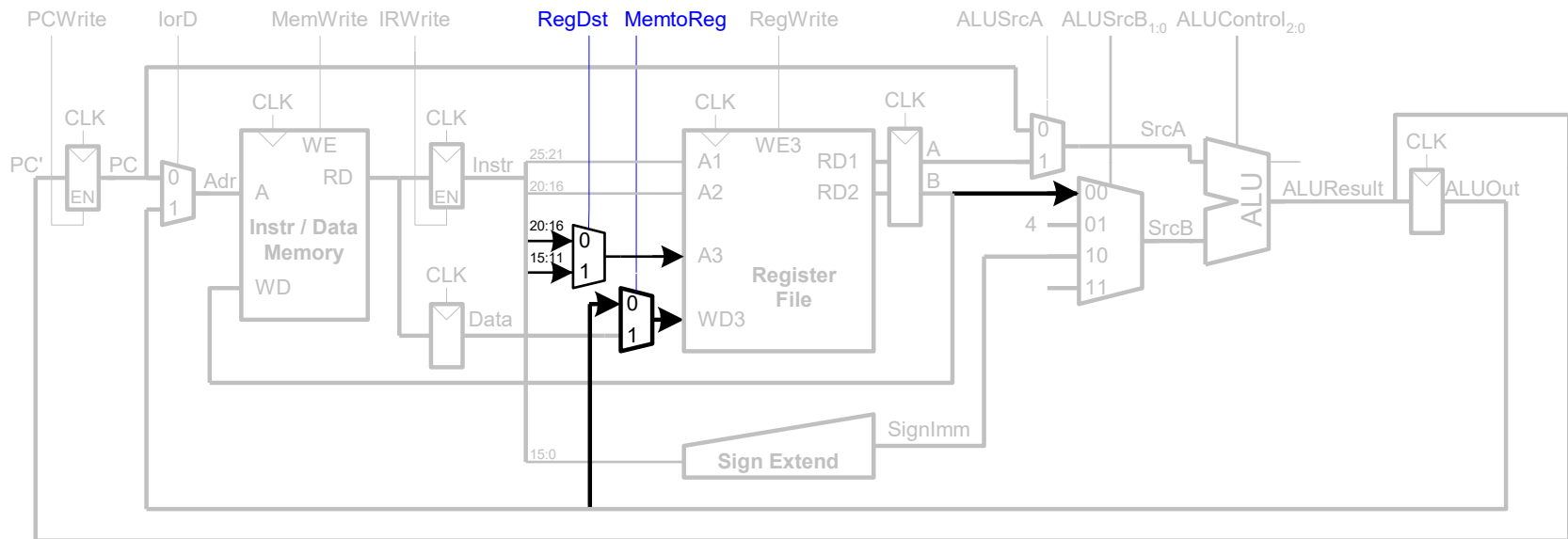
Multi-Cycle Datapath: sw

■ Write data in rt to memory



Multi-Cycle Datapath: R-type Instructions

- Read from rs and rt
 - Write ALUOut to register file
 - Write to rd (instead of rt)

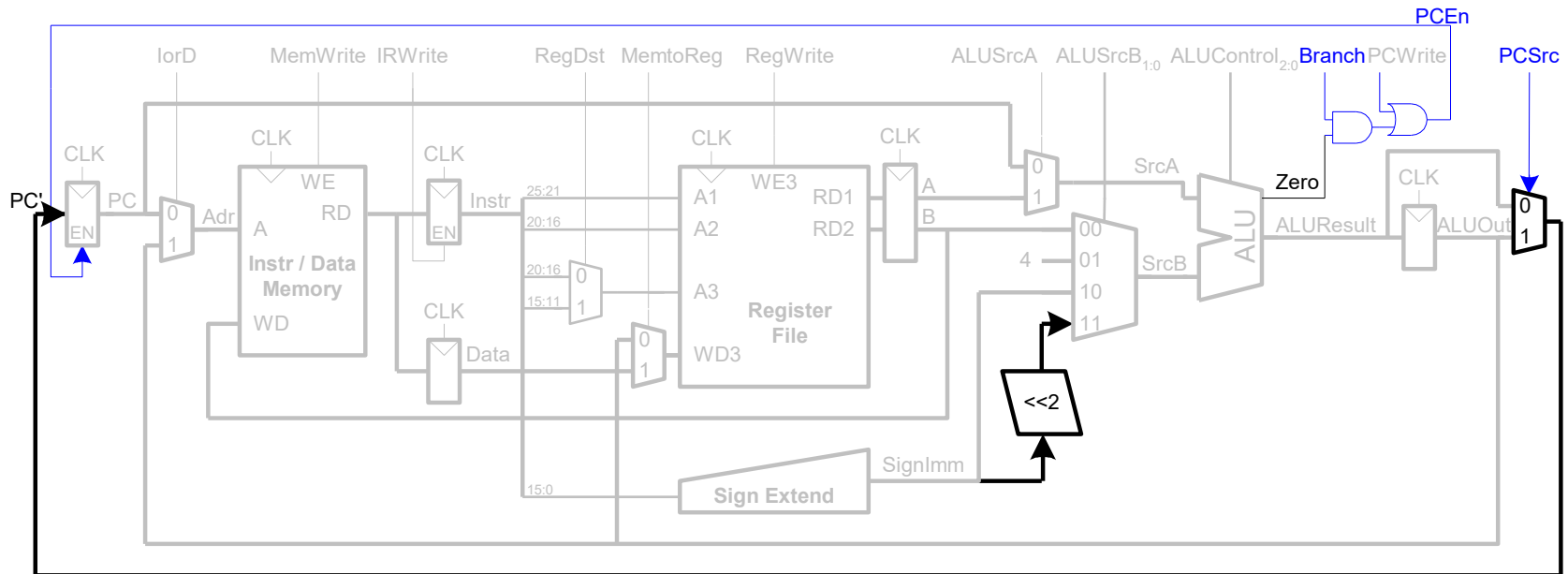


Multi-Cycle Datapath: beq

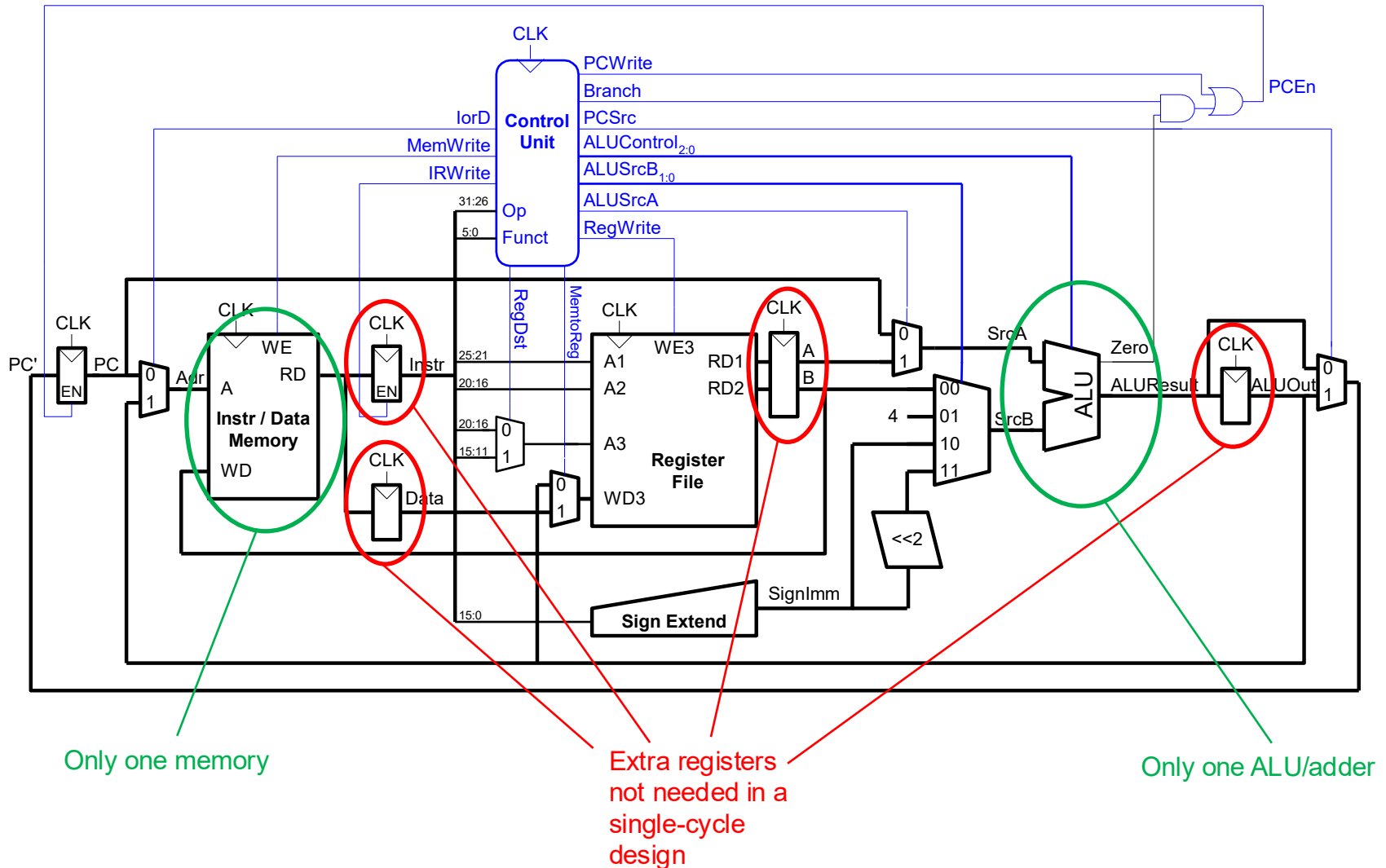
- Determine whether values in rs and rt are equal

- Calculate branch target address:

Target Address = (sign-extended immediate << 2) + (PC+4)

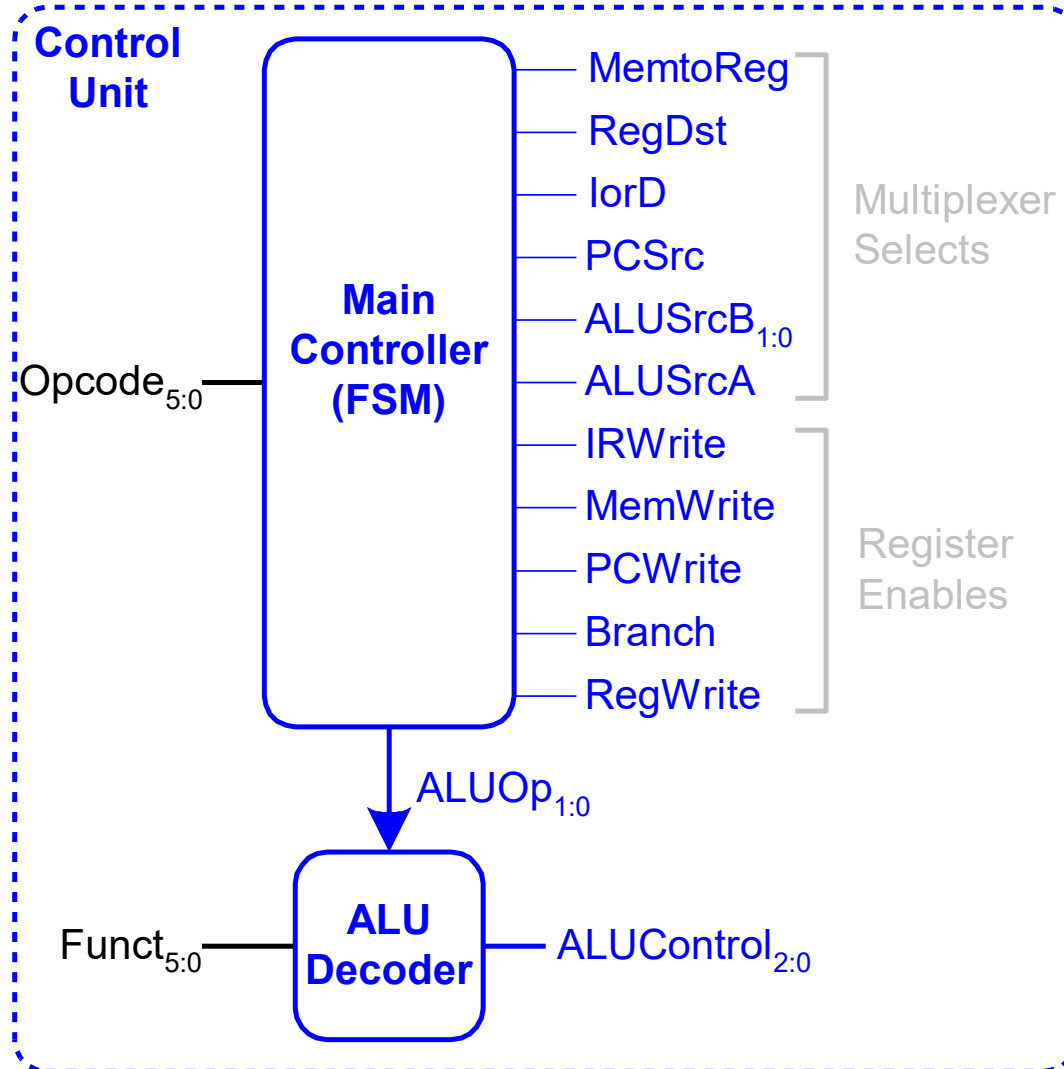


Complete Multi-Cycle Processor



Let's Construct the Multi-Cycle Control Logic

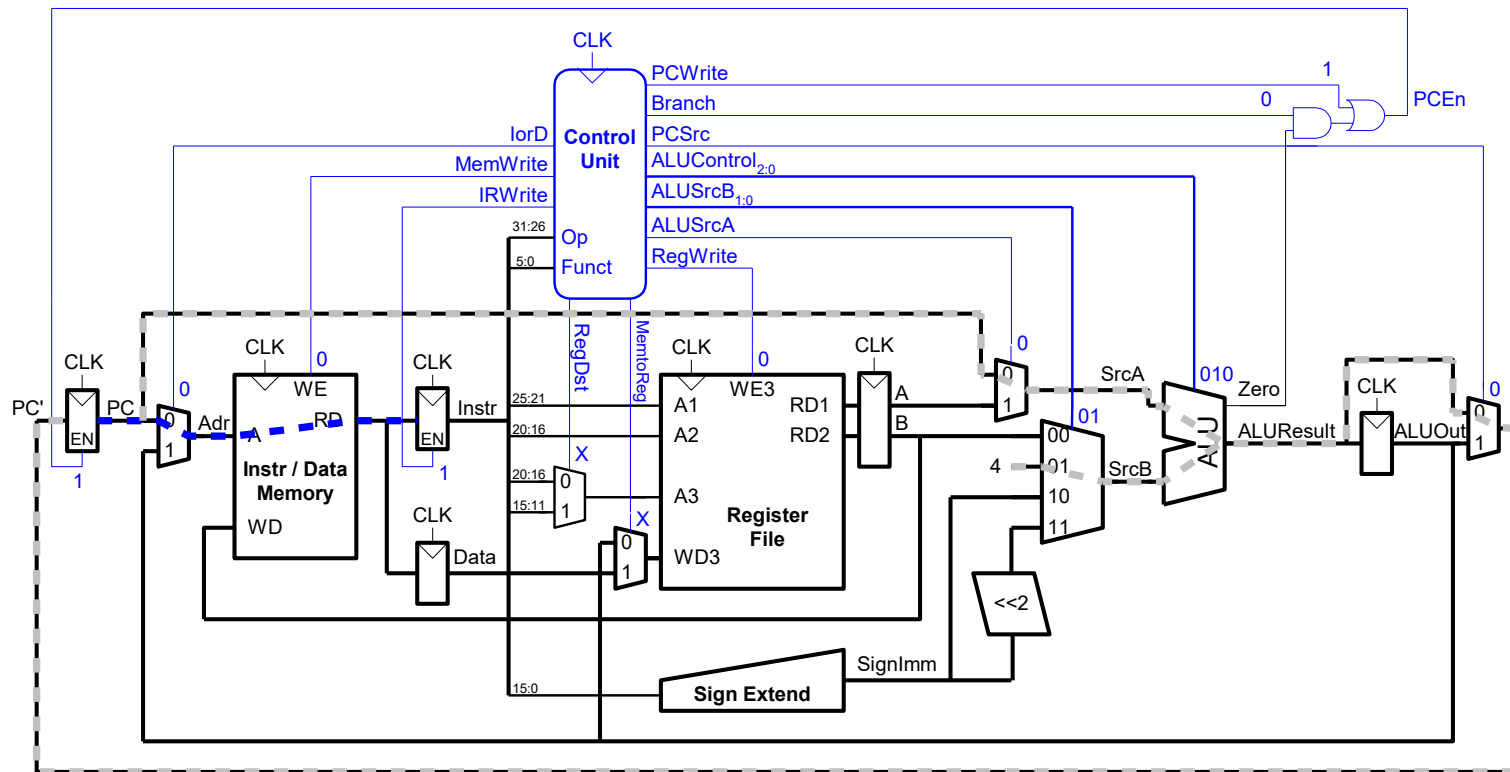
Control Unit



Main Controller FSM: Fetch

S0: Fetch

Reset

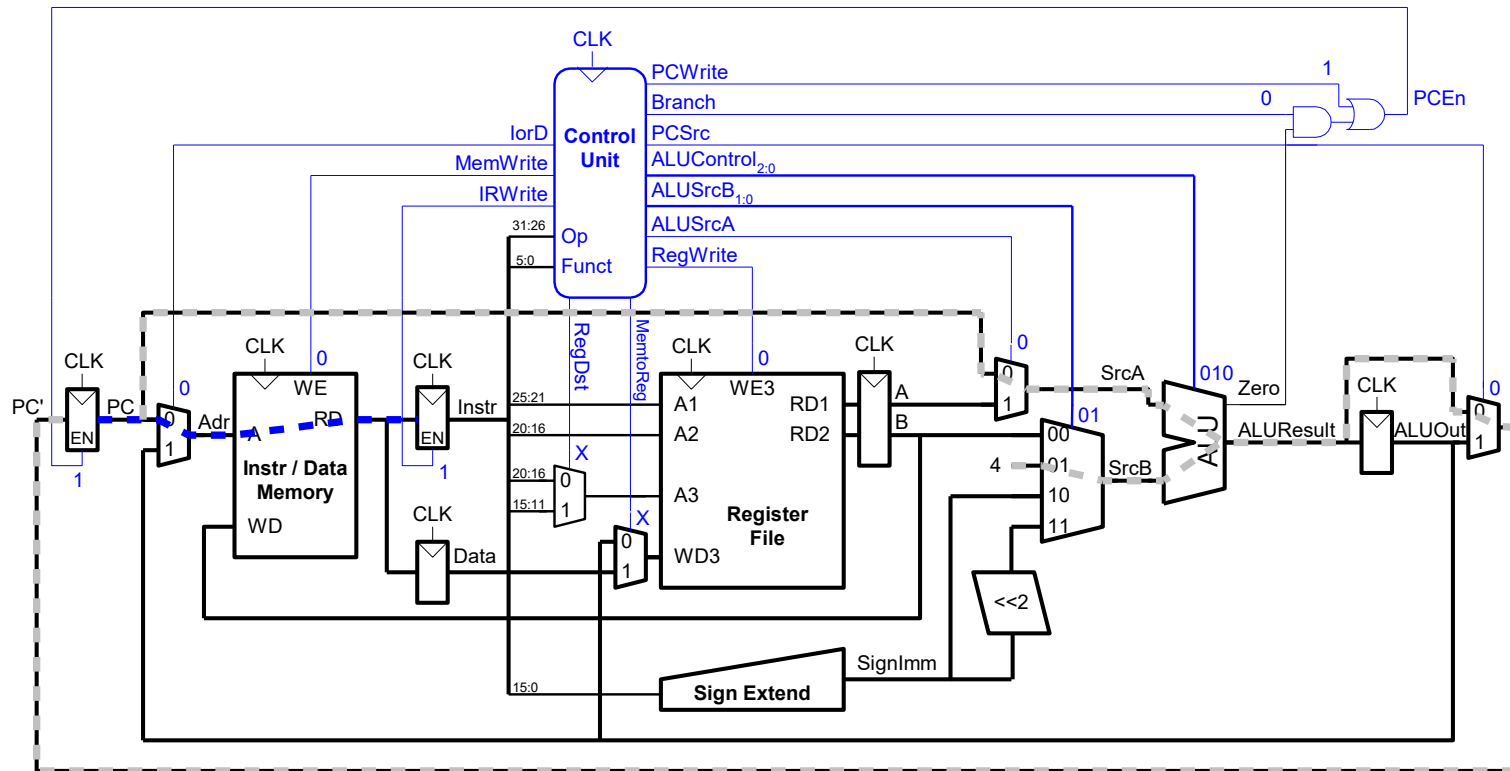


Main Controller FSM: Fetch

S0: Fetch

Reset

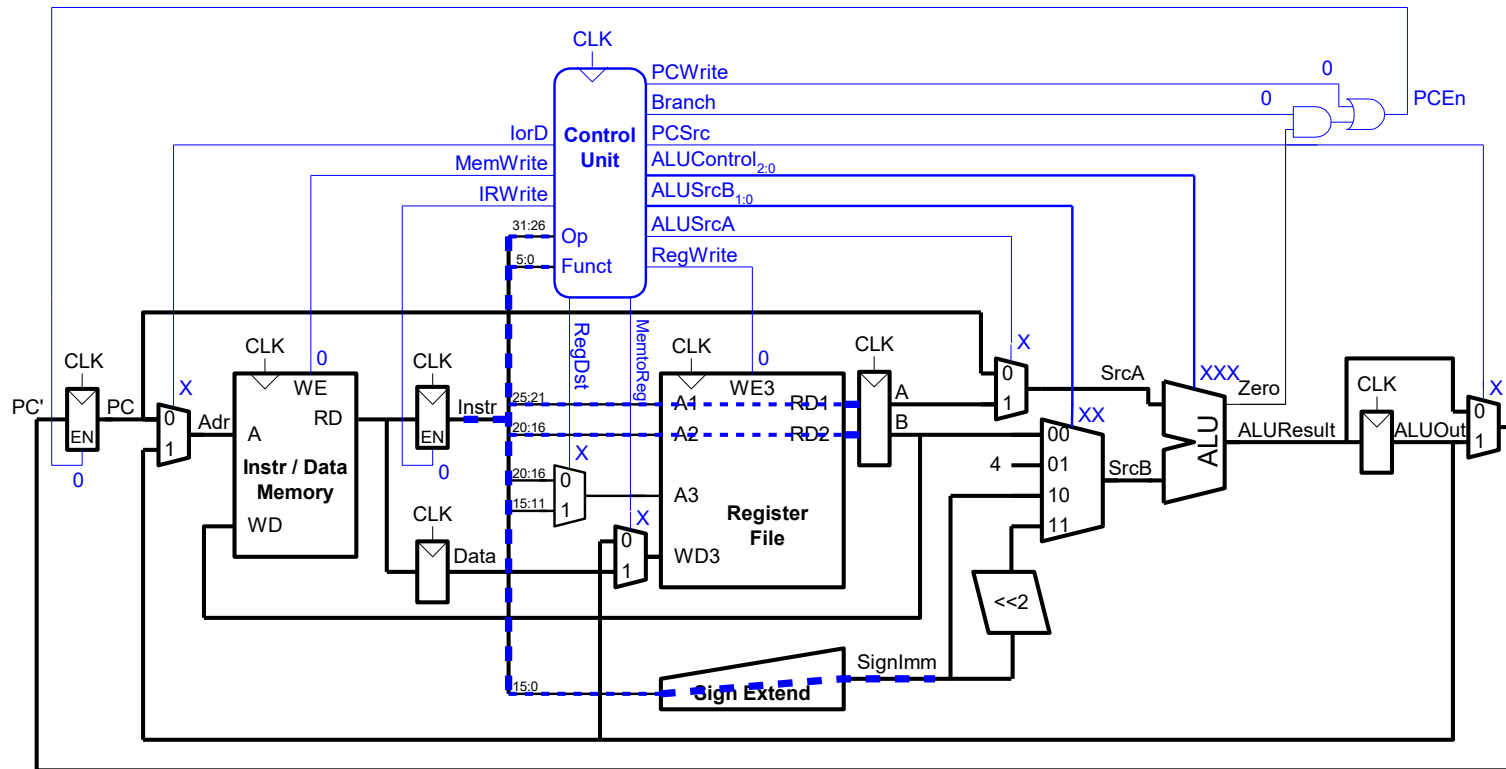
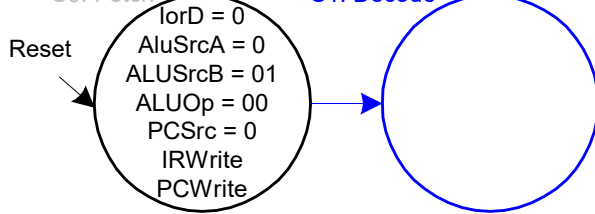
lorD = 0
AluSrcA = 0
ALUSrcB = 01
ALUOp = 00
PCSrc = 0
IRWrite
PCWrite



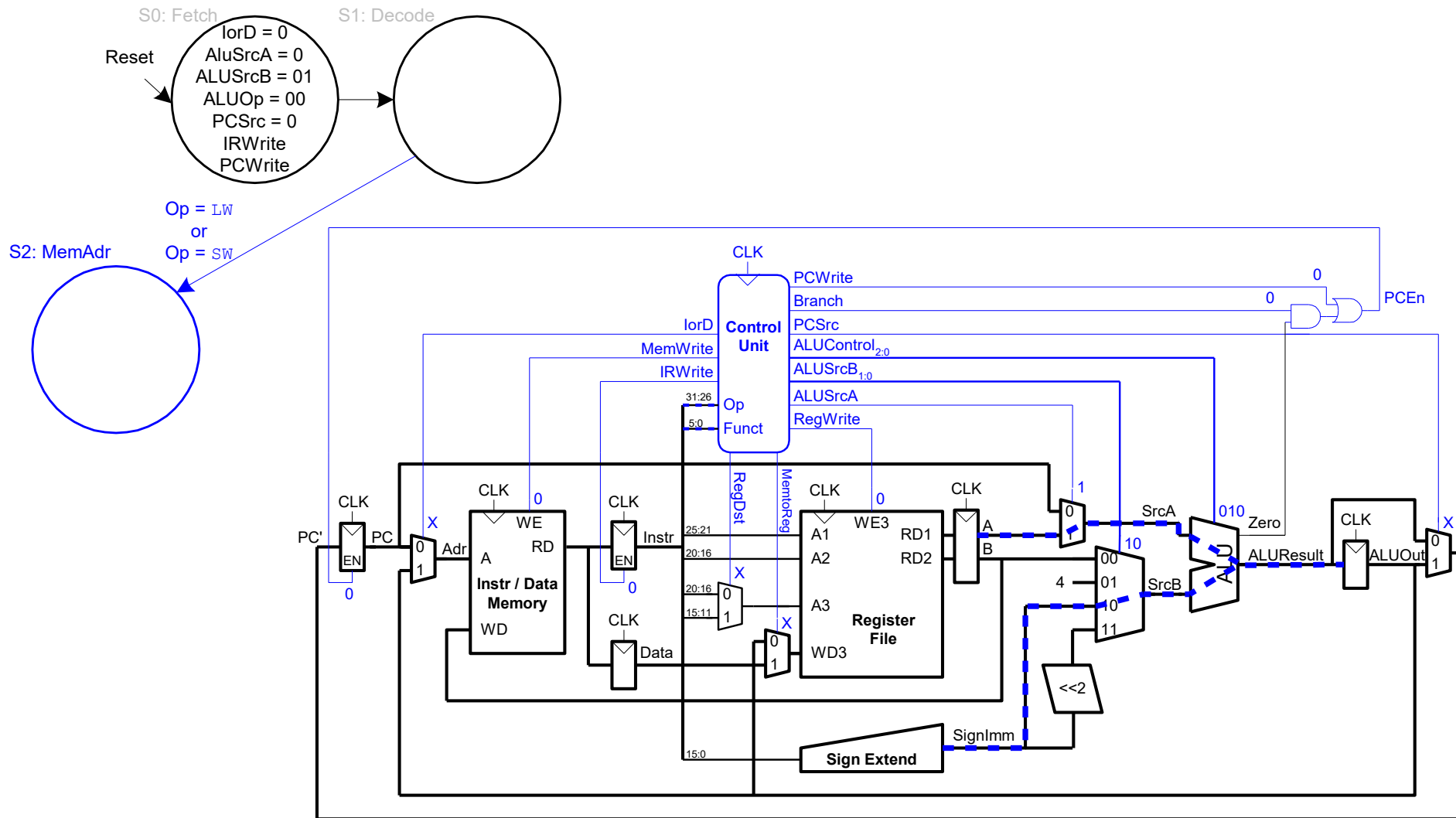
Main Controller FSM: Decode

S0: Fetch

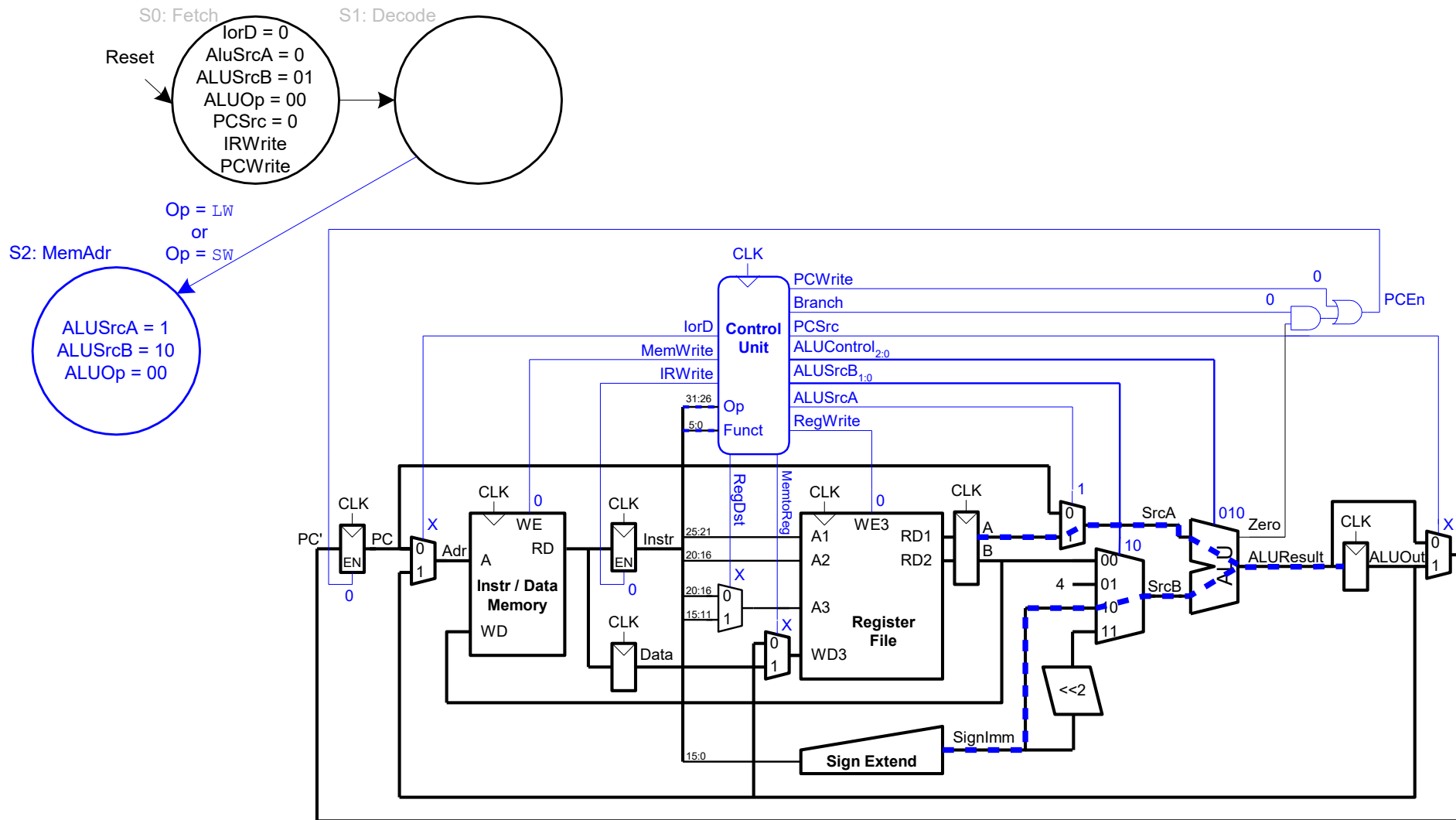
S1: Decode



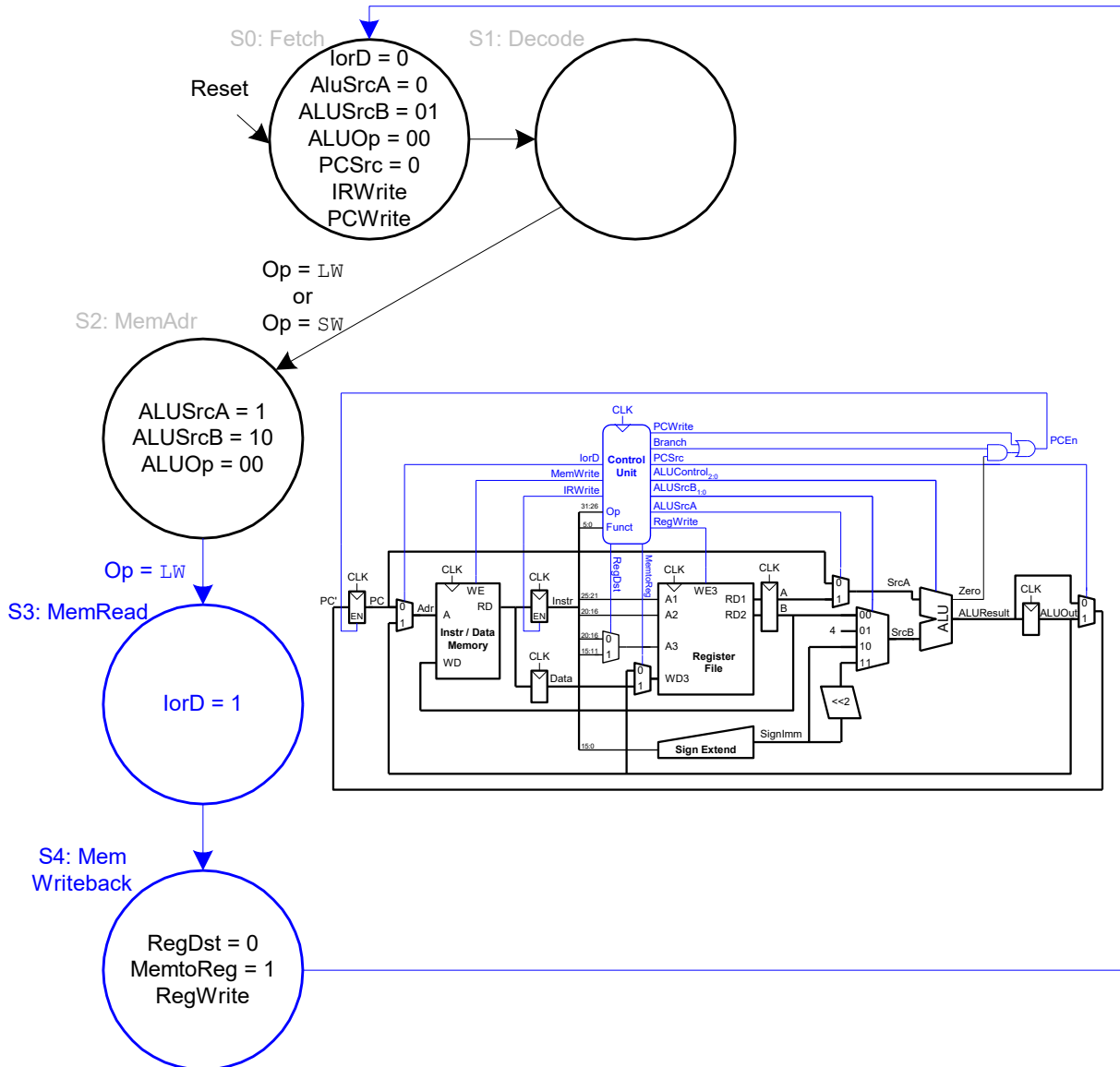
Main Controller FSM: Address Calculation



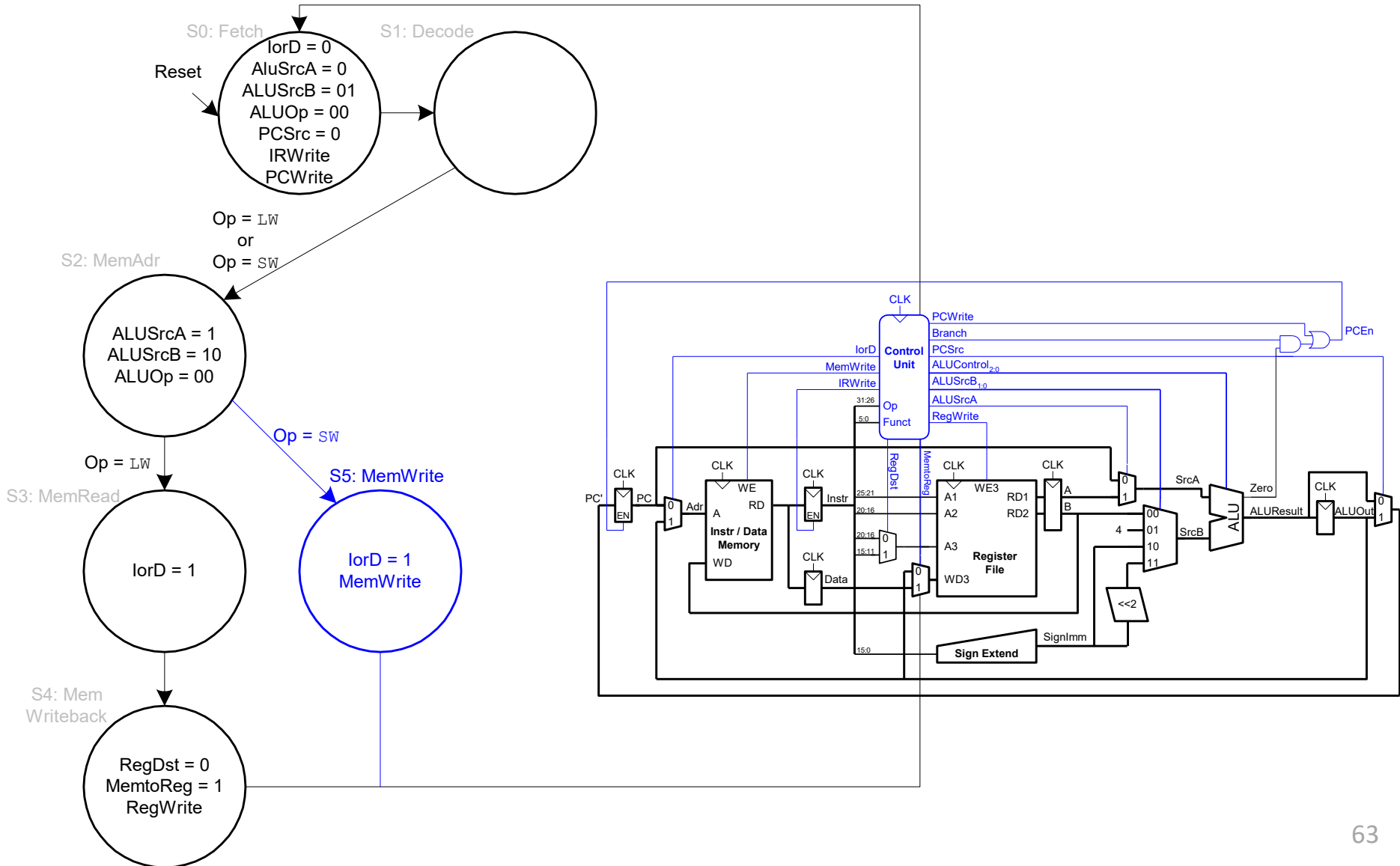
Main Controller FSM: Address Calculation



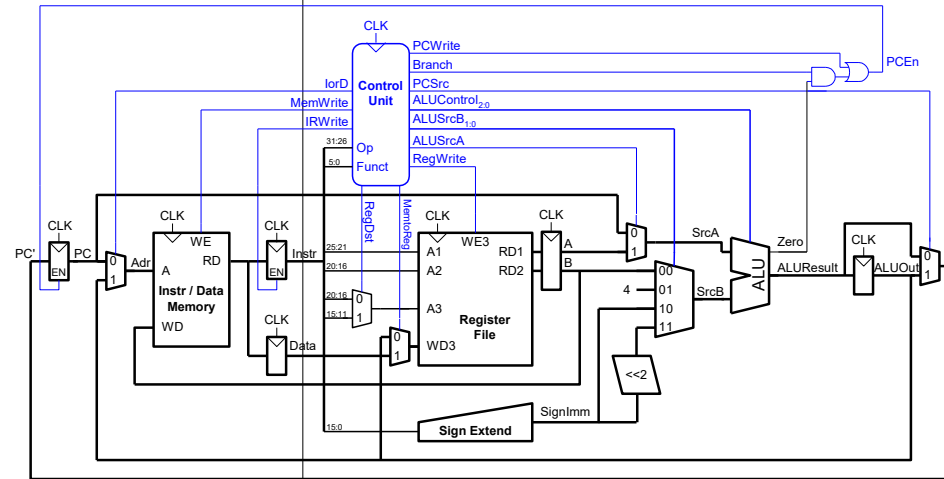
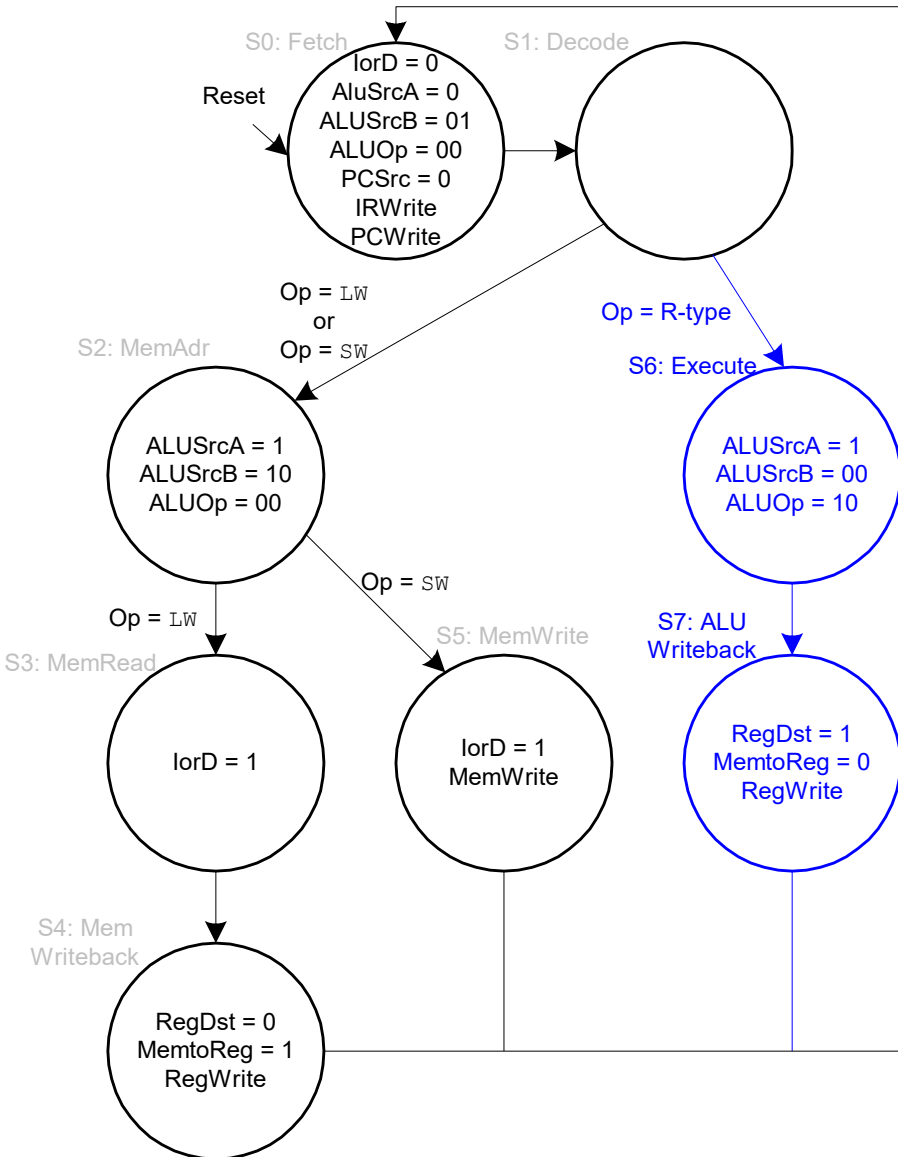
Main Controller FSM: lw



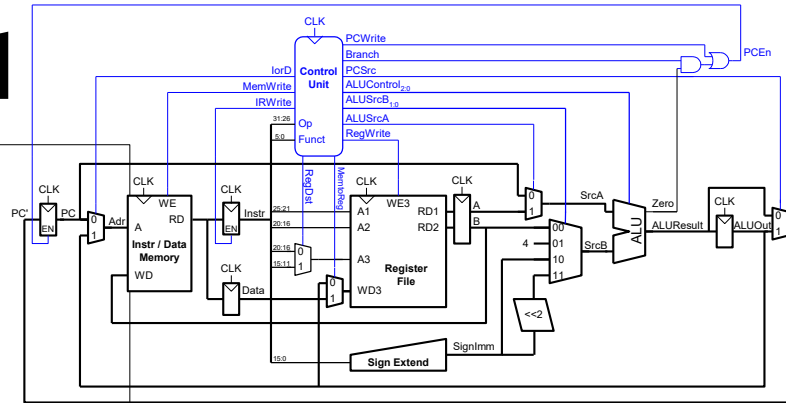
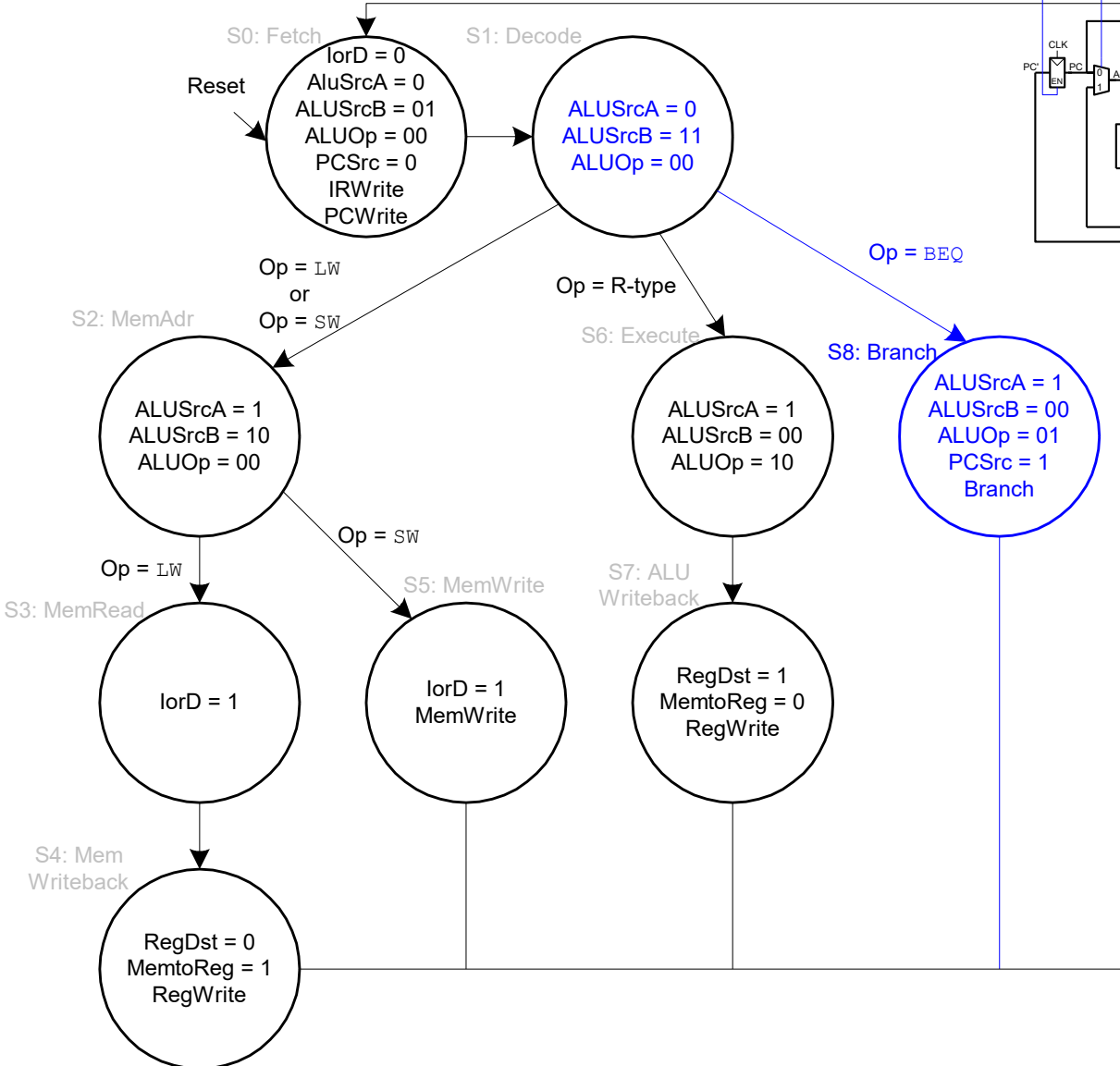
Main Controller FSM: sw



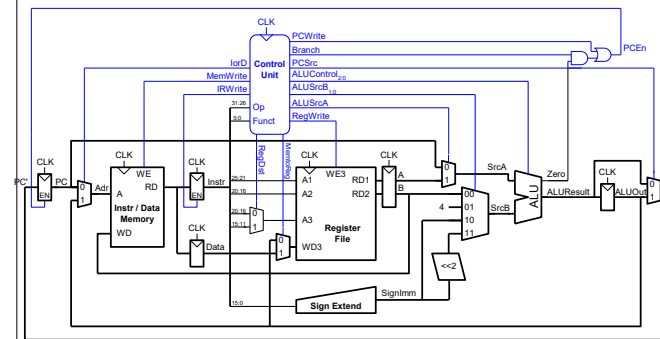
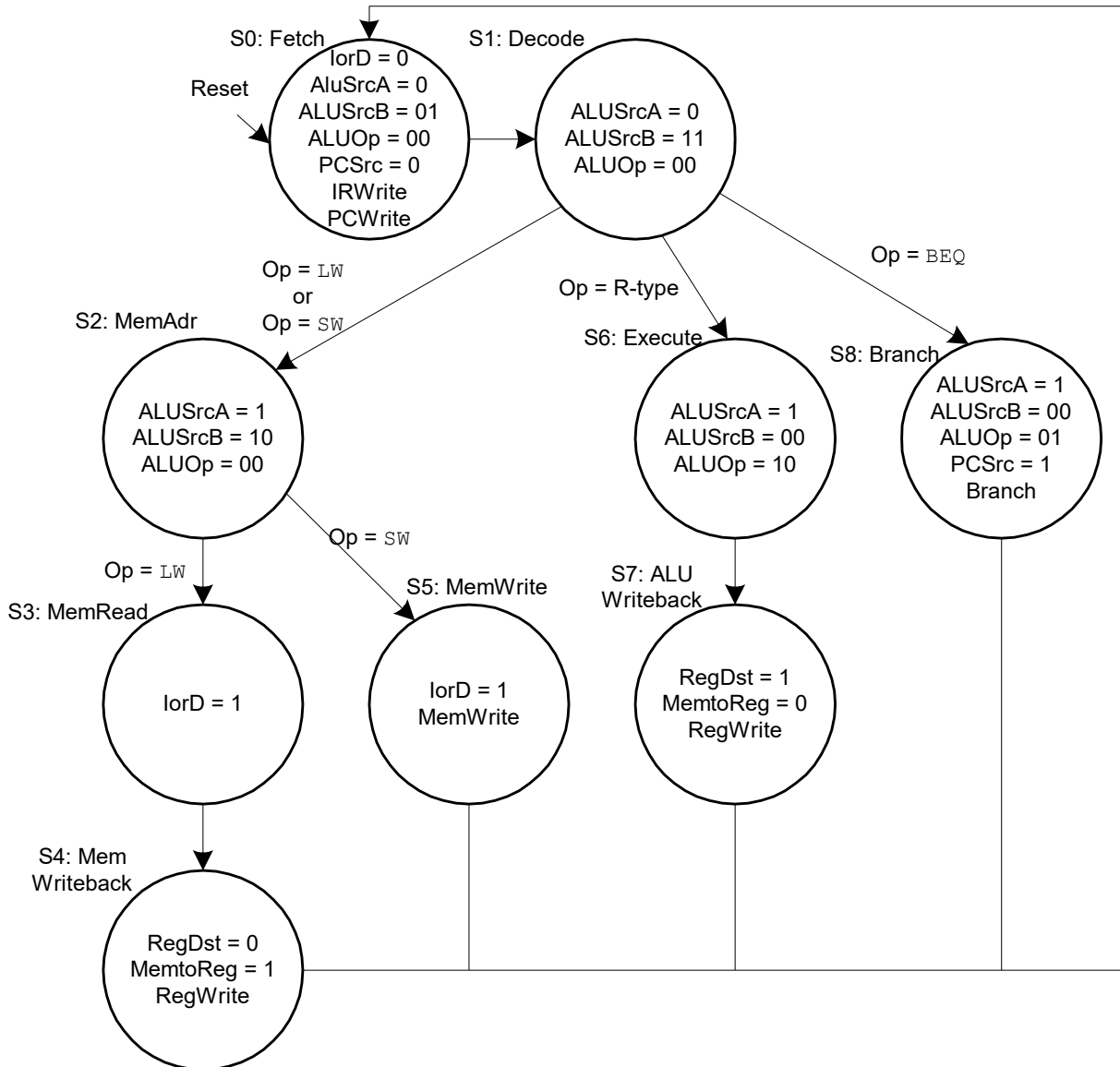
Main Controller FSM: R-Type



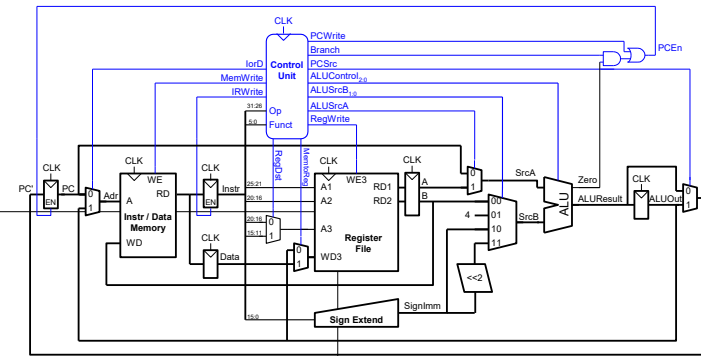
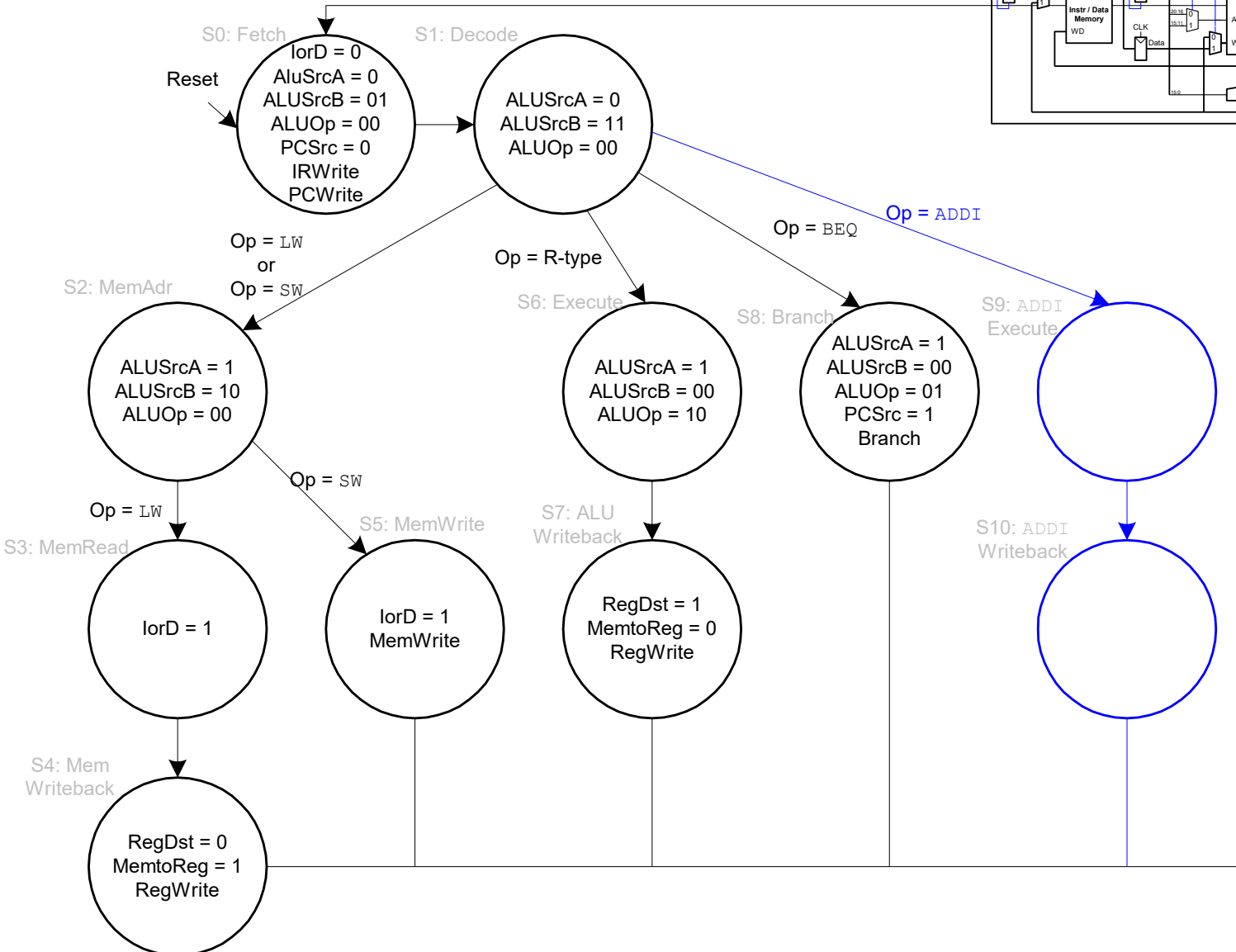
Main Controller FSM: beq



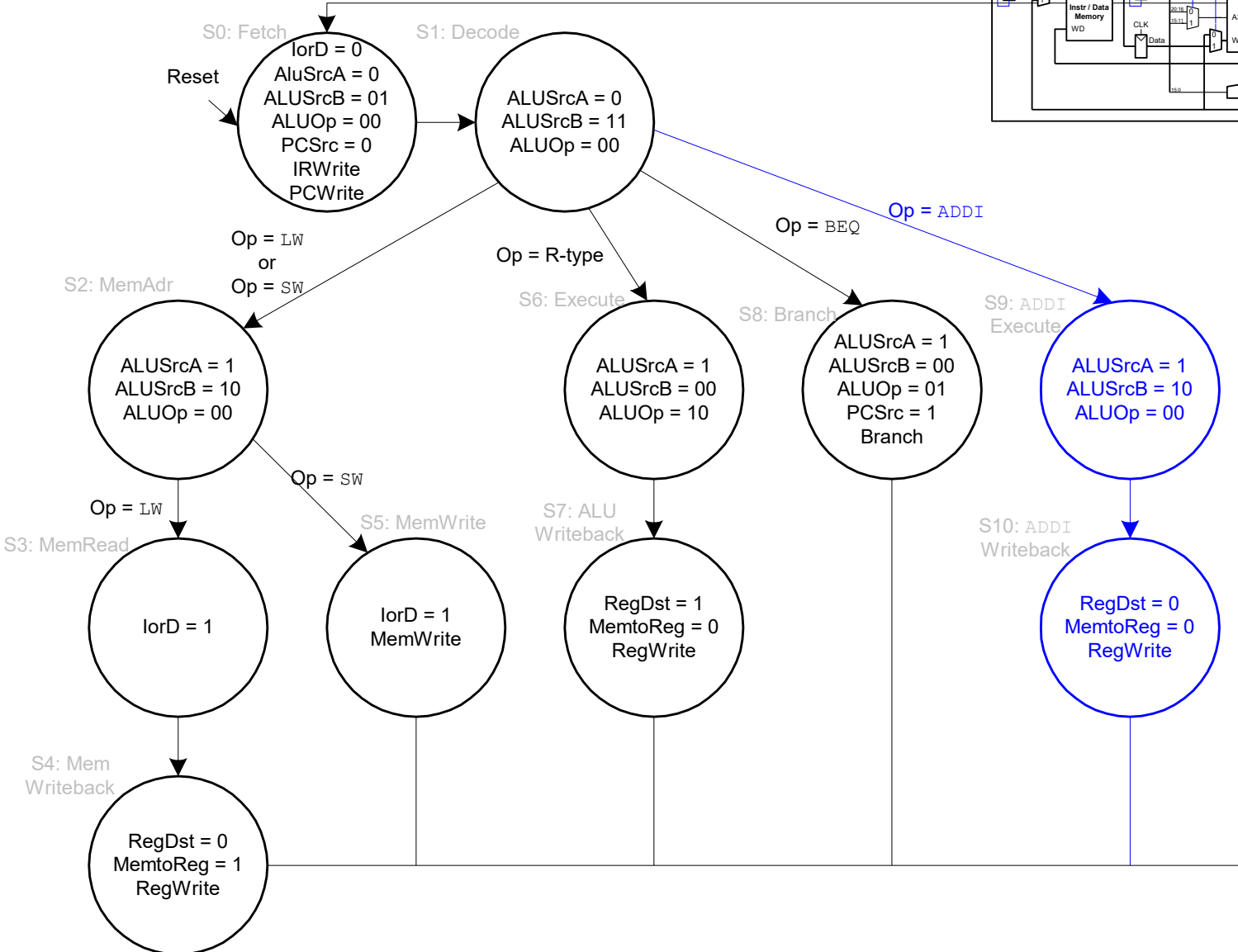
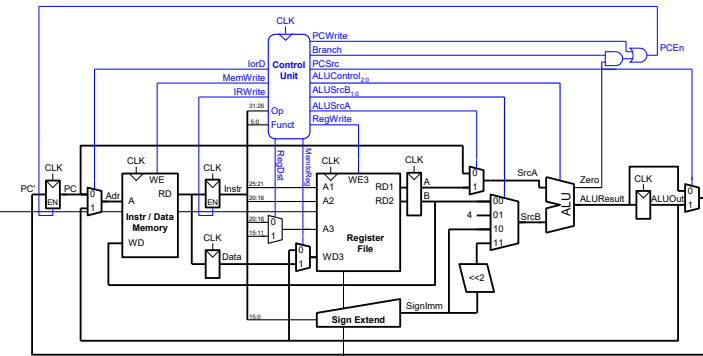
Complete Multi-Cycle Controller FSM



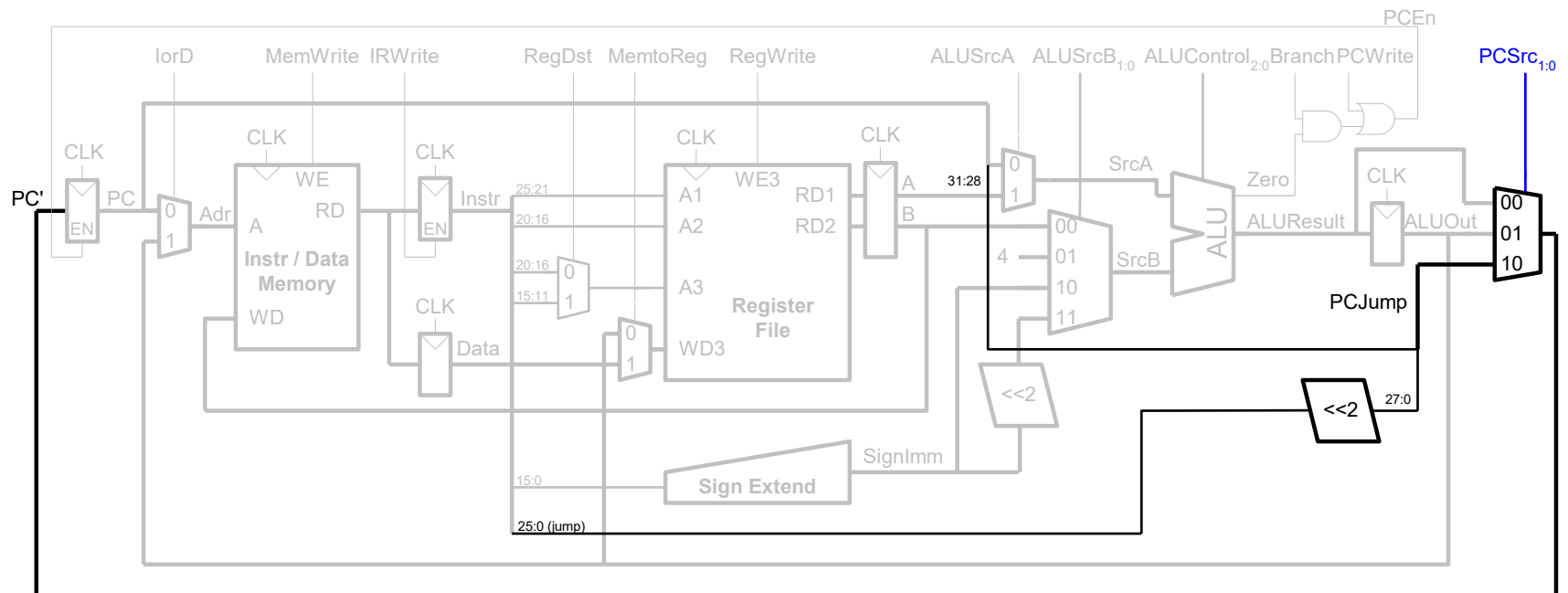
Main Controller FSM: addi



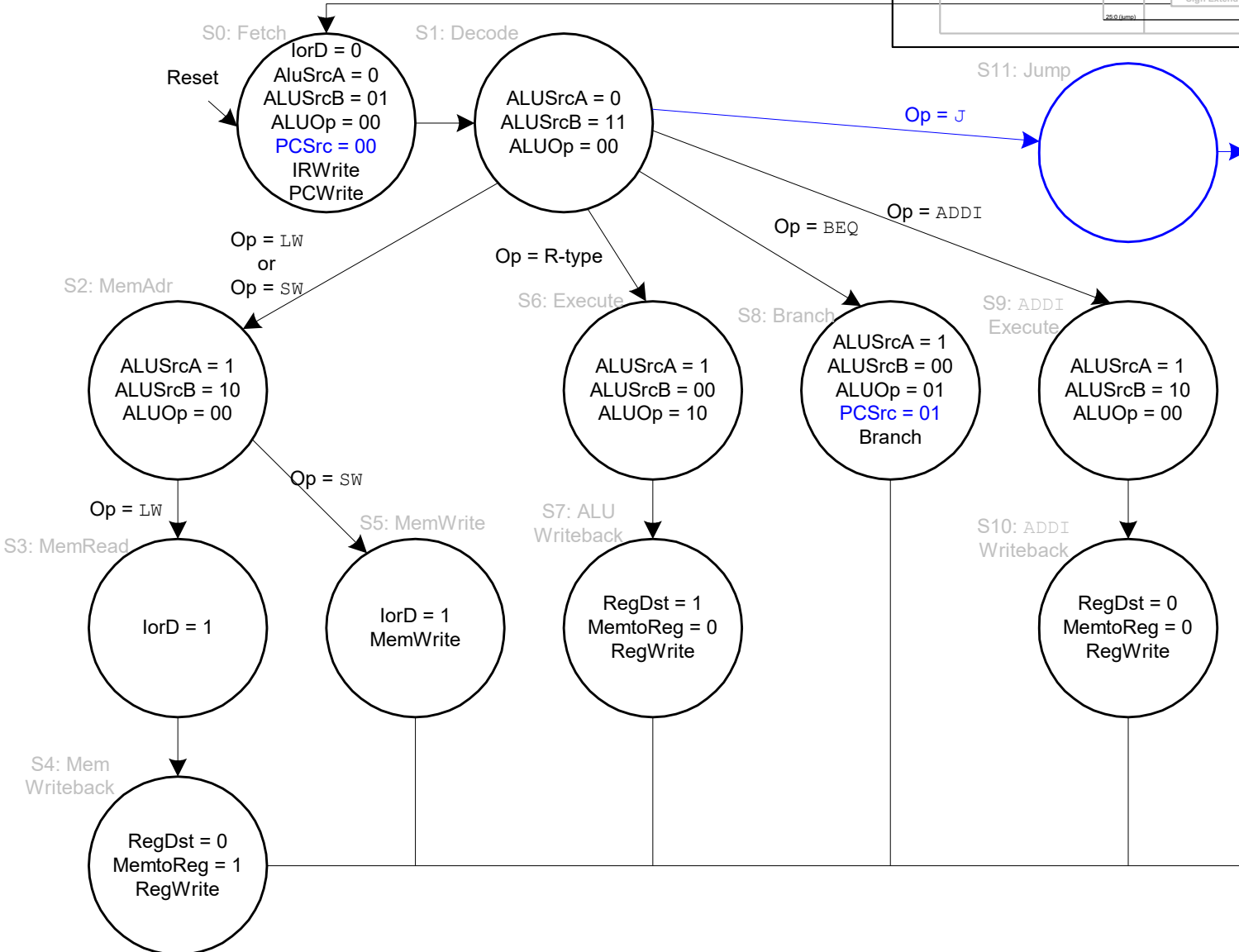
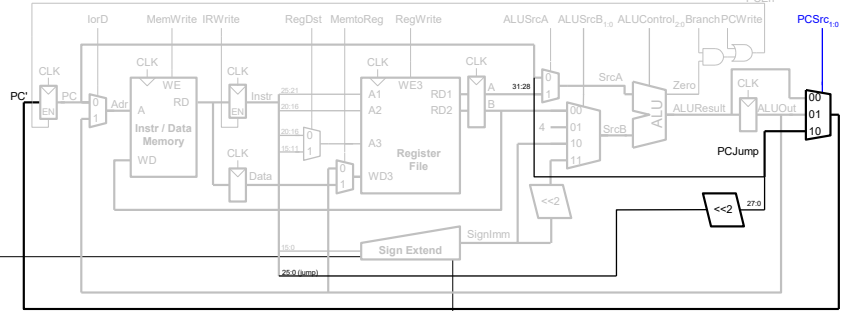
Main Controller FSM: addi



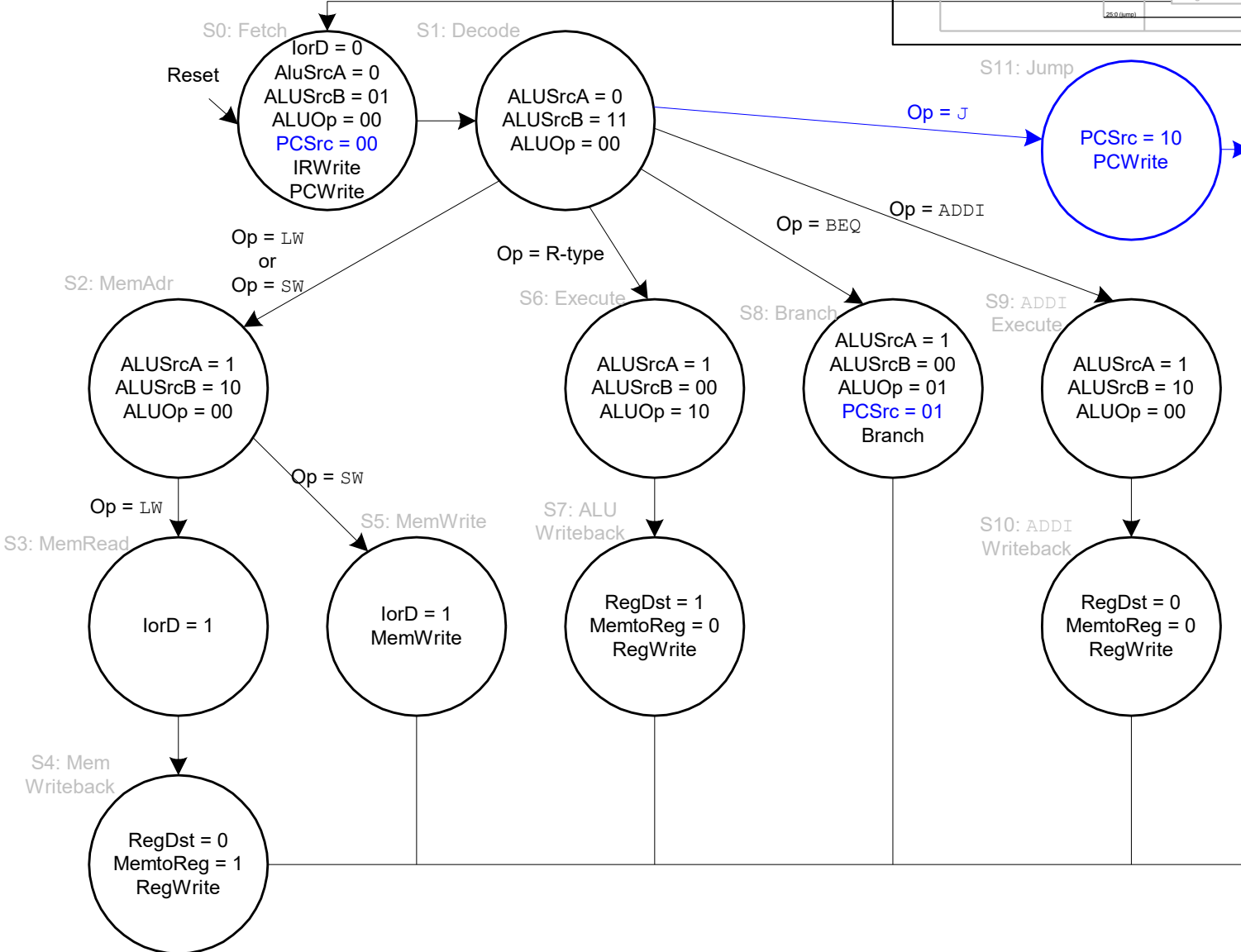
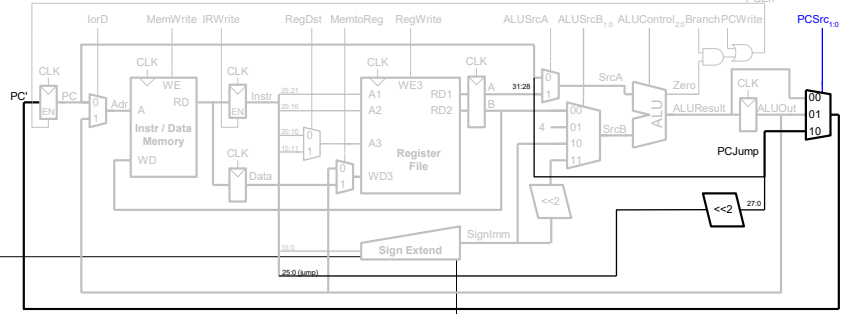
Extended Functionality: j



Control FSM: j

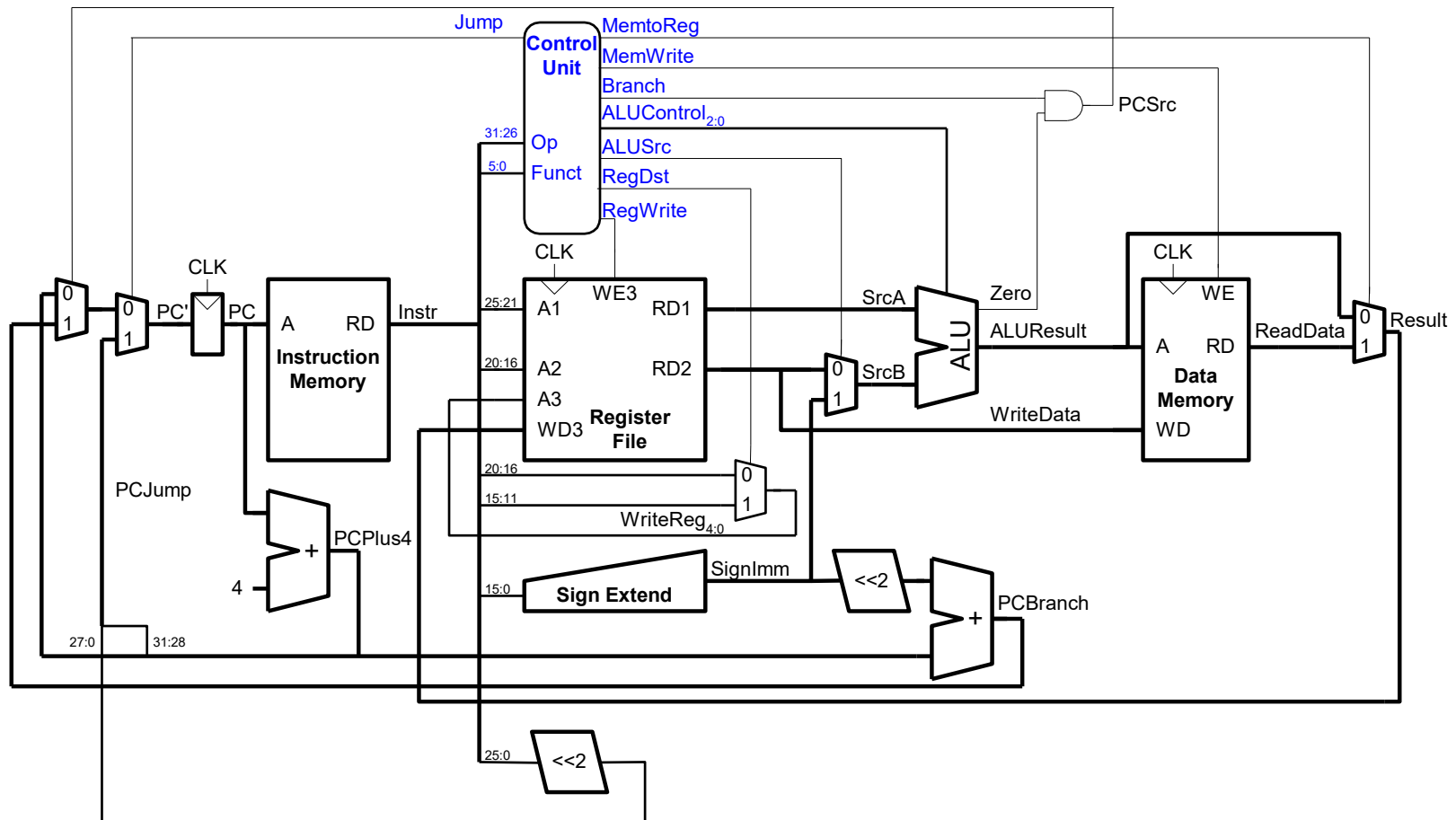


Control FSM: j



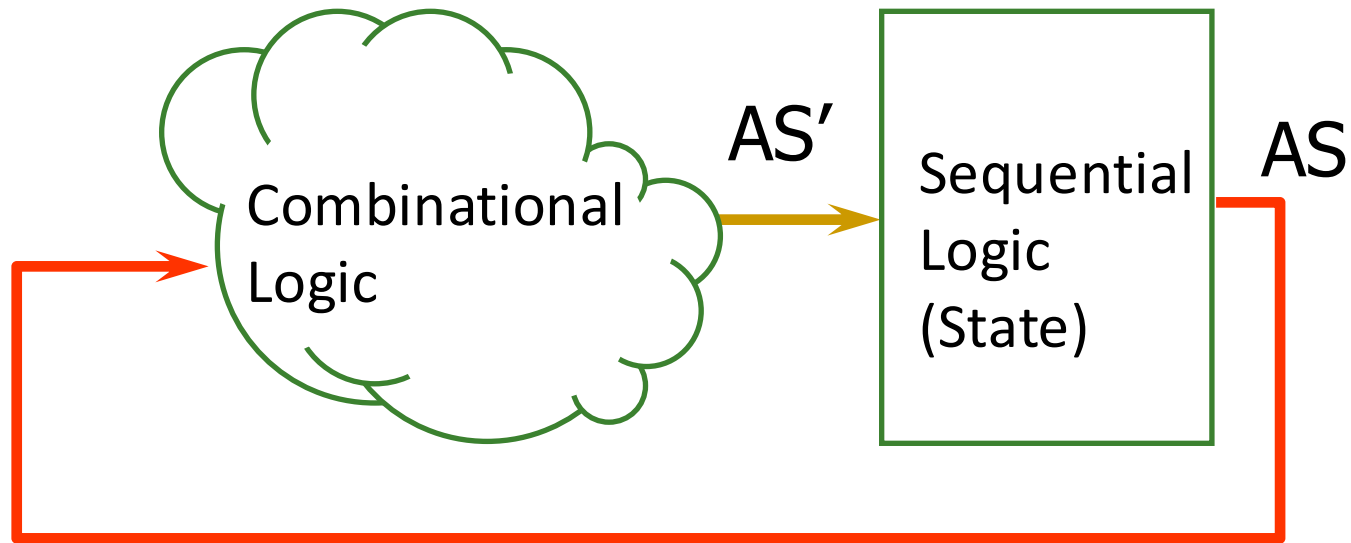
We Constructed a Multi-Cycle MIPS Microarchitecture

Review: Single-Cycle MIPS Microarchitecture

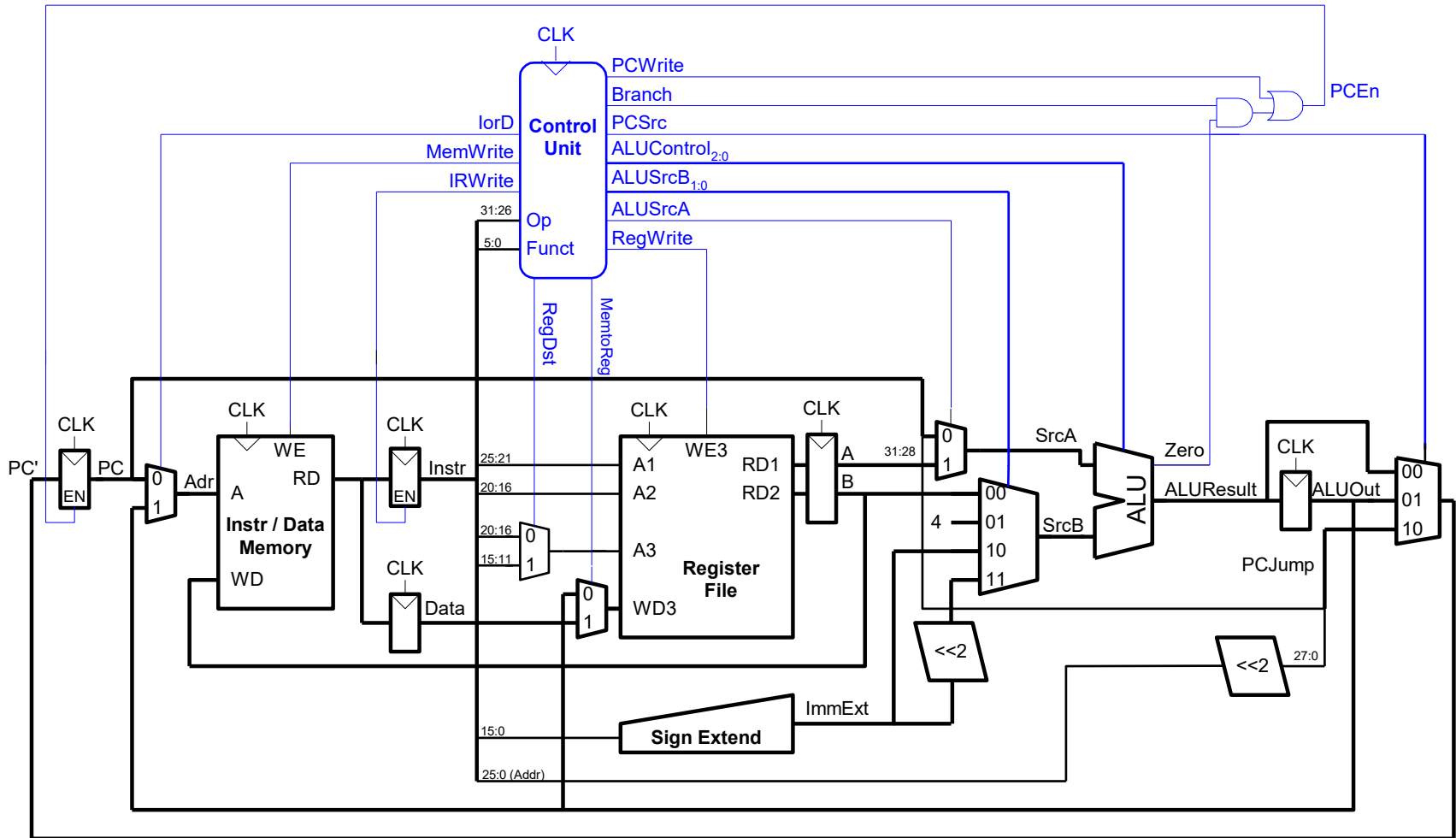


Review: Single-Cycle MIPS FSM

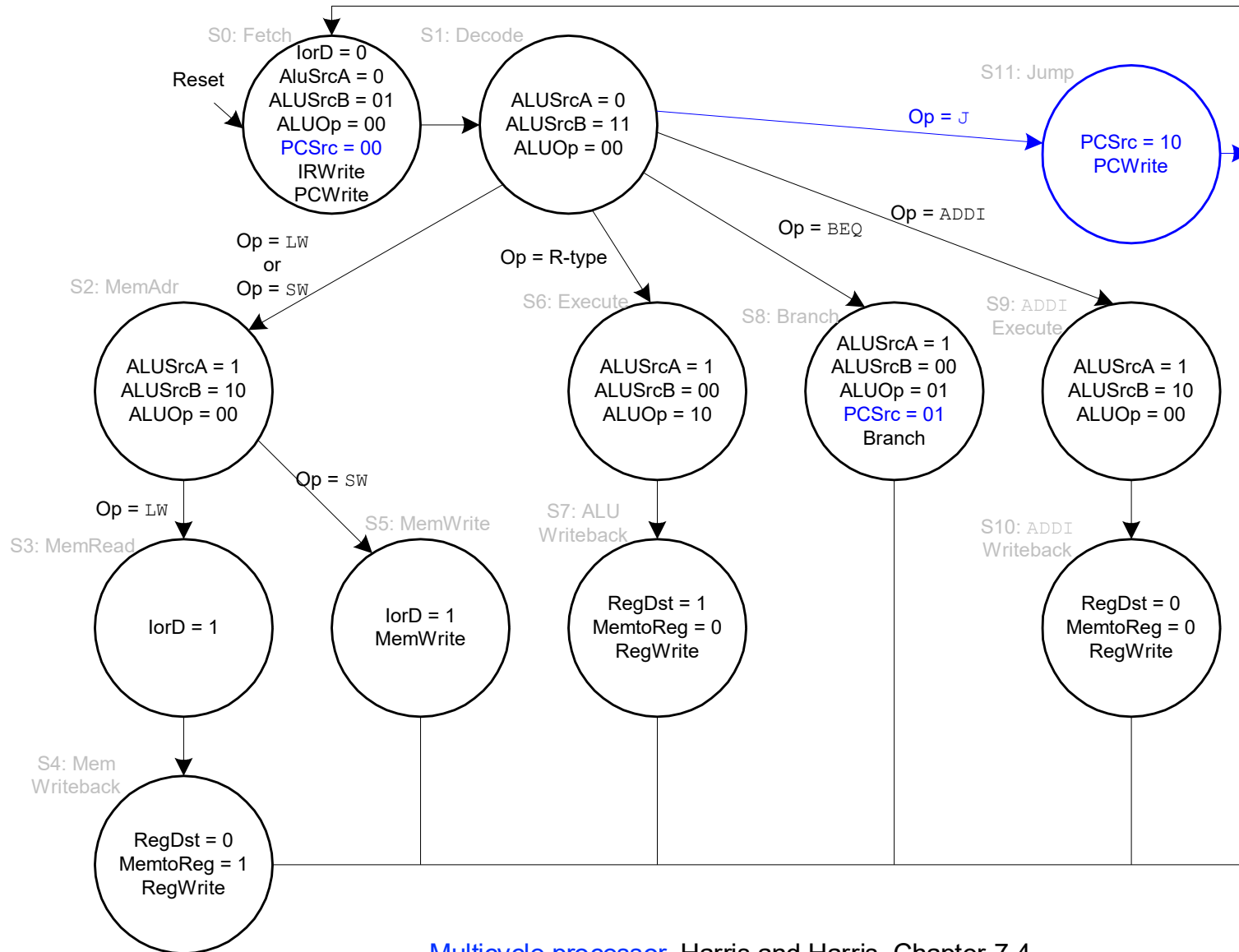
- Single-cycle machine



Review: Multi-Cycle MIPS Microarchitecture



Review: Multi-Cycle MIPS FSM



What does this design assume about memory?

What If Memory Takes $>$ One Cycle?

- Stay in the same “memory access” state until memory returns the data
- “Memory Ready?” bit is an input to the control logic that determines the next state

Recall: Full State Machine for LC-3b

Memory Access

Full FSM Controlling
a Multi-Cycle LC-3b
Microarchitecture

Memory Access

Memory Access

Memory Access

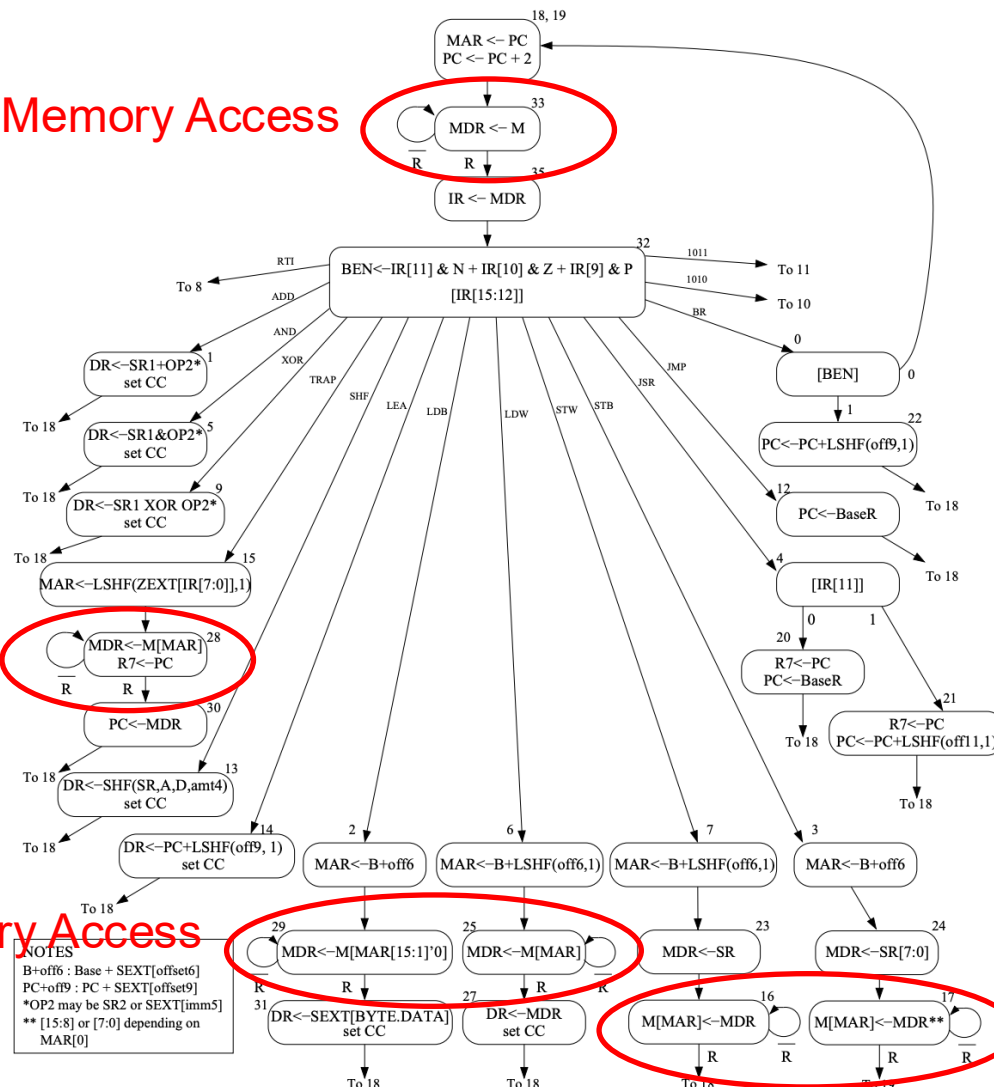


Figure C.2: A state machine for the LC-3b

Another Example:

**Microprogrammed Multi-Cycle
Microarchitecture**

Recall: An Elegant Multi-Cycle Processor Design

- Maurice Wilkes, “[The Best Way to Design an Automatic Calculating Machine](#),” Manchester Univ. Computer Inaugural Conf., 1951.

THE BEST WAY TO DESIGN AN AUTOMATIC CALCULATING MACHINE

By M. V. Wilkes, M.A., Ph.D., F.R.A.S.



- An elegant implementation:
 - [The concept of microcoded/microprogrammed machines](#)

Microprogrammed Control Terminology

- Control signals associated with the current state
 - Microinstruction

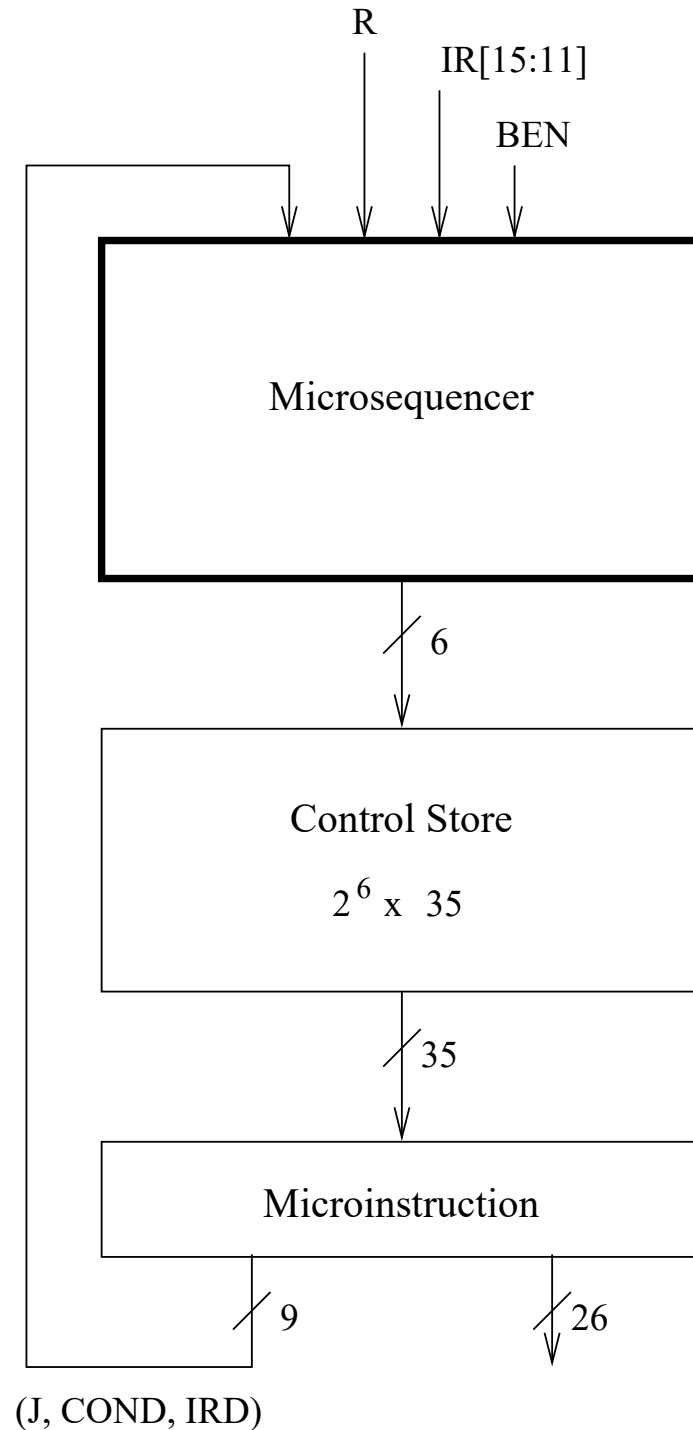
- Act of transitioning from one state to another
 - Determining the next state and the microinstruction for the next state
 - Microsequencing

- Control store stores control signals for every possible state
 - Store for microinstructions for the entire FSM

- Microsequencer determines which set of control signals will be used in the next clock cycle (i.e., next state)

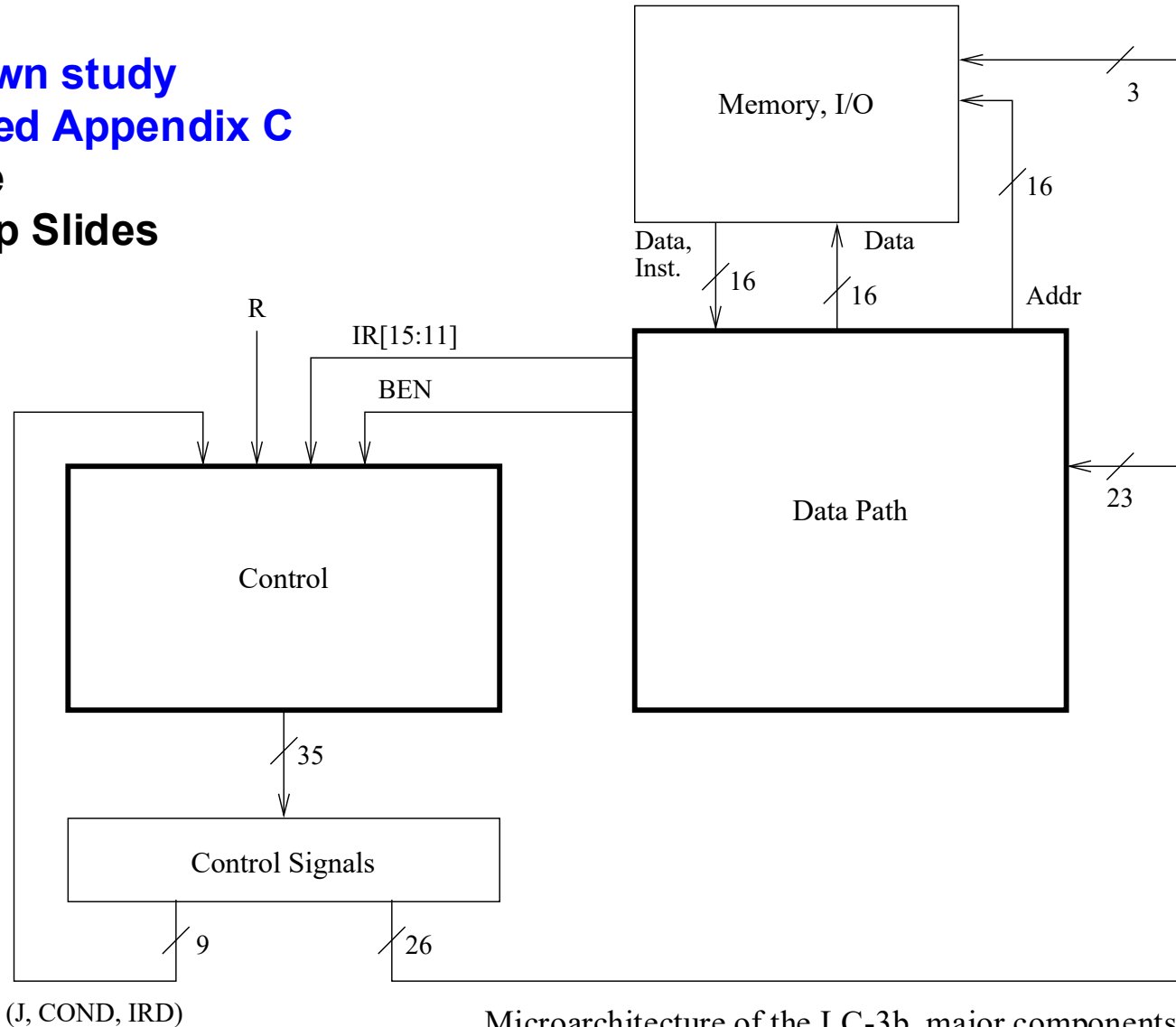
Example Control Structure

Simple Design
of the Control Structure



Example uProgrammed Control & Datapath

For your own study
P&P Revised Appendix C
On website
+ In Backup Slides



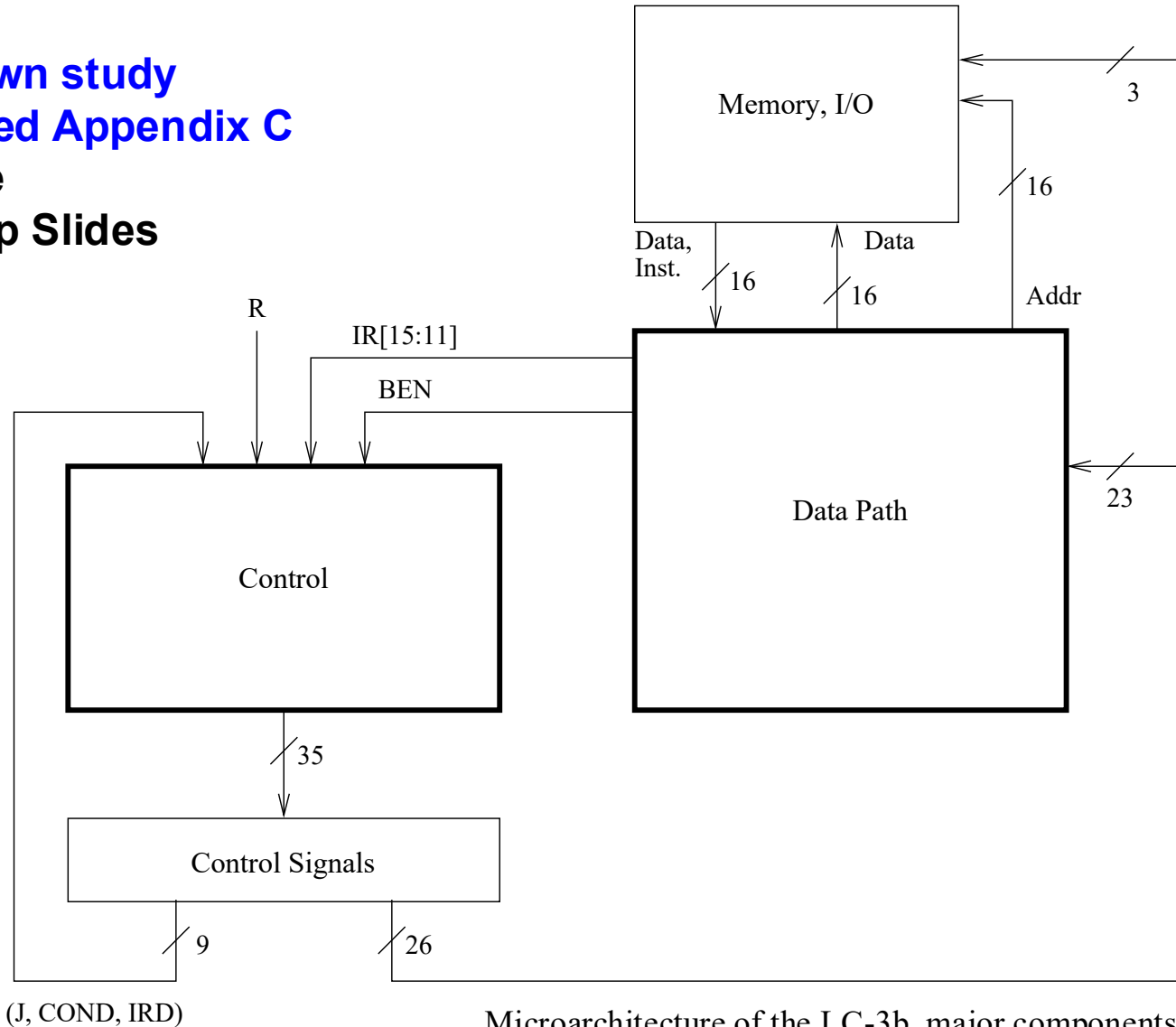
Microarchitecture of the LC-3b, major components

What Happens In A Clock Cycle?

- The control signals (microinstruction) for the current state control two things:
 - Processing in the data path
 - Generation of control signals (microinstruction) for the next cycle
- Datapath and microsequencer operate concurrently
- Question: why not generate control signals for the current cycle in the current cycle?
 - This could lengthen the clock cycle

Example uProgrammed Control & Datapath

For your own study
P&P Revised Appendix C
On website
+ In Backup Slides



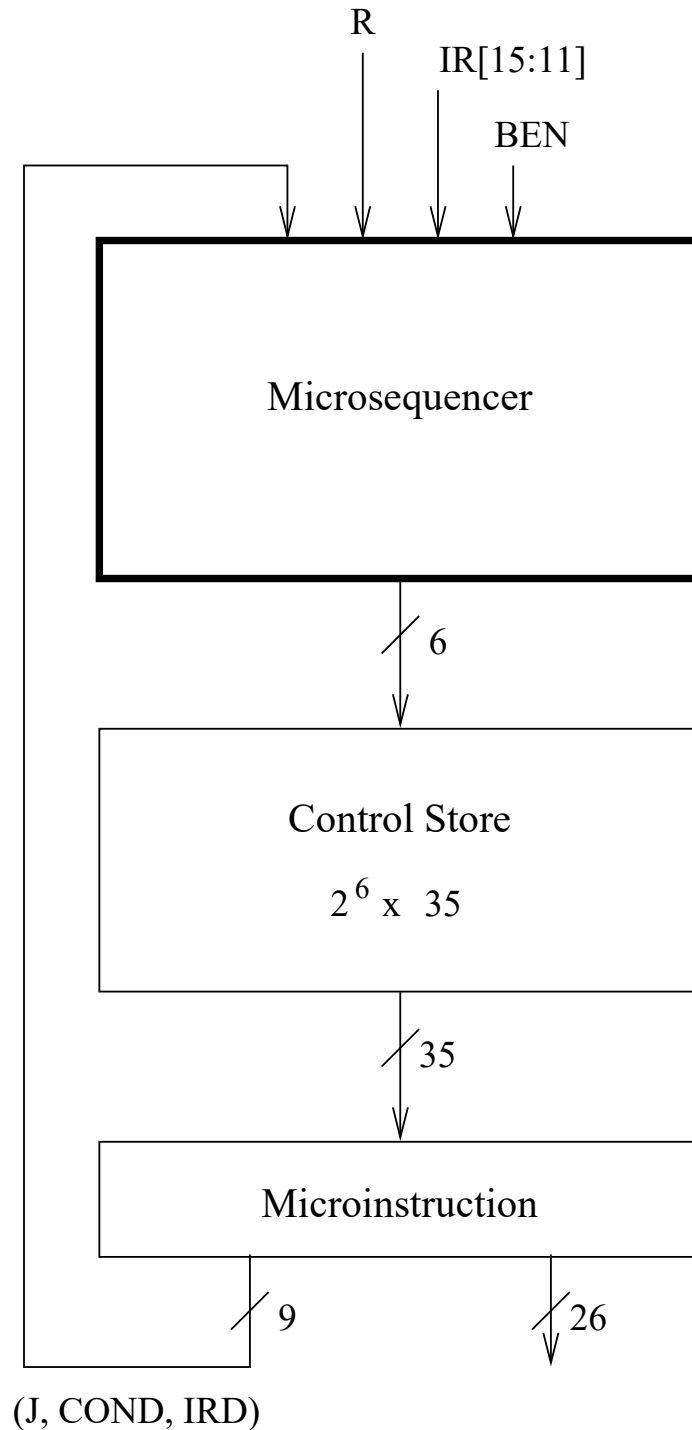
Microarchitecture of the LC-3b, major components

Microprogrammed Control Structure

- Three components: Microinstruction, Control store, Microsequencer
- **Microinstruction**: control signals that control the datapath (26 of them) and help determine the next state (9 of them)
- Each microinstruction is stored in a *unique location* in the **control store** (a special memory structure)
 - *Unique location*: address of the state corresponding to the microinstruction
 - Each state in the FSM corresponds to one microinstruction
- **Microsequencer** determines the address of the next microinstruction (i.e., next state)

uProgrammed Control Structure

Simple Design
of the Control Structure

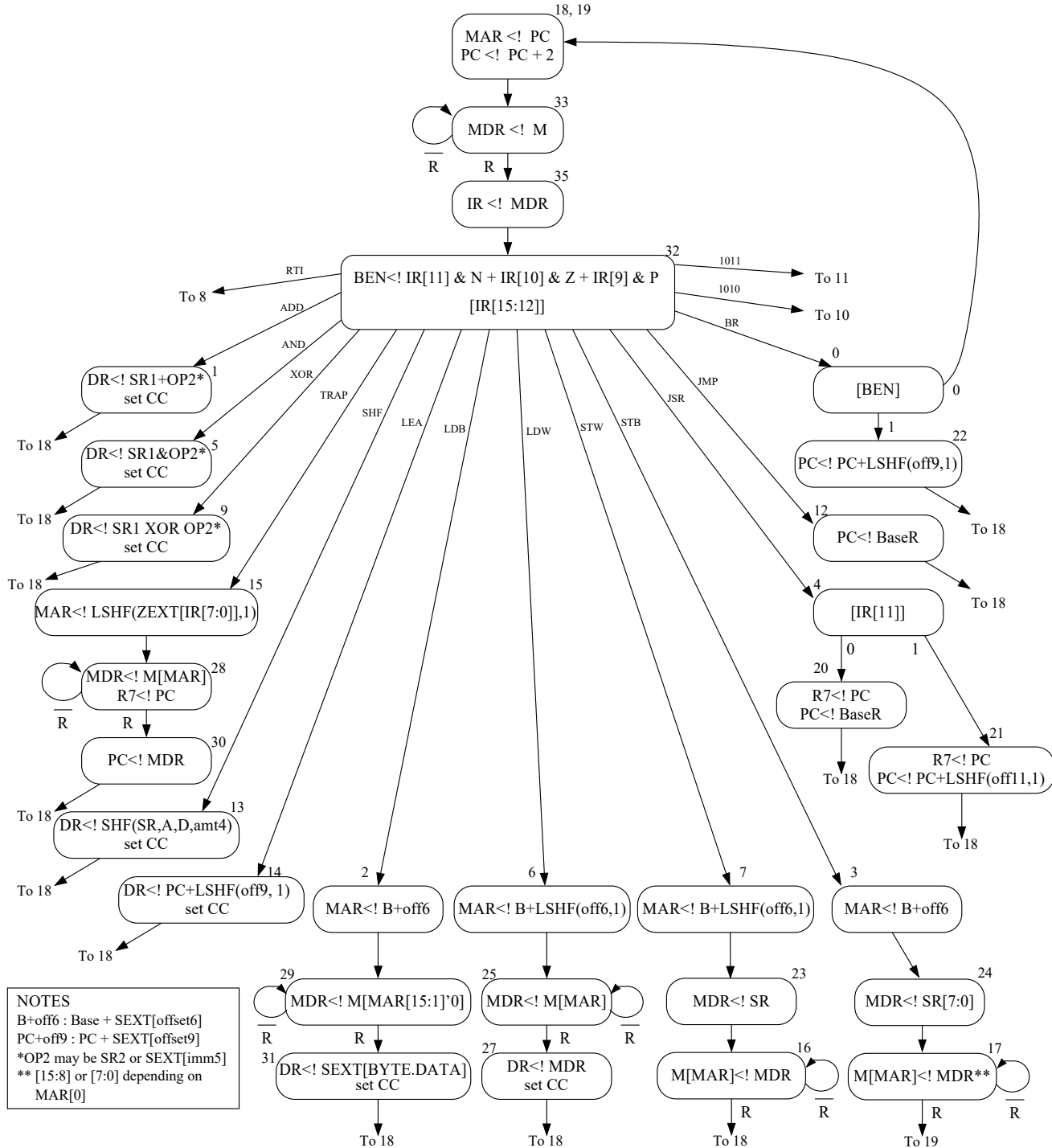


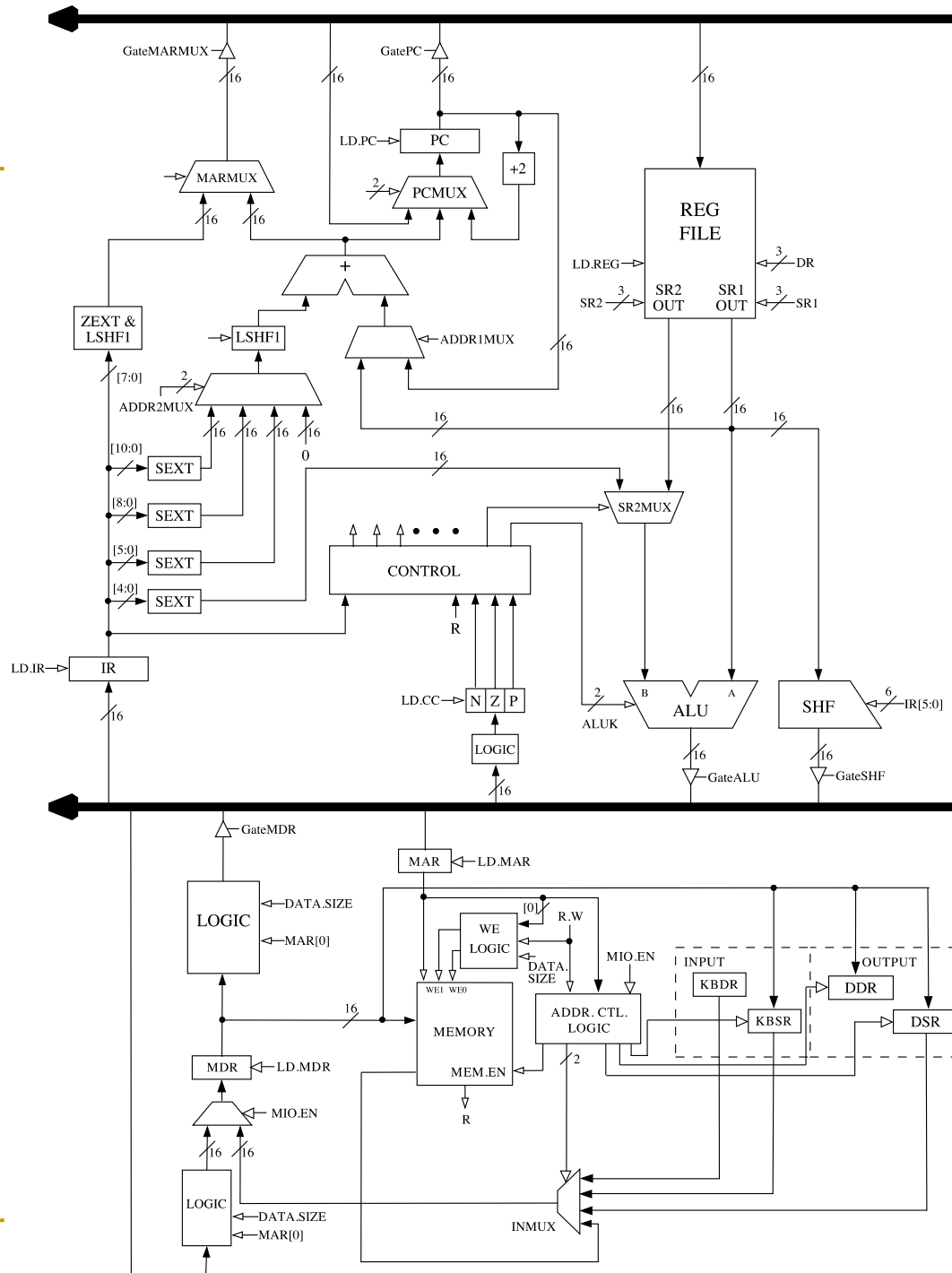
Control Store

IRD	Cond	J	LD MAR	LD MDR	LD IR	LD BEN	LD REG	LD PC	Gate PC	Gate MDR	Gate ALU	Gate SHIF	PCMUX	DRMUX	SRIMUX	ADDRIMUX	ADDRMUX	MARMUX	ALUX	MIDEN	RW	DATA SIZE	LSHIF1
																							000000 (State 0)
																							000001 (State 1)
																							000010 (State 2)
																							000011 (State 3)
																							000100 (State 4)
																							000101 (State 5)
																							000110 (State 6)
																							000111 (State 7)
																							001000 (State 8)
																							001001 (State 9)
																							001010 (State 10)
																							001011 (State 11)
																							001100 (State 12)
																							001101 (State 13)
																							001110 (State 14)
																							001111 (State 15)
																							010000 (State 16)
																							010001 (State 17)
																							010010 (State 18)
																							010011 (State 19)
																							010100 (State 20)
																							010101 (State 21)
																							010110 (State 22)
																							010111 (State 23)
																							011000 (State 24)
																							011001 (State 25)
																							011010 (State 26)
																							011011 (State 27)
																							011100 (State 28)
																							011101 (State 29)
																							011110 (State 30)
																							011111 (State 31)
																							100000 (State 32)
																							100001 (State 33)
																							100010 (State 34)
																							100011 (State 35)
																							100100 (State 36)
																							100101 (State 37)
																							100110 (State 38)
																							100111 (State 39)
																							101000 (State 40)
																							101001 (State 41)
																							101010 (State 42)
																							101011 (State 43)
																							101100 (State 44)
																							101101 (State 45)
																							101110 (State 46)
																							101111 (State 47)
																							110000 (State 48)
																							110001 (State 49)
																							110010 (State 50)
																							110011 (State 51)
																							110100 (State 52)
																							110101 (State 53)
																							110110 (State 54)
																							110111 (State 55)
																							111000 (State 56)
																							111001 (State 57)
																							111010 (State 58)
																							111011 (State 59)
																							111100 (State 60)
																							111101 (State 61)
																							111110 (State 62)
																							111111 (State 63)

Each entry in the control store is a microinstruction corresponding to the FSM state

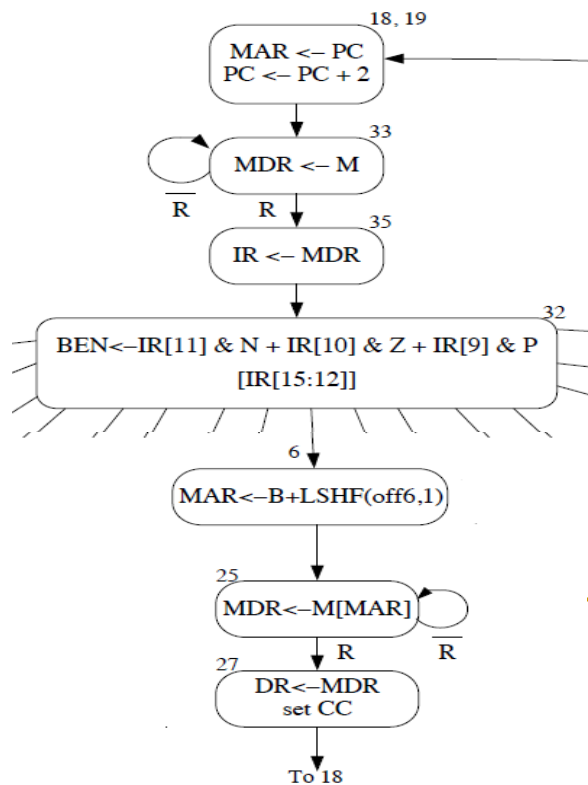
FSM state number is used to address the control store to get the relevant microinstruction



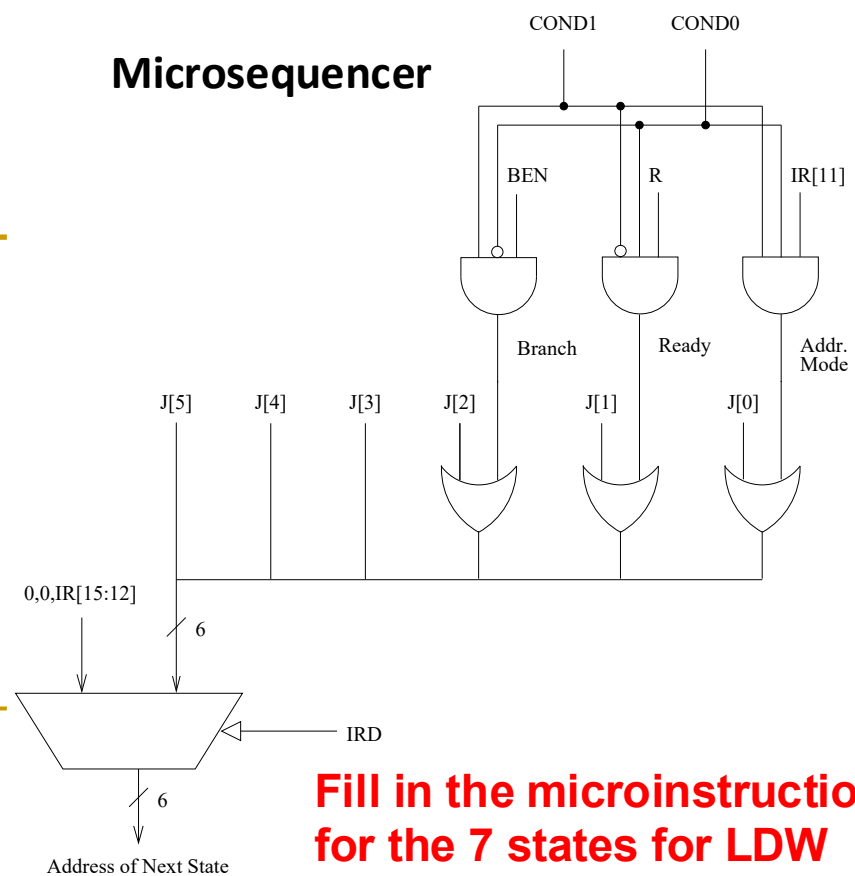


A Simple Datapath
Can Become
Very Powerful

State Machine for LDW



Microsequencer



Fill in the microinstructions for the 7 states for LDW

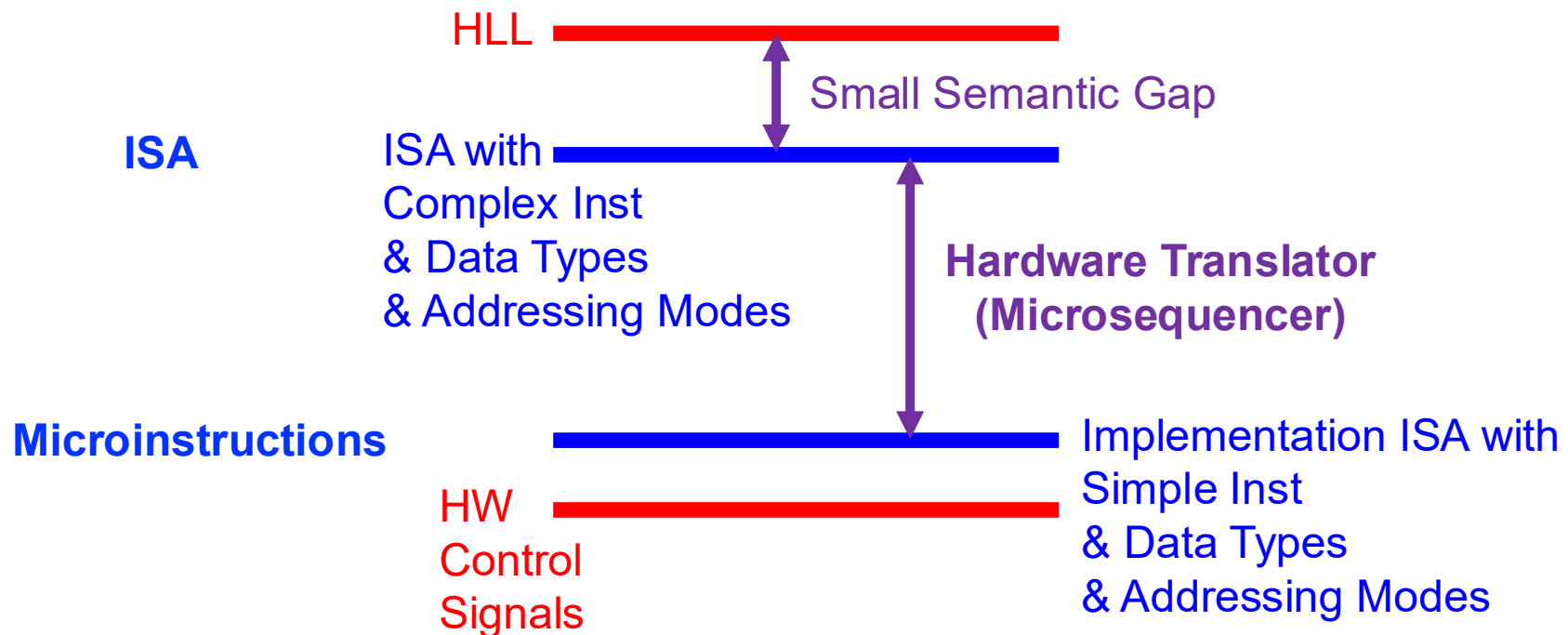
[illegible]

The Power of Abstraction

- The concept of a control store of microinstructions enables the hardware designer with a new abstraction:
microprogramming
- The designer can translate any desired operation to a sequence of microinstructions
- All the designer needs to provide is
 - The sequence of microinstructions needed to implement the desired operation
 - The ability for the control logic to correctly sequence through the microinstructions
 - Any additional datapath elements and control signals needed (no need if the operation can be “translated” into existing control signals)

How to Change the Semantic Gap Tradeoffs

- Translate from one ISA into a different “implementation” ISA



Advantages of Microprogrammed Control

- Allows a very simple design to do powerful computation by controlling the datapath (using a sequencer)
 - High-level ISA translated into microcode (sequence of u-instructions)
 - Microcode (u-code) enables a minimal datapath to emulate an ISA
 - Microinstructions can be thought of as a **user-invisible ISA (u-ISA)**
- Enables easy extensibility of the ISA
 - **Can support a new instruction by changing the microcode**
 - Can support complex instructions as a sequence of simple microinstructions (e.g., REP MOVS, Multi-dimensional Array Update)
- Enables update of machine behavior
 - **A buggy implementation of an instruction can be fixed by changing the microcode in the field**
 - Easier if datapath provides ability to do the same thing in different ways

Update of Machine Behavior

- The ability to update/patch microcode in the field (after a processor is shipped) enables
 - Ability to add new instructions without changing the processor!
 - Ability to “fix” buggy hardware implementations
- Historical Examples
 - IBM 370 Model 145: microcode stored in main memory, can be updated after a reboot
 - IBM System z: Similar to 370/145.
 - Heller and Farrell, “Millicode in an IBM zSeries processor,” IBM JR&D, May/Jul 2004.
 - B1700 microcode can be updated while the processor is running
 - User-microprogrammable machine!
 - Wilner, “Microprogramming environment on the Burroughs B1700”, CompCon 1972.
 - Systems today use microcode patches to fix HW bugs/issues

For More on Microprogrammed Designs

Microprogrammed Control Terminology

- Control signals associated with the current state
 - [Microinstruction](#)
- Act of transitioning from one state to another
 - Determining the next state and the microinstruction for the next state
 - [Microsequencing](#)
- [Control store](#) stores control signals for every possible state
 - Store for microinstructions for the entire FSM
- [Microsequencer](#) determines which set of control signals will be used in the next clock cycle (i.e., next state)

28



19:53 / 1:35:29



Design of Digital Circuits - Lecture 13: Microprogramming (ETH Zürich, Spring 2018)

2,301 views • Apr 14, 2018

30 0 SHARE SAVE ...



Onur Mutlu Lectures
15.4K subscribers

ANALYTICS

EDIT VIDEO

Design of Digital Circuits, ETH Zürich, Spring 2018 (<https://safari.ethz.ch/digitaltechnik/>)

Lecture 13: Microprogramming

Lecturer: Professor Onur Mutlu (<http://people.inf.ethz.ch/omutlu>)

Date: April 13, 2018

<https://www.youtube.com/onurmutlulectures>

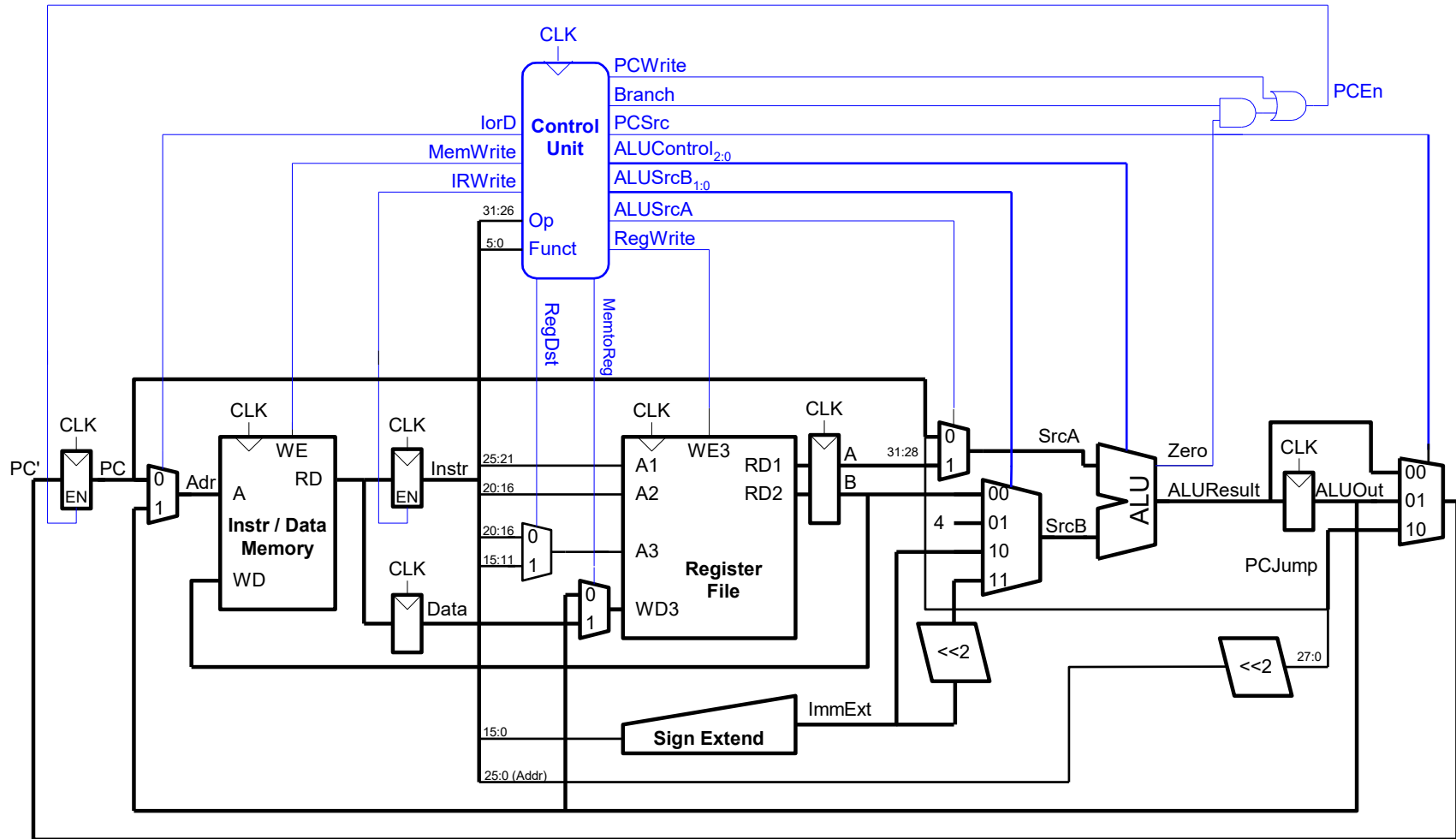
Detailed Lectures on Microprogramming

- Design of Digital Circuits, Spring 2018, Lecture 13
 - Microprogramming (ETH Zürich, Spring 2018)
 - https://www.youtube.com/watch?v=u4GhShuBP3Y&list=PL5Q2soXY2Zi_QedyPWtRmFUJ2F8DdYP7I&index=13

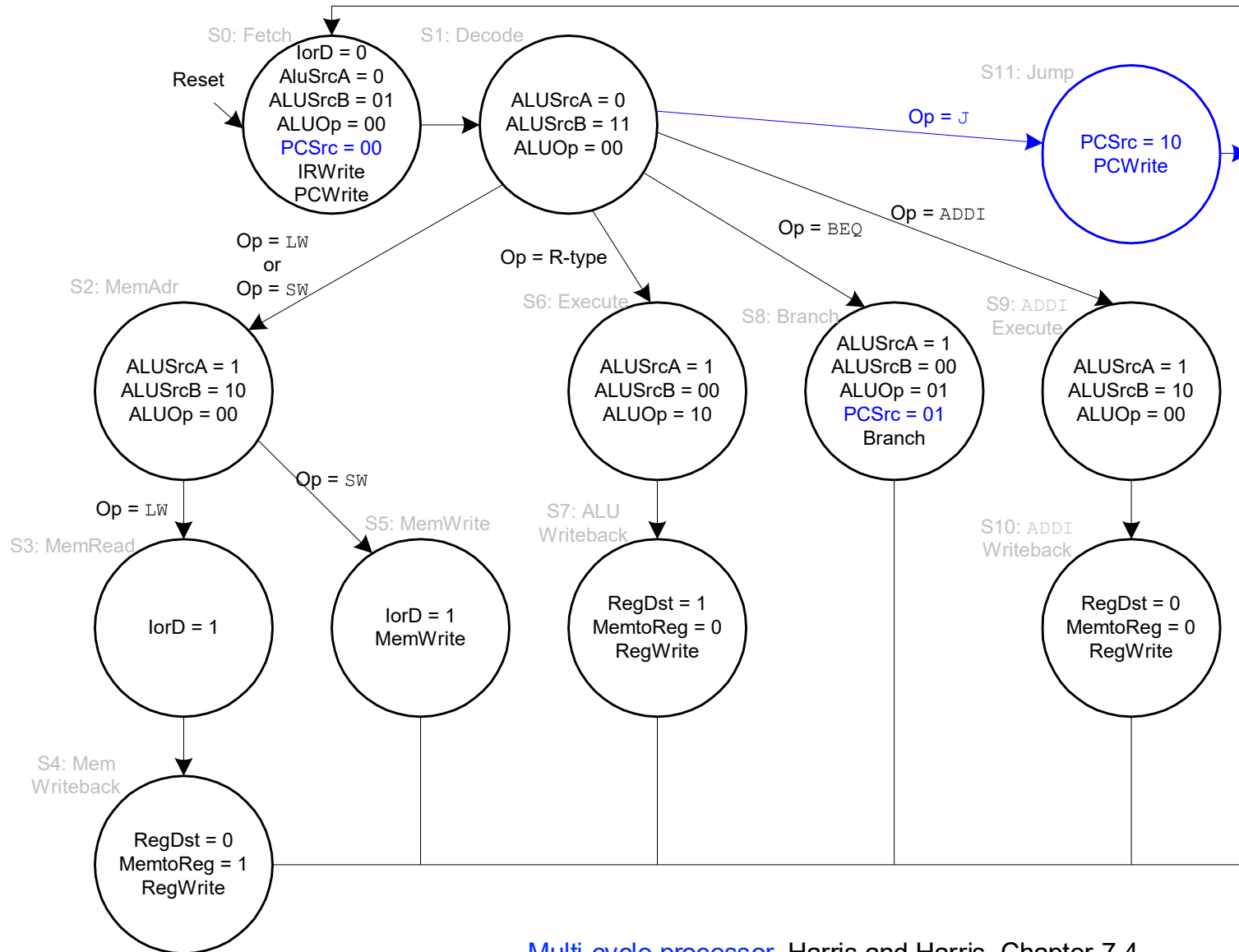
- Computer Architecture, Spring 2013, Lecture 7
 - Microprogramming (CMU, Spring 2013)
 - https://www.youtube.com/watch?v=_igvSI5h8cs&list=PL5PHm2jkkXmidJOd59REog9jDnPDTG6IJ&index=7

Can We Do Better?

Review: Multi-Cycle MIPS Processor



Review: Multi-Cycle MIPS FSM



We can easily handle multi-cycle memory access

Can We Do Better?

Can We Do Better?

- What limitations do you see with the multi-cycle design?
- Limited concurrency
 - Some hardware resources are idle during different phases of instruction processing cycle
 - “Fetch” logic is idle when an instruction is being “decoded” or “executed”
 - Most of the datapath is idle when a memory access is happening

Can We Use the Idle Hardware to Improve Concurrency?

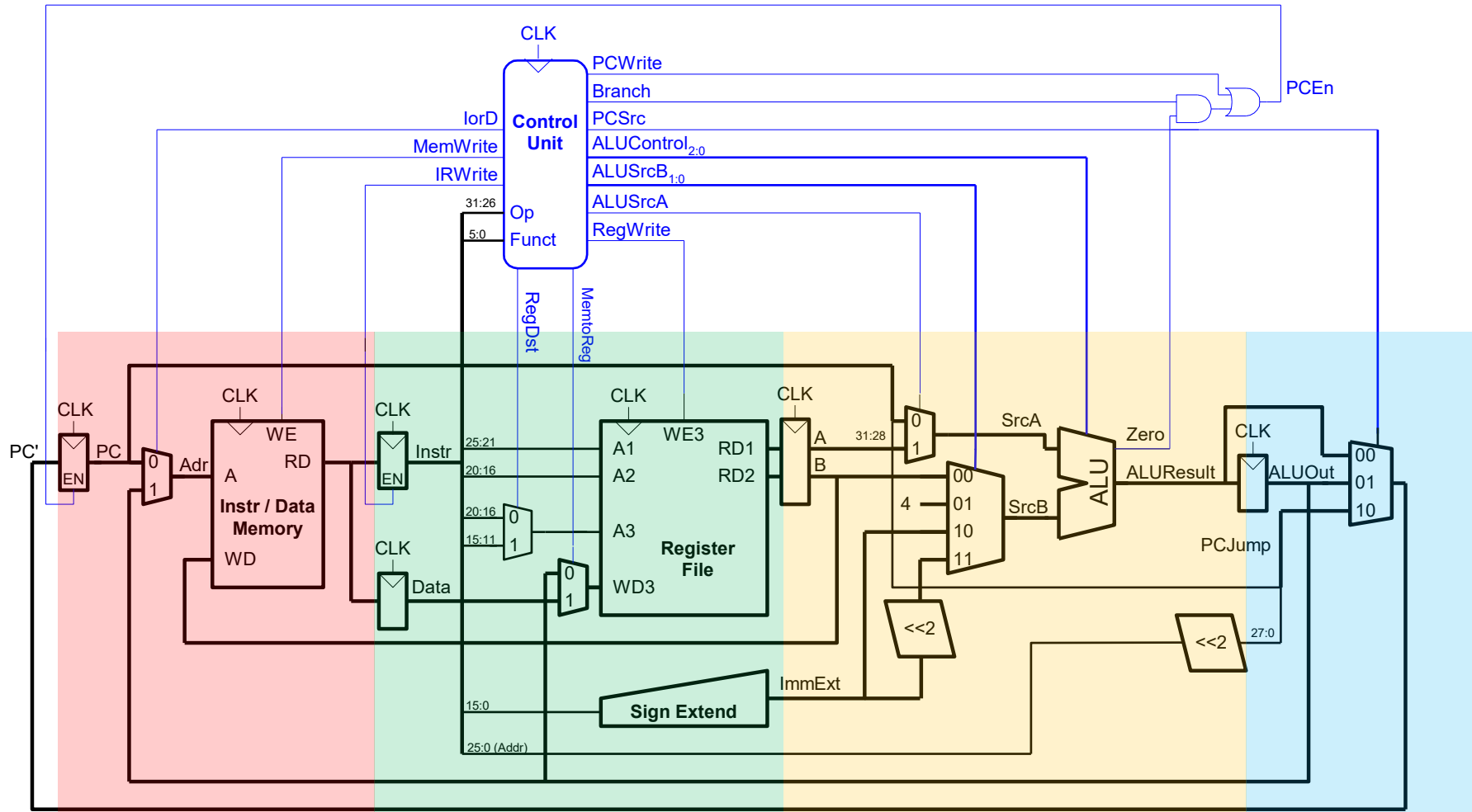
- Goal: **More concurrency** → **Higher instruction throughput** (i.e., more “work” completed in one cycle)
- Idea: When an instruction is using some resources in its processing phase, **process other instructions on idle resources** not needed by that instruction
 - E.g., when an instruction is being decoded, fetch the next instruction
 - E.g., when an instruction is being executed, decode another instruction
 - E.g., when an instruction is accessing data memory (ld/st), execute the next instruction
 - E.g., when an instruction is writing its result into the register file, access data memory for the next instruction

Can Have Different Instructions in Different Stages

- ❑ Fetch
- ❑ Decode
- ❑ Evaluate Address
- ❑ Fetch Operands
- ❑ Execute
- ❑ Store Result

1. Instruction fetch (IF)
2. Instruction decode and register operand fetch (ID/RF)
3. Execute/Evaluate memory address (EX/AG)
4. Memory operand fetch (MEM)
5. Store/writeback result (WB)

Can Have Different Instructions in Different Stages



Of course, we need to be more careful than this!

Pipelining

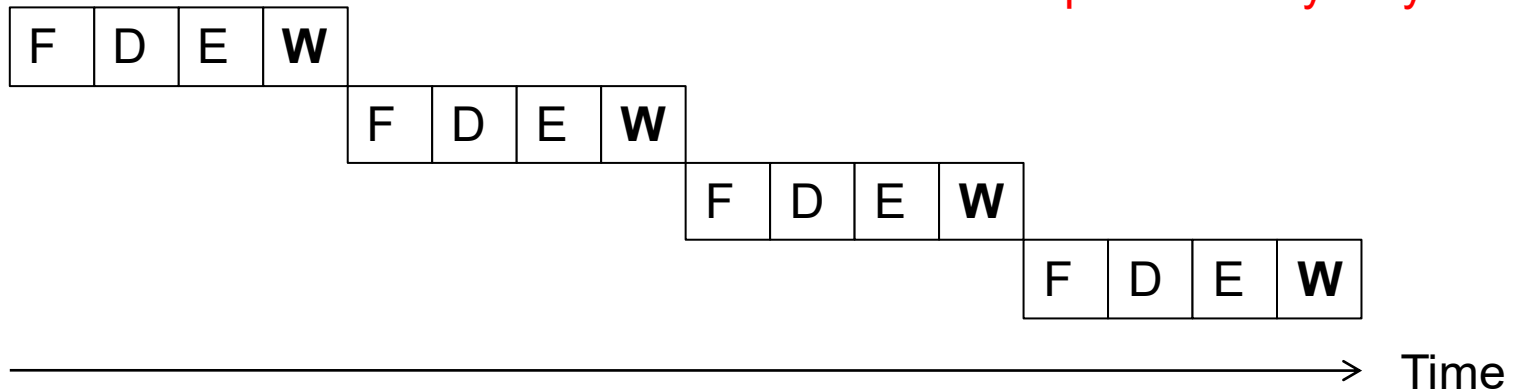
Pipelining: Basic Idea

- More systematically:
 - Pipeline the execution of multiple instructions
 - Analogy: “Assembly line processing” of instructions
- Idea:
 - Divide the instruction processing cycle into distinct “stages” of processing
 - Ensure there are enough hardware resources to process one instruction in each stage
 - Process a **different** instruction in each stage
 - Instructions consecutive in program order are processed in consecutive stages
- Benefit: **Increases instruction processing throughput (1/CPI)**
- Downside: Start thinking about this...

Example: Execution of Four Independent ADDs

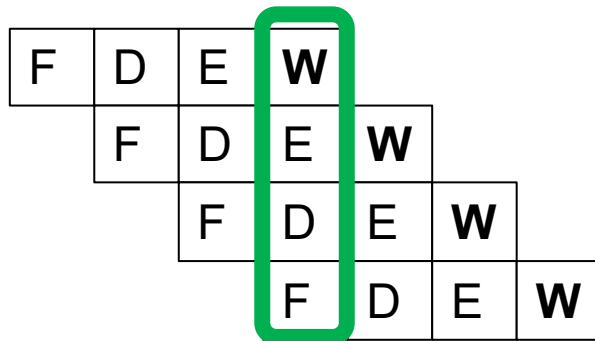
- Multi-cycle: 4 cycles per instruction

1 instruction completed every 4 cycles



- Pipelined: 4 cycles per 4 instructions (steady state)

1 instruction completed every 1 cycle



Is life always this beautiful?

We Covered Until Here in Lecture

Digital Design & Computer Arch.

Lecture 11: Multi-Cycle and Pipelined Processor Design

Dr. Mohammad Sadrosadati

Prof. Onur Mutlu

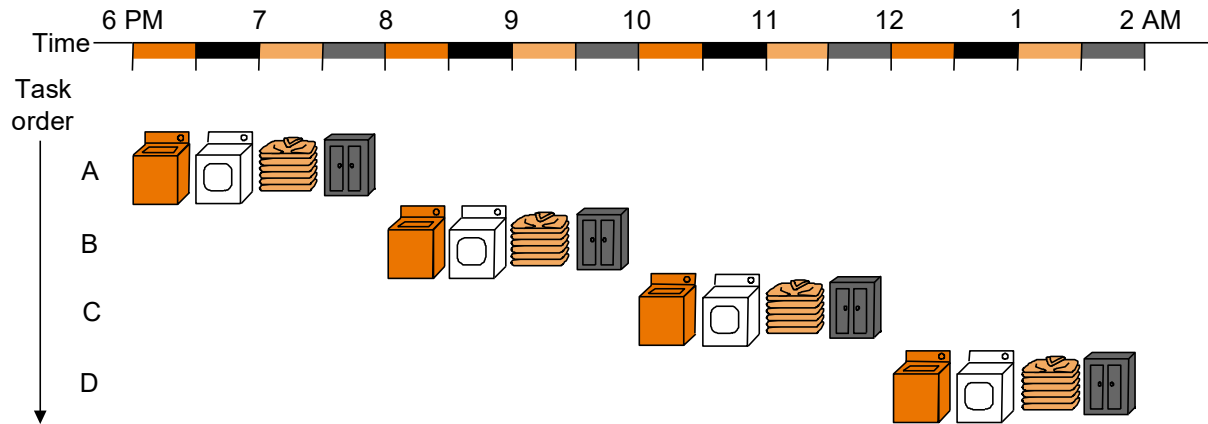
ETH Zürich

Spring 2026

26 March 2026

**Further Slides for Your Own Study
(May Be Covered in Future Lectures)**

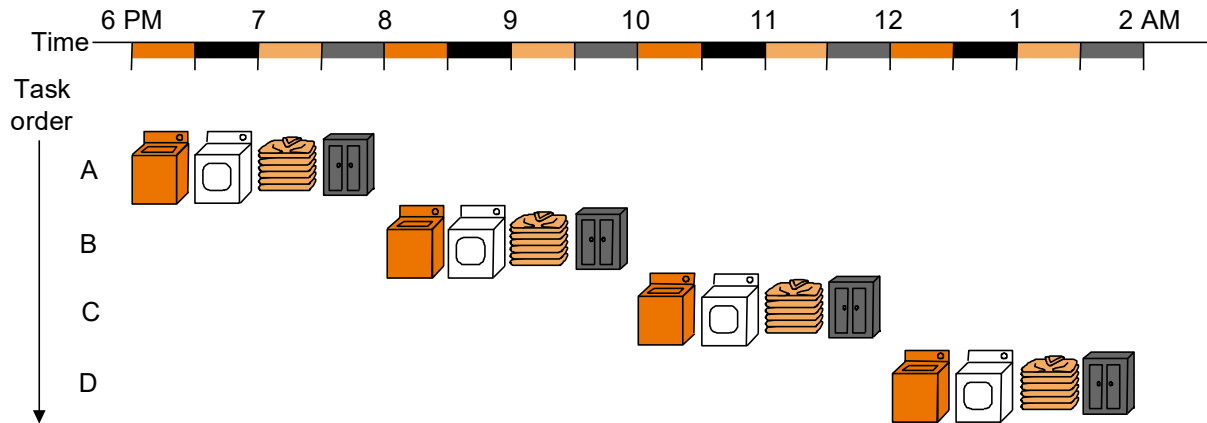
The Laundry Analogy



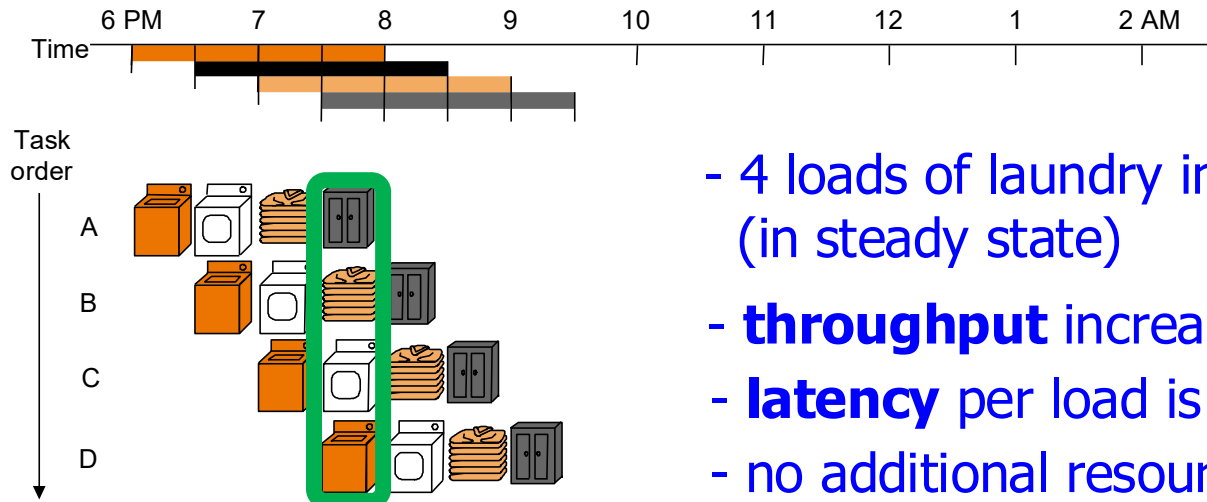
- “place one dirty load of clothes in the washer”
 - “when the washer is finished, place the wet load in the dryer”
 - “when the dryer is finished, take out the dry load and fold”
 - “when folding is finished, put the clothes away”
- steps to do a load are sequentially dependent
 - no dependence between different loads
 - different steps do not require the same resource(s)

Pipelining Multiple Loads of Laundry

1 load
every
120 mins

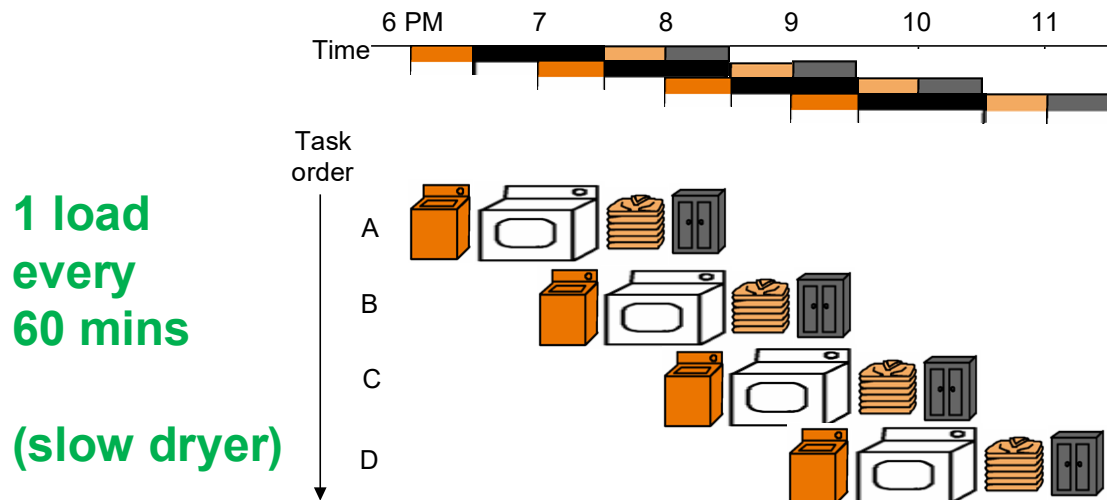
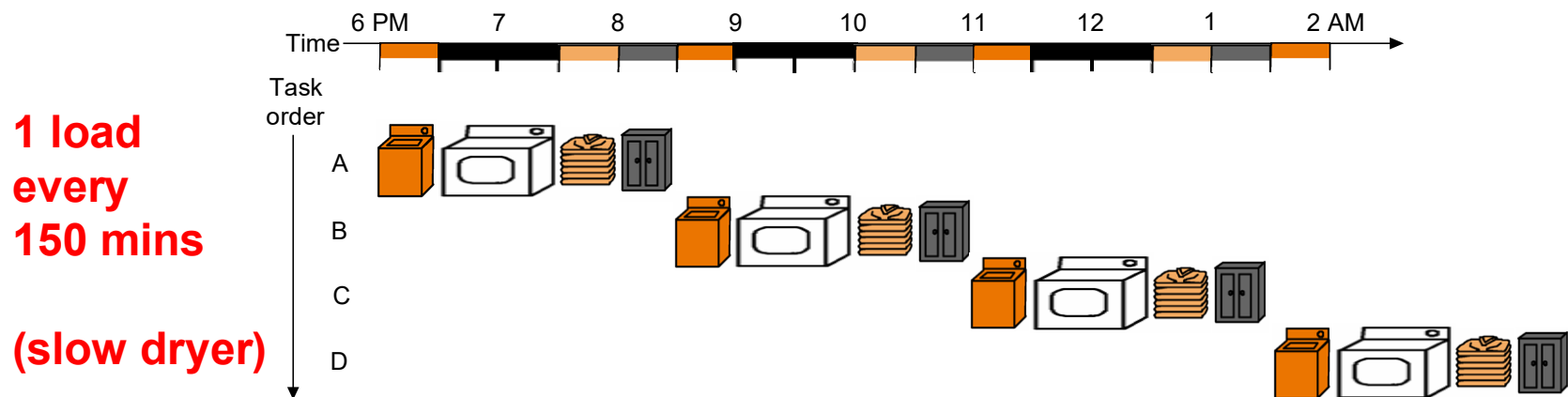


1 load
every
30 mins



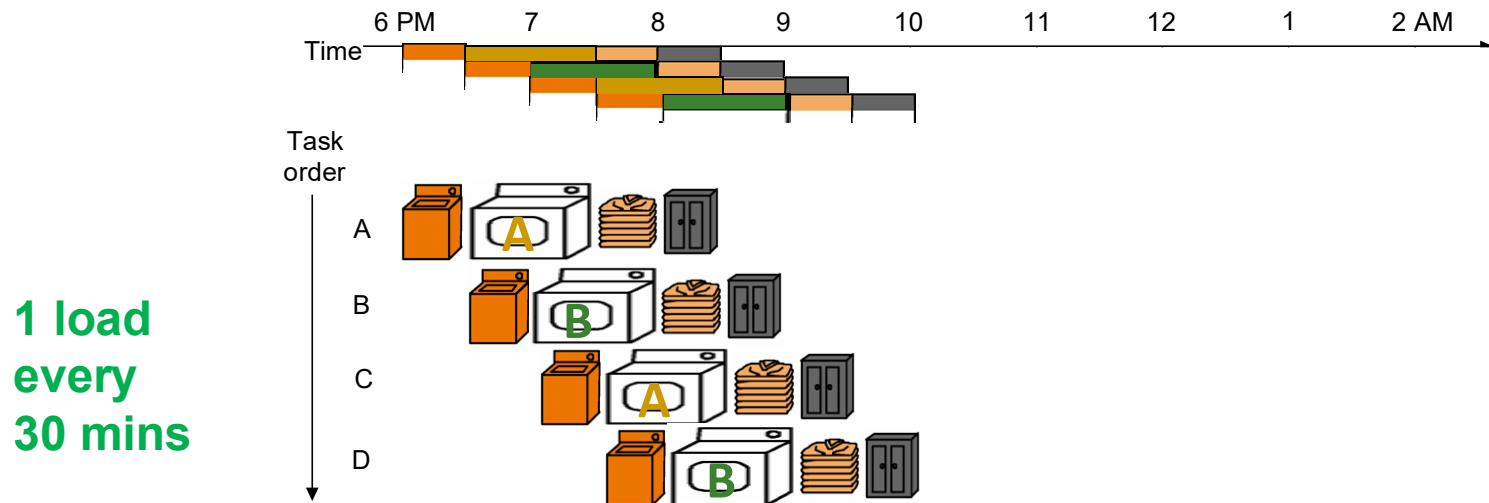
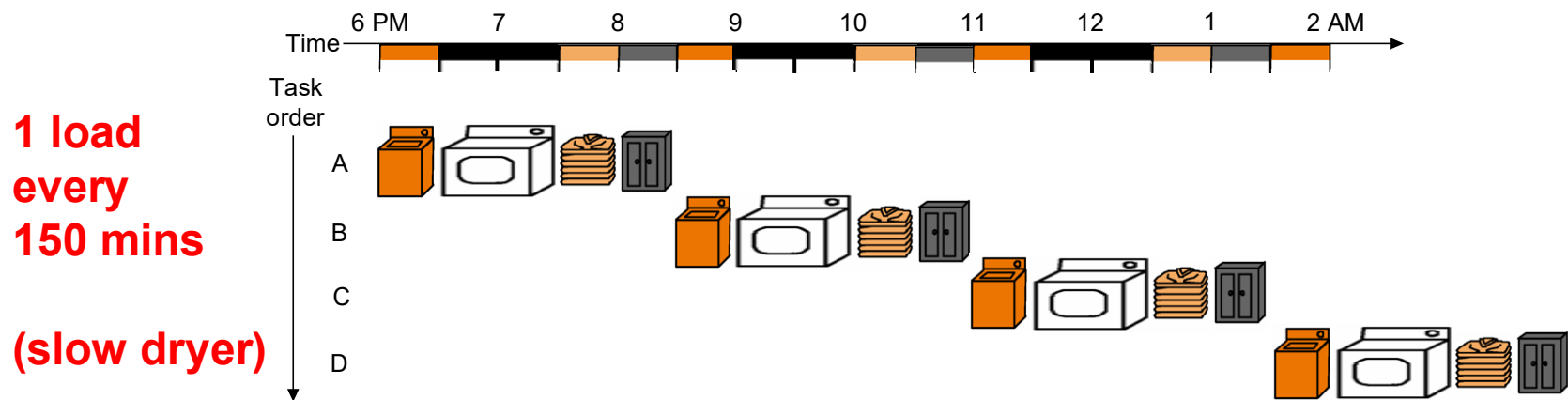
- 4 loads of laundry in parallel (in steady state)
- **throughput** increased by 4x
- **latency** per load is the same
- no additional resources

Pipelining Multiple Loads of Laundry: In Practice



the slowest step (the dryer) decides throughput

Pipelining Multiple Loads of Laundry: In Practice



throughput restored (1 load per 30-min) using 2 dryers

A Real-Life Pipeline: Automobile Assembly

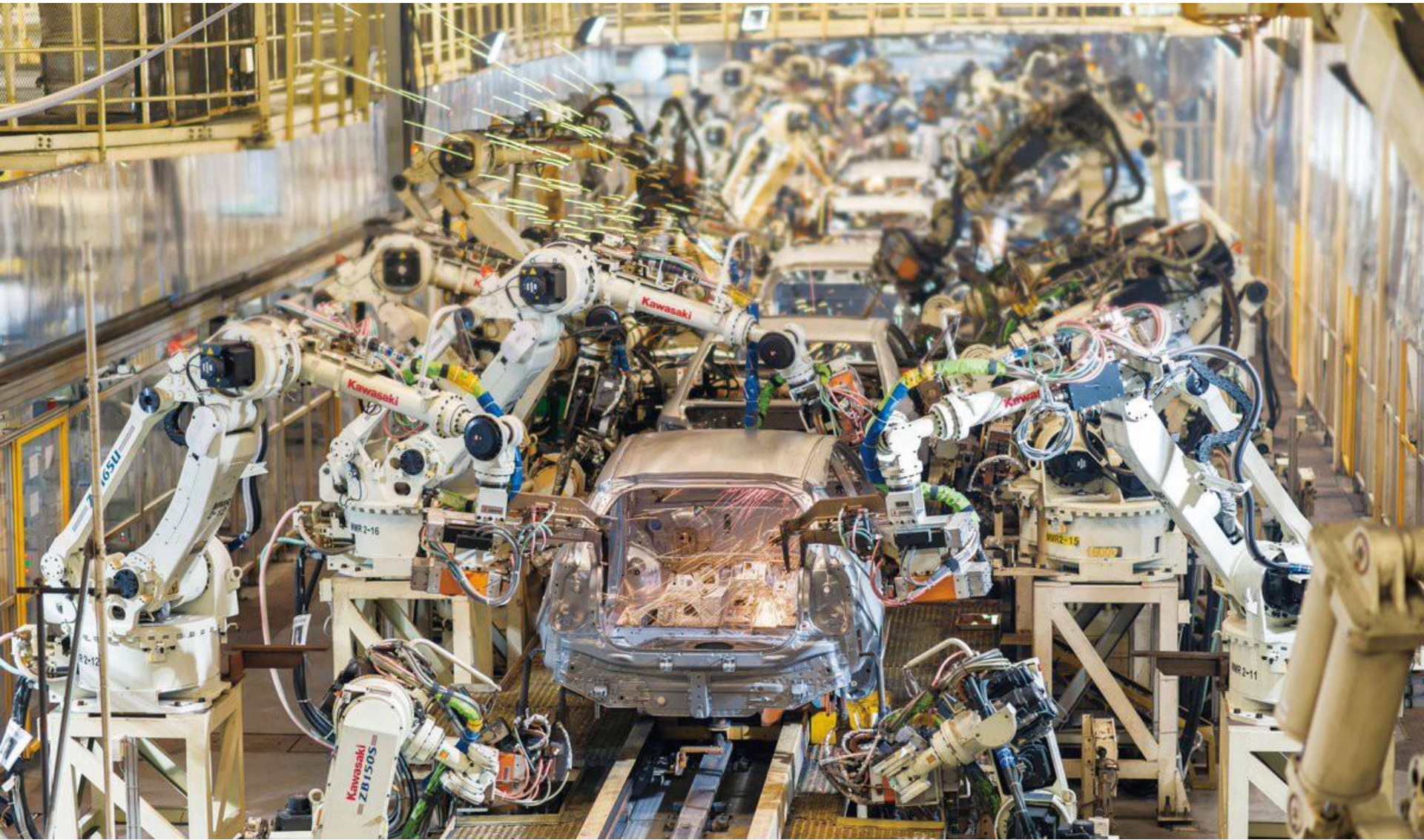
The guinea pig was the T's magneto, a component that supplied ignition energy to the engine before generators became common. A complex and innovative component that was one of the early Model T's technological advantages, Ford's magneto was integrated with the engine's flywheel and involved many pieces. Under the old system, each magneto was assembled by one worker. On average, that worker could assemble 35 of them in a nine-hour shift, or roughly one every 15 minutes.



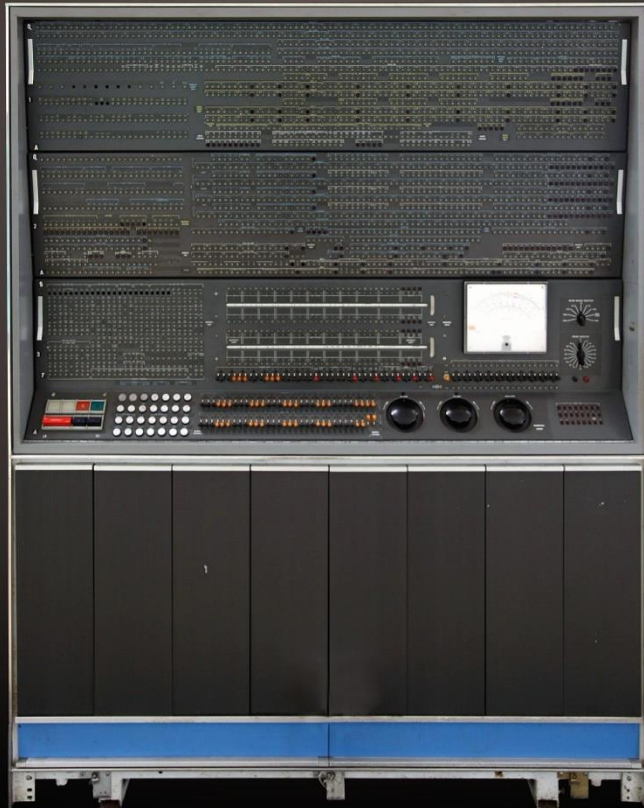
From the collections of the Henry Ford

Ford's transition to moving assembly lines began in April 1913 with the integrated (and complex) flywheel/magneto. With each worker assigned to complete a few specific tasks rather than build the entire unit, Ford reduced magneto assembly time from about 15 minutes to 5, and the required workforce decreased from 29 to 14.

A Real-Life Pipeline: Automobile Assembly



An Old Pipelined Computer: IBM Stretch



Design

Manufacturer	IBM
Designer	Gene Amdahl
Release date	May 1961
Units sold	9
Price	US\$7,780,000 (equivalent to \$67,380,000 in 2020)

Casing

Weight	70,000 pounds (35 short tons; 32 t) ^[1]
Power	100 kW ^[1] @ 110 V

System

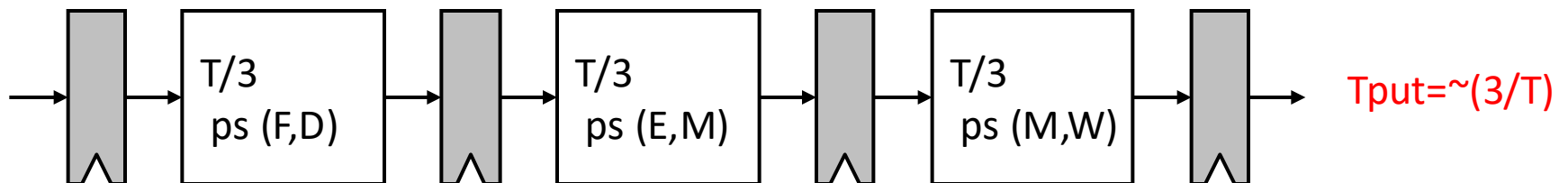
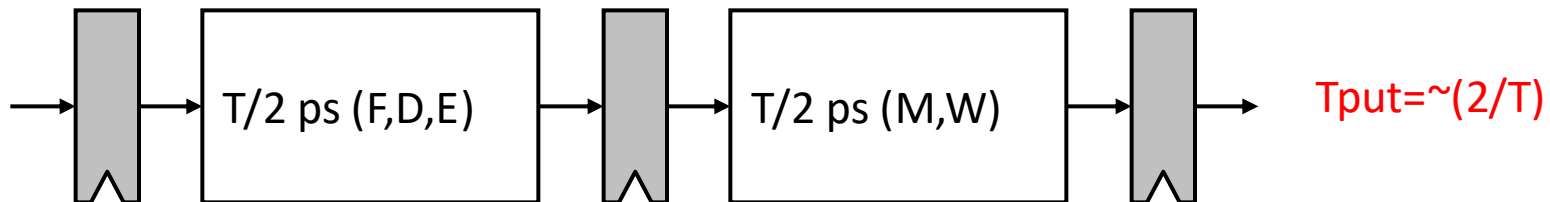
Operating system	MCP
CPU	64-bit processor
Memory	2048 kilobytes (262144 x 64bits) ^[1]
MIPS	1.2 MIPS

An Ideal Pipeline

- Goal: **Increase throughput with little increase in cost**
(hardware cost, in case of instruction processing)
- Repetition of **identical operations**
 - The same operation is repeated on a large number of different inputs (e.g., all laundry loads go through the same steps)
- Repetition of **independent operations**
 - No dependences between repeated operations
- **Uniformly partitionable suboperations**
 - Processing can be evenly divided into uniform-latency suboperations (that do not share resources)
- Fitting examples: automobile assembly line, doing laundry
 - What about the instruction processing “cycle”?

Ideal Pipelining

$T_{\text{put}} = \text{Throughput}$



More Realistic Pipeline: Throughput

- Nonpipelined version with delay T

$T_{\text{put}} = 1 / (T + S)$ where S = register (sequential logic) delay

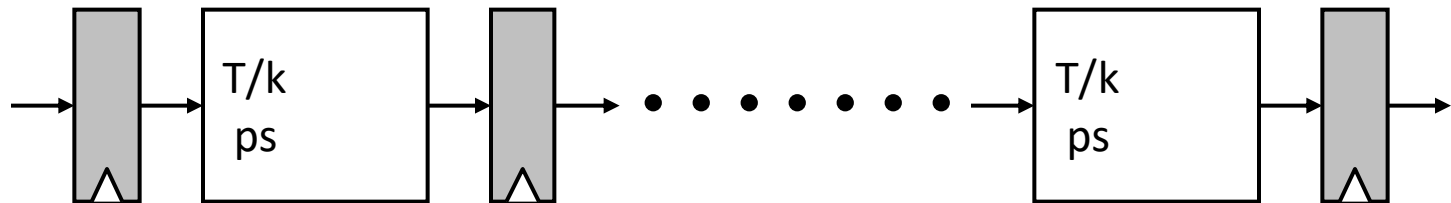


- k-stage pipelined version

$$T_{\text{put}_{k\text{-stage}}} = 1 / (T/k + S)$$

$$T_{\text{put}_{\text{max}}} = 1 / (1 \text{ gate delay} + S)$$

**Register delay reduces throughput
(sequencing overhead b/w stages)**



This picture assumes “perfect division of work between stages (T/k)”

More Realistic Pipeline: Cost

- Nonpipelined version with combinational cost G

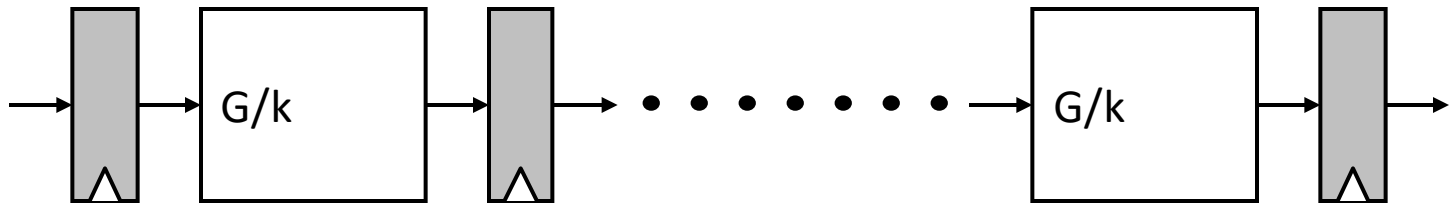
$\text{Cost} = G + R$ where R = register cost



- k -stage pipelined version

$\text{Cost}_{k\text{-stage}} = G + Rk$

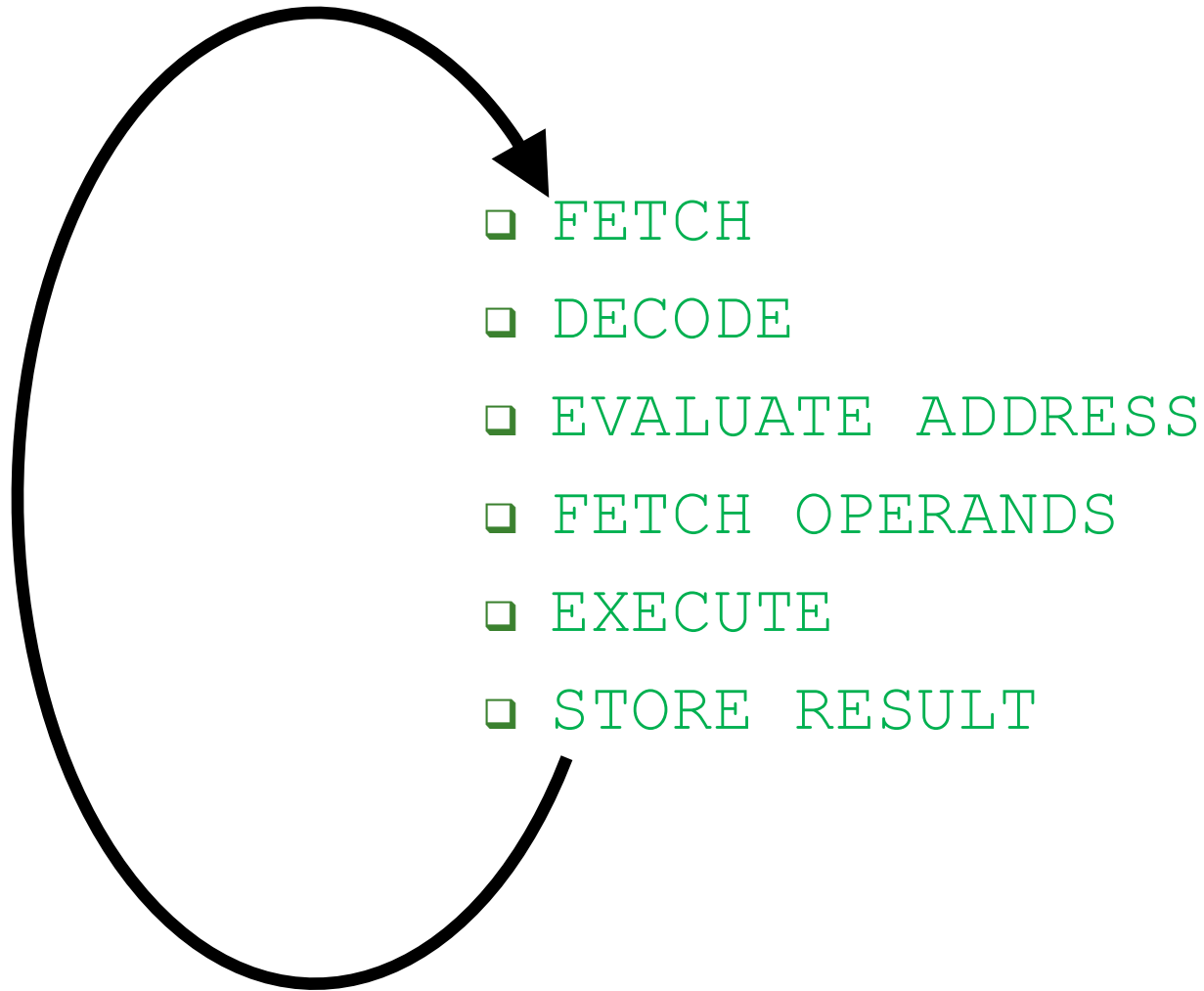
Registers increase hardware cost



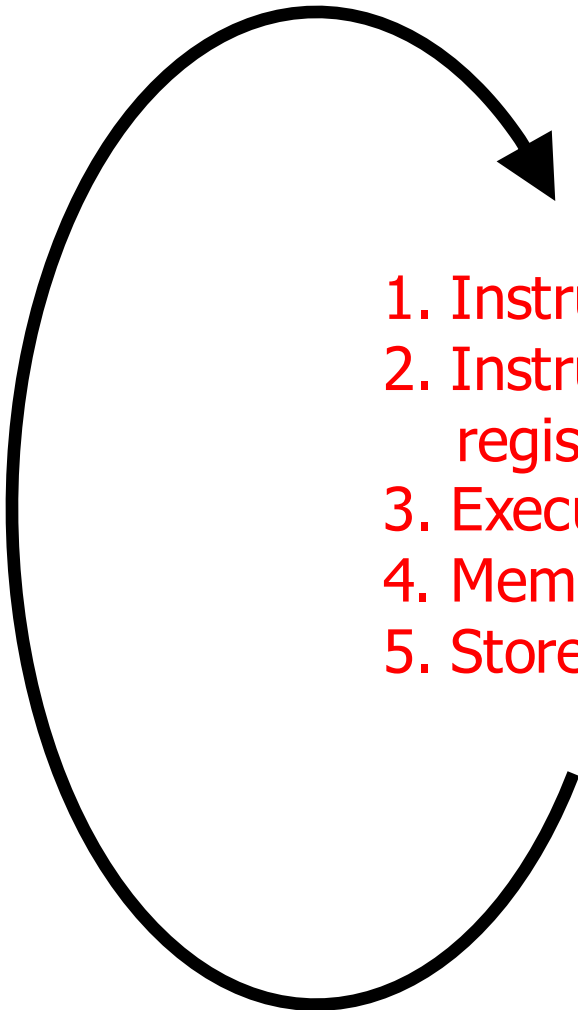
This picture ignores resource and register replication that is likely needed (G/k and Rk)

Pipelining Instruction Processing

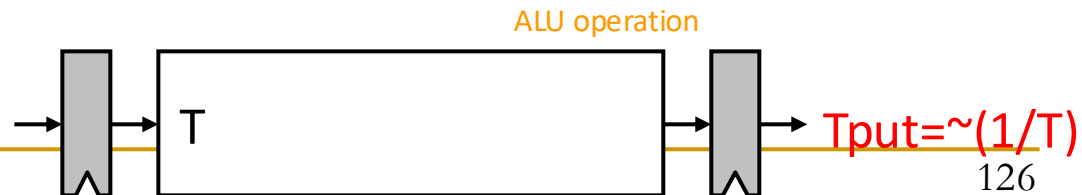
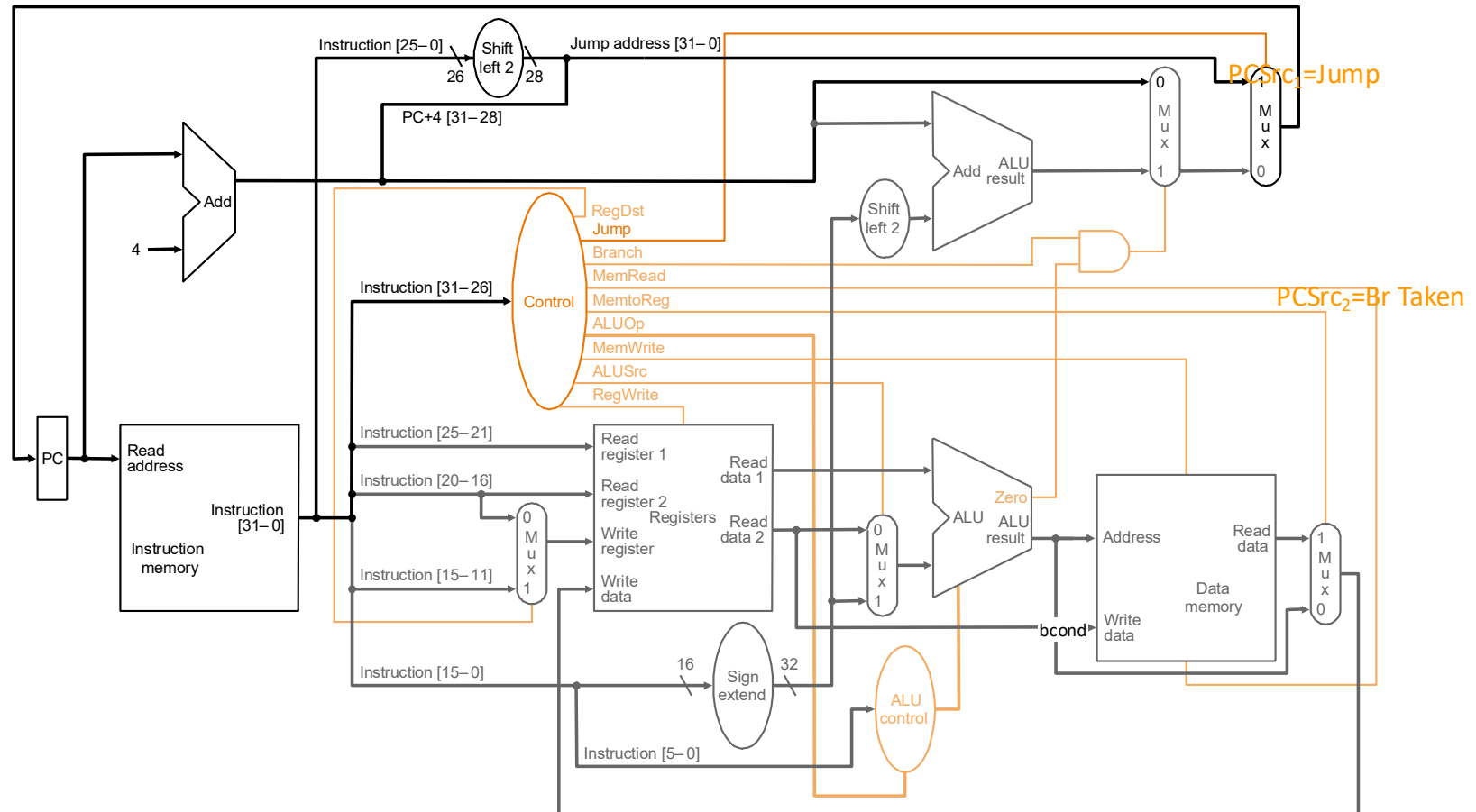
Remember: The Instruction Processing Cycle



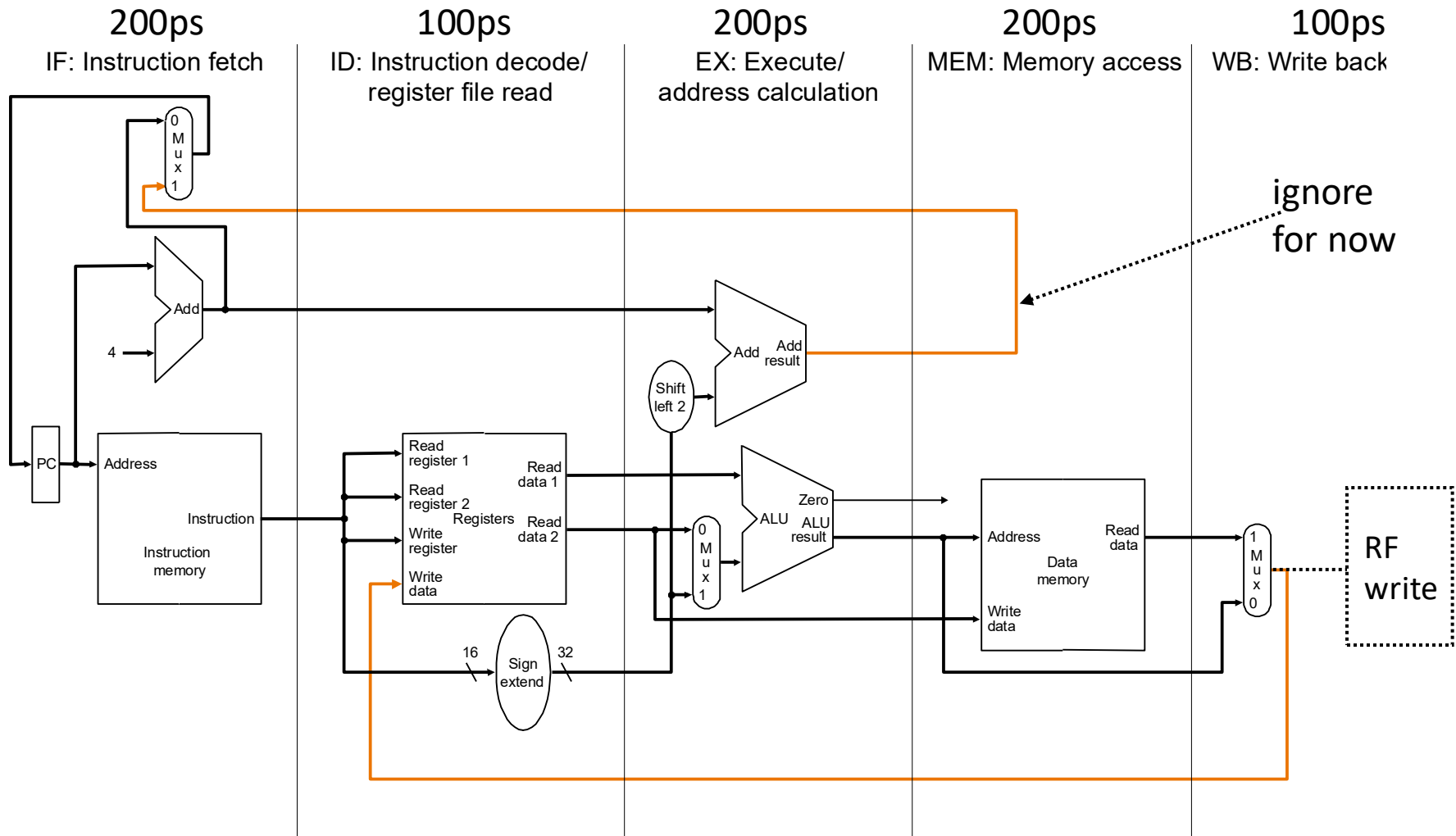
Remember: The Instruction Processing Cycle

- 
1. Instruction fetch (IF)
 2. Instruction decode and register operand fetch (ID/RF)
 3. Execute/Evaluate memory address (EX/AG)
 4. Memory operand fetch (MEM)
 5. Store/writeback result (WB)

Remember the Single-Cycle Microarchitecture



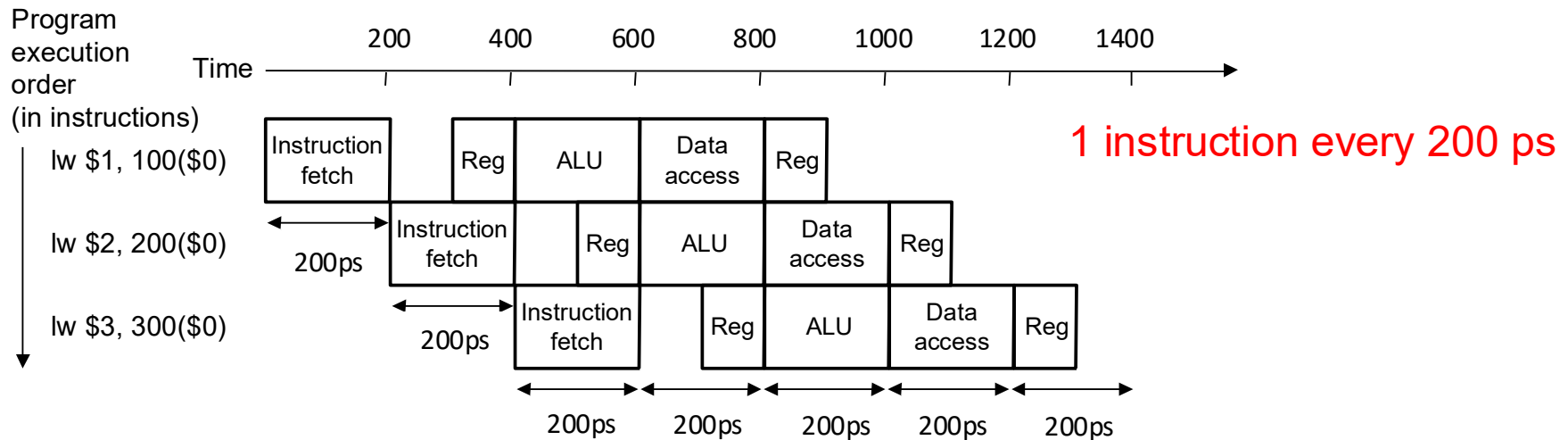
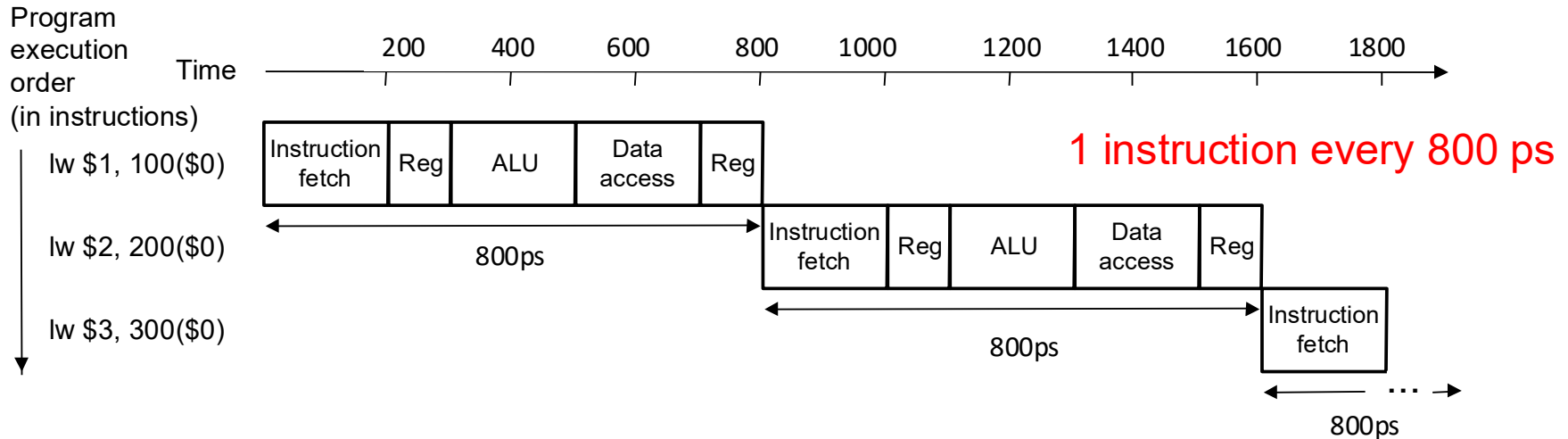
Dividing the Single-Cycle Uarch Into Stages



Is this the correct partitioning?

Why not 4 or 6 stages? Why not different boundaries?

Instruction Pipeline Throughput

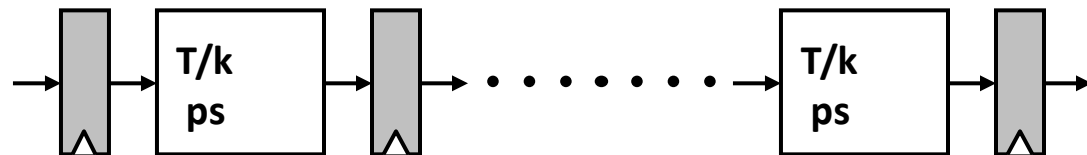
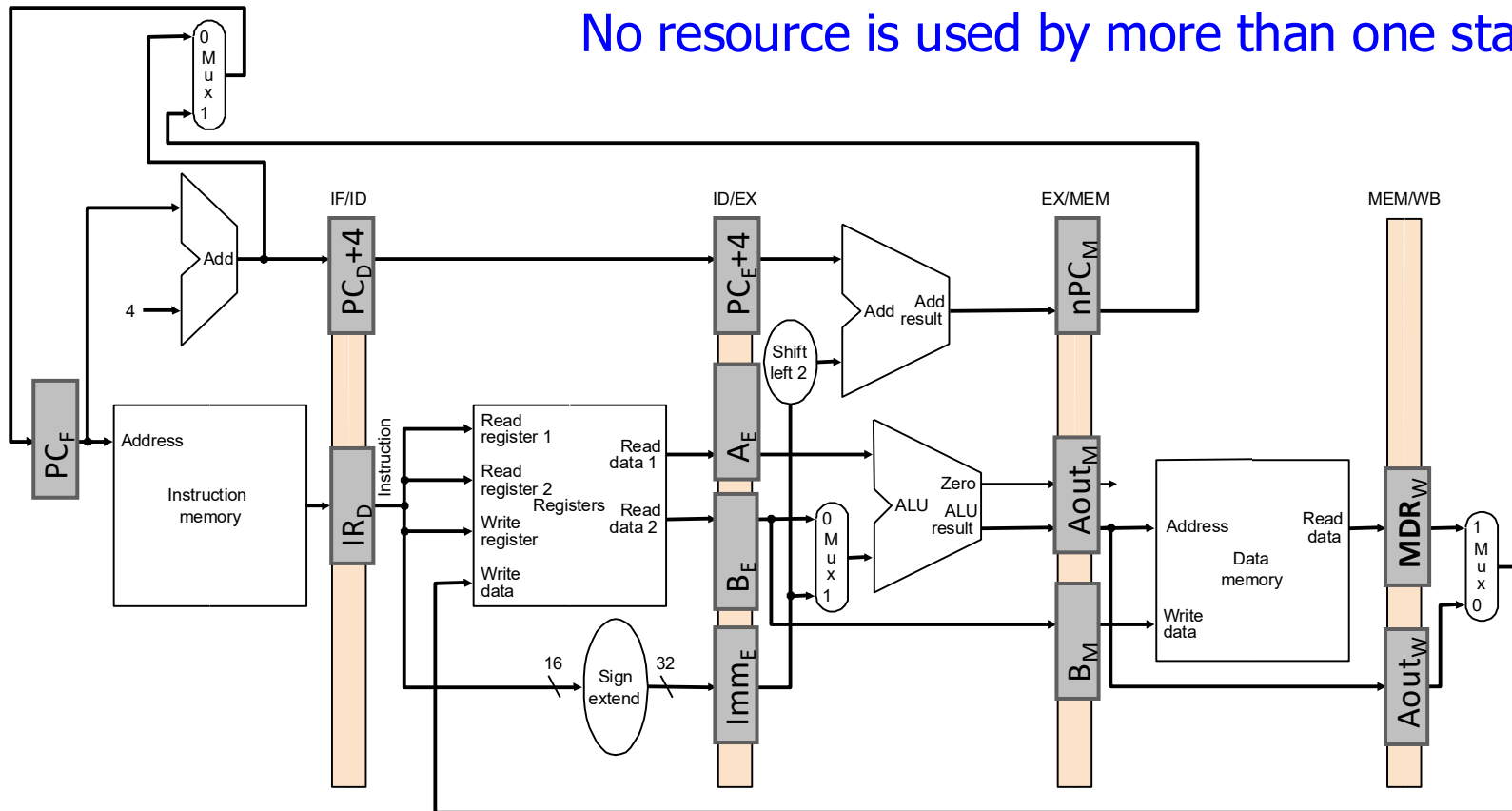


5-stage speedup is 4, not 5 as predicted by the ideal model. Why?

Half of the clock cycle is wasted in two stages (ID/RF and WB)

Enabling Pipelined Processing: Pipeline Registers

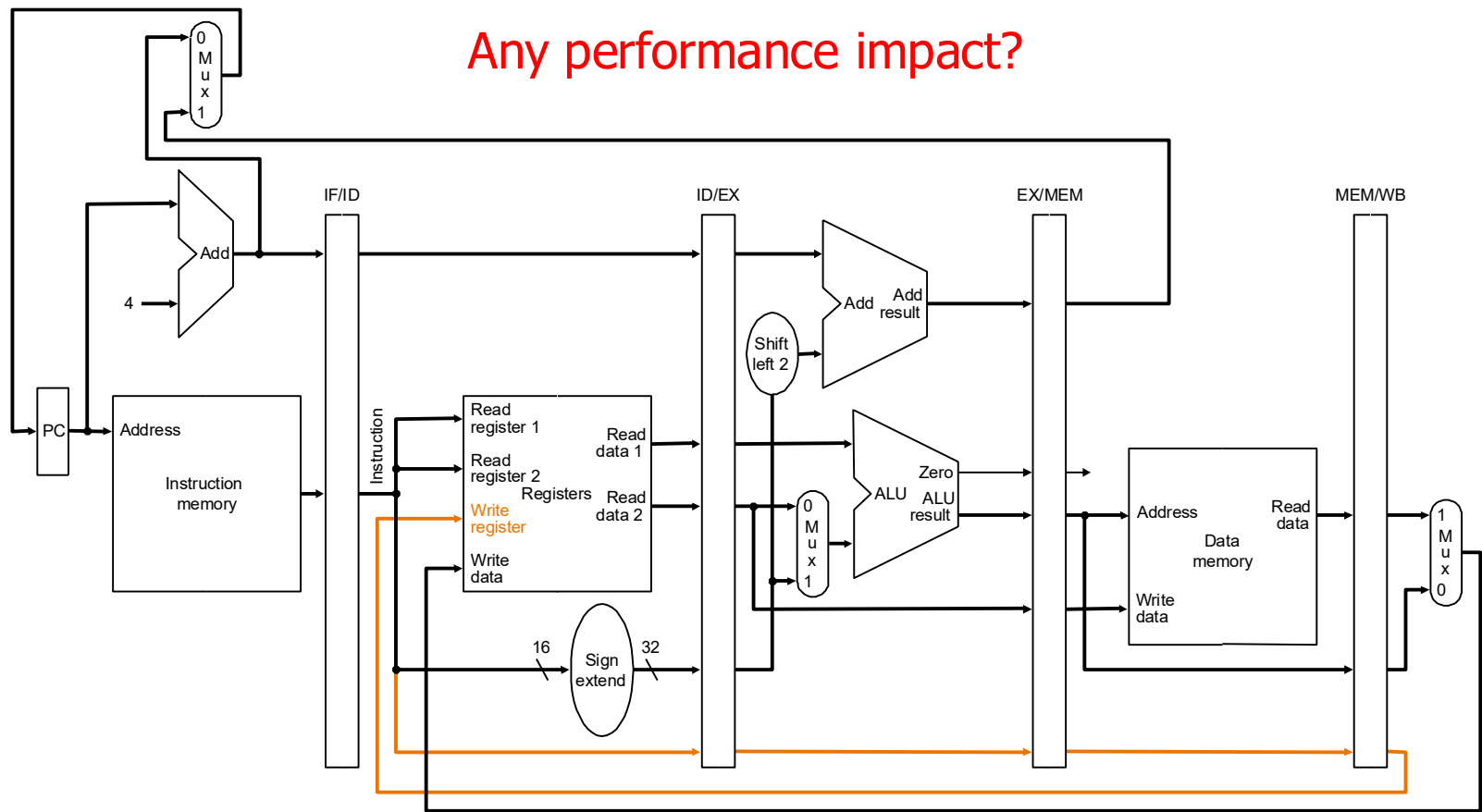
No resource is used by more than one stage



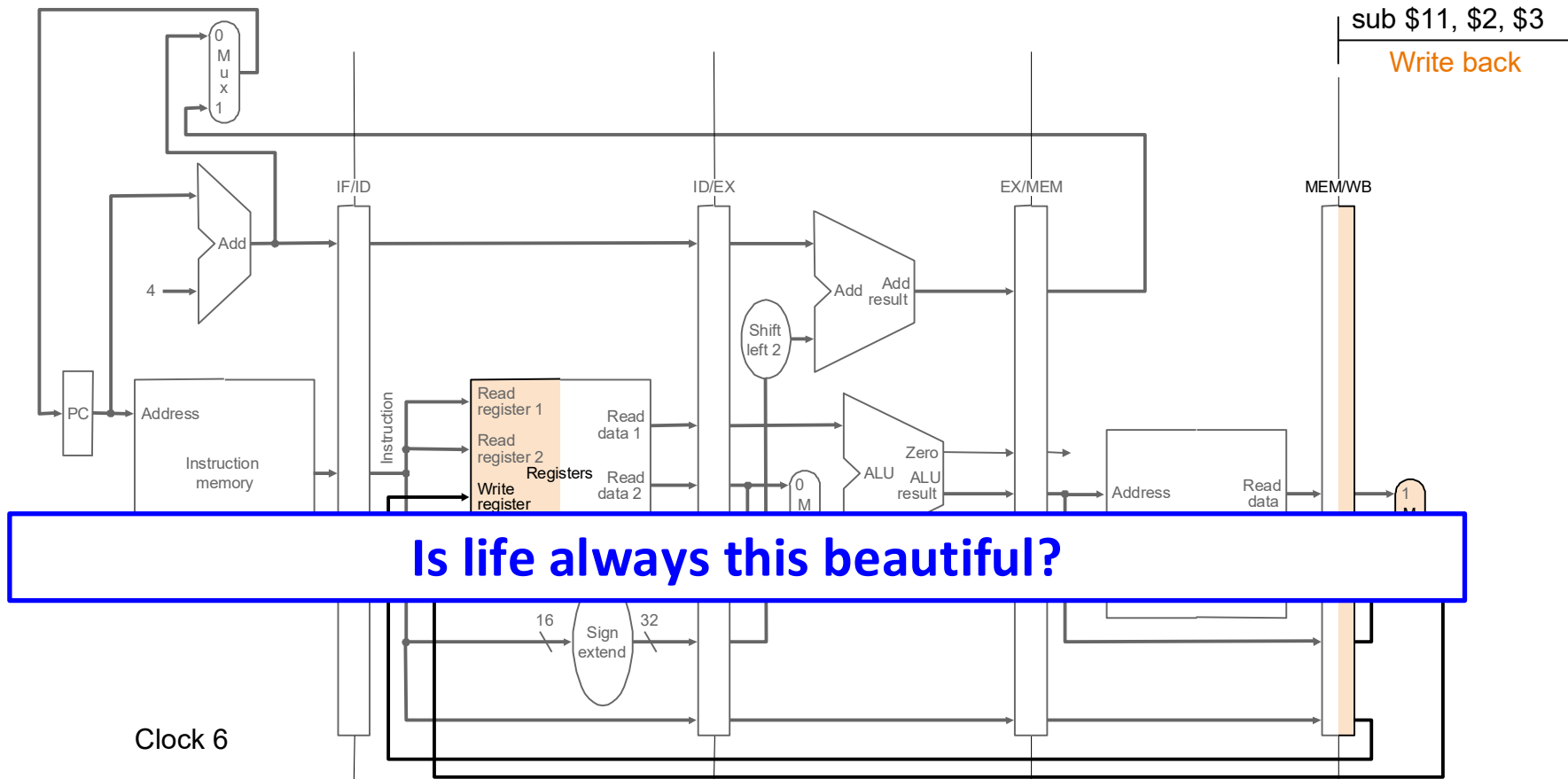
Pipelined Operation Example

All instruction classes must follow the same path and timing through the pipeline stages

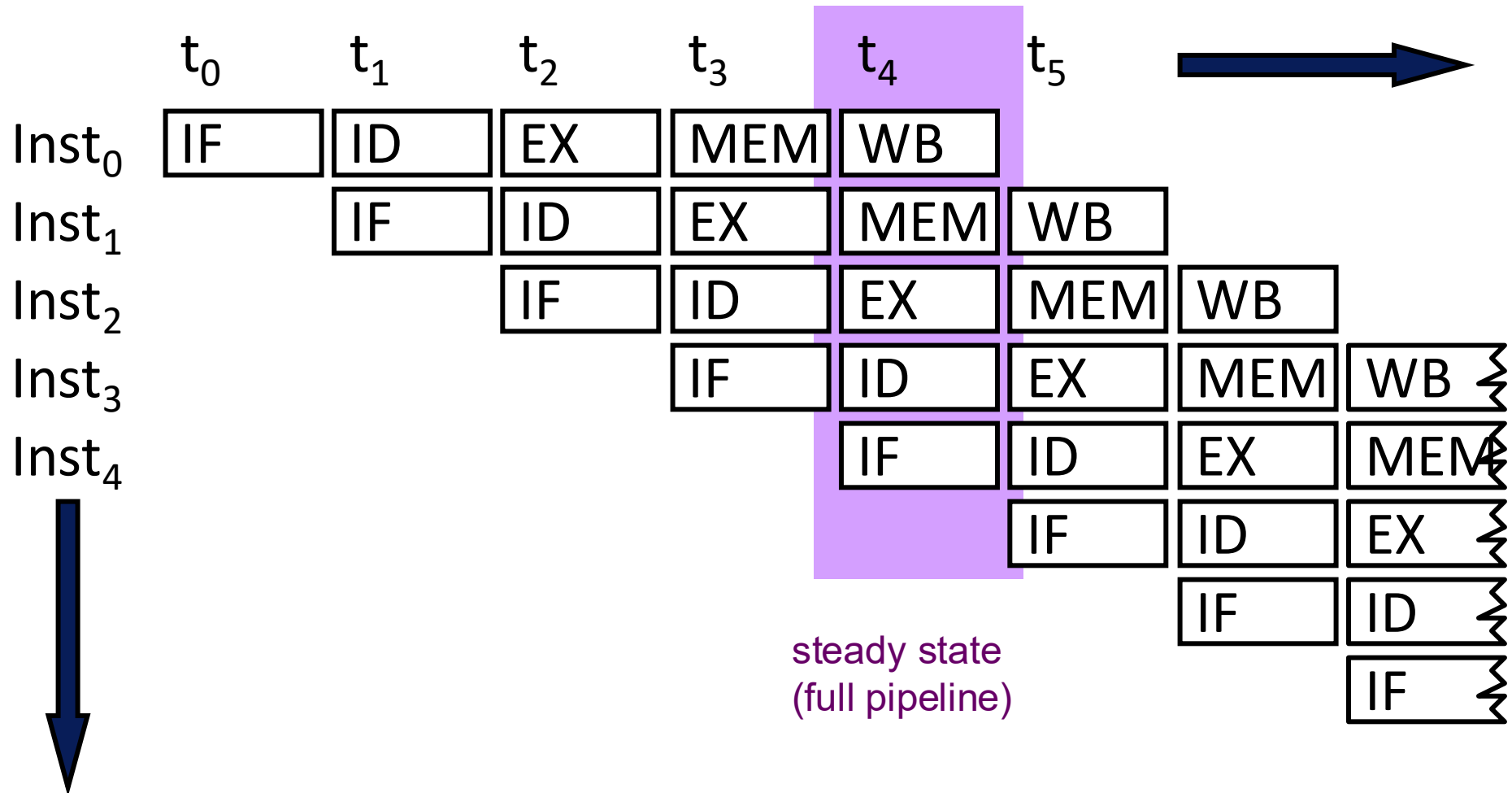
Any performance impact?



Pipelined Operation Example

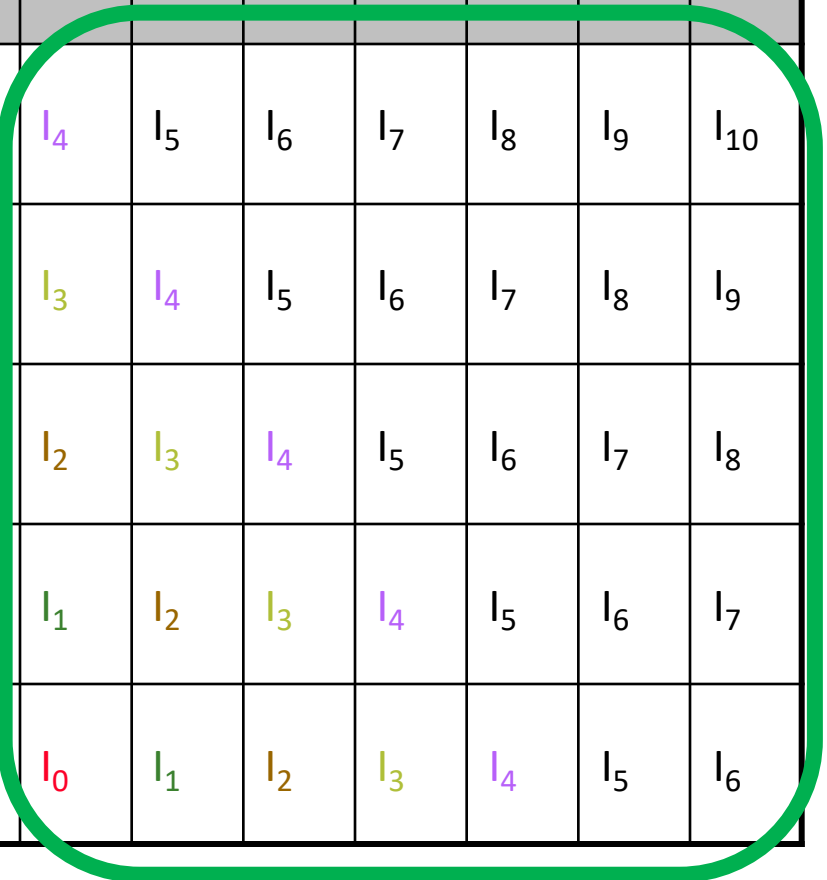


Illustrating Pipeline Operation: Operation View

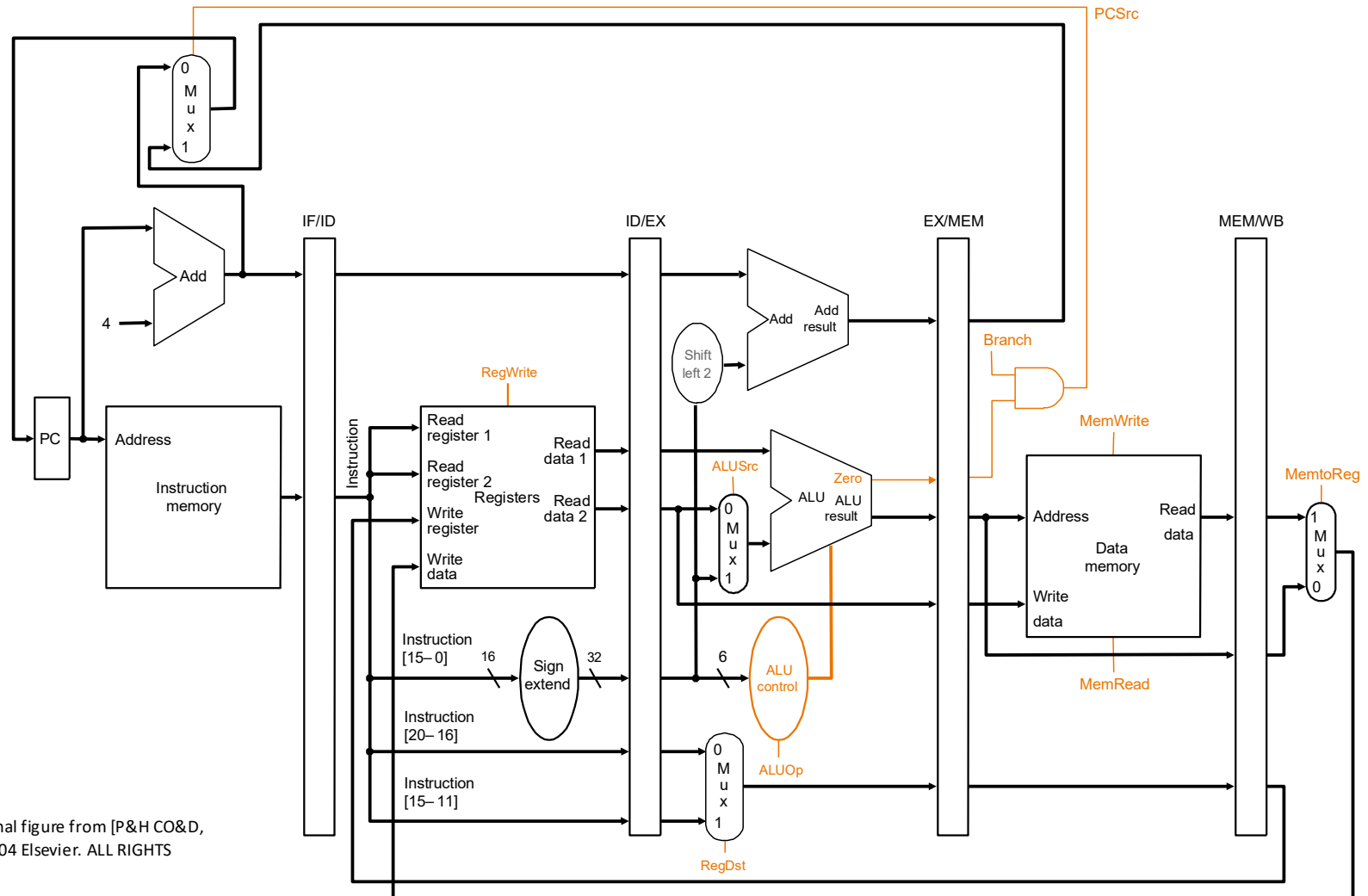


Illustrating Pipeline Operation: Resource View

	t ₀	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈	t ₉	t ₁₀
IF	I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	I ₈	I ₉	I ₁₀
ID		I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	I ₈	I ₉
EX			I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	I ₈
MEM				I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇
WB					I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆



Control Points in a Pipeline



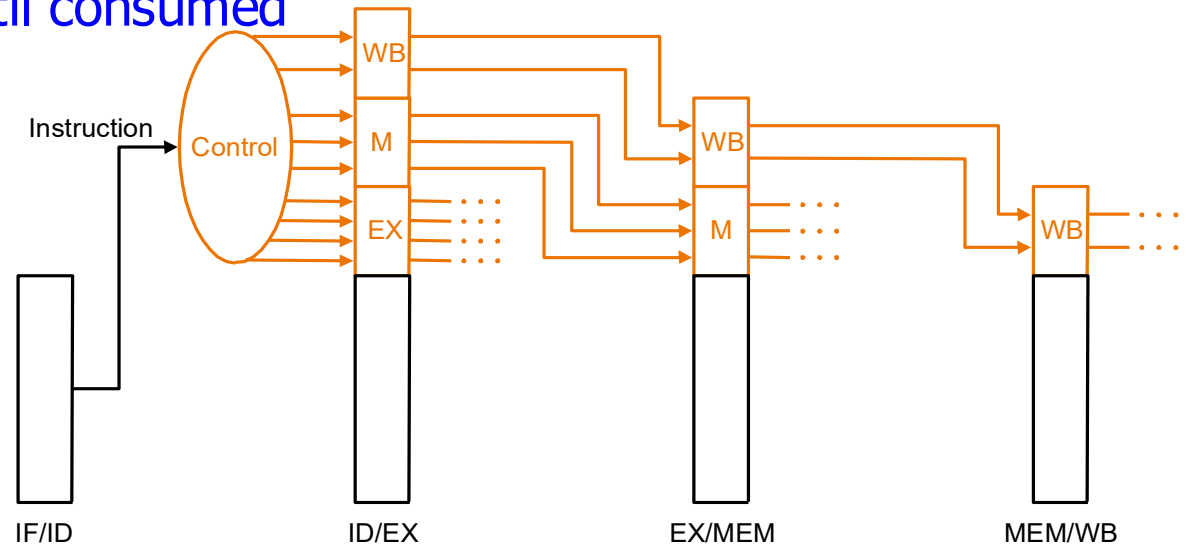
Based on original figure from [P&H CO&D,
COPYRIGHT 2004 Elsevier. ALL RIGHTS
RESERVED.]

Identical set of control points as the single-cycle datapath

Control Signals in a Pipeline

■ For a given instruction

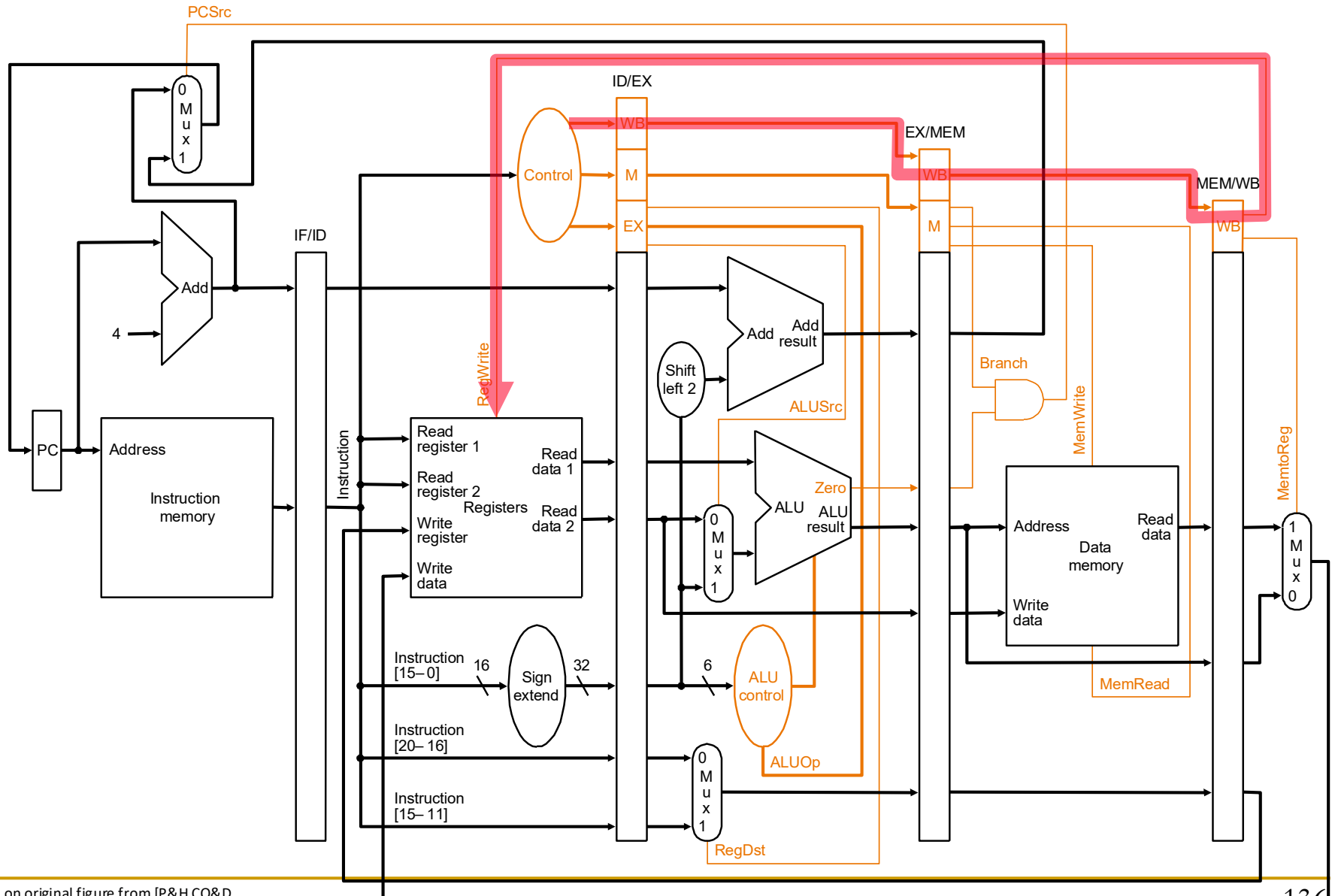
- same control signals as single-cycle, but
 - control signals required at different cycles, depending on stage
- ⇒ Option 1: decode once using the same logic as single-cycle and buffer signals until consumed



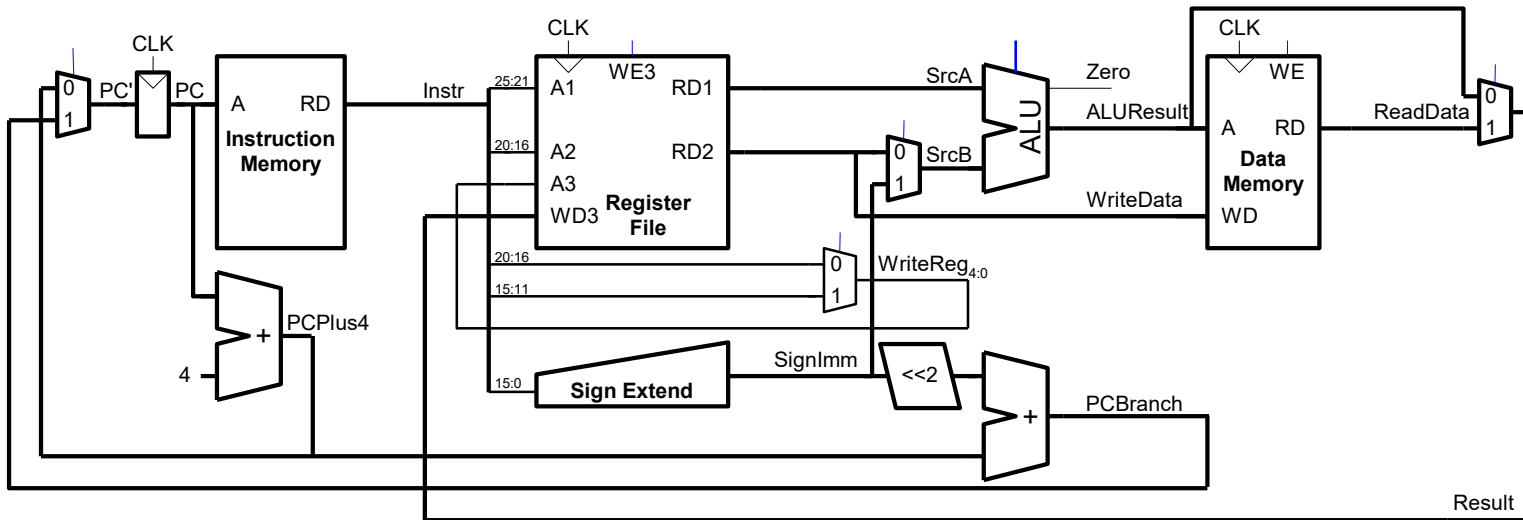
- ⇒ Option 2: carry relevant “instruction word/field” down the pipeline and decode locally within each or in a previous stage

Which one is better?

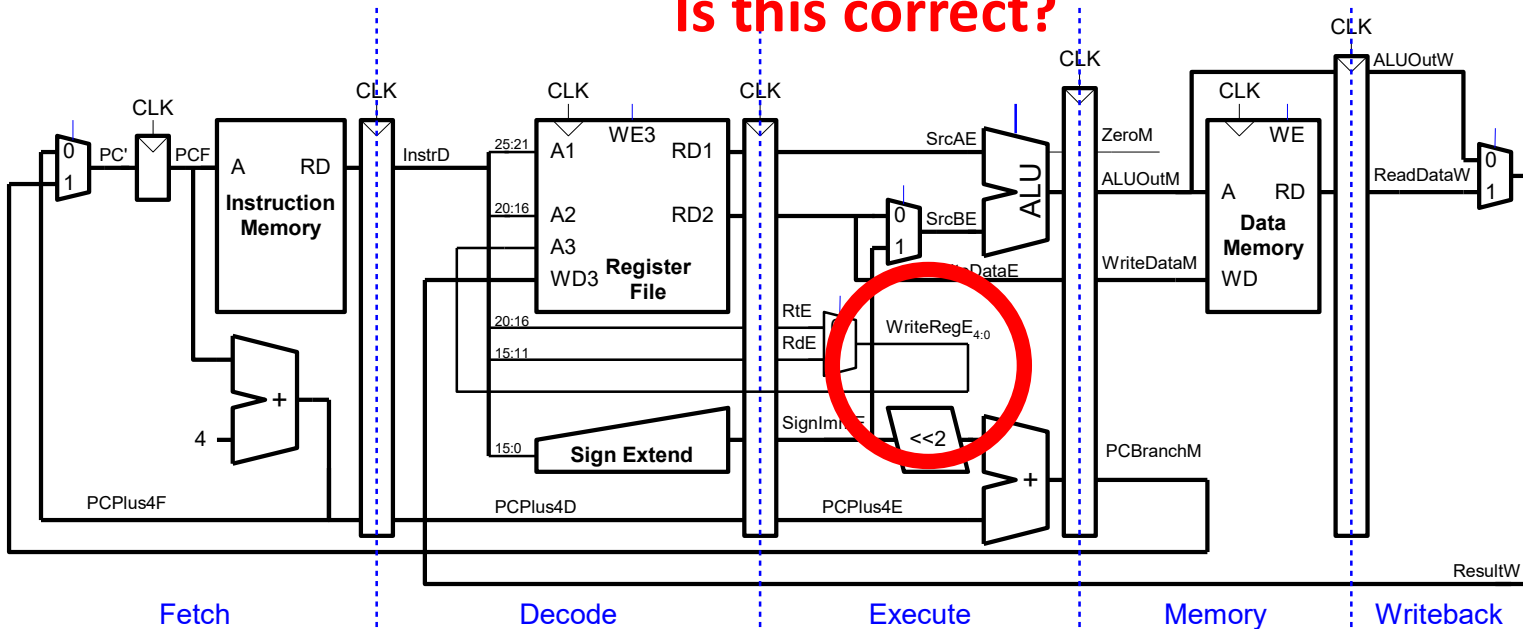
Pipelined Control Signals



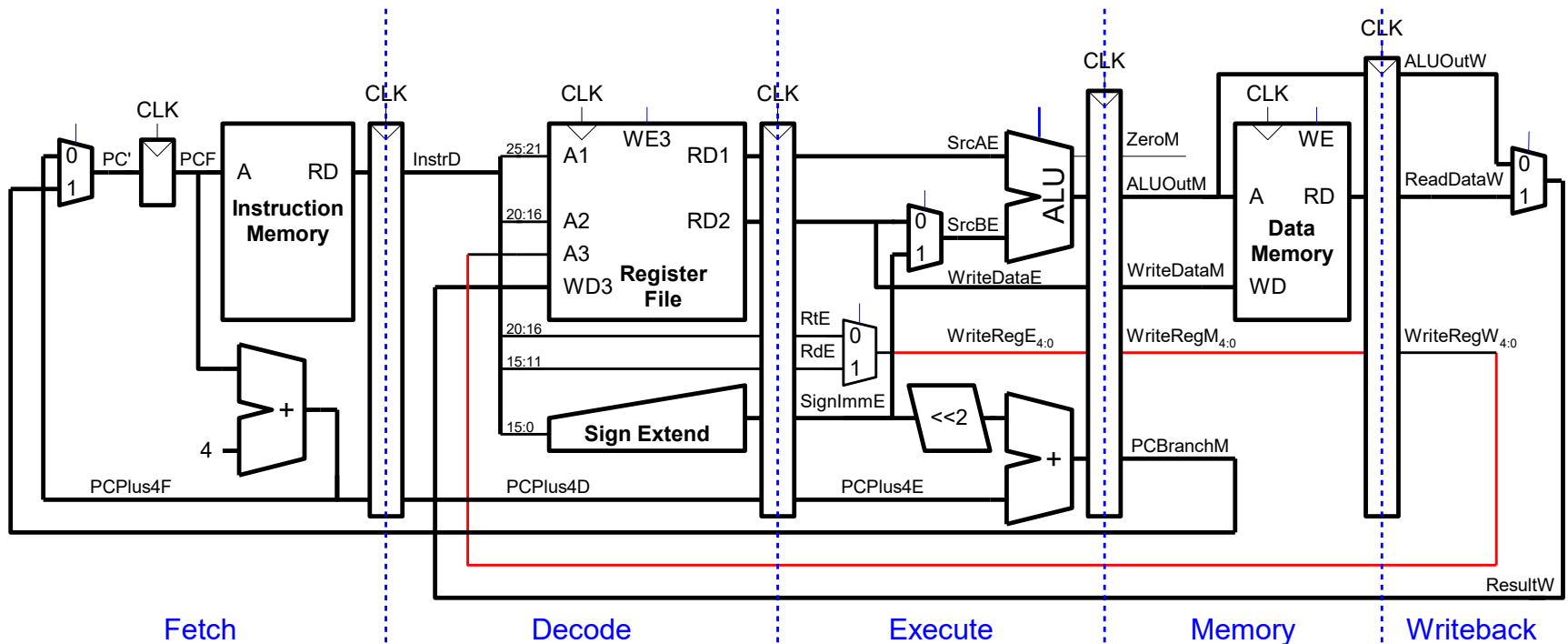
Another Example: Single-Cycle and Pipelined



Is this correct?

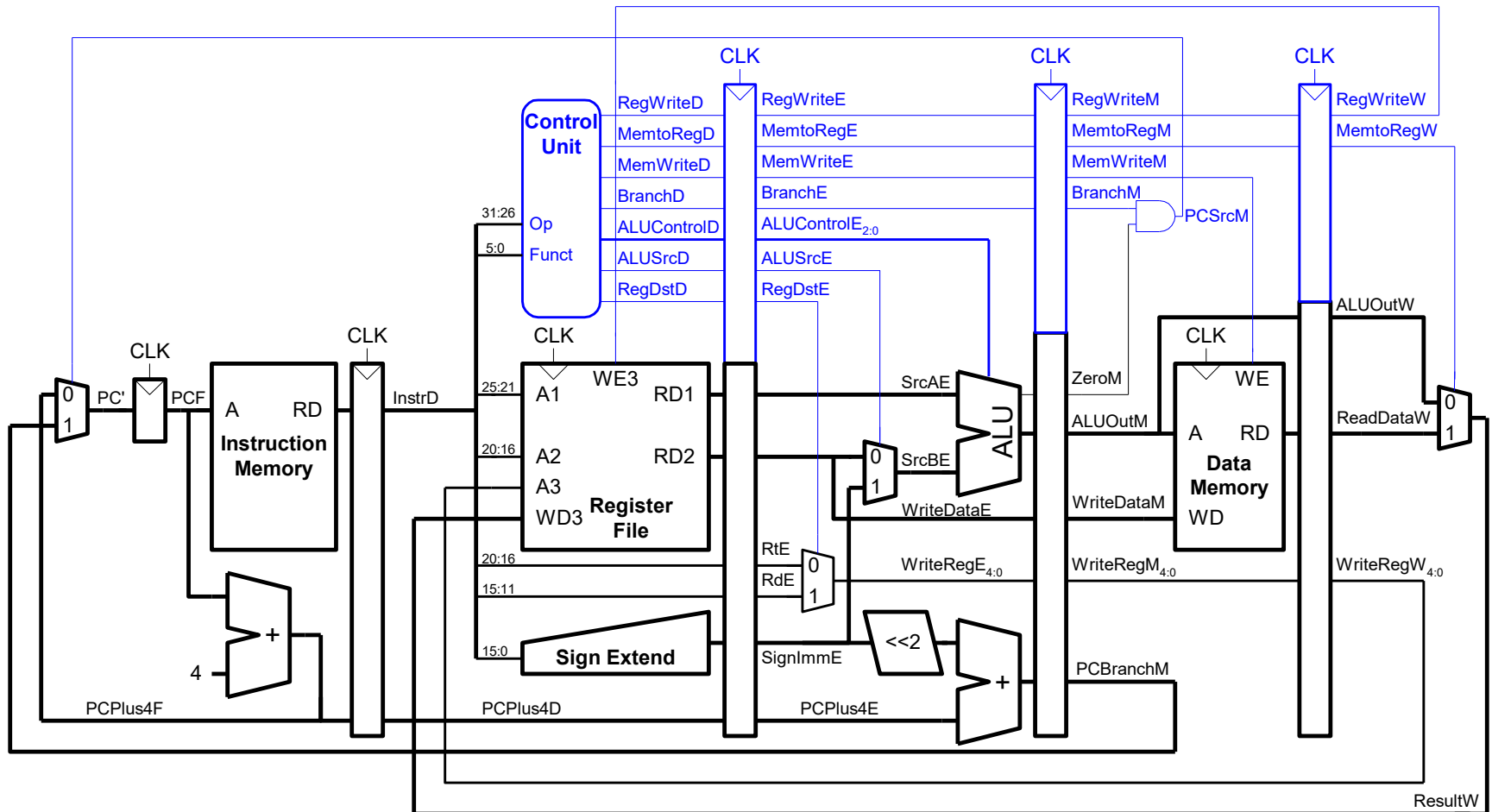


Another Example: **Correct** Pipelined Datapath



- **WriteReg control signal must arrive at the same time as Result**

Another Example: Pipelined Control



- Same control unit as single-cycle processor
- Control delayed to proper pipeline stage

Remember: An Ideal Pipeline

- Goal: **Increase throughput with little increase in cost** (hardware cost, in case of instruction processing)
- Repetition of **identical operations**
 - The same operation is repeated on a large number of different inputs (e.g., all laundry loads go through the same steps)
- Repetition of **independent operations**
 - No dependencies between repeated operations
- **Uniformly partitionable suboperations**
 - Processing can be evenly divided into uniform-latency suboperations (that do not share resources)
- Fitting examples: automobile assembly line, doing laundry
 - **What about the instruction processing “cycle”?**

Instruction Pipeline: Not An Ideal Pipeline

- Identical operations ... NOT!

- ⇒ different instructions → not all need the same stages

- Forcing different instructions to go through the same pipe stages

- external fragmentation (some pipe stages idle for some instructions)

- Uniform suboperations ... NOT!

- ⇒ different pipeline stages → not the same latency

- Need to force each stage to be controlled by the same clock

- internal fragmentation (some pipe stages are fast but still have to take the same clock cycle time)

- Independent operations ... NOT!

- ⇒ instructions are not independent of each other

- Need to detect and resolve inter-instruction dependences to ensure the pipeline provides correct results

- pipeline stalls (pipeline is not always moving)

Issues in Pipeline Design

- Balancing work in pipeline stages
 - How many stages and what is done in each stage
- Keeping the pipeline correct, moving, and full in the presence of events that disrupt pipeline flow
 - Handling dependences
 - Data
 - Control
 - Handling resource contention
 - Handling long-latency (multi-cycle) operations
- Handling exceptions, interrupts
- Advanced: Improving pipeline throughput
 - Minimizing *stalls*

Causes of Pipeline *Stalls*

- Stall: A condition when the pipeline stops moving
- Resource contention
- Dependences (between instructions)
 - Data
 - Control
- Long-latency (multi-cycle) operations

Dependences and Their Types

- Also called “dependency” or *less desirably* “hazard”
- Dependences dictate ordering requirements between instructions
- Two types
 - Data dependence
 - Control dependence
- Resource contention is sometimes called resource dependence
 - However, this is not fundamental to (dictated by) program semantics, so we will treat it separately

Handling Resource Contention

- Happens when instructions in two pipeline stages need the same resource
- Solution 1: Eliminate the cause of contention
 - Duplicate the resource or increase its throughput
 - E.g., use separate instruction and data memories (caches)
 - E.g., use multiple ports for memory structures
- Solution 2: Detect the resource contention and stall one of the contending stages
 - Which stage do you stall?
 - Example: What if you had a single read and write port for the register file?

Example Resource Dependence: RegFile

- With careful design, the register file can be read and written in the same cycle:
 - write takes place during the 1st half of the cycle
 - read takes place during the 2nd half of the cycle => no problem!!!
 - read/write from/to the register file has only *half a clock cycle* to complete

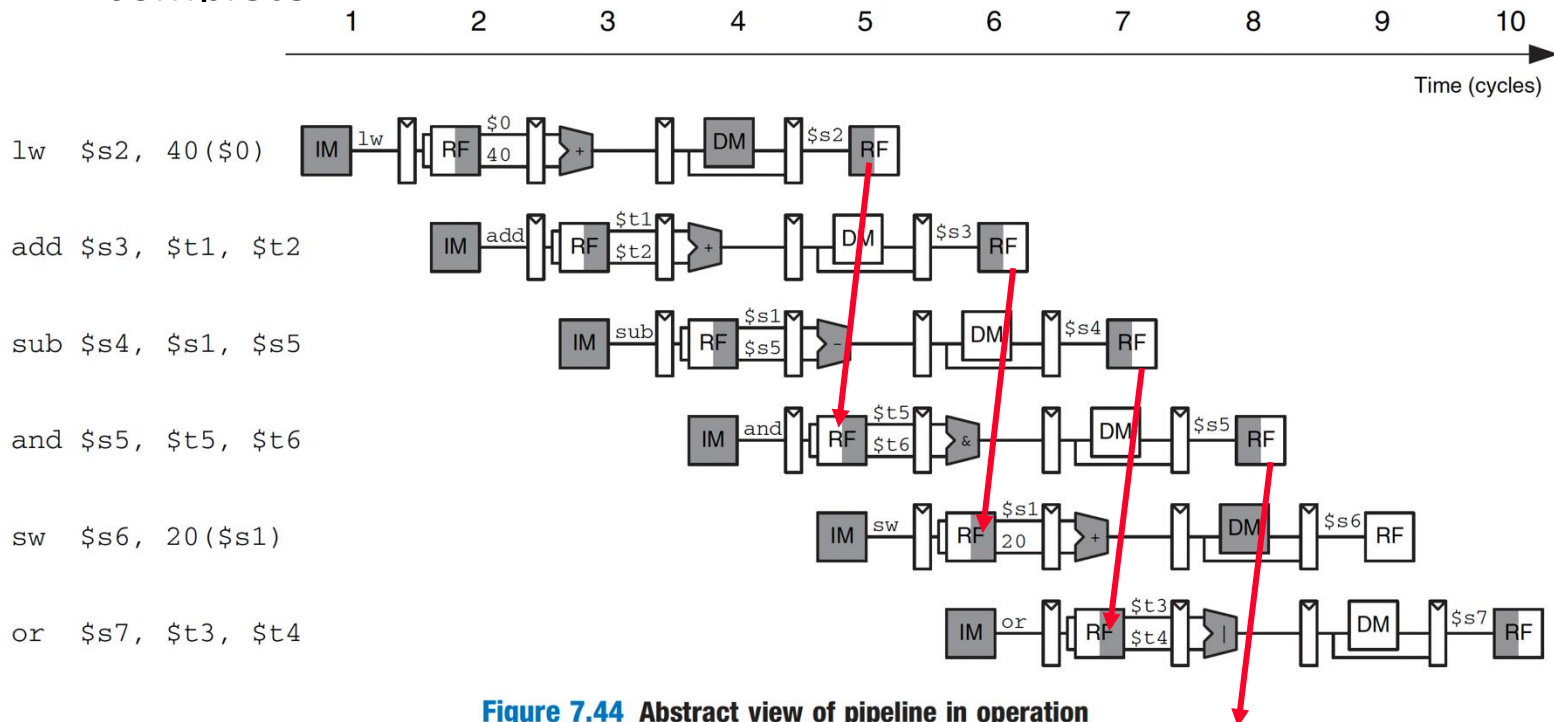


Figure 7.44 Abstract view of pipeline in operation

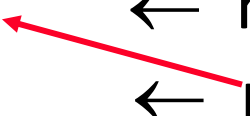
Data Dependences

- Data dependence types
 - Flow dependence (true data dependence – read after write)
 - Anti dependence (write after read)
 - Output dependence (write after write)
- Which ones cause stalls in a pipelined machine?
 - For all of them, we need to ensure semantics of the program is correct
 - Flow dependences always need to be obeyed because they constitute true dependence on a value
 - Anti and output dependences exist due to limited number of architectural registers
 - They are dependence on a name, not a value
 - We will later see what we can do about them

Data Dependence Types

Flow dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$



Read-after-Write
(RAW)

Anti dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_1 \leftarrow r_4 \text{ op } r_5$



Write-after-Read
(WAR)

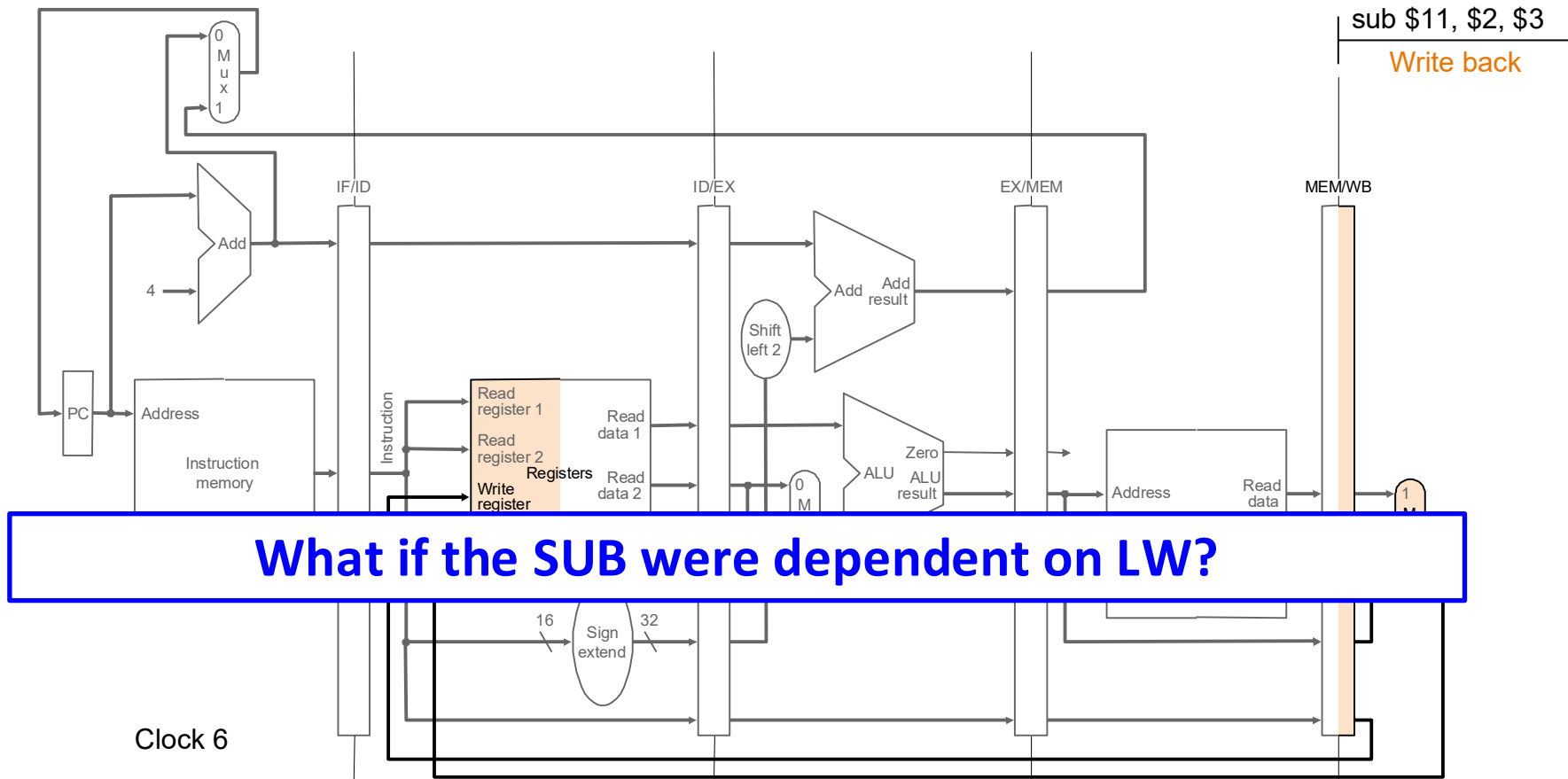
Output dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$
 $r_3 \leftarrow r_6 \text{ op } r_7$



Write-after-Write
(WAW)

Pipelined Operation Example



Data Dependence Handling

Readings for Next Few Lectures

- H&H, Chapter 7.5-7.9
- Smith and Sohi, “[The Microarchitecture of Superscalar Processors](#),” Proceedings of the IEEE, 1995
 - More advanced pipelining
 - Interrupt and exception handling
 - Out-of-order and superscalar execution concepts

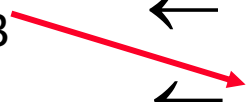
How to Handle Data Dependences

- Anti and output dependences are easier to handle
 - write to the destination only in last stage and in program order
- Flow dependences are more interesting & challenging
- Six fundamental ways of handling flow dependences
 - Detect and wait until value is available in register file
 - Detect and forward/bypass data to dependent instruction
 - Detect and eliminate the dependence at the software level
 - No need for the hardware to detect dependence
 - Detect and move it out of the way for independent instructions
 - Predict the needed value(s), execute “speculatively”, and verify
 - Do something else (fine-grained multithreading)
 - No need to detect

Recall: Data Dependence Types

Flow dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$



Read-after-Write
(RAW)

Anti dependence

$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_1 \leftarrow r_4 \text{ op } r_5$



Write-after-Read
(WAR)

Output dependence

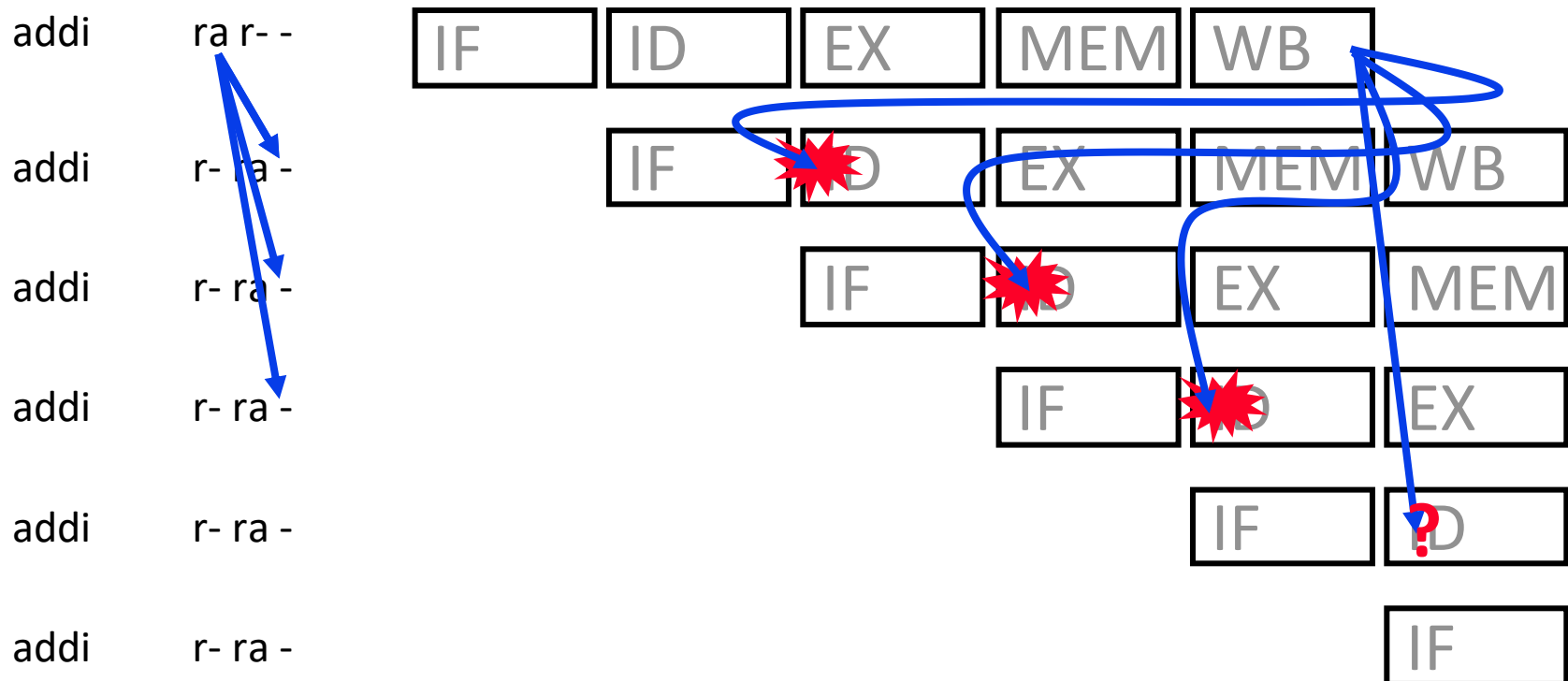
$r_3 \leftarrow r_1 \text{ op } r_2$
 $r_5 \leftarrow r_3 \text{ op } r_4$
 $r_3 \leftarrow r_6 \text{ op } r_7$



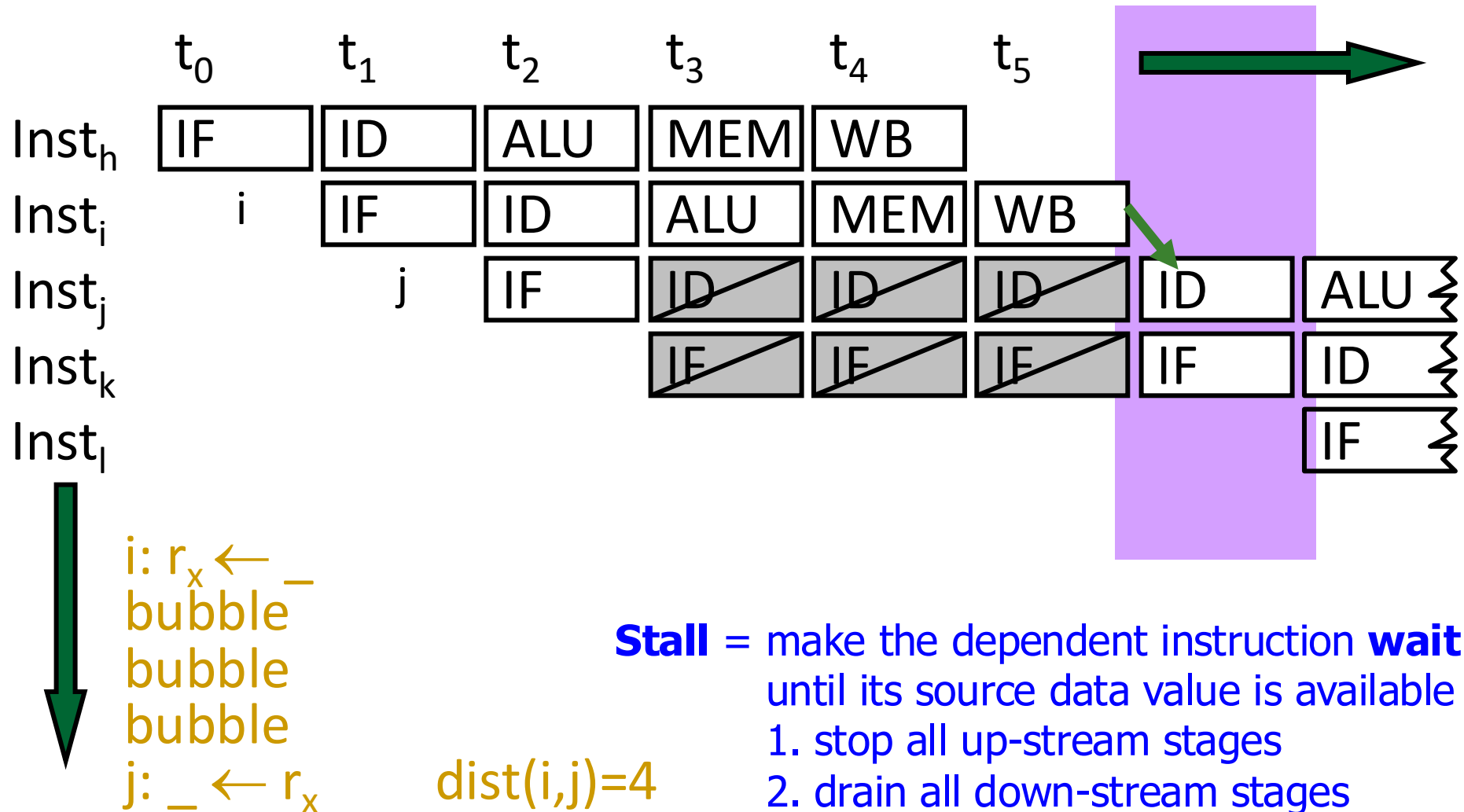
Write-after-Write
(WAW)

RAW Dependence Handling

- Which one of the following flow dependences lead to conflicts in the 5-stage pipeline?



Pipeline Stall: Resolving Data Dependence



Interlocking

- Interlocking: Detection of dependence between instructions in a pipelined processor to guarantee correct execution
- Software based interlocking
vs.
- Hardware based interlocking
- MIPS acronym?

Approaches to Dependence Detection (I)

■ Scoreboarding

- ❑ Each register in register file has a Valid bit associated with it
- ❑ An instruction that is writing to the register resets the Valid bit
- ❑ An instruction in Decode stage checks if all its source and destination registers are Valid
 - Yes: No need to stall... No dependence
 - No: Stall the instruction

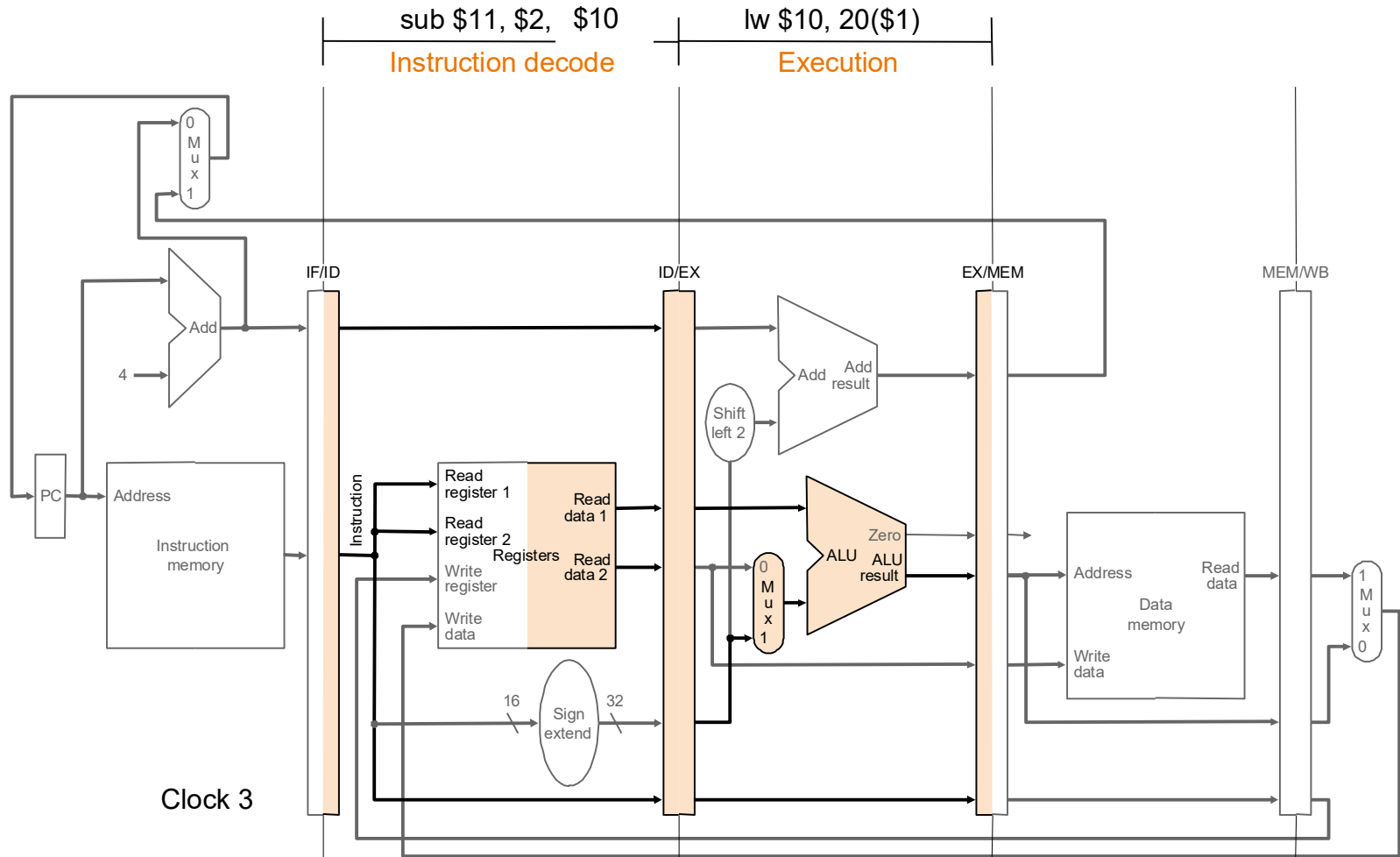
■ Advantage:

- ❑ Simple. 1 bit per register

■ Disadvantage:

- ❑ Need to stall for all types of dependences, not only flow dep.

Pipelined Operation Example



Now assume SUB is dependent on LW

Approaches to Dependence Detection (II)

■ Combinational dependence check logic

- ❑ Special logic checks if any instruction in later stages is supposed to write to any source register of the instruction that is being decoded
- ❑ Yes: stall the instruction/pipeline
- ❑ No: no need to stall... no flow dependence

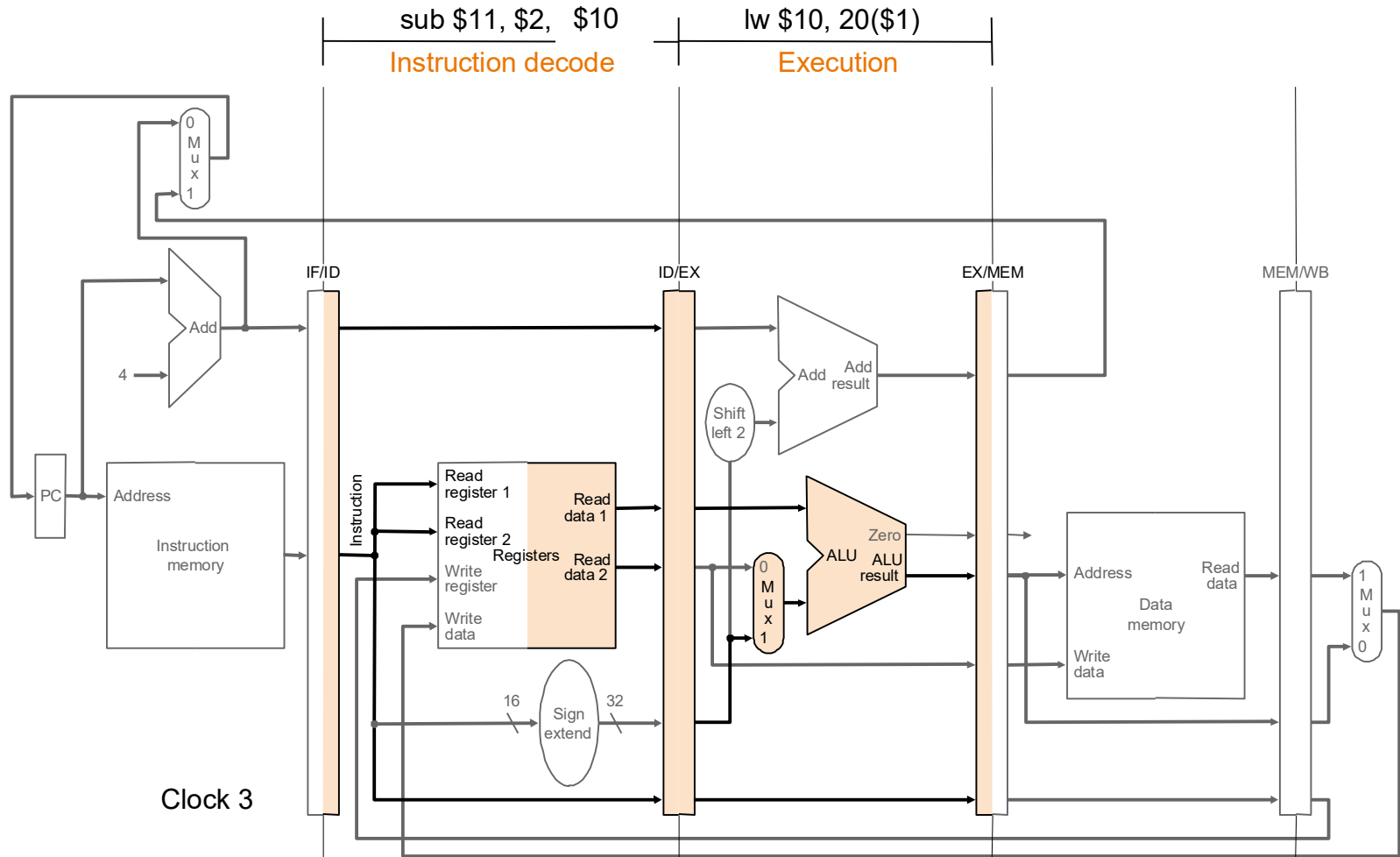
■ Advantage:

- ❑ No need to stall on anti and output dependences

■ Disadvantage:

- ❑ Logic is more complex than a scoreboard
- ❑ Logic becomes more complex as we make the pipeline deeper and wider (flash-forward: superscalar execution, later lecture)

Pipelined Operation Example



Now assume SUB is dependent on LW

Once You Detect the Dependence in Hardware

- What do you do afterwards?
- Observation: Dependence between two instructions is detected before the communicated data value becomes available
- Option 1: Stall the dependent instruction right away
- Option 2: Stall the dependent instruction only when necessary → data forwarding/bypassing
- Option 3: ...

Data Forwarding/Bypassing

- Problem: A consumer (dependent) instruction has to wait in decode stage until the producer instruction writes its value in the register file
- Goal: We do not want to stall the pipeline unnecessarily
- Observation: The data value needed by the consumer instruction can be supplied directly from a later stage in the pipeline (instead of only from the register file)
- Idea: Add additional dependence check logic and data forwarding paths (buses) to supply the producer's value to the consumer right after the value is available
- Benefit: Consumer can move in the pipeline until the point the value can be supplied → **less stalling**

Data Dependence Handling: Concepts and Implementation

■ Stall

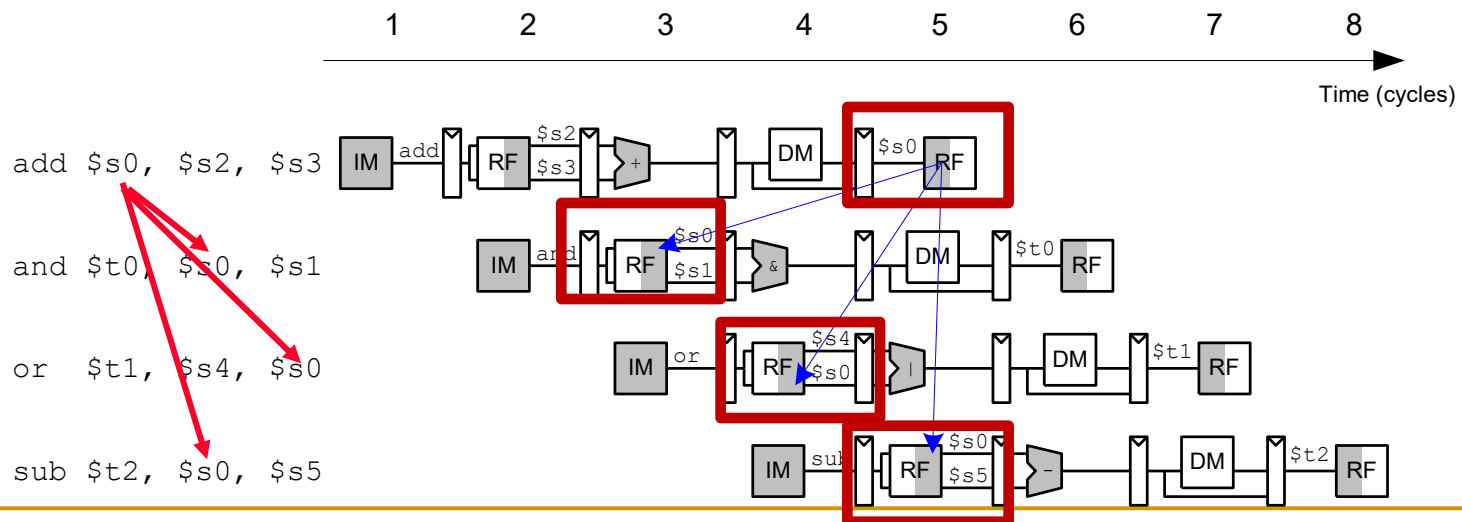
- 164

RAW Data Dependence Example

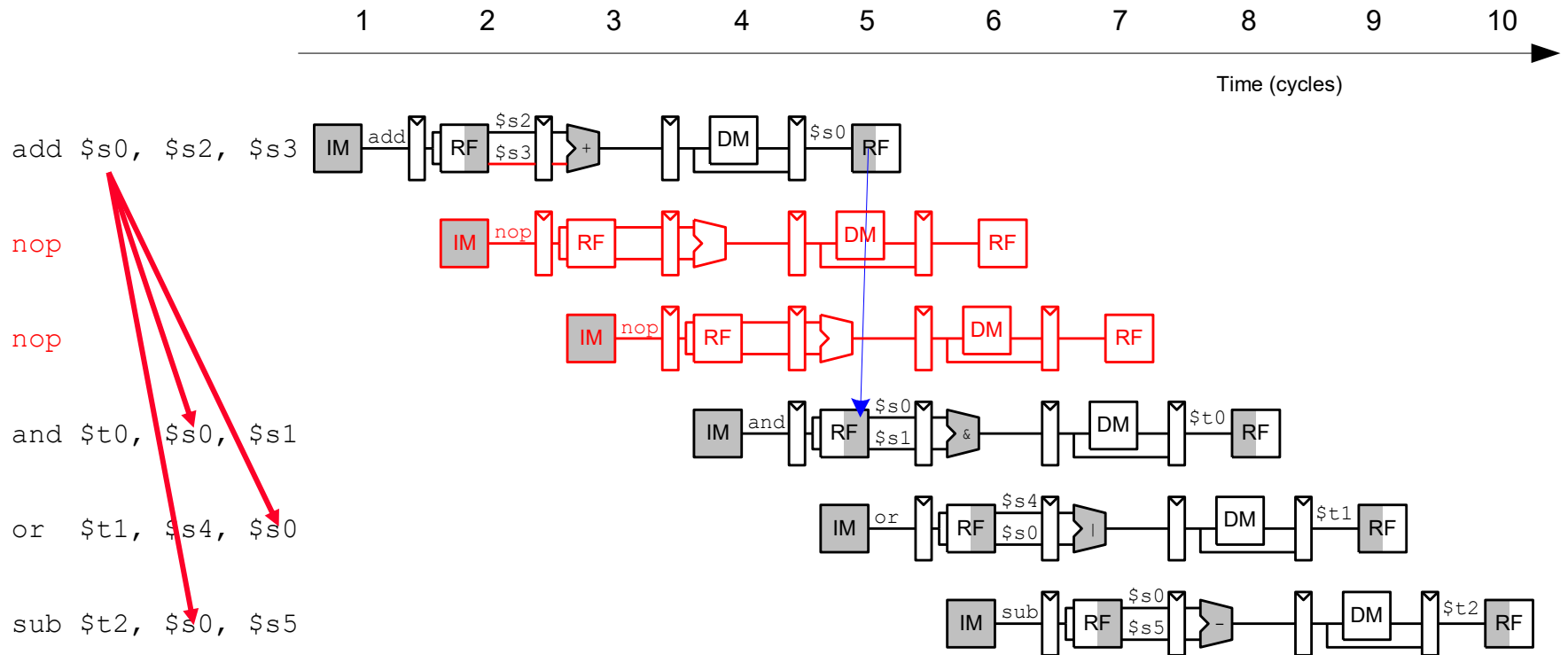
- One instruction writes a register (\$s0) and next instructions read this register => read after write (RAW) dependence.

**Wrong results happen only if
the pipeline handles
data dependences incorrectly!**

- subsequent instructions read the correct value of \$s0



Compile-Time Detection and Elimination

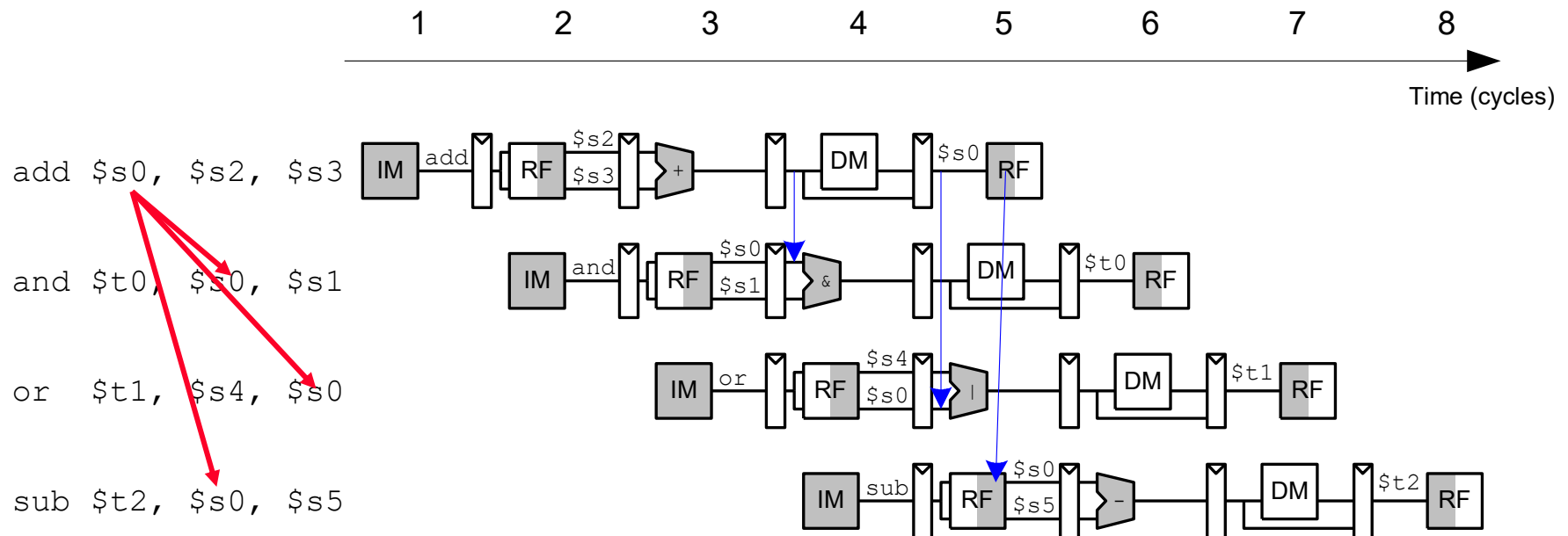


- Insert enough independent instructions for the required result to be ready by the time it is needed by a dependent one
 - Reorder/reschedule/insert instructions at the compiler level

Data Forwarding

- Also called Data Bypassing
 - Forward the result value to the dependent instruction as soon as the value is available
 - We have already seen the basic idea before
 - Remember dataflow?
 - Data value is supplied to dependent instruction as soon as it is available
 - Instruction executes when all its operands are available
 - Data forwarding brings a pipeline closer to dataflow execution principles
-

Data Forwarding: Locations in Datapath



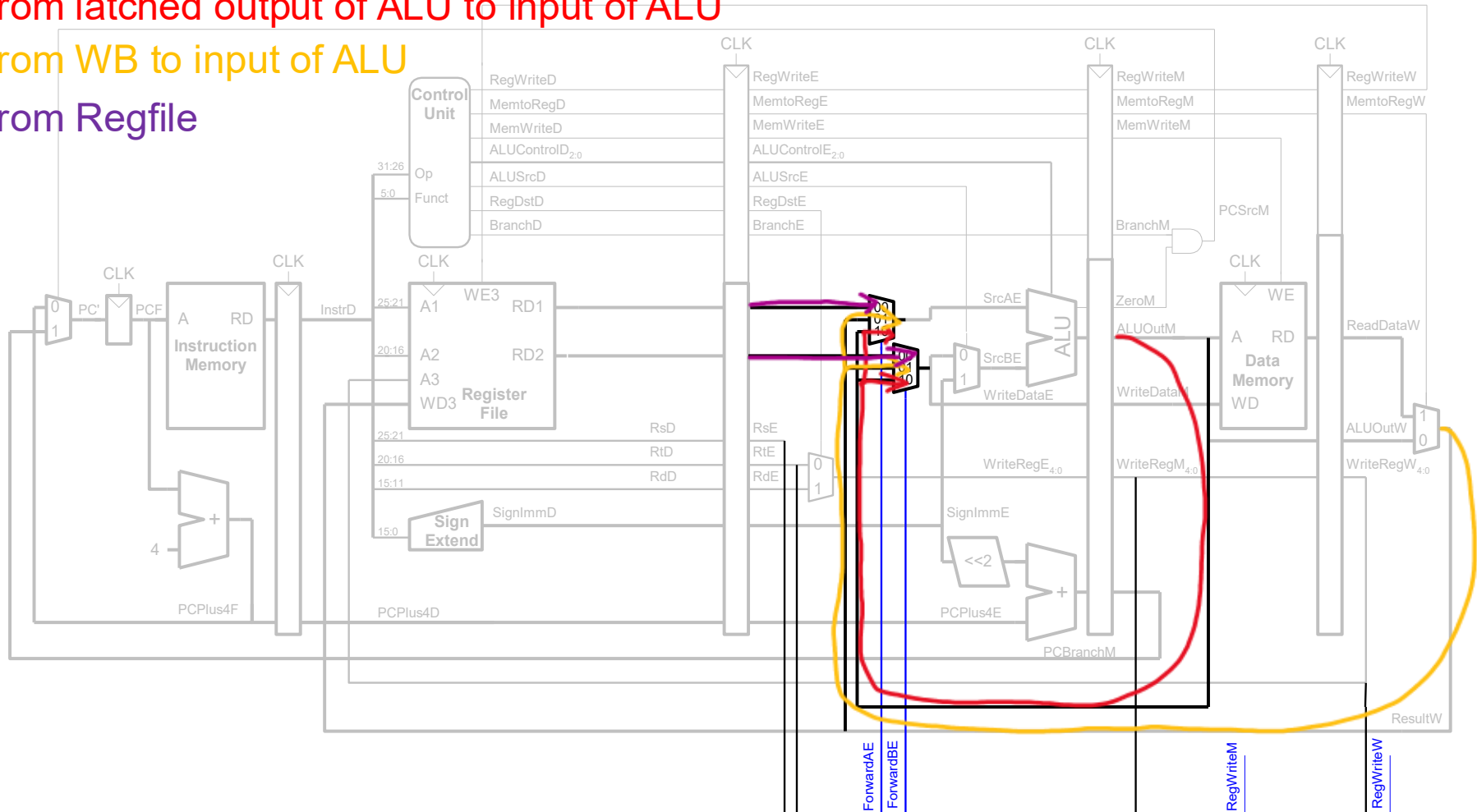
From latched output of ALU to input of ALU
From WB to input of ALU
From WB to RF (internal in Register File)

Data Forwarding: Datapath & Control

From latched output of ALU to input of ALU

From WB to input of ALU

From Regfile



Dependence Detection Logic

Data Forwarding: Implementation

- Forward to Execute stage from either:
 - Memory stage or
 - Writeback stage
 - When should we forward from either Memory or Writeback stage?
 - If that stage will write to a destination register and the destination register matches the source register
 - If both the Memory & Writeback stages contain matching destination registers, Memory stage has priority to forward its data, because it contains the *more recently executed* instruction
-

Data Forwarding (in Pseudocode)

- Forward to Execute stage from either:

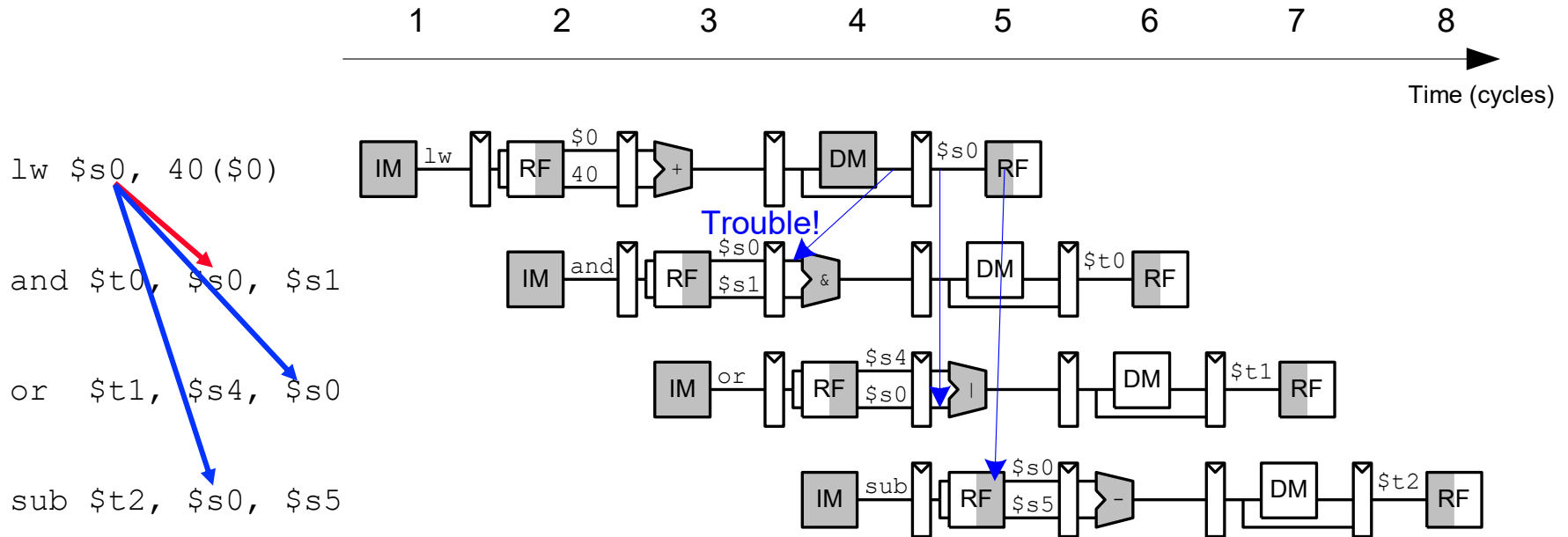
- Memory stage or
- Writeback stage

- Forwarding logic for *ForwardAE* (*pseudo code*):

```
if ((rsE != 0) AND (rsE == WriteRegM) AND RegWriteM) then
    ForwardAE = 10  # forward from Memory stage
else if ((rsE != 0) AND (rsE == WriteRegW) AND RegWriteW) then
    ForwardAE = 01  # forward from Writeback stage
else
    ForwardAE = 00  # no forwarding
```

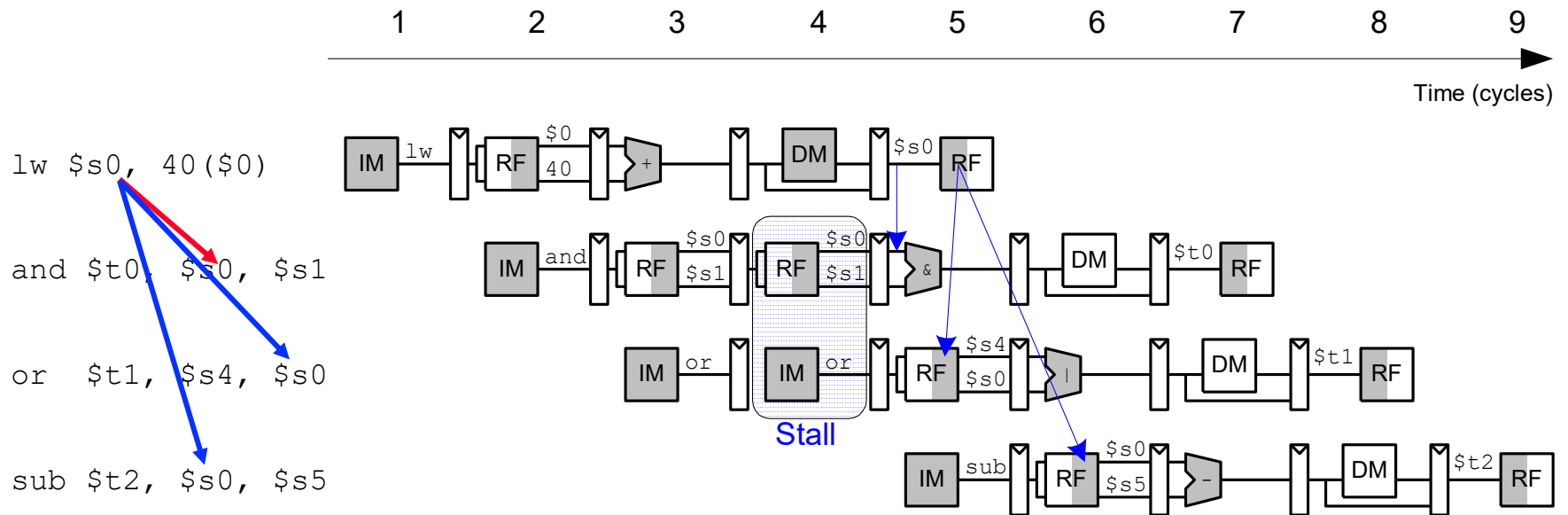
- Forwarding logic for *ForwardBE* same, but replace *rsE* with *rtE*
-

Data Forwarding Is **Not** Always Possible

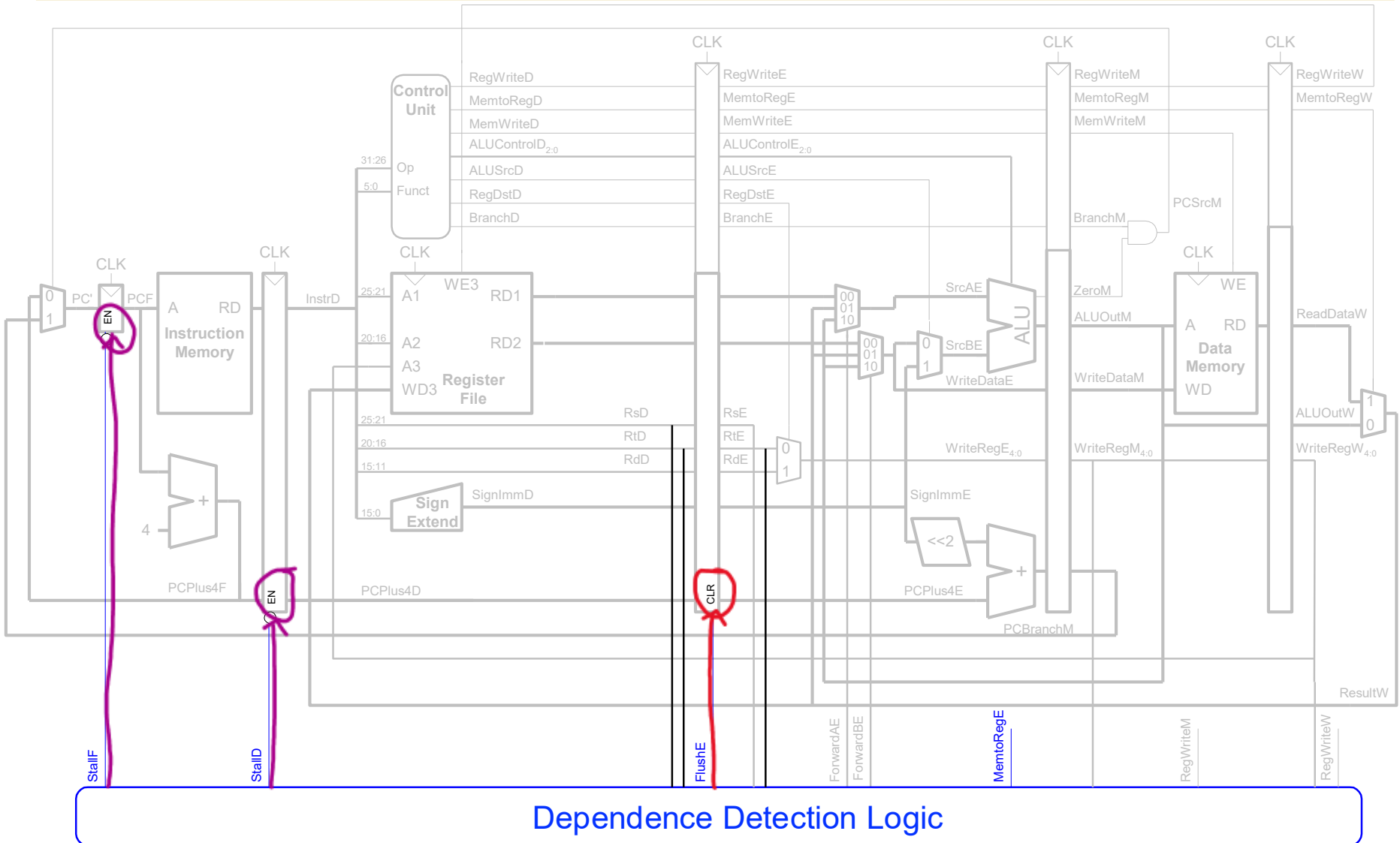


- Forwarding is usually sufficient to resolve RAW data dependences
- Unfortunately, there are cases when forwarding is **not possible**
 - due to pipeline design and instruction latencies
 - The 1w instruction **does not finish** reading data until the end of Memory stage
 - its result **cannot be forwarded** to the Execute stage of the next instruction unless we want a long critical path → **breaks critical path design principle**

Stalling Necessary for MEM-EX Dependence



Stalling and Dependence Detection Hardware



Hardware Needed for Stalling

- Stalls are supported by adding
 - enable inputs (EN) to the Fetch and Decode pipeline registers
 - synchronous reset/clear (CLR) input to the Execute pipeline register
 - or an INV bit associated with each pipeline register, indicating that contents are INValid

 - When a lw stall occurs
 - Keep the values in the Decode and Fetch stage pipeline registers
 - StallD and StallF are asserted
 - Clear the contents of the Execute stage register, introducing a bubble
 - FlushE is also asserted
-

A Special Case of Data Dependence

- Control dependence
 - Data dependence on the Instruction Pointer / Program Counter

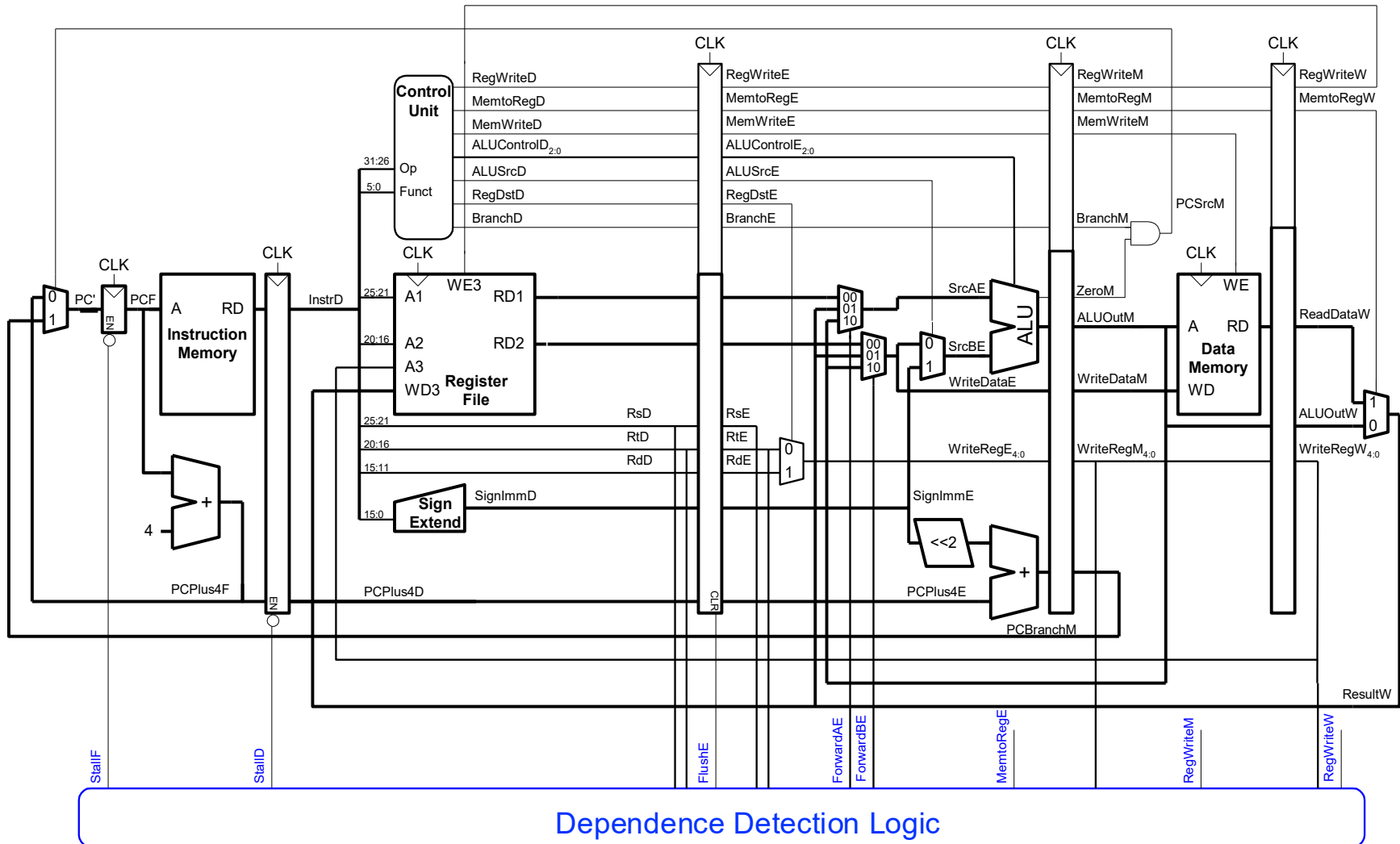
Control Dependence

- Question: **What should the fetch PC be in the next cycle?**
- Answer: The address of the next instruction
 - All instructions are control dependent on previous ones. Why?
- If the fetched instruction is a non-control-flow instruction:
 - Next Fetch PC is the address of the next-sequential instruction
 - Easy to determine if we know the size of the fetched instruction
- **If the instruction that is fetched is a control-flow instruction:**
 - How do we determine the next Fetch PC?
- In fact, how do we know whether or not the fetched instruction is a control-flow instruction?

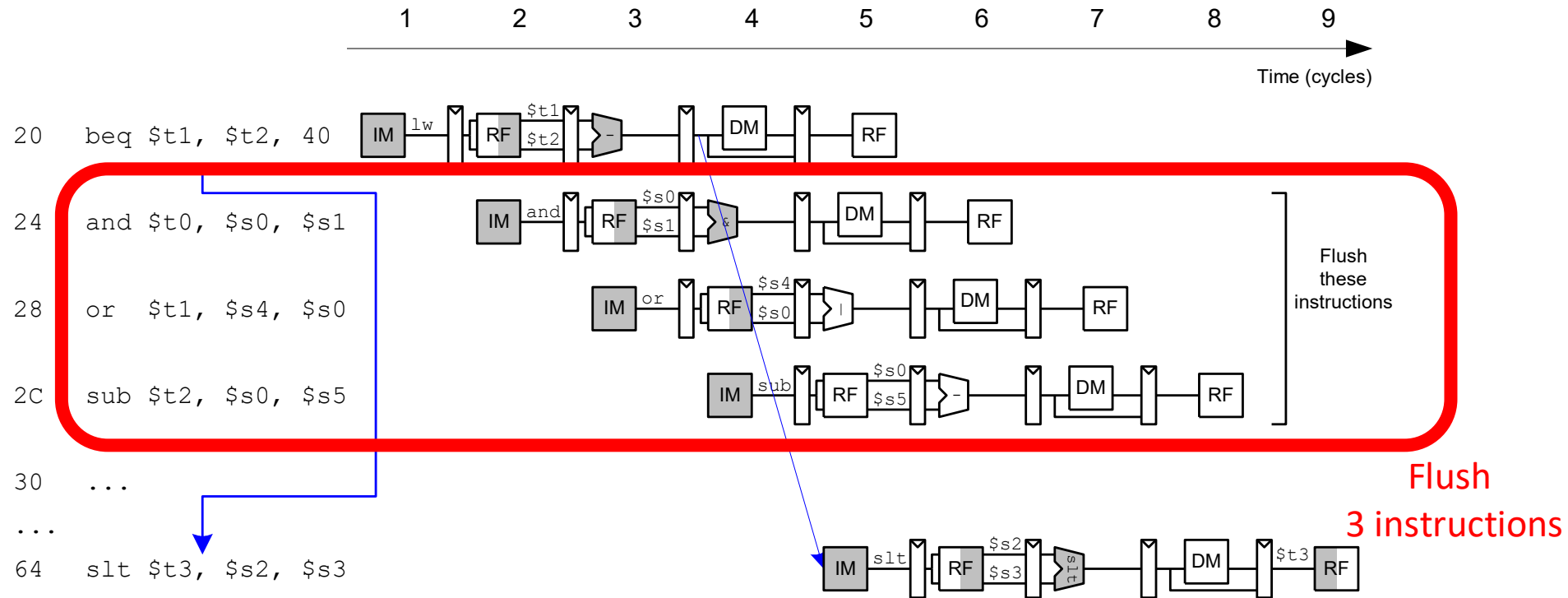
Branch Prediction

- Special case of data dependence: **dependence on PC**
- **beq:**
 - Conditional branch is not resolved until the fourth stage of the pipeline
 - **Instructions after the branch are fetched before branch is resolved**
 - Simple “**branch prediction**” example:
 - Always predict that the next sequential instruction is fetched
 - Called “**Always not taken**” prediction
 - Flush (invalidate) “not-taken path” instructions if the branch is taken
- **Branch misprediction penalty**
 - **number of instructions flushed when branch is incorrectly predicted**
 - Penalty can be reduced by resolving the branch earlier
 - Called “**Early branch resolution**”

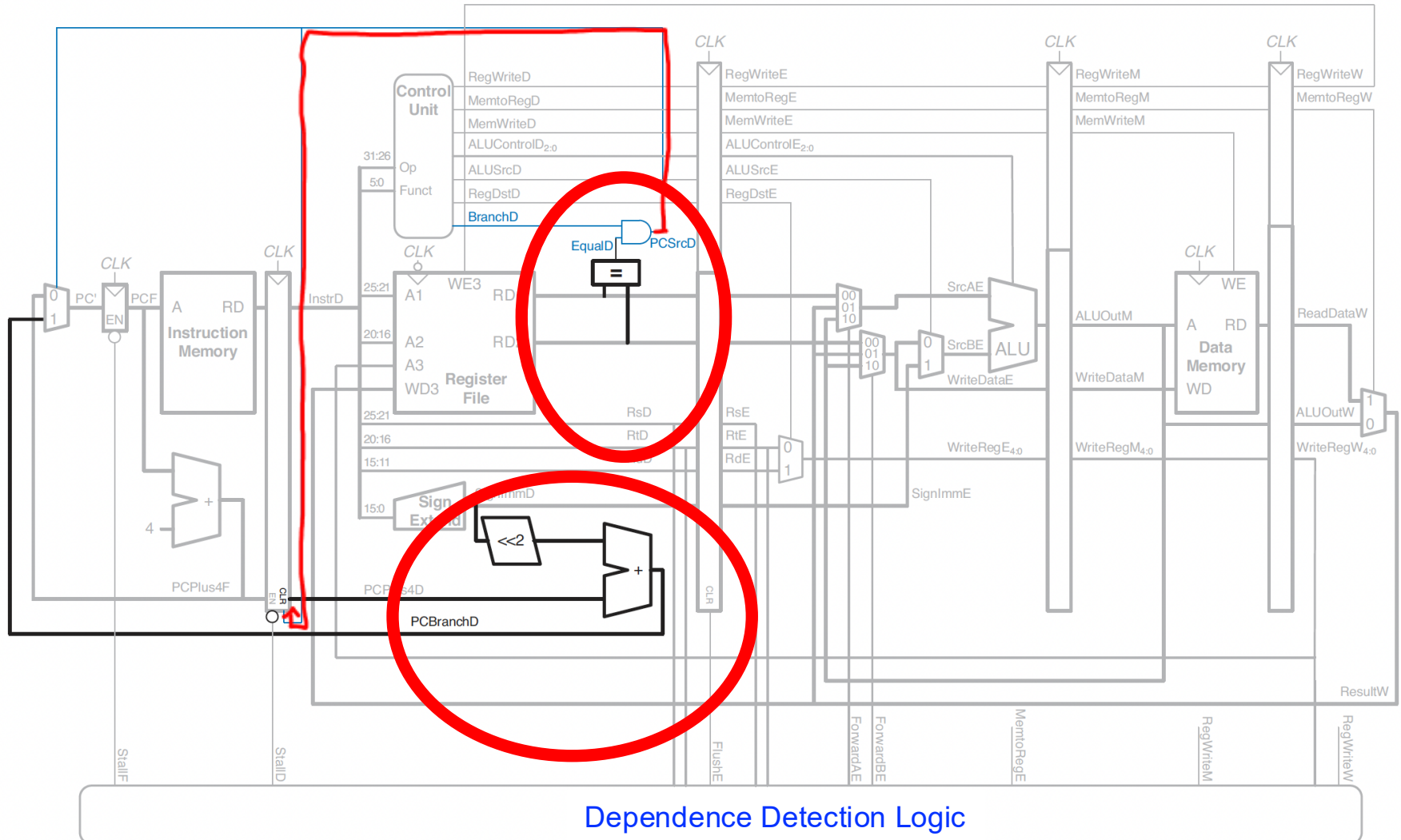
Our Pipeline So Far



Control Dependence: Flush on Misprediction

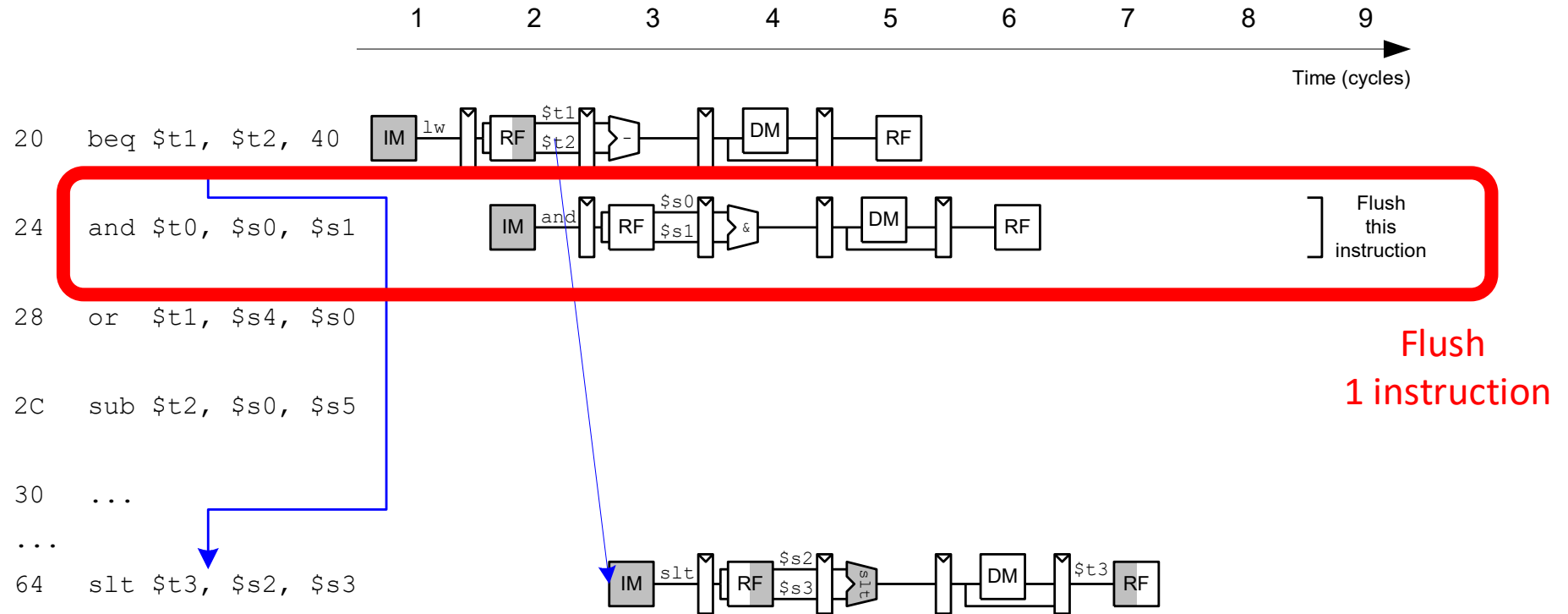


Pipeline with Early Branch Resolution



Need to calculate branch target and condition in the Decode Stage

Early Branch Resolution



Early Branch Resolution: Good Idea?

■ Advantages

- Reduced branch misprediction penalty
 - Reduced CPI (cycles per instruction)

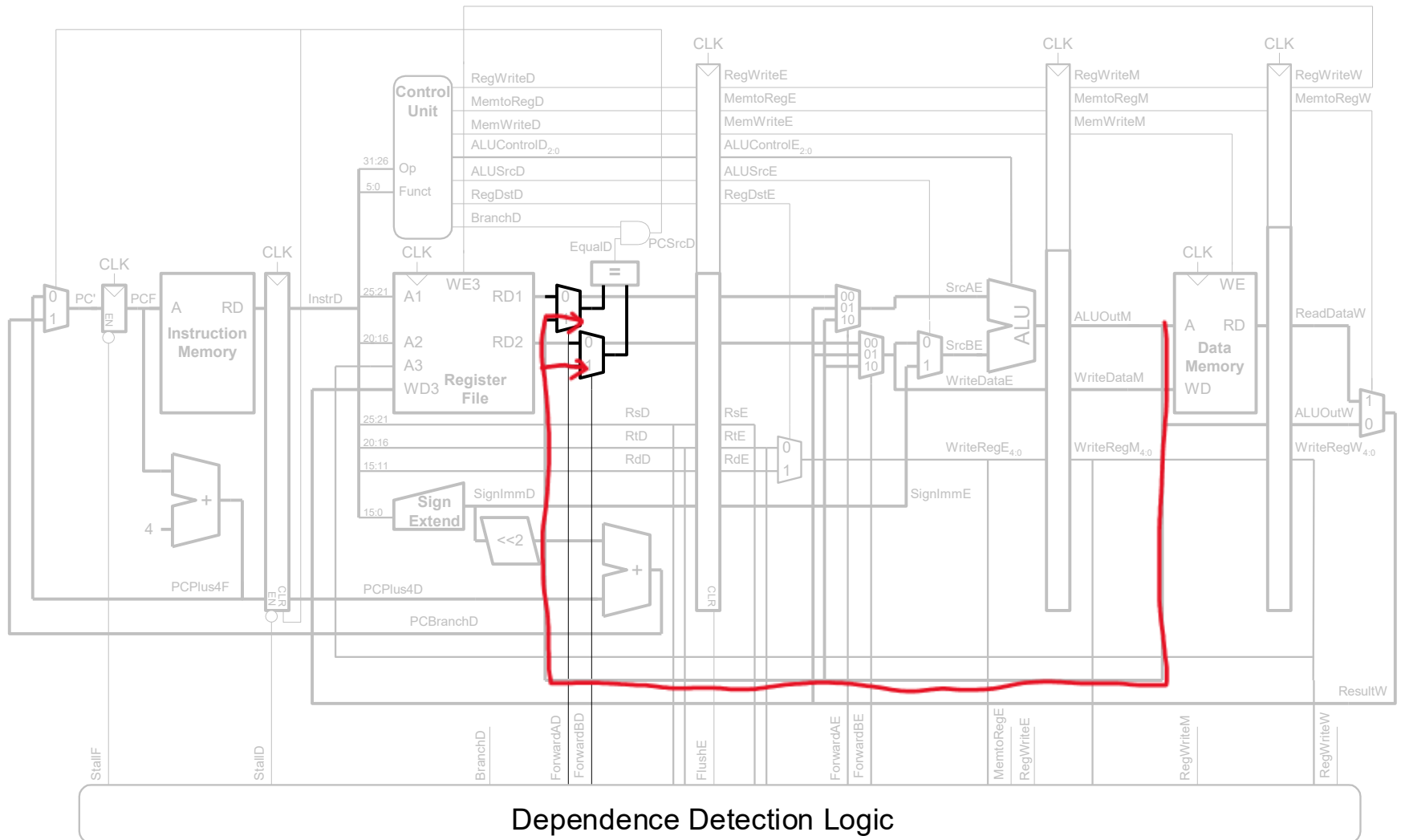
■ Disadvantages

- Potential increase in clock cycle time?
 - Higher clock period and lower frequency?
- Additional hardware cost
 - Specialized and likely not used by other instructions

Recall: Performance Analysis Basics

- Execution time of a single instruction
 - **{CPI} x {clock cycle time}**
 - CPI: Number of cycles it takes to execute an instruction
- Execution time of an entire program
 - Sum over all instructions [**{CPI} x {clock cycle time}**]
 - **{# of instructions} x {Average CPI} x {clock cycle time}**

Data Forwarding for Early Branch Resolution



Data forwarding for early branch resolution adds even more complexity

Forwarding and Stalling Hardware Control

```
// Forwarding logic:
assign ForwardAD = (rsD != 0) & (rsD == WriteRegM) & RegWriteM;
assign ForwardBD = (rtD != 0) & (rtD == WriteRegM) & RegWriteM;

//Stalling logic:
assign lwstall = ((rsD == rtE) | (rtD == rtE)) & MemtoRegE;

assign branchstall = (BranchD & RegWriteE &
                      (WriteRegE == rsD | WriteRegE == rtD))
                    |
                      (BranchD & MemtoRegM &
                      (WriteRegM == rsD | WriteRegM == rtD));

// Stall signals;
assign StallF = lwstall | branchstall;
assign StallD = lwstall | branchstall;
assign FlushE = lwstall | branchstall;
```

Final Pipelined MIPS Processor (H&H)

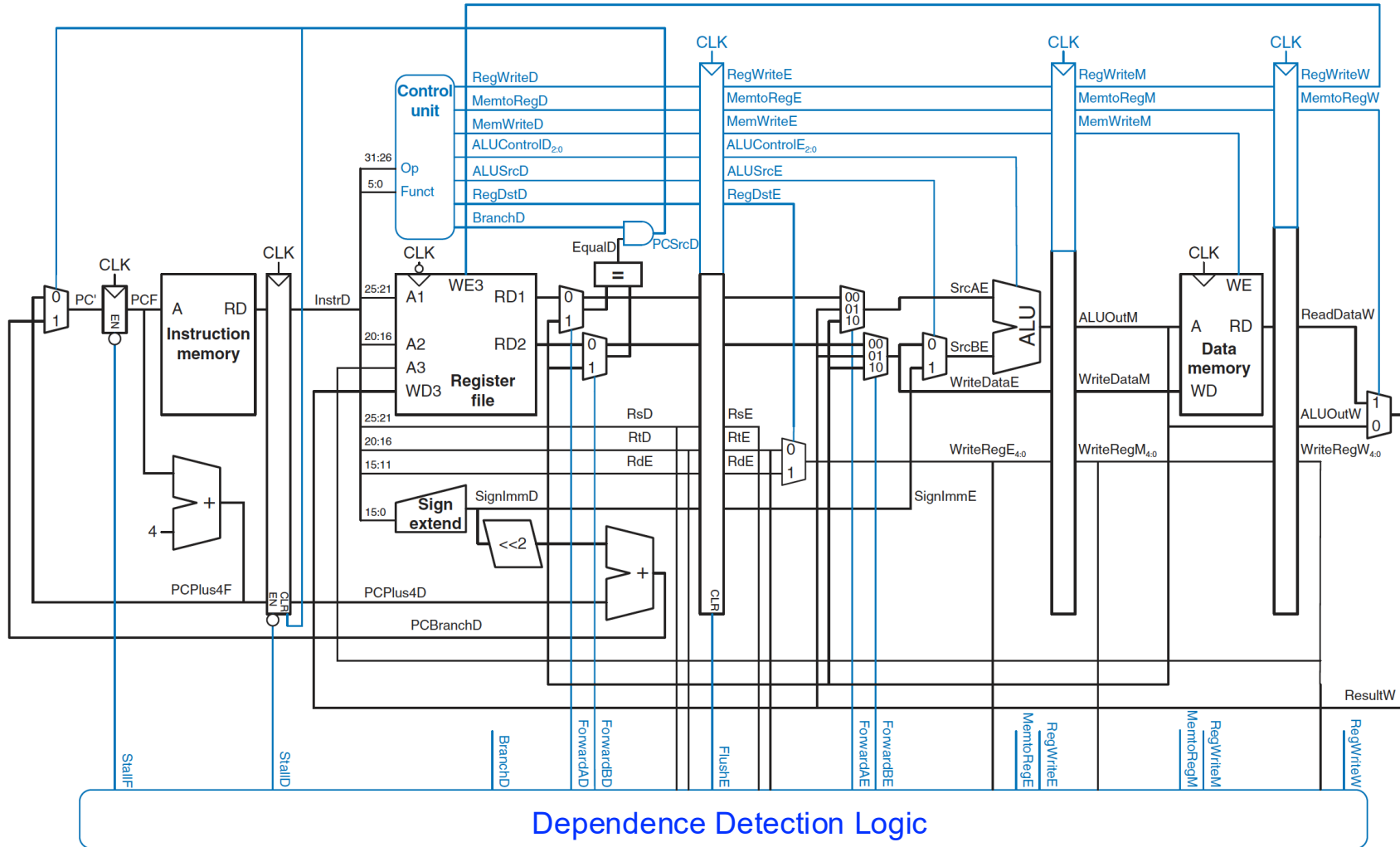


Figure 7.58 Pipelined processor with full hazard handling

Includes always-not-taken br prediction, early branch resolution, forwarding, stall logic

Doing Better: Smarter Branch Prediction

- **Guess whether or not branch will be taken**
 - Backward branches are usually taken (loops iterate many times)
 - History of whether branch was previously taken can improve the guess
- **Accurate branch prediction reduces the fraction of branches requiring a flush**
- **Many sophisticated techniques are employed in modern processors**
 - Including simple machine learning methods (perceptrons)
 - We will see them in Branch Prediction lectures

More on Branch Prediction (I)

Importance of The Branch Problem

- Assume $N = 20$ (20 pipe stages), $W = 5$ (5 wide fetch)
- Assume: 1 out of 5 instructions is a branch
- Assume: Each 5 instruction-block ends with a branch
- How long does it take to fetch 500 instructions?
 - 100% accuracy
 - 100 cycles (all instructions fetched on the correct path)
 - No wasted work; $IPC = 500/100$
 - 99% accuracy
 - 100 (correct path) + $20 * 1$ (wrong path) = 120 cycles
 - 20% extra instructions fetched; $IPC = 500/120$
 - 90% accuracy
 - 100 (correct path) + $20 * 10$ (wrong path) = 300 cycles
 - 200% extra instructions fetched; $IPC = 500/300$
 - 60% accuracy
 - 100 (correct path) + $20 * 40$ (wrong path) = 900 cycles
 - 800% extra instructions fetched; $IPC = 500/900$



11

19:43 / 1:05:13

ETH ZÜRICH

Digital Design & Computer Architecture - Lecture 16b: Branch Prediction I (ETH Zürich, Spring 2020)

2,612 views • Apr 20, 2020

Onur Mutlu Lectures
15.5K subscribers

Digital Design and Computer Architecture, ETH Zürich, Spring 2020
(<https://safari.ethz.ch/digitaltechnik...>)

Lecture 16b: Branch Prediction I
Lecturer: Professor Onur Mutlu (<https://people.inf.ethz.ch/omutlu/>)
Date: April 23, 2020

47 0 SHARE SAVE ...

ANALYTICS EDIT VIDEO

More on Branch Prediction (II)

Intel Pentium M Predictors: Loop and Jump

The advanced branch prediction in the Pentium M processor is based on the Intel Pentium® 4 processor's [6] branch predictor. On top of that, two additional predictors to capture special program flows, were added: a Loop Detector and an Indirect Branch Predictor.

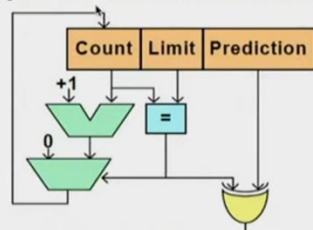


Figure 2: The Loop Detector logic

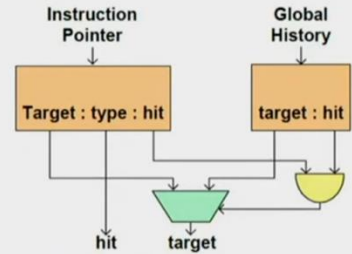


Figure 3: The Indirect Branch Predictor logic

Gochman et al.,
"The Intel Pentium M Processor: Microarchitecture and Performance,"
Intel Technology Journal, May 2003.

39

ETH ZÜRICH

Digital Design & Computer Architecture - Lecture 17: Branch Prediction II (ETH Zürich, Spring 2020)

2,846 views • Apr 20, 2020



Onur Mutlu Lectures
15.5K subscribers

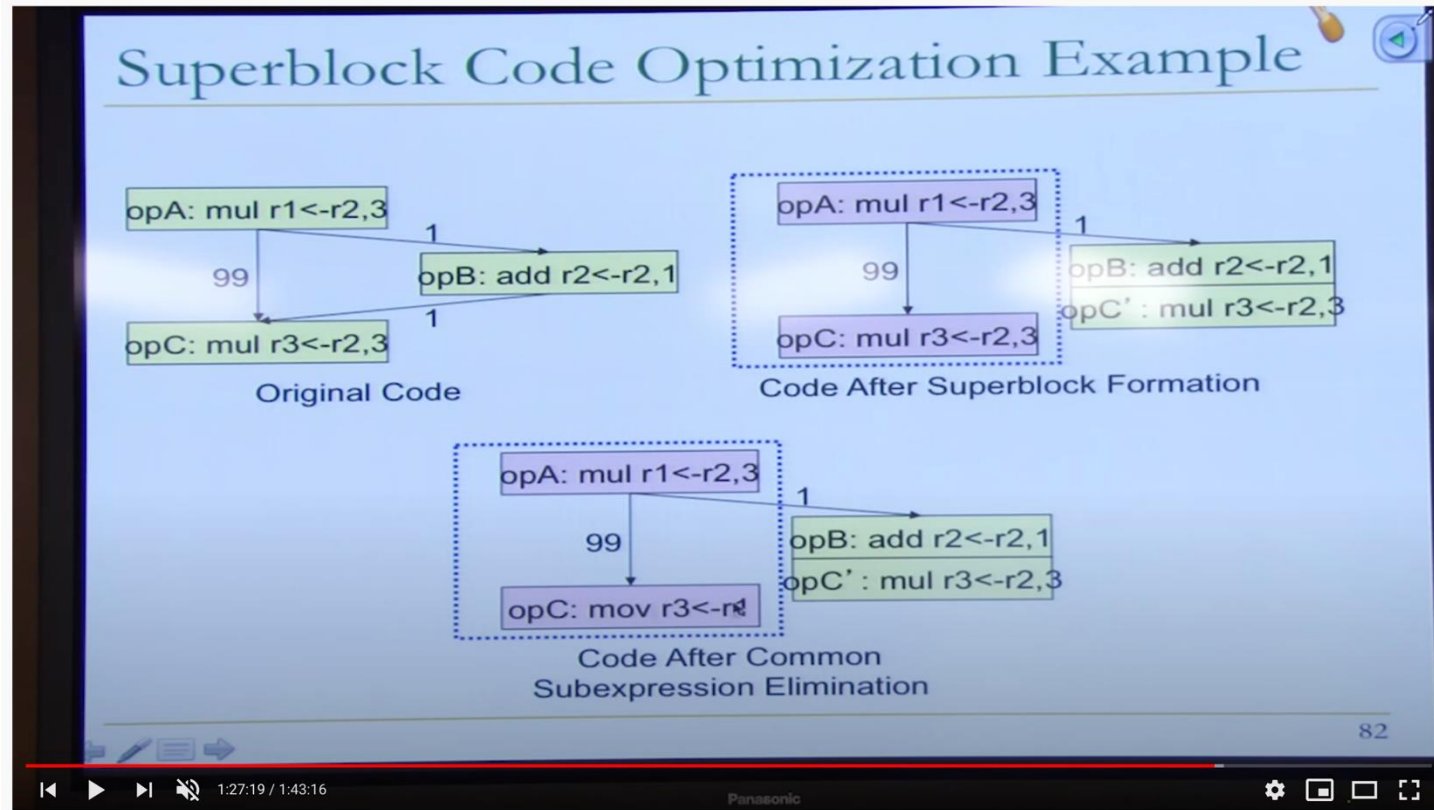
Digital Design and Computer Architecture, ETH Zürich, Spring 2020
(<https://safari.ethz.ch/digitaltechnik...>)

Lecture 17: Branch Prediction II
Lecturer: Professor Onur Mutlu (<https://people.inf.ethz.ch/omutlu/>)
Date: April 24, 2020

42 0 SHARE SAVE ...

ANALYTICS EDIT VIDEO

More on Branch Prediction (III)



18-740 Computer Architecture - Advanced Branch Prediction - Lecture 5

4,696 views • Sep 23, 2015

41 0 SHARE SAVE ...



Carnegie Mellon Computer Architecture
23K subscribers

Lecture 5: Advanced Branch Prediction

Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)

Date: September 16, 2014.

Lecture 5 slides (pdf): <http://www.ece.cmu.edu/~ece740/f15/li...>

Lecture 5 slides (ppt): <http://www.ece.cmu.edu/~ece740/f15/li...>

<https://www.youtube.com/watch?v=yDjsr-jT0tk&list=PL5PHm2jkkXmgVhh8CHAU9N76TShJqfYDt&index=4>

Lectures on Branch Prediction

- Digital Design & Computer Architecture, Spring 2020, Lecture 16b
 - Branch Prediction I (ETH Zurich, Spring 2020)
 - https://www.youtube.com/watch?v=h6l9yYSyZHM&list=PL5Q2soXY2Zi_FRrloMa2fUYWPGiZUBQo2&index=22
- Digital Design & Computer Architecture, Spring 2020, Lecture 17
 - Branch Prediction II (ETH Zurich, Spring 2020)
 - https://www.youtube.com/watch?v=z77VpggShvg&list=PL5Q2soXY2Zi_FRrloMa2fUYWPGiZUBQo2&index=23
- Computer Architecture, Spring 2015, Lecture 5
 - Advanced Branch Prediction (CMU, Spring 2015)
 - <https://www.youtube.com/watch?v=yDjsr-jTOtk&list=PL5PHm2jkkXmgVhh8CHAU9N76TShJqfYDt&index=4>

Pipelined Performance Example

Pipelined Performance Example

- **An important program consists of:**

- 25% loads
- 10% stores
- 11% branches
- 2% jumps
- 52% R-type

- **Assume:**

- 40% of loads used by next instruction
- 25% of branches mispredicted; flushes the next instruction on misprediction
- All jumps flush the next instruction fetched

- **What is the average CPI?**

Pipelined Performance Example: CPI

- Load/Branch CPI = 1 when no stall/flush, 2 when stall/flush.

Thus:

- $CPI_{lw} = 1(0.6) + 2(0.4) = 1.4$

Average CPI for load

- $CPI_{beq} = 1(0.75) + 2(0.25) = 1.25$

Average CPI for branch

- And

- *Average CPI* =

Pipelined Performance Example: CPI

- Load/Branch CPI = 1 when no stall/flush, 2 when stall/flush.

Thus:

- $CPI_{lw} = 1(0.6) + 2(0.4) = 1.4$

Average CPI for load

- $CPI_{beq} = 1(0.75) + 2(0.25) = 1.25$

Average CPI for branch

- And

- *Average CPI* = $(0.25)(1.4) +$
 $(0.1)(1) +$
 $(0.11)(1.25) +$
 $(0.02)(2) +$
 $(0.52)(1)$

load
store
beq
jump
r-type

= **1.15**

Pipelined Performance Example: Cycle Time

- There are 5 stages, and 5 different timing paths:

$$T_c = \max \{$$

$$t_{pcq} + t_{mem} + t_{setup}$$

$$2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup})$$

$$t_{pcq} + t_{mux} + t_{mux} + t_{ALU} + t_{setup}$$

$$t_{pcq} + t_{memwrite} + t_{setup}$$

$$2(t_{pcq} + t_{mux} + t_{RFwrite})$$

}

fetch

decode

execute

memory

writeback

- The clock cycle *depends* on the *slowest stage*
- Decode and Writeback use register file and have only half a clock cycle to complete → that is why there is a 2 in front of them

Final Pipelined MIPS Processor (H&H)

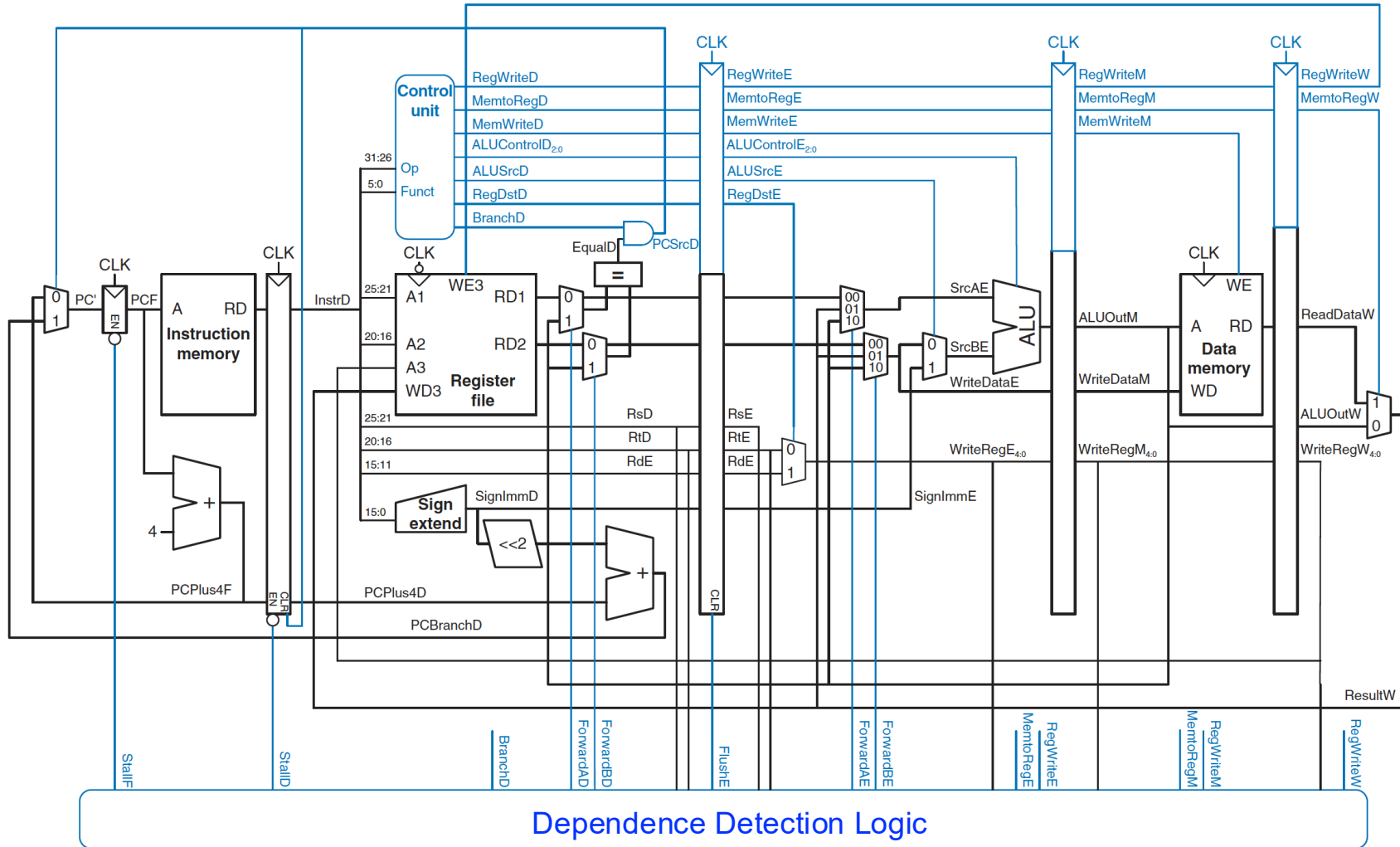


Figure 7.58 Pipelined processor with full hazard handling

Includes always-not-taken br prediction, early branch resolution, forwarding, stall logic

Pipelined Performance Example: Cycle Time

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20
Equality comparator	t_{eq}	40
AND gate	t_{AND}	15
Memory write	$T_{memwrite}$	220
Register file write	$t_{RFwrite}$	100

$$\begin{aligned}T_c &= 2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup}) \\&= 2[150 + 25 + 40 + 15 + 25 + 20] \text{ ps} \\&= 550 \text{ ps}\end{aligned}$$

Pipelined Performance Example: Exec Time

- For a program with 100 billion instructions executing on a pipelined MIPS processor:
 - $CPI = 1.15$
 - $T_c = 550 \text{ ps}$
- Execution Time $= (\# \text{ instructions}) \times CPI \times T_c$
 $= (100 \times 10^9)(1.15)(550 \times 10^{-12})$
 $= 63 \text{ seconds}$

Performance Summary for 3 MIPS microarch.

Processor	Execution Time (seconds)	Normalized Execution Time (single-cycle is baseline)
Single-cycle	95	1
Multicycle	133	0.71
Pipelined	63	1.51

- **Pipelined implementation is the fastest of 3 implementations**
- **Even though we have a 5-stage pipeline, speedup is not 5X over the single-cycle or the multi-cycle system!**

Recall: How to Handle Data Dependences

- Anti and output dependences are easier to handle
 - write to the destination only in last stage and in program order
- Flow dependences are more interesting
- Six fundamental ways of handling flow dependences
 - Detect and wait until value is available in register file
 - Detect and forward/bypass data to dependent instruction
 - Detect and eliminate the dependence at the software level
 - No need for the hardware to detect dependence
 - Detect and move it out of the way for independent instructions
 - Predict the needed value(s), execute “speculatively”, and verify
 - Do something else (fine-grained multithreading)
 - No need to detect

Recall: How to Handle Data Dependences

- Anti and output dependences are easier to handle
 - write to the destination only in last stage and in program order
- Flow dependences are more interesting
- Six fundamental ways of handling flow dependences
 - Detect and wait until value is available in register file
 - Detect and forward/bypass data to dependent instruction
 - Detect and eliminate the dependence at the software level
 - No need for the hardware to detect dependence
 - Detect and move it out of the way for independent instructions
 - Predict the needed value(s), execute “speculatively”, and verify
 - Do something else (fine-grained multithreading)
 - No need to detect

Question to Ponder: Hardware vs. Software

- What is the role of the hardware vs. the software in data dependence handling?
 - Software based interlocking
 - Hardware based interlocking
 - Who inserts/manages the pipeline bubbles?
 - Who finds the independent instructions to fill “empty” pipeline slots?
 - What are the advantages/disadvantages of each?
 - Think of the performance equation as well

Question to Ponder: Hardware vs. Software

- What is the role of the hardware vs. the software in the order in which instructions are executed in the pipeline?
 - ❑ Software based instruction scheduling → static scheduling
 - ❑ Hardware based instruction scheduling → dynamic scheduling

- How does each impact different metrics?
 - ❑ Performance (and parts of the performance equation)
 - ❑ Complexity
 - ❑ Power consumption
 - ❑ Reliability
 - ❑ Cost
 - ❑ ...

More on Software vs. Hardware

- Software based scheduling of instructions → static scheduling
 - Hardware executes the **instructions in the compiler-dictated order**
 - Contrast this with **dynamic scheduling**: hardware can execute instructions out of the compiler-specified order
 - How does the compiler know the latency of each instruction?
- What information does the compiler not know that makes static scheduling difficult?
 - Answer: Anything that is determined at run time
 - **Variable-length operation latency, memory address, branch direction**
- How can the compiler alleviate this (i.e., estimate the unknown)?
 - Answer: Profiling (done statically or dynamically)

More on Static Instruction Scheduling

Trace Scheduling Idea

(a) (b) (c) (d)

TRACE SCHEDULING LOOP-FREE CODE

43

Lecture 16. Static Instruction Scheduling - Carnegie Mellon - Comp. Arch. 2015 - Onur Mutlu

7,136 views • Feb 26, 2015

46 0 SHARE SAVE ...



Carnegie Mellon Computer Architecture
23K subscribers

Lecture 16: Static Instruction Scheduling
Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)
Date: Feb 23rd, 2015

Lecture 16 slides (pdf): <http://www.ece.cmu.edu/~ece447/s15/li...>

SUBSCRIBED



<https://www.youtube.com/watch?v=isBEVkljgGA&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=18>

Lectures on Static Instruction Scheduling

- Computer Architecture, Spring 2015, Lecture 16

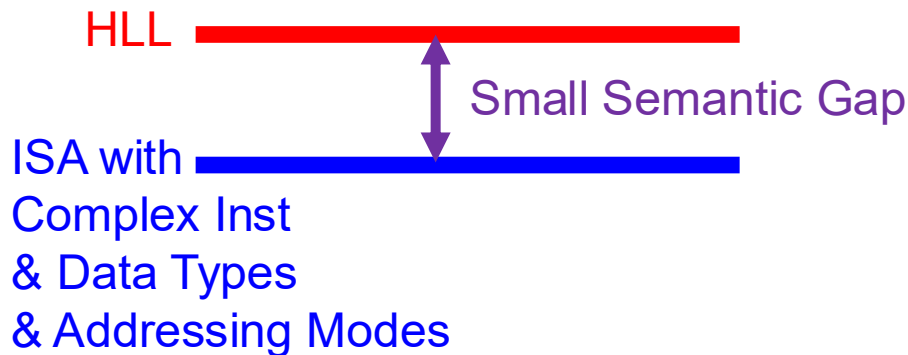
- Static Instruction Scheduling (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=isBEVkIjgGA&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=18>

- Computer Architecture, Spring 2013, Lecture 21

- Static Instruction Scheduling (CMU, Spring 2013)
- <https://www.youtube.com/watch?v=XdDUn2WtkRg&list=PL5PHm2jkkXmidJOd59REog9jDnPDTG6IJ&index=21>

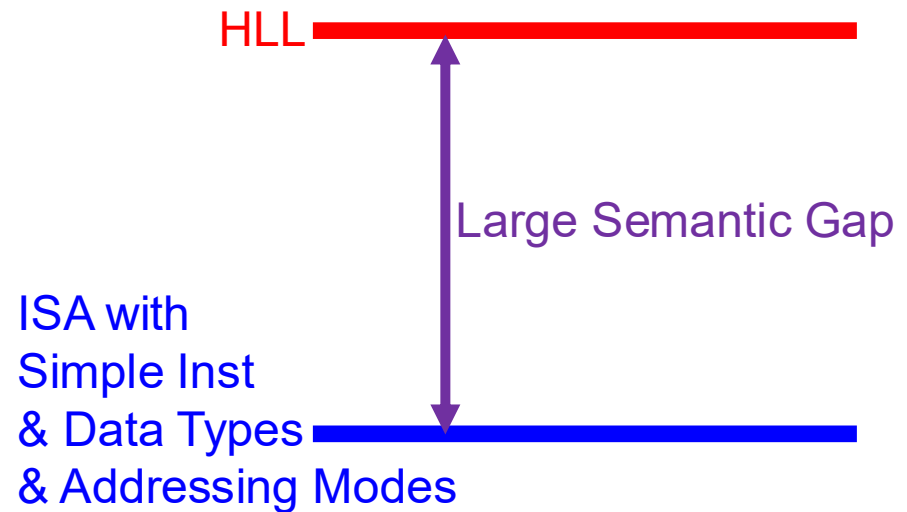
Recall: Semantic Gap

- How close instructions & data types & addressing modes are to high-level language (HLL)



HW Control Signals

Easier mapping of HLL to ISA
Less work for software designer
More work for hardware designer
Optimization burden on HW

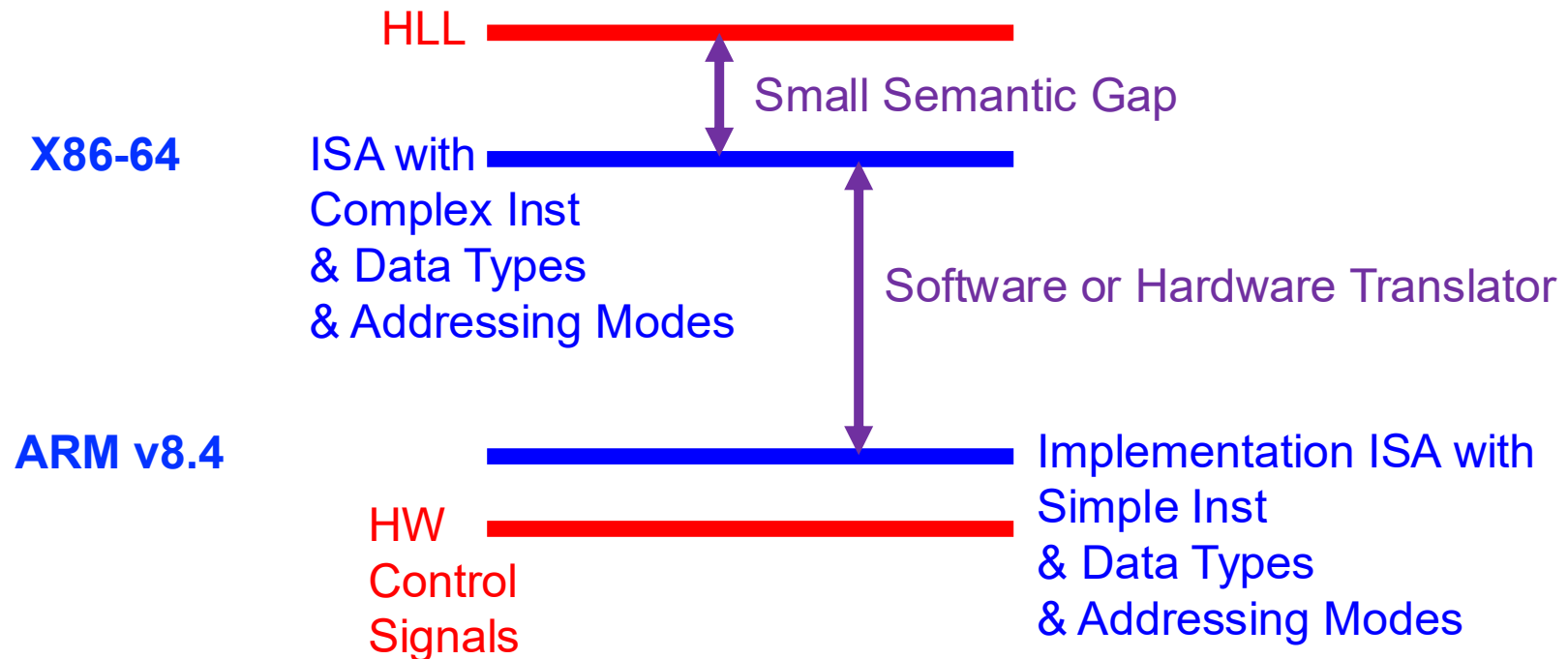


HW Control Signals

Harder mapping of HLL to ISA
More work for software designer
Less work for hardware designer
Optimization burden on SW

Recall: How to Change the Semantic Gap Tradeoffs

- Translate from one ISA into a different “implementation” ISA



An Example: Rosetta 2 Binary Translator

Rosetta 2 [\[edit \]](#)

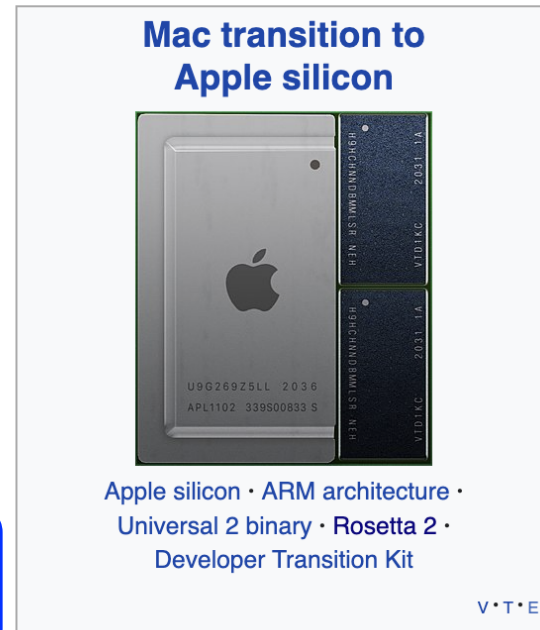
In 2020, Apple announced Rosetta 2 would be bundled with [macOS Big Sur](#), to aid in the [Mac transition to Apple silicon](#). The software permits many applications compiled exclusively for execution on x86-64-based processors to be translated for execution on Apple silicon.^{[2][8]}

In addition to the [just-in-time](#) (JIT) translation support, Rosetta 2 offers [ahead-of-time compilation](#) (AOT), with the x86-64 code fully translated, just once, when an application without a universal binary is installed on an Apple silicon Mac.^[9]

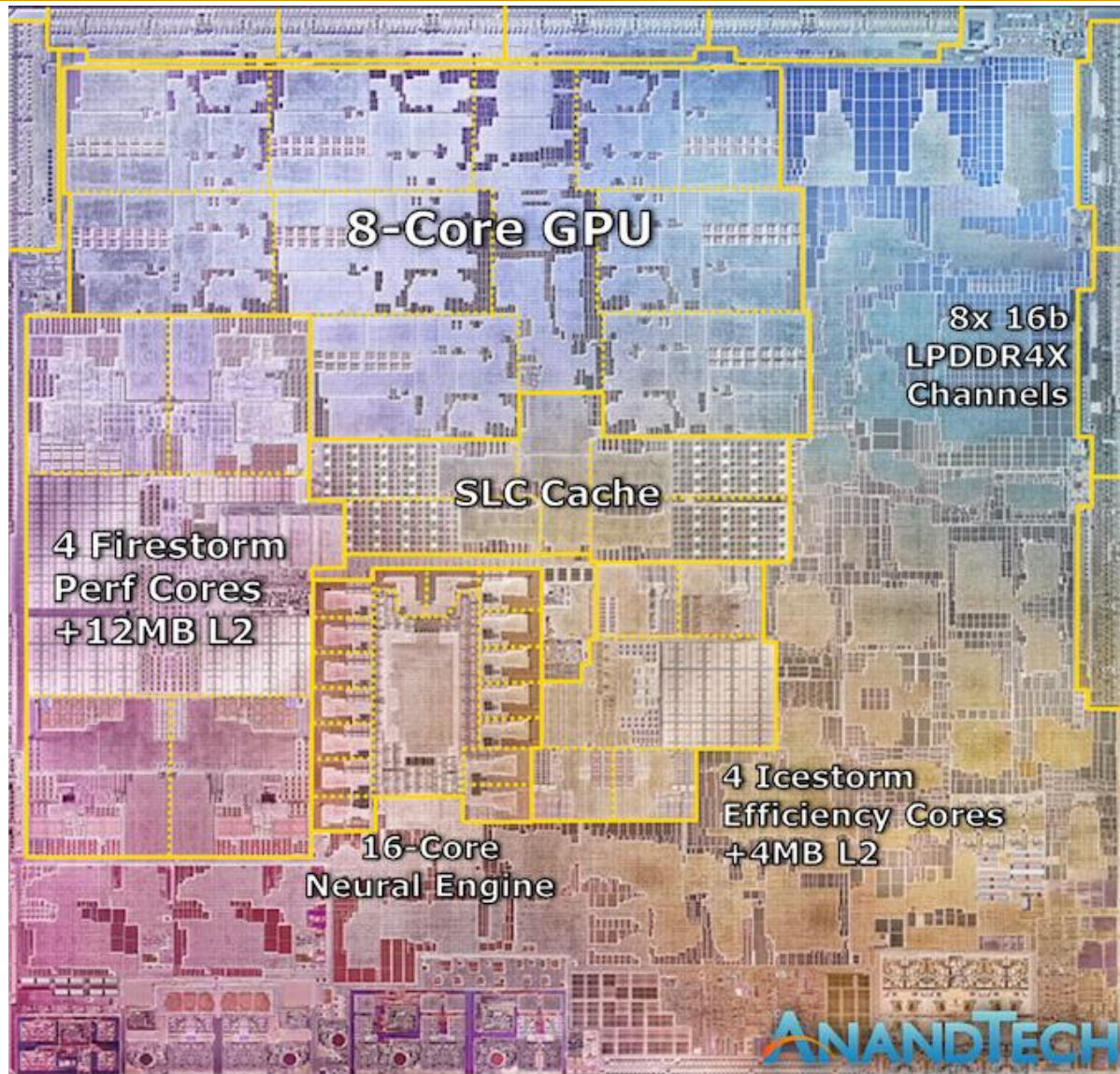
Rosetta 2's performance has been praised greatly.^{[10][11]} In some benchmarks, x86-64-only programs performed better under Rosetta 2 on a Mac with an Apple M1 SOC than natively on a Mac with an Intel x86-64 processor. One of the key reasons why Rosetta 2 provides such high level of translation efficiency is the support of x86-64 [memory ordering](#) in Apple M1 SOC.^[12]

Although Rosetta 2 works for most software, some software doesn't work at all^[13] or is reported to be "sluggish".^[14] A lot of software can be made compatible with the new Macs by the vendor recompiling the software, often a simple task; while for some software (such as software that includes [assembly language](#) code, or that generates [machine code](#)), the changes to make them work aren't simple and cannot be automated.

Similar to the first version, Rosetta 2 does not normally require user intervention. When a user attempts to launch an x86-64-only application for the first time, macOS prompts them to install Rosetta 2 if it is not already available. Subsequent launches of x86-64 programs will execute via translation automatically. An option also exists to force a universal binary to run as x86-64 code through Rosetta 2, even on an ARM-based machine.^[15]



An Example: Rosetta 2 Binary Translator



Apple M1,
2021

Another Example: NVIDIA Denver

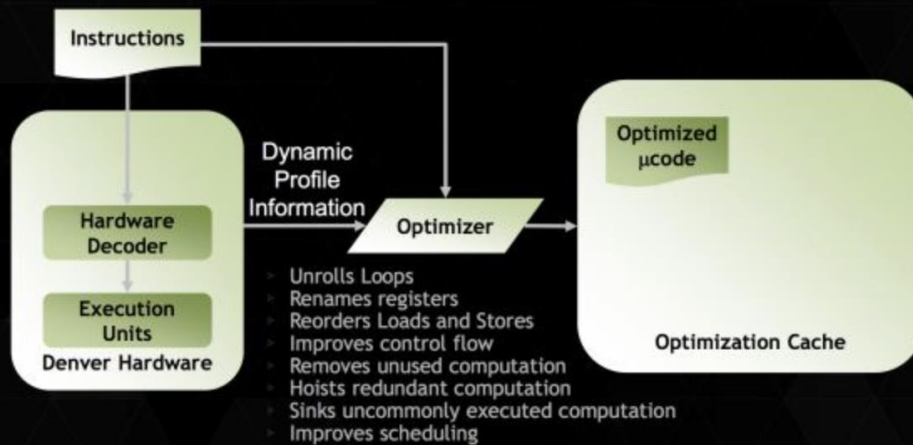
The Secret of Denver: Binary Translation & Code Optimization

As we alluded to earlier, NVIDIA's decision to forgo a traditional out-of-order design for Denver means that much of Denver's potential is contained in its software rather than its hardware. The underlying chip itself, though by no means simple, is at its core a very large in-order processor. So it falls to the software stack to make Denver sing.

Accomplishing this task is NVIDIA's dynamic code optimizer (DCO). The purpose of the DCO is to accomplish two tasks: to translate ARM code to Denver's native format, and to optimize this code to make it run better on Denver. With no out-of-order hardware on Denver, it is the DCO's task to find instruction level parallelism within a thread to fill Denver's many execution units, and to reorder instructions around potential stalls, something that is no simple task.



DYNAMIC CODE OPTIMIZATION OPTIMIZE ONCE, USE MANY TIMES



The DCO system employed in the Denver CPU is codesigned software that extends ideas from prior system-level binary translators.² The primary function is to execute the user's code. The secondary function is to profile execution, create, optimize, and manage regions of tens to thousands of ARM instructions to form equivalent microcode-optimized regions that execute efficiently on the underlying microarchitecture.

More on NVIDIA Denver Code Optimizer

DENVER: NVIDIA'S FIRST 64-BIT ARM PROCESSOR

NVIDIA'S FIRST 64-BIT ARM PROCESSOR, CODE-NAMED DENVER, LEVERAGES A HOST OF NEW TECHNOLOGIES, SUCH AS DYNAMIC CODE OPTIMIZATION, TO ENABLE HIGH-PERFORMANCE MOBILE COMPUTING. IMPLEMENTED IN A 28-NM PROCESS, THE DENVER CPU CAN ATTAIN CLOCK SPEEDS OF UP TO 2.5 GHZ. THIS ARTICLE OUTLINES THE DENVER ARCHITECTURE, DESCRIBES ITS TECHNOLOGICAL INNOVATIONS, AND PROVIDES RELEVANT COMPARISONS AGAINST COMPETING MOBILE PROCESSORS.

Codesigning a hardware processor with a DCO software system creates both additional validation exposure and benefits. The DCO system can be upgraded in the field to address functionality, performance, or security issues.

The Denver hardware decoder provides a mechanism for periodically profiling recently taken branches. This branch history is moved into a shared buffer that can be processed from other cores, thereby minimizing the latency of the interruption. The DCO system will then run a thread that uses this profile to evaluate the dynamic properties of code executing and to assemble a picture of which code regions are hottest across all the processors. On finding sufficiently hot code, the DCO system will begin an optimization process to turn this input ARM code into a microcode execution region. The optimization process uses well-known traditional³ and more speculative compiler techniques to reduce work and increase efficiency of execution on the underlying skewed pipeline. To keep the latency of interruptions to a minimum, the optimizer thread is time-sliced with ARM execution (if any) and runs in a mode that can be quickly interrupted.

Transmeta: x86 to VLIW Translation

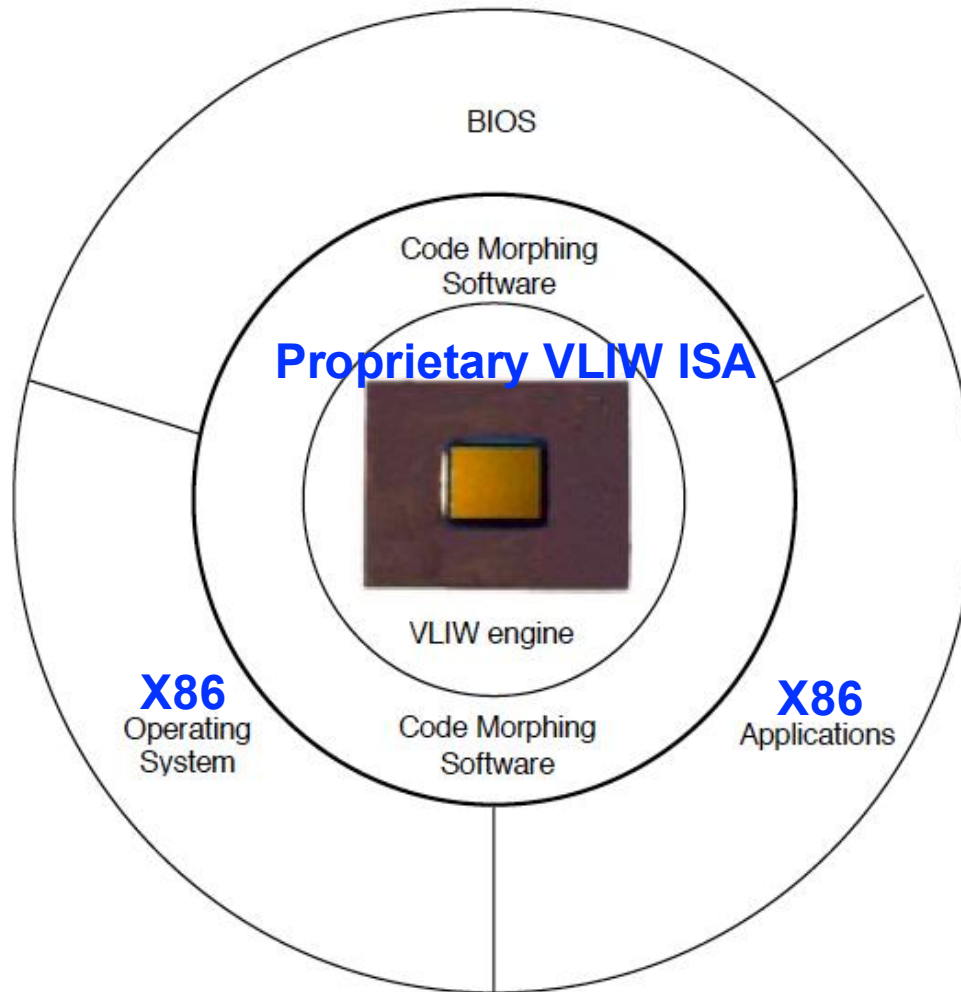


Figure 5. The Code Morphing software mediates between x86 software and the Crusoe processor.

Klaiber, “[The Technology Behind Crusoe Processors](#),” Transmeta White Paper 2000.

More on Static Instruction Scheduling

Trace Scheduling Idea

(a) (b) (c) (d)

TRACE SCHEDULING LOOP-FREE CODE

43

Lecture 16. Static Instruction Scheduling - Carnegie Mellon - Comp. Arch. 2015 - Onur Mutlu

7,136 views • Feb 26, 2015

46 0 SHARE SAVE ...



Carnegie Mellon Computer Architecture
23K subscribers

Lecture 16: Static Instruction Scheduling
Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)
Date: Feb 23rd, 2015

Lecture 16 slides (pdf): <http://www.ece.cmu.edu/~ece447/s15/li...>

<https://www.youtube.com/watch?v=isBEVkljgGA&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=18>

Lectures on Static Instruction Scheduling

■ Computer Architecture, Spring 2015, Lecture 16

- Static Instruction Scheduling (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=isBEVkJjgGA&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=18>

■ Computer Architecture, Spring 2013, Lecture 21

- Static Instruction Scheduling (CMU, Spring 2013)
- <https://www.youtube.com/watch?v=XdDUn2WtkRg&list=PL5PHm2jkkXmidJOd59REog9jDnPDTG6IJ&index=21>

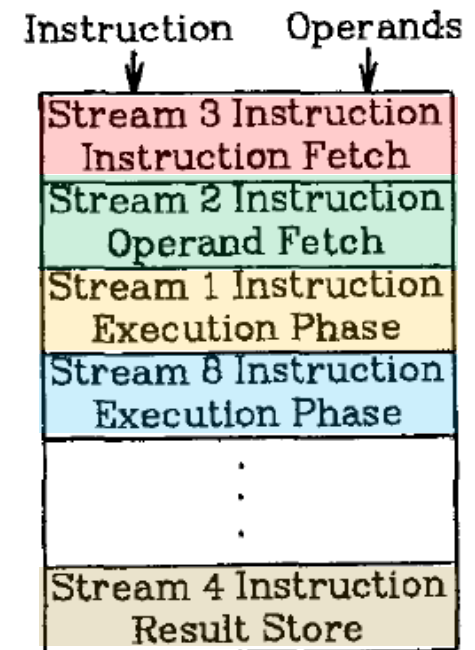
Recall: How to Handle Data Dependences

- Anti and output dependences are easier to handle
 - write to the destination only in last stage and in program order
- Flow dependences are more interesting
- Six fundamental ways of handling flow dependences
 - Detect and wait until value is available in register file
 - Detect and forward/bypass data to dependent instruction
 - Detect and eliminate the dependence at the software level
 - No need for the hardware to detect dependence
 - Detect and move it out of the way for independent instructions
 - Predict the needed value(s), execute “speculatively”, and verify
 - Do something else (fine-grained multithreading)
 - No need to detect

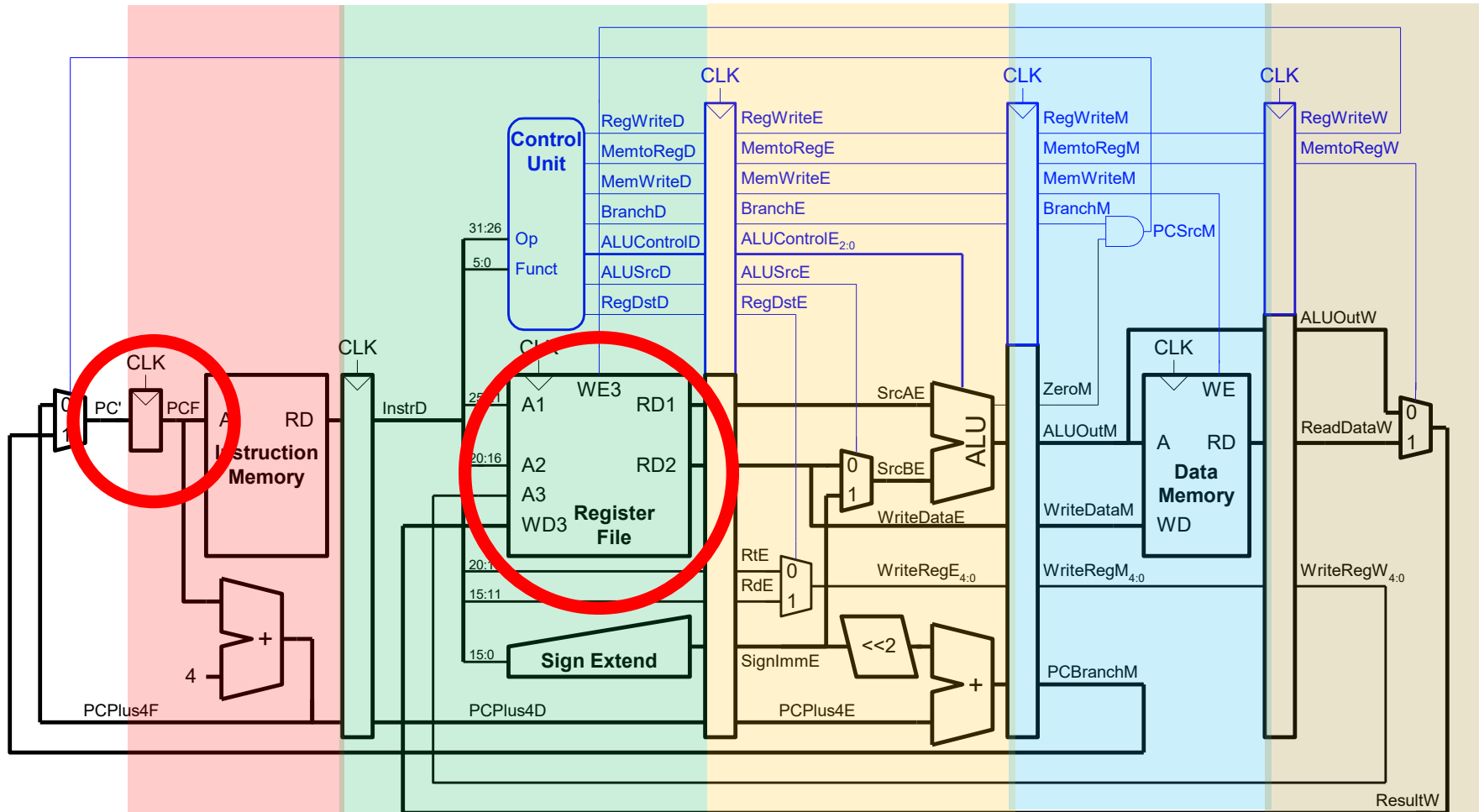
Fine-Grained Multithreading

Fine-Grained Multithreading

- Idea: Fetch from a different thread every cycle such that no two instructions from a thread are in the pipeline concurrently
 - ❑ Hardware has multiple thread contexts (PC+registers per thread)
 - ❑ Threads are completely independent
 - ❑ No instruction is fetched from the same thread until the prior branch/instruction from the thread completes
- + No logic needed for handling control and data dependences within a thread
- + High thread-level throughput
- Single thread performance suffers
- Extra logic for keeping thread contexts
- Throughput loss when there are not enough threads to keep the pipeline full



Each pipeline stage has an instruction from a different, completely-independent thread



Each pipeline stage has an instruction from a different, completely-independent thread

We need a PC and a register file for each thread + muxes and control

Fine-Grained Multithreading (II)

- Idea: Fetch from a different thread every cycle such that no two instructions from a thread are in the pipeline concurrently
- Tolerates control and data dependence resolution latencies by overlapping the latency with useful work from other threads
- Improves pipeline utilization by taking advantage of multiple threads
- Improves thread-level throughput but sacrifices per-thread throughput & latency
- Thornton, “Parallel Operation in the Control Data 6600,” AFIPS 1964.
- Smith, “A pipelined, shared resource MIMD computer,” ICPP 1978.

Fine-Grained Multithreading: History

- CDC 6600's peripheral processing unit is fine-grained multithreaded
 - Thornton, “[Parallel Operation in the Control Data 6600](#),” AFIPS 1964.
 - Processor executes a different I/O thread every cycle
 - An operation from the same thread is executed every 10 cycles

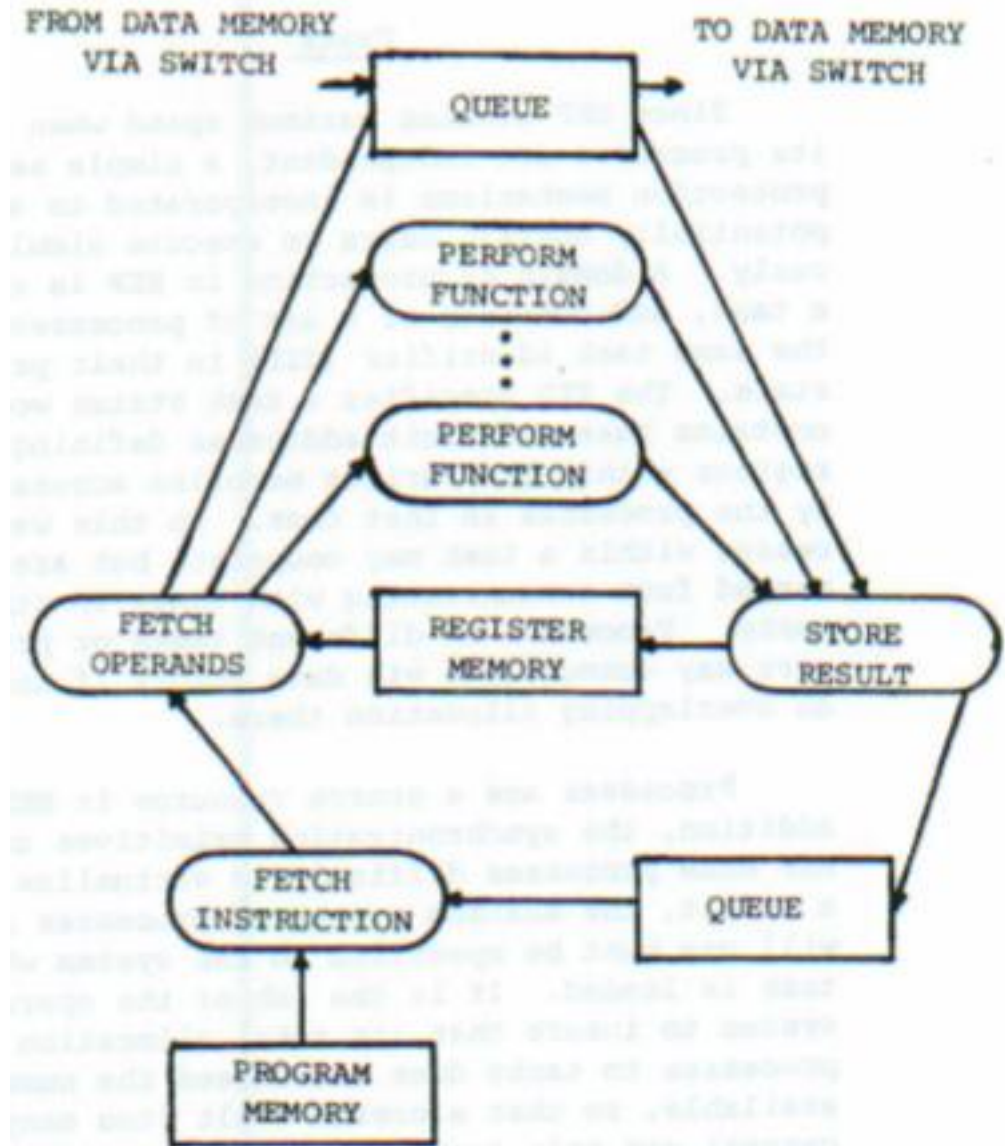
- Denelcor HEP (Heterogeneous Element Processor)
 - Smith, “[A pipelined, shared resource MIMD computer](#),” ICPP 1978.
 - 120 threads/processor
 - Available queue vs. unavailable (waiting) queue for threads
 - Each thread can have only 1 instruction in the processor pipeline
 - Each thread independent
 - To each thread, processor looks like a non-pipelined machine
 - [System throughput vs. single thread performance tradeoff](#)

Fine-Grained Multithreading in HEP

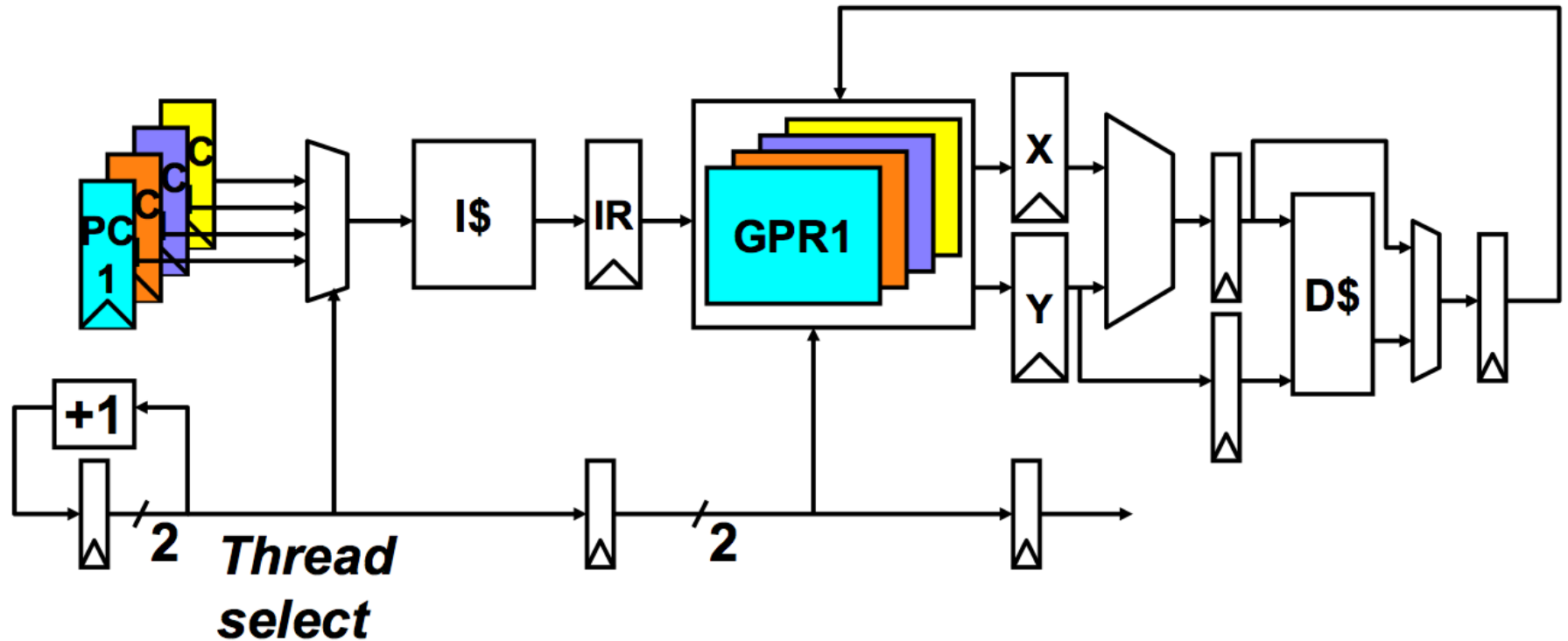
- Cycle time: 100ns
- 8 stages → 800 ns to complete an instruction
 - assuming no memory access
- No control and data dependence checking



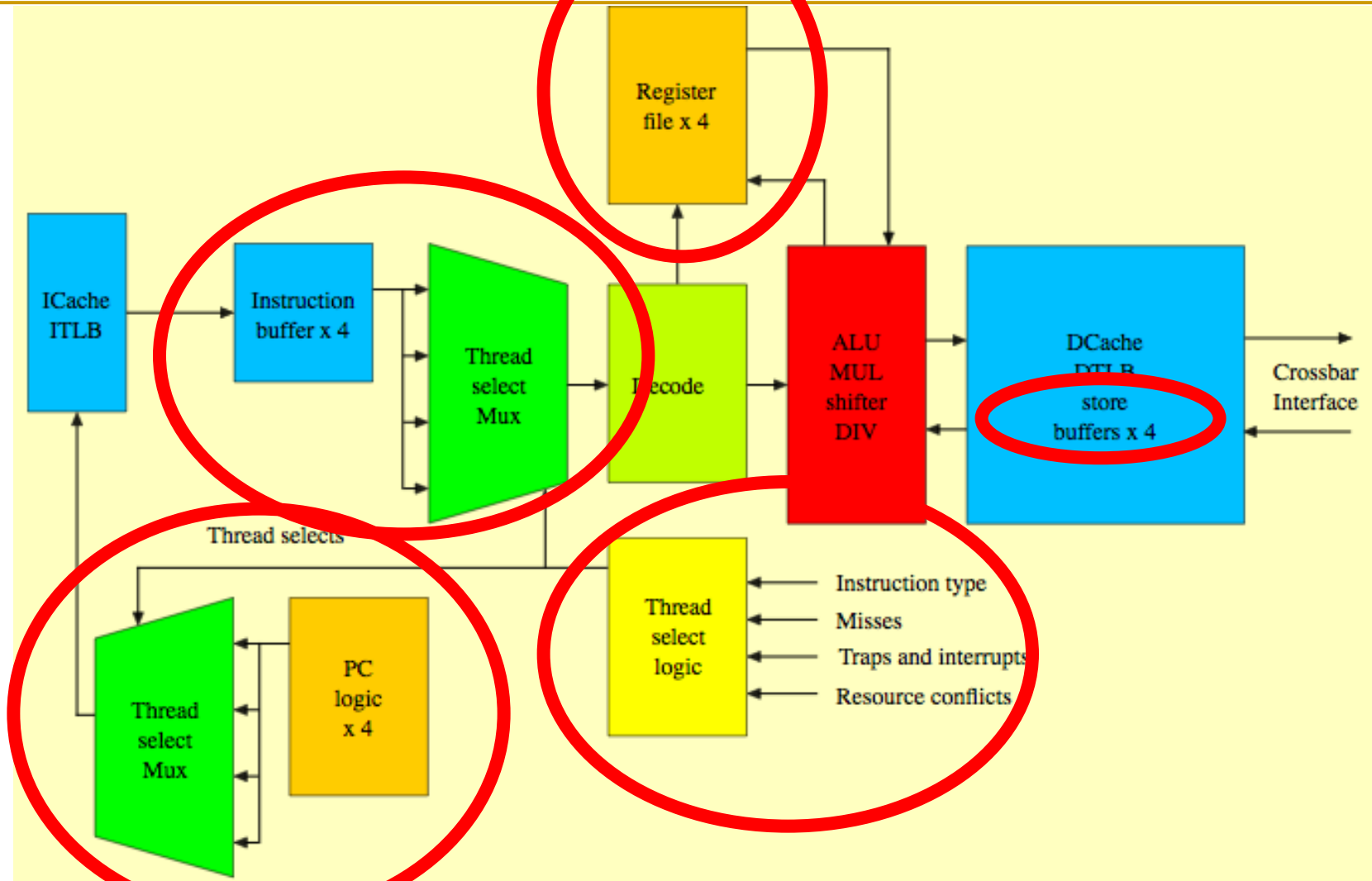
Burton Smith
(1941-2018)



Multithreaded Pipeline Example



Sun Niagara Multithreaded Pipeline



Kongetira et al., "Niagara: A 32-Way Multithreaded Sparc Processor," IEEE Micro 2005.

Fine-Grained Multithreading

■ Advantages

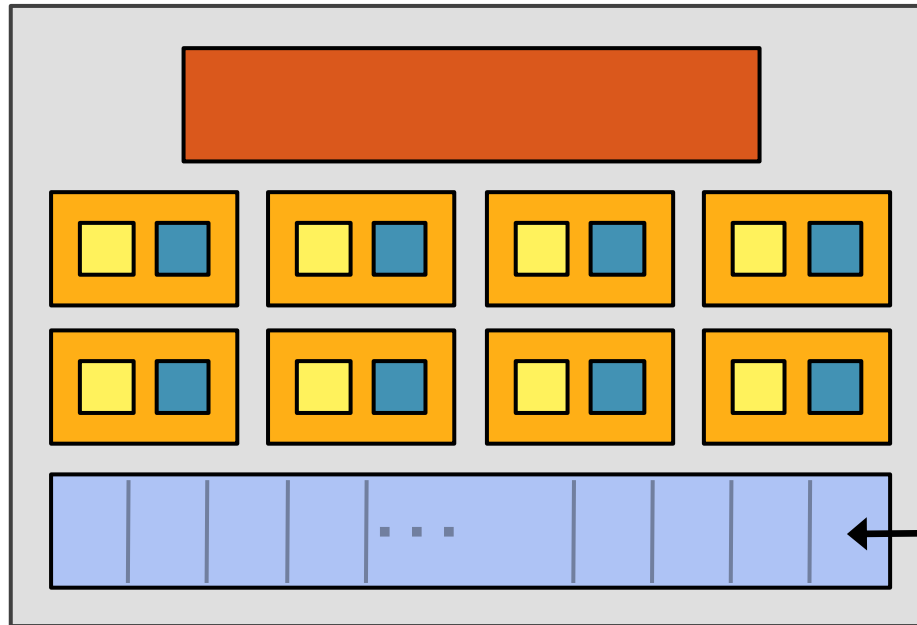
- + No need for dependence checking between instructions
(only one instruction in pipeline from a single thread)
- + No need for branch prediction logic
- + Otherwise-bubble cycles used for executing useful instructions from different threads
- + Improved system throughput, latency tolerance, pipeline utilization

■ Disadvantages

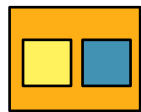
- Extra hardware complexity: multiple hardware contexts (PCs, register files, ...), thread selection logic
- Reduced single thread performance (one instruction fetched every N cycles from the same thread)
- Resource contention between threads in caches and memory
- Dependence checking logic *between* threads may be needed (load/store)

Modern GPUs are FGMT Machines

NVIDIA GeForce GTX 285 “core”



64 KB of storage
for thread contexts
(registers)



= data-parallel (SIMD) func. unit,
control shared across 8 units

■ = multiply-add
■ = multiply

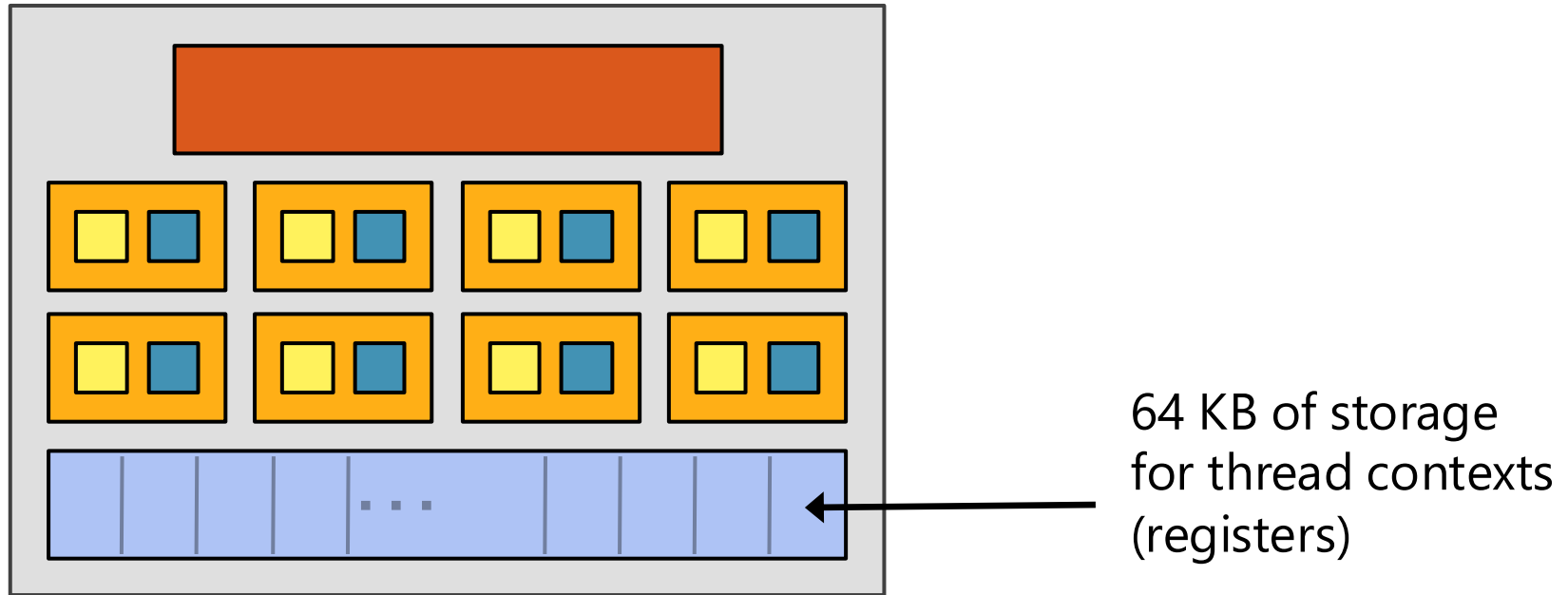


= instruction stream decode



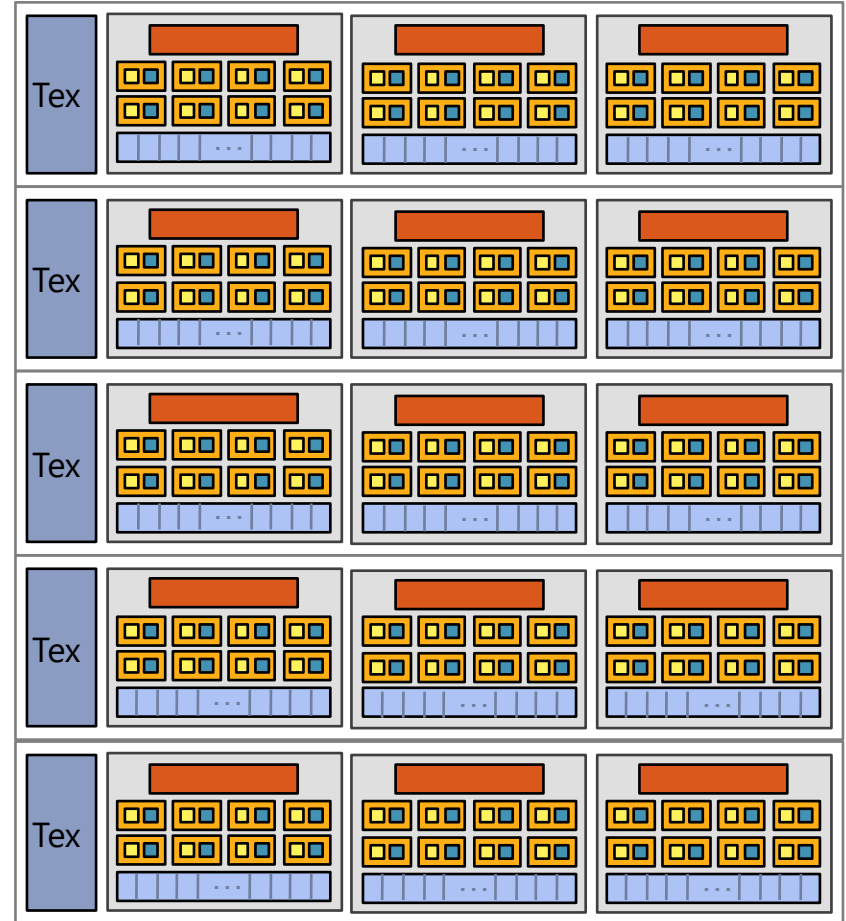
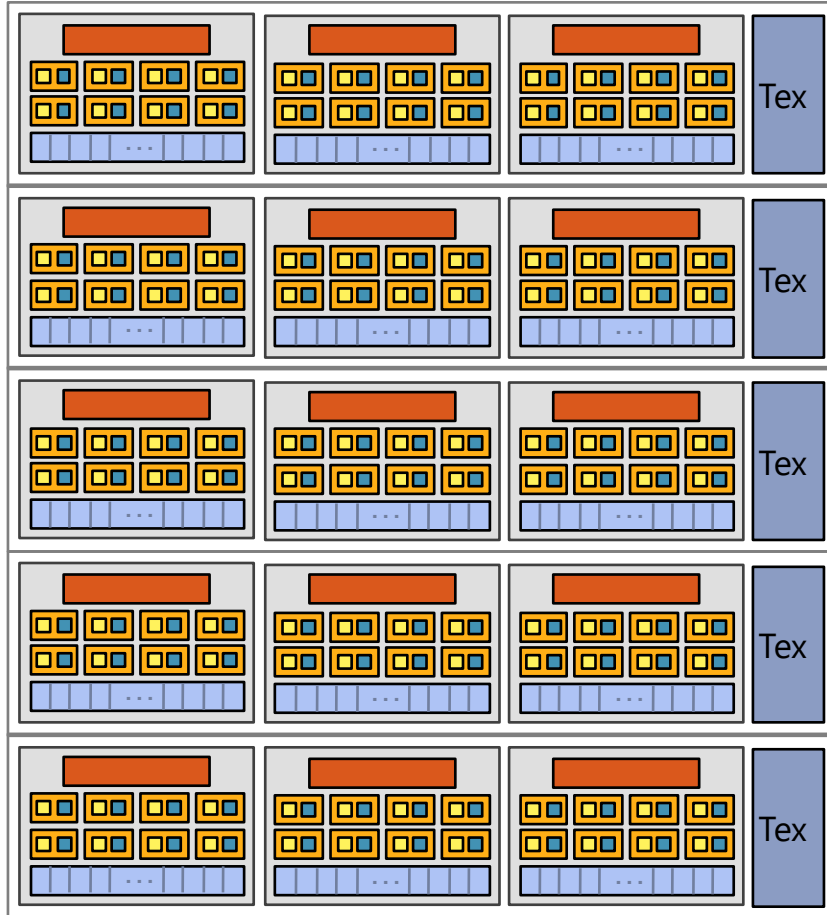
= execution context storage

NVIDIA GeForce GTX 285 “core”



- Groups of 32 **threads** share instruction stream (each group is a Warp): they execute the same instruction on different data
- **Up to 32 warps are interleaved in an FGMT manner**
- Up to 1024 thread contexts can be stored

NVIDIA GeForce GTX 285 (~2009)



30 cores on the GTX 285: 30,720 threads

Further Reading for the Interested (I)

A PIPELINED, SHARED RESOURCE MIMD COMPUTER

Burton J. Smith
Denelcor, Inc.
Denver, Colorado 80205



Burton Smith
(1941-2018)

Architecture and applications of the HEP multiprocessor computer system

Burton J. Smith
Denelcor, Inc., 14221 E. 4th Avenue, Aurora, Colorado 80011

Further Reading for the Interested (II)

The Tera Computer System*

Robert Alverson

David Callahan
Allan Porterfield

Daniel Cummings
Burton Smith

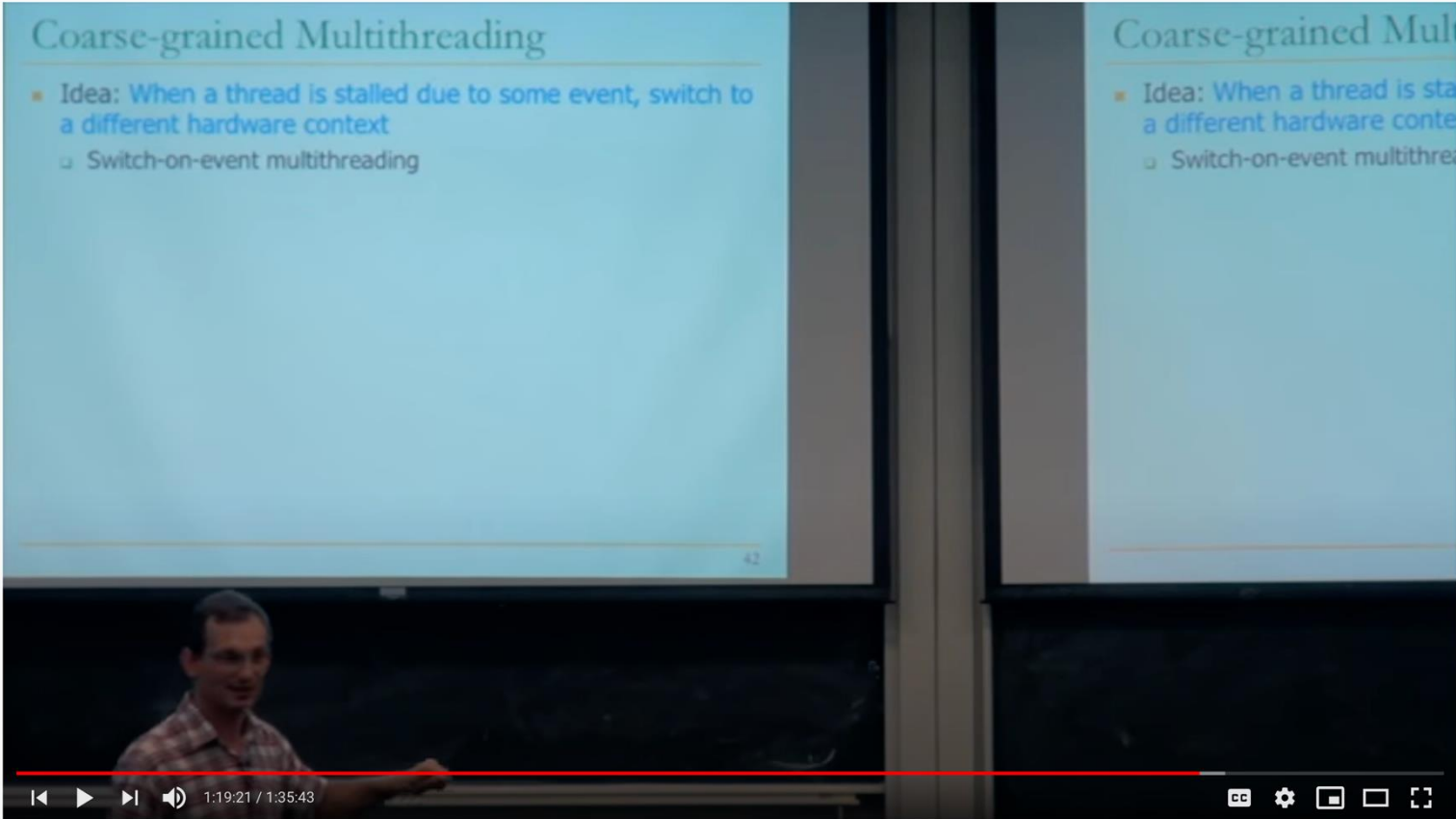
Brian Koblenz

Tera Computer Company
Seattle, Washington USA

4 Processors

Each processor in a Tera computer can execute multiple instruction streams simultaneously. In the current implementation, as few as one or as many as 128 program counters may be active at once. On every tick of the clock, the processor logic selects a stream that is ready to execute and allows it to issue its next instruction. Since instruction interpretation is completely pipelined by the processor and by the network and memories as well, a new instruction from a different stream may be issued in each tick without interfering with its predecessors. When an instruction finishes, the stream to which it belongs thereby becomes ready to execute the next instruction. As long as there are enough instruction streams in the processor so that the average instruction latency is filled with instructions from other streams, the processor is being fully utilized. Thus, it is only necessary to have enough streams to hide the expected latency (perhaps 70 ticks on average); once latency is hidden the processor is running at peak performance and additional streams do not speed the result.

More on Multithreading (I)



The video player shows a lecture slide titled "Coarse-grained Multithreading". The slide content is as follows:

- Idea: When a thread is stalled due to some event, switch to a different hardware context
 - Switch-on-event multithreading

The video player interface includes a progress bar at 1:19:21 / 1:35:43, a like count of 10, and a comment count of 0. The video is titled "Carnegie Mellon - Parallel Computer Architecture 2013 - Onur Mutlu - Lec 9 - Multithreading" and has 1,252 views as of Nov 19, 2013. The channel is "Carnegie Mellon Computer Architecture" with 23K subscribers. The video description includes "Lecture 9: Multithreading", "Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)", and "Date: September 26, 2013.".

Carnegie Mellon - Parallel Computer Architecture 2013 - Onur Mutlu - Lec 9 - Multithreading

1,252 views • Nov 19, 2013

10 0 SHARE SAVE ...

 **Carnegie Mellon Computer Architecture**
23K subscribers

Lecture 9: Multithreading
Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)
Date: September 26, 2013.

ANALYTICS EDIT VIDEO

More on Multithreading (II)

Intel Pentium 4 Hyperthreading

- Long latency load handling
 - Multi-level scheduling window
- More partitioned structures
 - I-TLB
 - Instruction Queues
 - Store buffer
 - Reorder buffer
- 5% area overhead due to SMT

Marr et al., "Hyper-Threading Technology Architecture and Microarchitecture," Intel Technology Journal 2002.

replay loop
4-out
soft line fetch
MICRO HT6

Carnegie Mellon -Parallel Computer Architecture 2012 - Onur Mutlu - Lecture 10 - Multithreading II

1,594 views • Sep 21, 2013

11 0 SHARE SAVE ...



Carnegie Mellon Computer Architecture
1.81K subscribers

Lecture 10: Multithreading II

Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)

Date: September 28, 2012.

SUBSCRIBED



More on Multithreading (III)

g (Tandem, Compaq Himalaya)

microprocessor

Input Multiplexer

Output Multiplexer

Memory covered by ECC
RAID array covered by parity
Servernet covered by CRC

the processor, compare the results of two
re committing an instruction

Lockstepping (Tandem, Compaq Himalaya)

microprocessor

Input Multiplexer

Output Multiplexer

Memory covered by ECC
RAID array covered by parity
Servernet covered by CRC

- Idea: Replicate the processor, compare the results of two processors before committing an instruction

Check $f(x) = 1, x, x^2, x^3$

Carnegie Mellon - Parallel Computer Architecture 2013 - Onur Mutlu - Lec 13-Multi-threading II

1,132 views • Sep 21, 2013

8 0 SHARE SAVE ...



Carnegie Mellon Computer Architecture
1.81K subscribers

Lecture 13: Multi-threading II

Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)

Date: October 5, 2012.

SUBSCRIBED



https://www.youtube.com/watch?v=7vkDpZ1-hHM&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnnyV6D&index=53

More on Multithreading (IV)



Carnegie Mellon - Parallel Computer Architecture 2013 - Onur Mutlu - Lec 15 - Speculation 1

915 views • Sep 21, 2013

9 0 SHARE SAVE ...



Carnegie Mellon Computer Architecture
1.81K subscribers

Lecture 15: Speculation I

Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)

Date: October 10, 2012.

SUBSCRIBED



https://www.youtube.com/watch?v=-hbmzIDe0sA&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnyV6D&index=54

Lectures on Multithreading

■ Parallel Computer Architecture, Fall 2012, Lecture 9

- ❑ Multithreading I (CMU, Fall 2012)
- ❑ https://www.youtube.com/watch?v=iqi9wFqFiNU&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnnyV6D&index=51

■ Parallel Computer Architecture, Fall 2012, Lecture 10

- ❑ Multithreading II (CMU, Fall 2012)
- ❑ https://www.youtube.com/watch?v=e8lfl6MbILg&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnnyV6D&index=52

■ Parallel Computer Architecture, Fall 2012, Lecture 13

- ❑ Multithreading III (CMU, Fall 2012)
- ❑ https://www.youtube.com/watch?v=7vkDpZ1-hHM&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnnyV6D&index=53

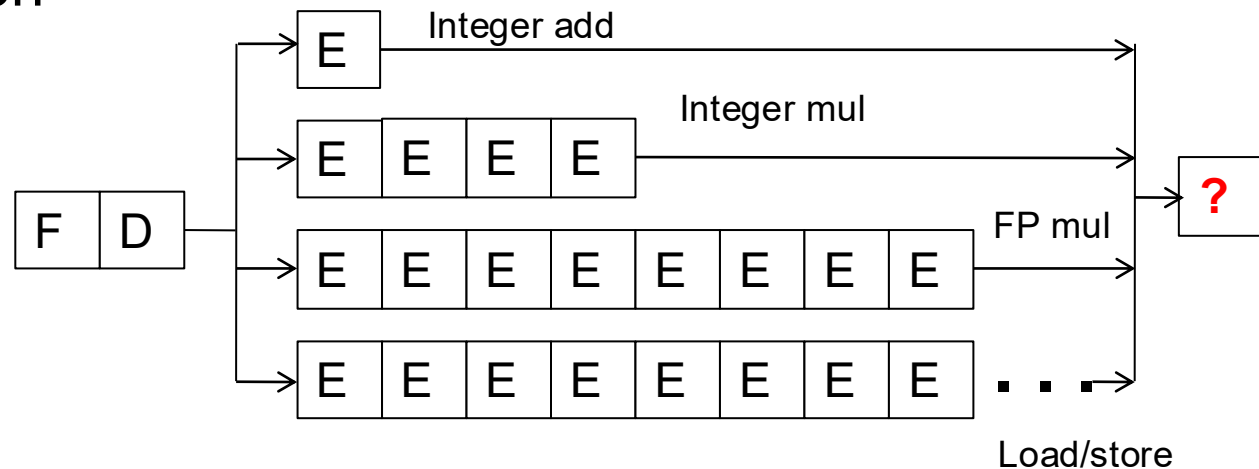
■ Parallel Computer Architecture, Fall 2012, Lecture 15

- ❑ Speculation I (CMU, Fall 2012)
- ❑ https://www.youtube.com/watch?v=-hbmzIDe0sA&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnnyV6D&index=54

Pipelining and Precise Exceptions: Preserving Sequential Semantics

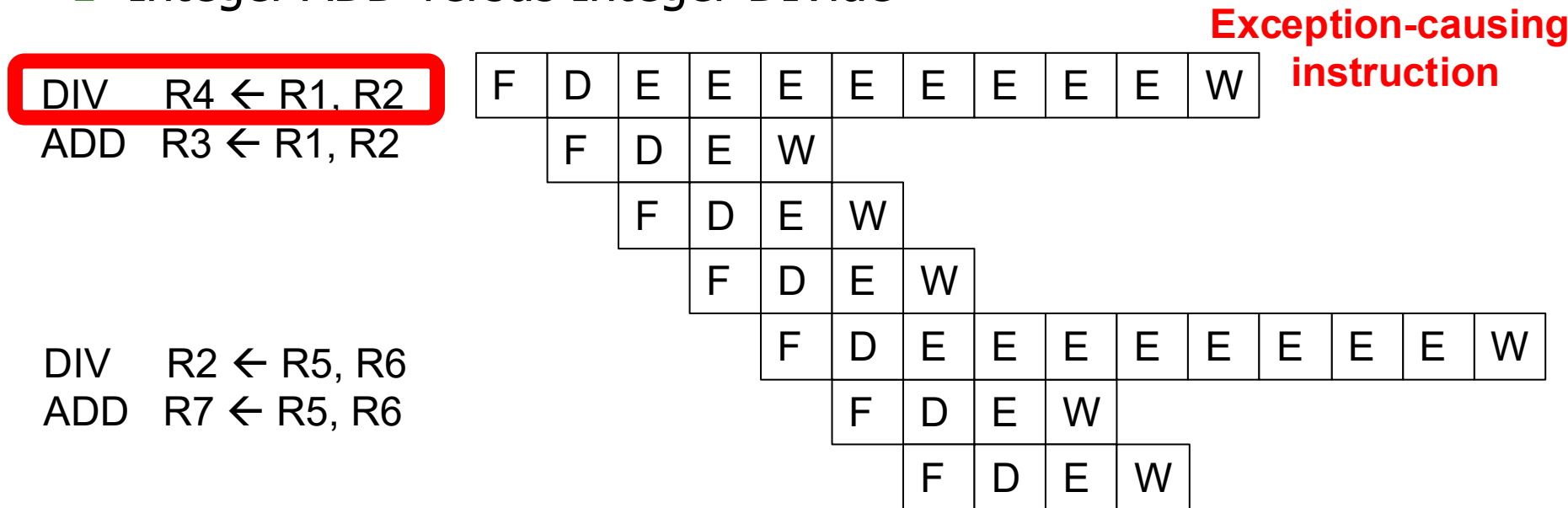
Multi-Cycle Execution

- Not all instructions take the same amount of time in the “execute stage” of the pipeline
- Idea: Have multiple different functional units that take different number of cycles
 - Can be pipelined or not pipelined
 - Can let independent instructions start execution on a different functional unit before a previous long-latency instruction finishes execution



Issues in Pipelining: Multi-Cycle Execute

- Instructions can take different number of cycles in EXECUTE stage
 - Integer ADD versus Integer DIVide



- What is wrong with this picture in a Von Neumann architecture?
 - Sequential semantics of the ISA NOT preserved!
 - What if `DIV` incurs an exception? (e.g., `DIV` by zero)

An Example Exception



Time: 12:55

An Example Exception



Time: 12:57

An Example Exception



Time: 12:58

An Example Exception



Time: 13:00

Another View



Exception Handled & Resolved...



Exceptions and Interrupts

- “Unplanned” changes or interruptions in program execution
- Due to **internal** problems in execution of the program
→ Exceptions
- Due to **external** events that need to be handled by the processor
→ Interrupts
- Both exceptions and interrupts require
 - ❑ stopping of the current program
 - ❑ saving the architectural state
 - ❑ handling the exception/interrupt → switch to handler
 - ❑ (if possible and makes sense) returning back to program execution